book 3

rust 1.99

a complete language manual

Raeliosol

version 1.0, compiled 10 2026

Contents

1. toolchain, cargo, and the rustup lane 7
   1.1. the rustup lane 7
   1.2. cargo's job and rustc's job 8
   1.3. rustc invoked directly 10
   1.4. the offline doc oracle 12
   1.5. clippy, rustfmt, and what each is for 12
   1.6. hosts: msvc and gnu 12
   1.7. the edition field 14
2. from edition 2015 to edition 2024 15
   2.1. what an edition is 15
   2.2. 2015 to 2018: keywords and paths 15
   2.3. 2018 to 2021: arrays, closures, panics 16
   2.4. 2021 to 2024: capture rules and unsafe hygiene 19
   2.5. the migration, measured 22
   2.6. the 1.0 to 1.99 timeline 23
3. lexical structure 26
   3.1. tokens, the grammar's shape 26
   3.2. identifiers and raw identifiers 26
   3.3. integer and float literals 27
   3.4. char, the four byte scalar 29
   3.5. strings, six forms 31
   3.6. comments 32
4. types and values 34
   4.1. the scalar inventory 34
   4.2. inference 35
   4.3. coercion sites and the as table 36
   4.4. tuples, arrays, slices 39
   4.5. unit and never 40
   4.6. const and static 41
   4.7. integer overflow 43
   4.8. type aliases 45
5. ownership and moves 47
   5.1. the ownership graph 47
   5.2. the double free, prevented 47
   5.3. moves in practice 48
   5.4. copy types and the Copy boundary 49
   5.5. drop order, demonstrated 51
   5.6. ownership at function boundaries 55
   5.7. Box, the first heap allocation 55
   5.8. closures capture ownership 57
6. borrowing and lifetimes 60
   6.1. shared and exclusive borrows 60
   6.2. the borrow checker, non lexical 61
   6.3. lifetime parameters 63
   6.4. elision, the three rules 64
   6.5. static 66
   6.6. the error vocabulary 69
7. structs, enums, and patterns 71

7.1. structs: named, tuple, unit ... 71
7.2. the newtype pattern ... 72
7.3. enums as sum types ... 73
7.4. match and exhaustiveness ... 73
7.5. guards, bindings, and at ... 75
7.6. if let, while let, let-else ... 75
7.7. match ergonomics ... 77
7.8. Option and Result ... 77
8. traits and trait objects ... 79
8.1. defining and implementing ... 79
8.2. bounds, supertraits, extension traits ... 80
8.3. associated types and consts ... 80
8.4. coherence and the orphan rule ... 81
8.5. dyn Trait and the vtable ... 82
8.6. derived traits ... 83
8.7. operator overloading ... 85
8.8. Any and downcasting ... 87
9. generics and macros ... 89
9.1. monomorphization versus dynamic dispatch ... 89
9.2. bounds, where, and the turbofish ... 90
9.3. const generics since 1.51 ... 90
9.4. phantom data and zero sized markers ... 92
9.5. macro_rules: repetition and specifiers ... 93
9.6. hygiene ... 94
9.7. proc macros in brief ... 95
10. closures and iterators ... 97
10.1. the three kinds ... 97
10.2. capture modes, measured ... 98
10.3. adapters and laziness ... 102
10.4. writing an iterator ... 103
10.5. collect and the turbofish ... 104
10.6. iter, iter_mut, into_iter ... 105
10.7. iterator invalidation ... 107
11. error handling ... 109
11.1. panic versus result ... 109
11.2. the question mark operator ... 111
11.3. combinators and the conversion traits ... 112
11.4. a custom error enum ... 113
11.5. catch_unwind and the panic machinery ... 116
12. memory and smart pointers ... 120
12.1. layout: repr(C) against repr(Rust) ... 120
12.2. Box: one heap owner ... 121
12.3. drop order ... 123
12.4. Rc: counted clones on one thread ... 124
12.5. Arc: the same count across threads ... 125
12.6. Cell and RefCell: the borrow rule at runtime ... 127
12.7. Weak and cycle breaking ... 129
12.8. Pin in one section ... 132
12.9. the no-gc contrast ... 132
13. concurrency: threads to atomics ... 133
13.1. spawn, join, and move ... 133

13.2. scoped threads, borrowing since 1.63 135
13.3. channels: mpsc 137
13.4. mutex, rwlock, and condvar 140
13.5. atomics and the orderings 143
13.6. Send, Sync, and the honest race claim 148
14. async: futures from first principles 152
14.1. the Future trait and poll 152
14.2. wakers and a tiny executor 154
14.3. async fn is a state machine 156
14.4. join, by hand 158
14.5. timers through a thread and channel bridge 159
14.6. select and cancellation 160
14.7. what tokio adds 162
15. the standard library 164
15.1. the core collections 164
15.2. strings and utf-8 boundaries 168
15.3. io: readers and writers 170
15.4. paths 170
15.5. time 172
15.6. sockets: a localhost echo pair 173
15.7. environment and processes 175
16. crates, modules, and workspaces 179
16.1. modules and paths 179
16.2. use and re-exports 182
16.3. the manifest, field by field 184
16.4. cargo test organization 185
17. unsafe and ffi 188
17.1. the five superpowers 188
17.2. split_at_mut, the trick in the standard library 190
17.3. uninitialized memory and transmute 191
17.4. calling into the c runtime, measured on msvc 193
17.5. earning unsafe: a safe wrapper 195
18. testing and idioms 199
18.1. the attributes and their meanings 199
18.2. doc tests 202
18.3. hand-rolled benches 203
18.4. clippy and rustfmt governance 206
18.5. the idiom roundup 206
19. http kernel 210
19.1. the socket is the whole server 210
19.2. the parser: request line, headers, bodies 211
19.3. the route table 214
19.4. the error envelope 215
19.5. json by hand 217
19.6. the decode ladder 219
19.7. the bounded pool and the drain 220
19.8. testing through a real socket 222
20. middleware 227
20.1. layers are functions, the chain composes them 227
20.2. request id, minted and quoted once 227
20.3. recover, the panic lane 228

20.4. the access log needs no wrapper 229
20.5. timing on the unwind 230
20.6. wiring order, and the tests that pin it 230
21. the user resource 232
21.1. the representation and its two projections 232
21.2. create, validation, and the 422 versus 409 boundary 234
21.3. the store seam 237
21.4. the password seam, primitives not crates 238
21.5. keyset pagination with frozen cursor bytes 239
21.6. get and the etag 241
21.7. the vectors drive the tests 242
22. authn 244
22.1. the token, three segments and one primitive 244
22.2. verification, order and constant time 246
22.3. the session family and reuse detection 248
22.4. login, prove then issue, both branches the same cost 251
22.5. refresh and logout, the routes that move the family 253
22.6. the identity layer and its two channels 254
22.7. the wiring and the tests 256
23. authz and rbac 260
23.1. authentication versus authorization 260
23.2. policies as functions 260
23.3. object-level authorization 262
23.4. roles as stored data and the never-zero-admins guard 262
23.5. the table and deny by default 265
23.6. the grant surface 267
23.7. the matrix and the races 269
24. store 273
24.1. the frame, crc32c by hand 273
24.2. replay and the torn tail 275
24.3. append then mutate, one writer 278
24.4. compaction, rewrite and atomic swap 278
24.5. the user store over the engine 279
24.6. the wiring and the reboot proof 282
25. concurrency control 284
25.1. the note, and the tag it carries 284
25.2. the conditional read 285
25.3. the guarded write 285
25.4. compare-and-swap over the engine 286
25.5. idempotency keys, the stored response window 287
25.6. the count 291
26. caching 292
26.1. ttl from first principles, lazy on the read 292
26.2. the bound and the eviction rank 292
26.3. the eager sweep 293
26.4. single flight, one fill per key 294
26.5. the recheck window and the failed fill 296
26.6. the layer at the wire 297
26.7. the count 299
27. rate limiting 300
27.1. why limits exist 300

27.2. the token bucket, all integer ... 300
27.3. the 429 contract through the ch19 writer ... 302
27.4. what to key on, and the address seam ... 303
27.5. bounded per-key state ... 303
27.6. the wiring and the stack order ... 304
27.7. the count ... 304
28. observability ... 306
28.1. the two series ... 306
28.2. the text exposition ... 307
28.3. the ops port ... 309
28.4. the layer at the edge ... 309
28.5. the ring and the exact tail ... 311
28.6. p99 from buckets ... 312
28.7. one request fully observed ... 313
29. load and p99 ... 314
29.1. the client, plain std::net ... 314
29.2. the two loops ... 315
29.3. the schedule is pure integer math ... 317
29.4. warmup and the stop signal ... 317
29.5. the report, every number an integer ... 317
29.6. the ladder, measured ... 319
29.7. the budget, measured on the shipped binary ... 319
29.8. what the run leaves behind ... 320
30. ship it ... 321
30.1. the ship as data ... 321
30.2. the image, two stages ... 322
30.3. the binary as its own doctor ... 323
30.4. compose, one command ... 323
30.5. the docker lane as a gate ... 324
30.6. the shipped checklist ... 325
31. the test suite ... 326
31.1. the inventory ... 326
31.2. the one lane gate ... 326
31.3. the two-lane contract ... 327
31.4. the deterministic seams ... 327
31.5. the adversarial history ... 327
31.6. what the suite does not hold ... 328
31.7. what the suite proves ... 328
32. capstone: an async runtime from scratch ... 329
32.1. the smallest executor ... 329
32.2. timers, the wheel, and select ordering ... 332
32.3. the blocking pool and the io bridge ... 335
32.4. the reactor and the concurrent server ... 336
32.5. what tokio adds ... 342
32.6. what the capstone proves ... 342
33. appendices: coverage and sources ... 344
33.1. keyword coverage ... 344
33.2. stdlib coverage ... 346
33.3. pinned sources ... 347

1. toolchain, cargo, and the rustup lane

Rust does not ship as one compiler binary. Rustup installs toolchains, each toolchain carries the compiler plus a set of components, and Cargo drives the compiler over a manifest. This book pins the stable msvc lane at rustc 1.99.0 (b940084d7 2026-09-28), rustup 1.29.1 manages it, and every sample in the corpus compiles as a single file under `rustc --edition 2024 -D warnings`. The gates resolve the toolchain by directory and assert the full version string before anything compiles, so a wrong toolchain is a loud red, never a silent drift. Go makes the opposite bet, one binary holding every subcommand, the trade the book 4, ch 1 (go 1.27, toolchain and modules) chapter documents from its side.

1.1. the rustup lane

A toolchain name is channel plus host triple: `stable-x86_64-pc-windows-msvc`. Rustup keeps one directory per toolchain under `RUSTUP_HOME`, defaulting to `~/.rustup`, and the `cargo` and `rustc` on PATH are proxies that dispatch to the active toolchain. Three rules pick the active one, in order: a directory override from `rustup override set`, a `rust-toolchain.toml` file naming a channel, then the default channel. Measured on this machine 2026-10-13: two toolchains installed, stable as the default plus a nightly the gates exclude, and dropping a `rust-toolchain.toml` containing `channel = "stable"` into a directory makes `rustup show active-toolchain` report the file as the override source.

```
fn edition_shipped(edition: &str) -> Option<(u32, u32)> {
    match edition {
        "2015" => Some((1, 0)),
        "2018" => Some((1, 31)),
        "2021" => Some((1, 56)),
        "2024" => Some((1, 85)),
        _ => None,
    }
}

fn split_triple(triple: &str) -> (String, String, String, Option<String>) {
    let mut parts = triple.split('-');
    let arch = parts.next().unwrap_or_default().to_string();
    let vendor = parts.next().unwrap_or_default().to_string();
    let os = parts.next().unwrap_or_default().to_string();
    let abi = parts.next().map(str::to_string);
    (arch, vendor, os, abi)
}

fn channel_of(spec: &str) -> &str {
    spec.split('-').next().unwrap_or_default()
}

fn rustup_home<'a>(override_home: Option<&'a str>, default_home: &'a str) -> &'a str {
    override_home.unwrap_or(default_home)
}
```

listing 1: the name grammar: edition to shipping version, triple fields, channel prefix, home override (source: rust/samples/src/Ch01/Toolchain.rs, lines 56-81)

Each component installs once per toolchain, and this lane carries 8 of them, measured with `rustup component list`: `cargo`, `clippy`, `llvm-tools`, `rust-docs`, `rust-src`, `rust-std`, `rustc`, `rustfmt`. The standard library is itself a component, `rust-std-<triple>`, one per target, which is how cross compilation pays its entry fee later in this chapter.

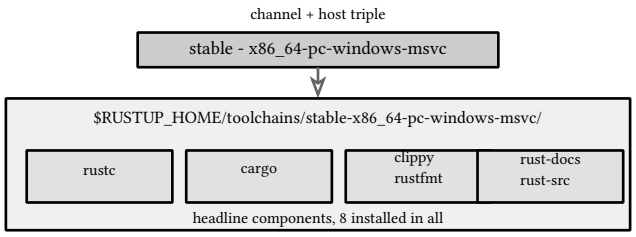


figure 1: one rustup lane: the name grammar on top, the directory it maps to, the components inside

NOTE: why the directory matters

The gates prepend the toolchain's `bin` directory to PATH themselves instead of trusting whatever `rustc` resolves to, then assert the exact pinned prefix of the version line. A PATH pointing at the nightly, or at a system rustc from a package manager, is rejected before a single sample compiles.

1.2. cargo's job and rustc's job

Cargo reads `Cargo.toml`, resolves the dependency graph, lowers build profiles into rustc flags, and invokes rustc once per crate in that graph. Rustc compiles exactly one crate root, the file it is given, and knows nothing about manifests or versions or lockfiles. The split is the whole configuration surface: dependency decisions live in the manifest, language decisions live in rustc flags, and Cargo is the translator between them.

The day to day command set is small. `cargo new` scaffolds the manifest and the `src` tree, `cargo build` compiles into `target/debug` or, with `--release`, into `target/release`, `cargo run` builds and then executes, `cargo test` runs the unit tests and then the doc tests as a second leg, and `cargo doc` renders html from doc comments. The doc test leg deserves one measured fact: on this lane a library crate with one fenced example in a doc comment reports `running 0 tests` for the unit leg and `1 passed` for a doctest leg that compiled the fence as its own binary with the crate's public items in scope. A doc comment is a compiled artifact, not a comment. Chapter 18 owns testing whole.

`cargo new hello` on this lane writes a manifest with `edition = "2024"`, measured 2026-10-13 with cargo 1.99.0. New projects get the newest stable edition. A manifest with no edition field compiles as 2015, forever, because the edition is an opt in and its default never moves under an existing project.

```
fn parse_manifest(text: &str) -> Manifest {
    let mut name = String::new();
    let mut edition = String::new();
    let mut dependencies = Vec::new();
    let mut section = "";
    for raw in text.lines() {
        let line = raw.trim();
        if line.starts_with('[') && line.ends_with(']') {
            section = &line[1..line.len() - 1];
            continue;
        }
        if line.is_empty() || line.starts_with('#') {
            continue;
        }
        if let Some((key, value)) = line.split_once('=') {
            let key = key.trim();
            let value = value.trim().trim_matches('"');
            match (section, key) {
                ("package", "name") => name = value.to_string(),
                ("package", "edition") => edition = value.to_string(),
                ("dependencies", _) => dependencies.push(key.to_string()),
                _ => {}
            }
        }
    }
    Manifest {
        name,
        edition,
        dependencies,
    }
}

fn effective_edition(m: &Manifest) -> &str {
    if m.edition.is_empty() {
        "2015"
    } else {
        &m.edition
    }
}
```

listing 2: a manifest reader: package fields, the dependency table, and the 2015 default when the edition field is absent (source: rust/samples/src/Ch01/CargoVsRustc.rs, lines 19-57)

The profiles are the other half of the translation. Cargo's book fixes the defaults: `dev` runs at opt-level 0 with debug assertions on, `release` at opt-level 3 with them off, and each profile owns one output directory under `target`. The dev profile's directory is `target/debug`, the name differs from the profile on purpose.

```
struct Profile {
    name: &'static str,
    opt_level: u8,
    debug_assertions: bool,
}

fn profile(name: &str) -> Option<Profile> {
    match name {
        "dev" => Some(Profile {
            name: "dev",
            opt_level: 0,
            debug_assertions: true,
        }),
        "release" => Some(Profile {
            name: "release",
            opt_level: 3,
            debug_assertions: false,
        }),
        _ => None,
    }
}

fn rustc_args(p: &Profile) -> String {
    format!(
        "--edition 2024 -C opt-level={} -C debug-assertions={}",
        p.opt_level,
        if p.debug_assertions { "on" } else { "off" }
    )
}

fn target_dir(p: &Profile) -> String {
    let dir = if p.name == "dev" { "debug" } else { "release" };
    format!("target/{dir}")
}
```

listing 3: the two profiles this book uses and their lowering into rustc codegen flags (source: rust/samples/src/Ch01/CargoVsRustc.rs, lines 59-92)

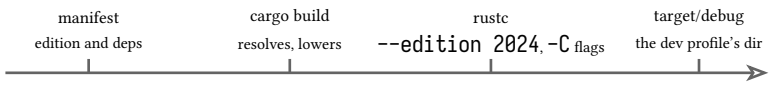


figure 2: cargo's translation: manifest and profile in, one rustc invocation per crate out

1.3. rustc invoked directly

Rustc alone is the right tool for single-file programs with zero dependencies, for learning the language without a build system in the way, and for probing the compiler itself. All three describe this book's samples lane: `tools/run-rust-samples.ps1` compiles each sample with `rustc --edition 2024 -D warnings`, runs the binary, and fails on any error or on a binary that prints no `ok` lines. The pinning law that guards the lane:

```
$tcBin = Join-Path $rustupHome "toolchains\stable-x86_64-pc-windows-msvc\bin"
$env:PATH = "$tcBin;$env:PATH"
$versionLine = & $rustc --version | Select-Object -First 1
if (-not "$versionLine".StartsWith("rustc 1.99.0 (b940084d7")) {
```

```
  throw "verify-rust: expected rustc 1.99.0 (b940084d7...), found: $versionLine"
}
```

fragment, illustrative, not executed

Rustc also answers questions about the machine. `--print cfg` dumps the host's cfg values, `target_arch="x86_64"`, `target_env="msvc"`, `target_os="windows"`, `target_vendor="pc"`, all measured on this lane. `--print target-list` counted 331 targets on the same day. The version line itself is a parseable contract:

```
struct Version {
    major: u32,
    minor: u32,
    patch: u32,
    pre: Option<String>,
}

impl Version {
    fn parse(text: &str) -> Option<Version> {
        let body = text.strip_prefix("rustc ")?;
        let core = body.split(' ').next()?;
        let mut parts = core.split('-');
        let numbers = parts.next()?;
        let pre = parts.next().map(str::to_string);
        let mut fields = numbers.split('.');
        let major = fields.next()?.parse().ok()?;
        let minor = fields.next()?.parse().ok()?;
        let patch = fields.next()?.parse().ok()?;
        if fields.next().is_some() {
            return None;
        }
        Some(Version {
            major,
            minor,
            patch,
            pre,
        })
    }

    fn precedence(&self, other: &Version) -> Ordering {
        (self.major, self.minor, self.patch)
            .cmp(&(other.major, other.minor, other.patch))
            .then_with(|| match (&self.pre, &other.pre) {
                (None, None) => Ordering::Equal,
                (None, Some(_)) => Ordering::Greater,
                (Some(_), None) => Ordering::Less,
                (Some(a), Some(b)) => a.cmp(b),
            })
    }
}
```

listing 4: the version string as a type: parse, pre-release precedence, numeric field order (source: rust/samples/src/Ch01/Toolchain.rs, lines 15-54)

1.4. the offline doc oracle

`rustup doc --book --path` prints `~/.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html/book/index.html` on this machine, measured 2026-10-13. The `rust-docs` component installs that whole html tree: `book/`, `reference/`, `rust-by-example/`, `edition-guide/`, `cargo/`, `std/`, the release notes as `releases.md`, and the error index. This book grounds every claim against that tree and cites it in the stable URL form, `https://doc.rust-lang.org/stable/book/` and its siblings, with the access date recorded in each chapter's sources paragraph.

The oracle cannot drift from the compiler because both arrive in one toolchain: the docs describe 1.99.0, the compiler is 1.99.0, and upgrading the toolchain upgrades both in the same atomic step. The edition guide is the spine for chapter 2, `releases.md` is the spine for the 1.0 through 1.99 timeline, and the reference settles grammar questions the book chapters only summarize.

1.5. clippy, rustfmt, and what each is for

Clippy 0.1.99 (b940084d7e 2026-09-28, the same commit as rustc, measured) is the lint collection above rustc's own: idiom lints, correctness lints, and pedantic groups, driven with `cargo clippy` and repaired with `cargo clippy --fix`. Its history runs in both directions, because rustc lints get uplifted from clippy when they prove themselves, and `releases.md` records the receipts: 1.72 lifted `clippy::cast_ref_to_mut` into `invalid_reference_casting`, 1.86 lifted `clippy::double_neg` into `double_negations`. Scoping came with the edition system itself: since 1.31, tool lints let an attribute like `#[allow(clippy::too_many_arguments)]` target a foreign tool's lint by name without a compiler flag.

Rustfmt 1.10.0 is the formatter, driven with `cargo fmt`. Unlike go's single non-negotiable format, rustfmt is configurable and editions carry their own style editions, the 2024 edition adjusts raw identifier sorting and version sorting among others, so a reformat belongs at the end of an edition migration.

NOTE: the corpus split

The samples lane never runs rustfmt or clippy: samples hold to rustc lints only, `-D warnings`, because a formatting pass would churn the pinned line ranges every chapter lists. The service part's crates run the full chain instead, `cargo fmt --check` and `cargo clippy --all-targets -- -D warnings`, under `make verify-rust`.

1.6. hosts: msvc and gnu

Windows has two tier 1 hosts and the triple's fourth field names the difference. `x86_64-pc-windows-msvc` targets the MSVC ABI and links with the Visual Studio toolchain, `x86_64-pc-windows-gnu` rides MinGW-w64 and links with gcc, and `x86_64-pc-windows-gnullvm`, tier 2, is the LLVM ABI row. One cfg value separates the first two, `target_env="msvc"` against `target_env="gnu"`, and building the gnu host from an msvc machine is a cross: the target's `rust-std` component must be installed first through `rustup target add`. This book pins the msvc host, the tier 1 default rustup chooses on this machine.

```
fn parse_target(triple: &str) -> Option<Target> {
    let parts: Vec<&str> = triple.split('-').collect();
    match parts.as_slice() {
        [arch, vendor, os] => Some(Target {
            arch: arch.to_string(),
            vendor: vendor.to_string(),
            os: os.to_string(),
            abi: None,
        }),
        [arch, vendor, os, abi] => Some(Target {
            arch: arch.to_string(),
            vendor: vendor.to_string(),
            os: os.to_string(),
            abi: Some(abi.to_string()),
        }),
        _ => None,
    }
}
```

listing 5: the triple grammar: three fields plus an optional abi, anything else rejected (source: rust/samples/src/Ch01/Triples.rs, lines 20-37)

```rust
fn pointer_width(arch: &str) -> Option<u32> {
    match arch {
        "x86_64" | "aarch64" => Some(8),
        "i686" | "armv7" => Some(4),
        _ => None,
    }
}

fn tier(triple: &str) -> Option<u8> {
    match triple {
        "x86_64-pc-windows-msvc" | "x86_64-pc-windows-gnu" | "x86_64-unknown-linux-gnu" => Some(1),
        "x86_64-pc-windows-gnullvm" => Some(2),
        _ => None,
    }
}

fn is_cross(host: &str, target: &str) -> bool {
    host != target
}

fn cfg_env(triple: &str) -> &'static str {
    match triple {
        "x86_64-pc-windows-msvc" => "msvc",
        "x86_64-pc-windows-gnu" => "gnu",
        _ => "unknown",
    }
}

fn std_component(triple: &str) -> String {
    format!("rust-std-{triple}")
}

fn toolchain_spec(channel: &str, host_triple: &str) -> String {
    format!("{channel}-{host_triple}")
}
```

listing 6: pointer widths, tier lookup, the cfg environment split, cross detection, component and toolchain naming (source: rust/samples/src/Ch01/Triples.rs, lines 39-74)

1.7. the edition field

The manifest's `edition` field is the whole edition system from Cargo's side. It is per crate, it defaults to 2015 when absent, and it never affects other crates in the same build: a 2015 crate and a 2024 crate link into one binary without ceremony, because every edition compiles down to the same internal representation in the compiler. What the field buys is the right to the incompatible changes each rung introduced, new keywords, new capture rules, denied footguns, and the ladder from 2015 to 2024 with its worked migrations is chapter 2.

sources: doc.rust-lang.org/stable/edition-guide, editions index and transitioning chapters, accessed 2026-10-13. doc.rust-lang.org/stable/cargo, profiles reference and manifest reference, accessed 2026-10-13. doc.rust-lang.org/stable/rustc, platform support tiers, accessed 2026-10-13. releases.md from the offline rust-docs component, 1.31, 1.72, and 1.86 sections, read 2026-10-13. Machine facts measured live on the pinned lane 2026-10-13: rustc and cargo and rustfmt and clippy version lines, rustup 1.29.1 show, toolchain list, component list, rust-toolchain.toml override probe, cargo new manifest capture, cargo build –edition rejection, rustup doc –book –path, rustc –print cfg, rustc –print target-list count. Sample evidence: Ch01 4 files 50 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1`
-Chapter Ch01, 2026-10-13.

2. from edition 2015 to edition 2024

Every feature in this book carries the version or edition that shipped it: “async/await arrived stable in 1.39”, “edition 2021 changed array IntoIterator”. This chapter is where those numbers come from. Editions are Rust’s mechanism for incompatible change without breaking the ecosystem. There are four of them so far, and this chapter walks all four as worked migrations, 2015 to 2018, 2018 to 2021, 2021 to 2024, then closes with the compiler timeline the rungs hang from. Java walks its own version ladder the same way in book 5, ch 2 (java 27, from java 8 to java 27).

2.1. what an edition is

Rust 1.0 landed in may 2015 with a promise, stability without stagnation: once a feature ships on stable, every later compiler keeps compiling it. But some changes are incompatible by nature. A new keyword breaks every variable already named with it, and a closure that captures less than before can change when destructors run. The edition system is the escape valve: incompatible changes pile up behind an opt in, and a crate states which rule set it compiles under through one manifest field.

The constraint that shapes every edition decision is that editions never split the ecosystem. A 2015 crate links against a 2024 crate without ceremony, because the choice is private to each crate and all editions compile down to the same internal representation in the compiler. The changes an edition may carry are correspondingly shallow in machine terms, a keyword here, a capture rule there, a denied lint, even when they rewrite how every file in a project reads.

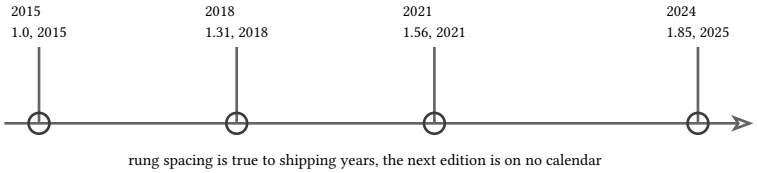


figure 3: four rungs in ten years, each ships in the release that stabilized it

Migration between rungs is tooling, not archaeology: the `cargo fix` subcommand, run with `--edition`, applies the migration lints for the next edition and rewrites the code. The whole hop is five commands:

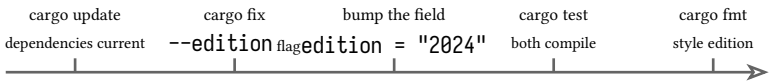


figure 4: the edition hop as cargo runs it, five commands from warning free to reformatted

After the hop the code compiles under both editions, which is why a large dependency graph migrates crate by crate with nothing waiting on anything else.

2.2. 2015 to 2018: keywords and paths

Edition 2018 shipped in 1.31 (2018-12-06) under RFC 2052, the RFC that also proposed the edition system, and it front-loaded two language cleanups the first three years had earned.

`dyn` became a strict keyword. Bare trait object syntax, `Box<Trait>`, was ambiguous with generic syntax and read badly beside `impl Trait`, so trait objects gained their own marker:

```
// 2015
let sink: Box<Write> = Box::new(Vec::new());

// 2018
let sink: Box<dyn Write> = Box::new(Vec::new());
```

fragment, illustrative, not executed

The `dyn Trait` syntax arrived in 1.27 (2018-06-21), usable in every edition, and 2018 made it the required spelling while reserving `async` and `await` as strict keywords in 2018 crates, ahead of async/await stabilizing in 1.39 (2019-11-07), and `try` for a feature that has still not shipped.

The module system got the bigger rework. `extern crate` disappeared from ordinary code, paths in `use` declarations started working the same as paths everywhere else, `crate::` names the current crate's root from anywhere, and a module's submodules stopped needing the special name `mod.rs`:

```
// 2015
extern crate serde;
use serde::Serialize;

// 2018
use serde::Serialize;
```

fragment, illustrative, not executed

```
2015                        2018
src/                        src/
+- lib.rs                   +- lib.rs
+- foo/                     +- foo.rs
   +- mod.rs                +- foo/
   +- bar.rs                   +- bar.rs
```

fragment, illustrative, not executed

A dependency now enters through `Cargo.toml` alone, and the compiler resolves its name in every module. Renamed crates moved from `extern crate futures as f` to `use futures as f`, per module.

2.3. 2018 to 2021: arrays, closures, panics

Edition 2021 shipped in 1.56 (2021-10-21) and its three visible changes all removed traps rather than adding features.

Arrays became iterable by value. Until 1.53 only `&[T; N]` and `&mut [T; N]` implemented `IntoIterator`, so `for e in [1, 2, 3]` failed to compile. The trait implementation landed in 1.53 (2021-06-17) in every edition, with one hack: method call syntax, `array.into_iter()`, kept resolving to `(&array).into_iter()` in 2015 and 2018 crates, because too much code depended on the borrow. Edition 2021 removed the hack, and the method call finally yields owned values:

```
// 2018: x is &u8, the call secretly borrows
for x in array.into_iter() {}

// 2021: x is u8, the call consumes
for x in array.into_iter() {}
```

fragment, illustrative, not executed

```rust
fn main() {
    let mut total = 0;
    for e in [1, 2, 3] {
        total += takes_value(e);
    }
    ok(
        total == 6,
        "for over an array yields i32 by value since 1.53, all editions",
    );
    let owned: Vec<i32> = [1, 2, 3].into_iter().collect();
    ok(
        owned == vec![1, 2, 3],
        "array.into_iter() yields owned values in edition 2021 and later",
    );
    let fully_qualified: Vec<i32> = IntoIterator::into_iter([4, 5, 6]).collect();
    ok(
        fully_qualified == vec![4, 5, 6],
        "the fully qualified call also yields values, no method-call hack applies",
    );
    let refs: Vec<i32> = [1, 2, 3].iter().map(|r| takes_ref(r)).collect();
    ok(
        refs == vec![1, 2, 3],
        "iter() still borrows, references in every edition",
    );
    let zipped: i32 = [1, 2, 3]
        .iter()
        .zip([10, 20, 30])
        .map(|(a, b)| a * b)
        .sum();
    ok(
        zipped == 140,
        "zip over an array argument consumes it by value since 1.53",
    );
    let words = ["a".to_string(), "b".to_string()];
    let joined = words.into_iter().collect::<Vec<_>>().join("-");
    ok(
        joined == "a-b",
        "into_iter moves non-Copy elements out of the array",
    );
    let survivors = [7, 8, 9];
```

listing 7: the 2021 behaviors asserted: by-value for and into_iter, the fully qualified form, iter's borrow, zip consuming an array (source: rust/samples/src/Ch02/Arrays2021.rs, lines 21-60)

The `array_into_iter` lint warns on every call that changes meaning, on all editions since 1.41, so warning-free 2018 code was already 2021-ready.

Closures started capturing disjoint fields, RFC 2229. In 2018 and before, a closure mentioning `a.x` captures all of `a`, which blocks moves of sibling fields and drags non-Send fields into threads. From 2021 the closure captures `a.x` and nothing else:

```rust
// 2018: error, the closure wants all of a
drop(a.x);
let c = || println!("{}", a.y);

// 2021: compiles, the closure captures a.y only
```

```
drop(a.x);
let c = || println!("{}", a.y);
```

fragment, illustrative, not executed

Capturing less can change drop order and which auto traits a closure implements, the two behaviors the sample measures directly:

```
fn main() {
    {
        let pair = (Tag("disjoint-0"), Tag("disjoint-1"));
        {
            let c = || drop(pair.0);
            c();
        }
        log_event("mid");
    }
    ok(
        log_snapshot() == ["disjoint-0", "mid", "disjoint-1"],
        "disjoint capture drops only pair.0 with the closure, pair.1 waits for the scope",
    );
    {
        let pair = (Tag("whole-0"), Tag("whole-1"));
        {
            let c = || {
                let _ = &pair;
                drop(pair.0);
            };
            c();
        }
        log_event("mid2");
    }
    ok(
        log_snapshot().ends_with(&["whole-0", "whole-1", "mid2"]),
        "the dummy let that cargo fix inserts forces the whole pair into the closure",
    );
    let parts = Parts {
        name: "editions".to_string(),
        count: 4,
    };
    drop(parts.name);
    let read = || parts.count + 1;
    ok(
        read() == 5,
        "a closure may read one field after a sibling field moved out",
    );
    let mixed = Mixed {
        tag: 9,
```

listing 8: drop event logs prove the capture granularity, the dummy let cargo fix inserts forces the whole capture, a Clone field makes a Clone closure (source: rust/samples/src/Ch02/DisjointCapture.rs, lines 47-86)

`panic!` became consistent with `format!`. The 2018 macro only applied formatting when handed more than one argument, so `panic!("{}", 1)` formatted its message to 1 while a single argument bypassed formatting entirely: `panic!(a)` panicked with the raw payload, and `panic!(123)` produced the famously unhelpful `Box<Any>` message.

Edition 2021 made the first argument always a format string, killed the arbitrary-payload form, and moved non-string panics to `std::panic::panic_any`:

```
// 2018
panic!("{}");
panic!(a);

// 2021
panic!("{{}}");
panic!("{}", a);
std::panic::panic_any(a);
```

fragment, illustrative, not executed

NOTE: why the consistency could not wait

The 2021 panic rules landed one release before captured format arguments, `{ident}` inside any format string, arrived in 1.58 (2022-01-13). Without the fix, `panic!("failed at {stage}")` would have printed the literal braces while `println!` interpolated, the exact confusion the edition change erased.

2.4. 2021 to 2024: capture rules and unsafe hygiene

Edition 2024 shipped in 1.85 (2025-02-20) under RFC 3501, and its changes share one theme, making the compiler honest about what code already depended on by accident.

`gen` became a reserved keyword, clearing room for gen blocks, a future iterator-building feature, before the next edition needs it. Identifiers named `gen` become `r#gen` through raw identifiers, the same escape hatch `r#async` used in the 2018 hop. The `try` keyword from 2018 is still waiting in the same reserved state.

Return-position `impl Trait` changed what it captures. In 2021 and earlier, an RPIT captured a lifetime parameter only when it appeared syntactically in the opaque type's bounds, which forced the `Captures` trick and the outlives trick onto serious codebases. Rust 1.82 (2024-10-17) stabilized `use<..>` precise capturing in all editions, and edition 2024 flipped the default: an RPIT without a `use<..>` bound now captures every in-scope lifetime, matching what RPITIT and `async fn` already did in every edition:

```
// the outlives trick, before
fn f<'a, T: 'a>(x: &'a (), y: T) -> impl Sized + 'a { (x, y) }

// precise capturing, since 1.82 in all editions
fn f<'a, T>(x: &'a (), y: T) -> impl Sized + use<'a, T> { (x, y) }

// 2024, the capture is implicit
fn f<'a, T>(x: &'a (), y: T) -> impl Sized { (x, y) }
```

fragment, illustrative, not executed

```rust
fn require_static<T: 'static>(value: &T) -> bool {
    let _ = value;
    true
}

fn no_lifetime(probe: &u32) -> impl Sized + use<> {
    *probe
}

fn borrow_kept(probe: &u32) -> impl Debug + use<'_> {
    *probe
}

fn pair<'a, A: Copy + Debug, B: Copy + Debug>(a: &'a A, b: B) -> impl Debug + use<'a, A, B> {
    (*a, b)
}

trait Captures<T: ?Sized> {}

impl<T: ?Sized, U: ?Sized> Captures<T> for U {}

fn captures_trick<'a, A: Copy + Debug, B: Copy + Debug>(
    a: &'a A,
    b: B,
) -> impl Debug + Captures<(&'a A, B)> {
    (*a, b)
}

struct Holder<T> {
    value: T,
}

impl<T: Copy + Debug> Holder<T> {
    fn take(self) -> impl Debug + use<T> {
        self.value
    }
}
```

listing 9: use<> beside the Captures trick, an impl method capturing its outer generic, Debug as the only window into an opaque (source: rust/samples/src/Ch02/RpitCapture2024.rs, lines 15-51)

Attributes with soundness requirements became `unsafe` syntax. `no_mangle`, `export_name`, and `link_section` reach into the global symbol namespace, where a collision can break linking with nothing but safe code in sight. Since 1.82 any attribute could opt into the `unsafe(...)` spelling, and 2024 requires it:

```rust
// 2021
#[no_mangle]
pub fn example() {}

// 2024
#[unsafe(no_mangle)]
pub fn example() {}
```

fragment, illustrative, not executed

References to `static mut` are denied. The `static_mut_refs` lint, warn-by-default before, is deny in 2024, because forming even an unused reference to a mutable static violates the aliasing rules instantly and proving it sound requires reasoning about the whole program. The replacement is the raw borrow operators, `&raw const` and `&raw mut`, stabilized in 1.82:

```
// 2021
unsafe { let r = &X; }

// 2024
unsafe { let p = &raw const X; }
```

fragment, illustrative, not executed

```rust
static mut COUNTER: u32 = 0;
static mut BUFFER: [u8; 4] = [0, 0, 0, 0];

#[unsafe(no_mangle)]
extern "C" fn book_native() -> u32 {
    41
}

#[unsafe(export_name = "rust_book_export")]
extern "C" fn book_export() -> u32 {
    42
}

unsafe extern "C" {
    fn rust_book_export() -> u32;
}

fn main() {
    unsafe {
        let ptr = &raw mut COUNTER;
        *ptr += 1;
        *ptr += 1;
    }
    unsafe {
        let view = &raw const COUNTER;
        ok(
            std::ptr::read(view) == 2,
            "static mut reaches the program through &raw mut and &raw const, never a reference",
        );
    }
    unsafe {
        let slot = &raw mut BUFFER[1];
        std::ptr::write(slot, 9);
        let read_back = &raw const BUFFER[1];
        ok(
            std::ptr::read(read_back) == 9,
            "raw borrows address places inside a static mut",
        );
    }
    let declared = book_native();
    let exported = unsafe { rust_book_export() };
```

listing 10: the 2024-clean forms asserted: raw borrows on a static mut, the unsafe extern block, `r#gen`, the atomics replacement (source: rust/samples/src/Ch02/Unsafe2024.rs, lines 13-53)

2.5. the migration, measured

The chapter's claims about cargo hold on the pinned lane, measured 2026-10-13 with cargo 1.99.0: `cargo fix --edition` and `cargo fix --edition-idioms` exist, while `cargo build --edition` and `cargo run --edition` do not, the flag is rejected with an unexpected argument error. The edition lives in the manifest and in rustc, not in build subcommands. `cargo fix --edition` works by applying the `rust-2024-compatibility` lint group, the same lints the ladder sample names: `keyword_idents_2024` for `gen`, `impl_trait_overcaptures` for RPIT capture, `unsafe_attr_outside_unsafe` for the attributes, `static_mut_refs` for the references. Each hop in the ladder has its equivalent group, `keyword_idents_2018` for the first hop, `rust_2021_incompatible_closure_captures` among others for the second.

Two migrations have no automatic fix: the `static mut` rewrite, where the tooling refuses to guess between atomics, locks, and raw pointers, and the APIT cases of overcapture, where a type parameter must gain a name before a `use<..>` bound can mention it.

2.6. the 1.0 to 1.99 timeline

version	date	language	remembered for
1.0	2015-05-15	the baseline edition, ownership and borrowing as shipped	stability without stagnation
1.27	2018-06-21	`dyn Trait` trait object syntax	the marker before it was required
1.31	2018-12-06	edition 2018, tool lints, `const fn`, lifetime elision in impl headers	the edition system's debut
1.39	2019-11-07	`async fn`, `.await`	async/await for everyone
1.53	2021-06-17	`IntoIterator` for arrays in all editions, or patterns	the by-value fix with a hack around it
1.56	2021-10-21	edition 2021, `binding @ pattern` may bind	arrays, closures, panic
1.58	2022-01-13	captured format arguments `{ident}`	format strings name their arguments
1.75	2023-12-28	`async fn` and return-position `impl Trait` in traits	traits speak async
1.82	2024-10-17	`use<..>` bounds, `&raw const` and `&raw mut`, unsafe attribute opt-in, unsafe extern blocks	the 2024 groundwork
1.85	2025-02-20	edition 2024, async closures	the fourth rung
1.99	2026-10-01	C-variadic definitions stable, `#[my_macro] mod foo;`	the lane this book pins

The version line the samples parse carries the build stamp 2026-09-28 while the release notes date the release 2026-10-01, a commit-date versus release-date distinction the ladder sample's data keeps straight by citing the notes for dates and the version line for the pin.

```
struct Rung {
    edition: &'static str,
    version: (u32, u32),
    year: u32,
    keywords: &'static [&'static str],
    lint: &'static str,
}

const LADDER: &[Rung] = &[
    Rung {
        edition: "2015",
        version: (1, 0),
        year: 2015,
        keywords: &[],
        lint: "",
    },
    Rung {
        edition: "2018",
        version: (1, 31),
        year: 2018,
        keywords: &["async", "await", "dyn", "try"],
        lint: "keyword_idents_2018",
    },
    Rung {
        edition: "2021",
        version: (1, 56),
        year: 2021,
        keywords: &[],
        lint: "rust_2021_incompatible_closure_captures",
    },
    Rung {
        edition: "2024",
        version: (1, 85),
        year: 2025,
        keywords: &["gen"],
        lint: "keyword_idents_2024",
    },
];
```

listing 11: the ladder as data: version, year, reserved keywords, and the migration lint each hop rides (source: rust/samples/src/Ch02/EditionLadder.rs, lines 15-52)

```
const TIMELINE: &[(u32, u32, &str)] = &[
    (1, 0, "stability without stagnation, may 2015"),
    (1, 27, "dyn trait object syntax"),
    (1, 31, "edition 2018, tool lints, const fn"),
    (1, 39, "async and await stable"),
    (1, 53, "IntoIterator for arrays, all editions"),
    (1, 56, "edition 2021"),
    (1, 58, "captured format arguments"),
    (1, 75, "async fn and return-position impl Trait in traits"),
    (1, 82, "use bounds, raw borrows, unsafe attribute opt-in"),
    (1, 85, "edition 2024, async closures"),
    (1, 99, "the lane this book pins"),
];

fn rung(edition: &str) -> &'static Rung {
    LADDER.iter().find(|r| r.edition == edition).unwrap()
}

fn version_of(feature: &str) -> (u32, u32) {
    TIMELINE
        .iter()
        .find(|(_, _, text)| *text == feature)
        .map(|(major, minor, _)| (*major, *minor))
        .unwrap()
}

fn panic_message<F: FnOnce() + std::panic::UnwindSafe>(f: F) -> String {
    let err = panic::catch_unwind(f).unwrap_err();
    let payload = err.downcast_ref::<String>();
    match payload {
        Some(text) => text.clone(),
        None => err
            .downcast_ref::<&str>()
            .map(|text| text.to_string())
            .unwrap_or_default(),
    }
}
```

listing 12: the timeline table, rung and feature lookups, and the panic catcher that measures the 2021 macro (source: rust/samples/src/Ch02/EditionLadder.rs, lines 54-91)

The closing sample runs the ladder itself: 15 checks across the rungs, keyword reservations, lint names, the dyn trait object, captured format arguments, and five panic behaviors under `catch_unwind`, green on the pinned build 2026-10-13.

sources: doc.rust-lang.org/stable/edition-guide, editions index, transitioning chapter, and the 2018, 2021, 2024 chapter pages for keywords, paths, arrays, closures, panic, gen, RPIT capture, unsafe attributes, and static mut references, all accessed 2026-10-13 against the offline rust-docs copy. releases.md from the offline component, sections 1.0, 1.31, 1.39, 1.53, 1.56, 1.58, 1.75, 1.82, 1.85, 1.99, read 2026-10-13, plus the edition guide's dyn page, which redirects to the 1.27 blog post dated 2018-06-21. doc.rust-lang.org/stable/rustc lint listing, allowed and warn by default groups, accessed 2026-10-13, for the six lint names. Cargo behavior measured live 2026-10-13: `cargo fix --help` shows `--edition` and `--edition-idioms`, `cargo build --edition 2024` and `cargo run --edition 2024` are rejected. Sample evidence: Ch02 5 files 38 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1 -Chapter Ch02`, 2026-10-13.

3. lexical structure

Everything rustc sees before it understands types: the token grammar, identifiers across every unicode script, a literal system that grew one form per real need, and a `char` that is a unicode scalar value, never a byte. The reference's lexical section is the spine here, every number below comes out of a sample run on the pinned 1.99.0 lane.

3.1. tokens, the grammar's shape

Rust source lexes into keywords, identifiers, literals, lifetimes, punctuation, and delimiters, and the grammar is written as regular productions for each token kind. Two consequences show up in daily code. Delimiters are tokens, so the compiler never needs whitespace to separate a closing brace from the next token. And any literal token may carry any suffix at lex time, the restriction arrives later: a literal expression only accepts suffixes from the fixed list of integer and float type names, so `1suffix` passes a macro untouched and fails as an expression.

```
macro_rules! blackhole { ($l:literal) => () }
blackhole!(1suffix);   // fine, the macro never interprets it
let x = 1suffix;       // error, no such type suffix
```

fragment, illustrative, not executed

NOTE: prefixed literals are reserved, since the 2021 edition

An identifier glued to a string literal, `string"..."`, used to lex as two tokens and now fails as `unknown prefix`. The reservation is what made `c"..."` for C strings possible in 1.77, and the error message this toolchain prints is `prefix string is unknown` with the hint to insert whitespace, probed 2026-10-13. The lesson is to keep a space between a name and a string literal, and to trust that a new literal form can arrive without edition breakage.

The lifetime token `'a` is its own token kind, which is why the lexer never confuses a lifetime with a character literal, the grammar for `CHAR_LITERAL` requires a single quote on both sides and the grammar for `LIFETIME_TOKEN` requires an identifier after the quote. The classic confusion goes the other way, `'a` alone is a lifetime, `'a'` is the character a:

```
let label: &'static str = "statically";   // 'static is a lifetime token
let letter: char = 'a';                   // quotes on both sides, a char
let borrowed: &'a str = ask();            // 'a names a region, chapter 6
```

fragment, illustrative, not executed

3.2. identifiers and raw identifiers

An identifier starts with an XID_Start character or `_` and continues with XID characters, following Unicode Standard Annex 31. Vietnamese diacritics, Cyrillic, and CJK are all legal, and this sample leans on it:

```
    let tổng = 7;
    let москва = 2;
    let 東京 = 5;
    ok(
        tổng + москва + 東京 == 14,
        "identifiers follow the unicode XID rules, any script works",
    );
    let tông = tông();
    ok(
        tông == 9,
        "a decomposed definition and a precomposed call site are the same identifier after NFC
normalization",
    );
```

listing 13: three scripts, one namespace, NFC normalizes the spellings (source: rust/samples/src/Ch03/RawIdent.rs, lines 52-63)

Two identifiers are the same identifier when their NFC normalizations match. The sample defines a function with a decomposed name, an `o` followed by a combining circumflex, and calls it through the precomposed `o` with a circumflex. One name, two byte sequences.

Strict and reserved keywords cannot be identifiers, and the escape is the raw identifier `r#` plus the word. Raw identifiers landed in 1.30 as the mechanism that let the 2018 and 2024 editions promote words like `dyn` and `gen` without breaking variables named after them:

```
    let r#match = 3;
    ok(r#match == 3, "r#match names a variable although match is a strict keyword");
    ok(
        r#for(21) == 42,
        "r#for names a function although for is a strict keyword",
    );
    let r#type = "string";
    ok(
        r#type.len() == 6,
        "r#type works, type is a strict keyword in every edition",
    );
```

listing 14: `r#match`, `r#for`, `r#type`, `r#gen` as names, the keywords still parse as keywords (source: rust/samples/src/Ch03/RawIdent.rs, lines 22-32)

NOTE: the prefix is not part of the name

`r#match` and a hypothetical variable `match` in an older edition are the same identifier path. Four words refuse the escape outright, `r#self`, `r#Self`, `r#crate`, `r#super`, and `r#_`, because their meanings are positional and cannot be borrowed as names. The full census, 39 strict, 14 reserved, and the weak set, is frozen in the appendix keyword matrix and is not re-enumerated here.

3.3. integer and float literals

Integers come in four radices, decimal, `0x` hex, `0o` octal, `0b` binary, with `_` separators legal between any two digits and directly after the radix prefix. Suffixes pin the type, and the unsuffixed default is `i32`:

```
    let dec = 1_000;
    ok(dec == 1_000, "underscore separators group digits: 1_000 reads as 1000");
    ok(1_0_0 == 100, "underscores are legal between any two digits, not only in groups of 3");
    ok(
        ty(&dec) == "i32",
        "an unsuffixed integer literal defaults to i32",
    );
    let hex = 0xff;
    ok(hex == 255, "hex literal 0xff is 255");
    ok(0xFF_u8 == 255, "hex literals accept mixed case digits and a suffix");
    ok(0xf_f == 255, "hex literals accept underscore separators");
    ok(0x_ffu8 == 255, "underscores are legal right after the radix prefix");
    let bin = 0b1010;
    ok(bin == 10, "binary literal 0b1010 is 10");
    ok(0b1010_1010 == 170, "binary literals accept underscore separators");
    ok(0b_1010 == 10, "underscores are legal right after the 0b prefix");
    let oct = 0o777;
    ok(oct == 511, "octal literal 0o777 is 511");
    ok(0o7_7 == 63, "octal literals accept underscore separators");
```

listing 15: four radices, separators everywhere, the i32 default measured (source: rust/samples/src/Ch03/Literals.rs, lines 20-38)

The suffix grammar has one trap: an integer suffix may not begin with `e` or `E`, because that would read as a float exponent, which is why `2em` lexes as one broken token rather than `2` with suffix `em`. A decimal literal out of range for its suffix is a compile error, and the only sanctioned overflow spelling is the unary minus form, where the literal is truncated to its type:

```
    let wide = 1_000u64;
    ok(
        ty(&wide) == "u64",
        "the u64 suffix pins the literal to u64",
    );
    ok(wide == 1_000, "the suffixed literal carries the same value");
    let tiny = 42u8;
    ok(tiny == 42 && ty(&tiny) == "u8", "the u8 suffix pins the literal to u8");
    let j: i8 = -(128);
    ok(
        j == -128,
        "the literal inside unary minus is truncated to its type, -(128) is i8::MIN without a panic",
```

listing 16: suffixes pin types, and -(128) is how i8::MIN is written (source: rust/samples/src/Ch03/Literals.rs, lines 39-50)

Floats add the exponent forms and one grammar quirk worth seeing once, a trailing period makes a float and a leading dot does not:

```
    let float = 2.5e3;
    ok(float == 2_500.0, "the exponent form 2.5e3 is 2500.0");
    ok(1e3 == 1_000.0, "an integer mantissa with an exponent is a float literal");
    ok(5e-3 == 0.005, "negative exponents work");
    let two: f64 = 2.;
    ok(two == 2.0, "a trailing period makes a float, 2. is 2.0");
    ok(
        ty(&two) == "f64",
        "an unsuffixed float literal defaults to f64",
    );
    let single = 1.0f32;
    ok(single == 1.0 && ty(&single) == "f32", "the f32 suffix pins the literal to f32");
    let glued = 5f32;
    ok(glued == 5.0, "5f32 with no dot or exponent is a float literal");
    let via_int = 3_f64;
    ok(via_int == 3.0, "3_f64 suffixes an integer mantissa directly");
    ok(
```

listing 17: exponents, the f64 default, 2. is 2.0, 3_f64 and 5f32 both work (source: rust/samples/src/Ch03/Literals.rs, lines 52-68)

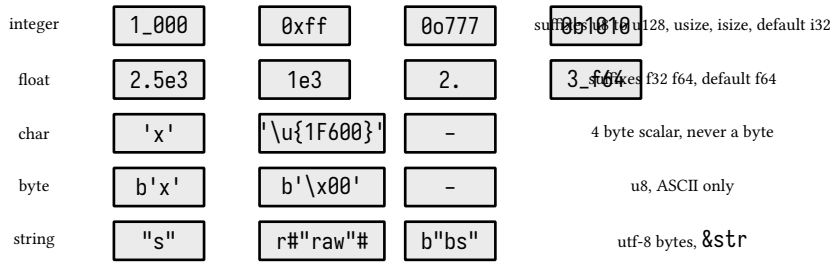


figure 5: the literal inventory: bases, separators, suffixes

3.4. char, the four byte scalar

A `char` is a unicode scalar value, a code point that is not a surrogate, stored as a 32 bit word with the size and alignment of `u32`. The range has a hole, `U+D800` through `U+DFFF` are reserved for utf-16 surrogates and no `char` holds them:

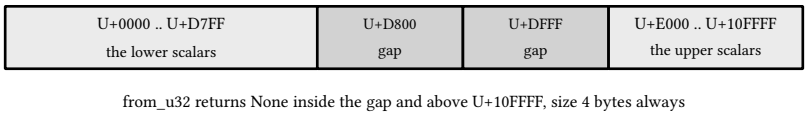


figure 6: the scalar value range with the surrogate gap

```
        char::from_u32(0xD800).is_none(),
        "U+D800, a high surrogate, is not a scalar value and makes no char",
    );
    ok(
        char::from_u32(0xDFFF).is_none(),
        "U+DFFF, a low surrogate, is rejected too",
    );
    ok(
        char::from_u32(0xE000).is_some(),
        "U+E000 is the first scalar above the gap",
    );
    ok(
        char::from_u32(0x10FFFF).is_some(),
        "U+10FFFF itself is a valid char",
    );
    ok(
```

listing 18: the surrogate gap and the range ends, measured (source: rust/samples/src/Ch03/CharScalar.rs, lines 41-56)

The byte literal `b'x'` is a different type entirely, a `u8`. The two worlds connect through `as`: `char` to integer always works, integer to `char` through `as` works only from `u8`, every wider integer goes through `char::from_u32`, whose associated form carries a 1.52 stability badge:

```
        "one past U+10FFFF is outside unicode entirely",
    );
    ok(
        ty(&'x') == "char" && ty(&b'x') == "u8",
        "'x' is char and b'x' is u8, the two literal forms never mix",
    );
    ok(
        'x' as u32 == 120 && b'x' as u8 == 120,
        "the two forms agree on the value while differing on the type",
    );
    ok(
        120u8 as char == 'x',
        "u8 as char is the one integer to char cast as allows",
    );
    ok(
        char::from_u32(120) == Some('x'),
        "wider integers use char::from_u32, since the associated form carries a 1.52 badge",
    );
    ok(
```

listing 19: char and u8, the cast door between them is one way for u8 (source: rust/samples/src/Ch03/CharScalar.rs, lines 58-76)

A `char` is not its utf-8 encoding. `'é'` is one `char` and two utf-8 bytes, `'😀'` is one `char` and four bytes, and casting a scalar to `u8` truncates the code point, it does not encode it:

```
        'é' as u8 == 233,
        "char as u8 truncates the scalar, 233 fits in a byte",
    );
    ok(
        '\u{1F600}' as u8 == 0,
        "a char above the byte range truncates to its low 8 bits, U+1F600 has low byte 0",
    );
    ok(
        'é'.len_utf8() == 2,
        "a char below U+0080 takes 1 utf-8 byte, this one takes 2",
    );
    ok(
        '\u{1F600}'.len_utf8() == 4,
        "the widest scalars take 4 utf-8 bytes",
```

listing 20: truncation versus encoding, the two facts that bite everyone once (source: rust/samples/src/Ch03/CharScalar.rs, lines 77-90)

PITFALL: s.len() counts bytes

`"héllo".len()` is 6, not 5, because `len` answers the byte count of the utf-8 buffer. `chars().count()` answers the scalar count. String indexing by number does not exist at all, `as_bytes()` hands back raw bytes and `0xC3` is a perfectly legal element.

3.5. strings, six forms

The string literal family grew one form per real need, and the raw delimiter counts its hashes to raise the nesting budget:

```
    let quoted = r#"she said "ok""#;
    ok(
        quoted == "she said \"ok\"",
        "a raw string with one # holds embedded quotes unescaped",
    );
    ok(
        r"C:\Users\lav" == "C:\\Users\\lav",
        "raw strings treat backslashes literally, the windows path form",
    );
    let hashes = r##"ends with a quote and hash "# inside"##;
    ok(
        hashes.ends_with("inside") && hashes.contains("\"# "),
        "more hashes in the delimiter raise the nesting budget for quotes and hashes",
    );
    ok(
        r"raw \n stays two characters".contains("\\n"),
        "raw strings process no escapes at all",
    );
```

listing 21: raw strings, one hash, two hashes, escapes off (source: rust/samples/src/Ch03/Strings.rs, lines 40-57)

Byte strings are borrowed byte arrays, and the C string forms, since 1.77 and the 2021 edition onward, hand a borrowed `CStr` with an implicit NUL terminator for ffi work:

```
    let bytes = b"ab";
    ok(
        ty(&bytes) == "&[u8; 2]",
        "a byte string literal is a borrowed byte array, not a str",
    );
    ok(b"\x00".len() == 1, "byte strings take \\x escapes over the full byte range");
    let raw_bytes = br##"raw byte "# string"##;
    ok(
        raw_bytes.len() == 18,
        "raw byte strings combine the b prefix with raw delimiters, two hashes to embed the quote hash
pair",
    );
    let cstr: &CStr = c"hi";
    ok(
        ty(&cstr).contains("CStr"),
        "a C string literal is a borrowed CStr, since 1.77",
    );
    ok(
        cstr.to_bytes() == b"hi",
        "the C string body holds the bytes without the terminator",
```

listing 22: byte strings, raw byte strings, C strings, and the NUL that rides along (source: rust/samples/src/Ch03/Strings.rs, lines 58-76)

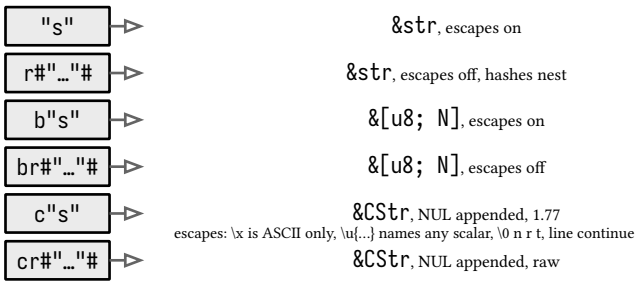


figure 7: the six string forms and what each is for

3.6. comments

Line comments // and block comments /* */, and block comments nest, which C programmers meet with relief. Documentation comments are their own token kinds: /// and /** */ attach to the item that follows, //! and /*! */ attach to the enclosing module, and both are sugar for #[doc = "..."] attributes rather than compiler ignorable text. Four slashes //// are an ordinary comment, the doc forms require exactly three, and /** */ with three or more leading asterisks stops being a doc comment too. A CR character is illegal inside a doc comment, and a block doc comment ends at the first */ no matter what the markdown inside promised.

```
mod example {
    //! module level doc, inner
    /// item level doc, outer
    pub fn documented() {}
    //// four slashes, just a comment
    /* /* block comments nest */ */
    pub fn nested() {}
}
```

fragment, illustrative, not executed

The lexer proof that nesting and the underscore prefix warnings behave as documented is the sample compile itself, the suite runs under `rustc --edition 2024 -D warnings` and every fact above that has a runtime face carries an ok line in `RawIdent.rs`.

sources: the rust reference, tokens and literals, identifiers, and comments pages, doc.rust-lang.org/stable/reference/tokens.html, identifiers.html, comments.html, and expressions/literal-expr.html, accessed 2026-10-13 against the pinned 1.99.0 offline rust-docs set. version attributions from the release log in that same offline set, releases.md: raw identifiers in 1.30, `'_` in 1.26, C string literals in 1.77, read 2026-10-13. claims verified by the Ch03 sample suite, 76 checks under `tools/run-rust-samples.ps1 -Chapter Ch03` on /.rustup/toolchains/stable-x86_64-pc-windows-msvc, plus rustc probes for the compile fail forms: the suffix rejection on `1suffix` as an expression, the `r#self` refusal, and the overflowing literal error on `128_i8`, all probed 2026-10-13.

4. types and values

The value layer: twelve integer types and two floats with measured ranges, inference that runs backwards from usage, one explicit cast operator, the composite trio of tuples, arrays, and slices, the two degenerate types `()` and `!`, and the two spelling of forever, `const` and `static`. The reference's types section and expressions section are the spine, the numbers come from the pinned 1.99.0 lane.

4.1. the scalar inventory

Five unsigned widths, five signed, two floats, `bool`, `char`, `usize`, and `isize`. The 128 bit integers joined the language in 1.26, everything else was there at 1.0. The machine dependent pair follows the pointer: `usize::BITS` reads 64 on this host, and the reference guarantees only at least 16:

```
    ok(i8::MIN == -128, "i8 spans -128 to 127");
    ok(i8::MAX == 127, "i8 max is 127");
    ok(i16::MIN == -32_768 && i16::MAX == 32_767, "i16 spans -32768 to 32767");
    ok(i32::MIN == -2_147_483_648, "i32 min is -2147483648");
    ok(i32::MAX == 2_147_483_647, "i32 max is 2147483647");
    ok(
        i64::MIN == -9_223_372_036_854_775_808,
        "i64 min is -9223372036854775808",
    );
```

listing 23: the signed ranges, measured against the reference table (source: rust/samples/src/Ch04/Scalars.rs, lines 16-24)

```
        u64::MAX == 18_446_744_073_709_551_615,
        "u64 max is 18446744073709551615",
    );
    ok(
        u128::MAX == 340_282_366_920_938_463_463_374_607_431_768_211_455,
        "u128 max is 2 to the 128 minus 1",
    );
    ok(
        usize::BITS == 64,
        "usize is the pointer width, 64 bit on the pinned host, measured",
    );
    ok(
        usize::MAX as u64 == u64::MAX,
        "on a 64 bit host usize and u64 share the range",
    );
    ok(
        isize::MIN as i64 == i64::MIN,
        "isize is the signed pointer width, matching i64 here",
```

listing 24: the unsigned ranges and the pointer width pair on this host (source: rust/samples/src/Ch04/Scalars.rs, lines 41-58)

NOTE: code that assumes 64 bit pointers

`usize::MAX` differs per platform and the reference warns that much of the ecosystem assumes 32 or 64 bit pointers, so 16 bit targets need opt in care. Everything this book measures is the pinned 64 bit msvc host, and the sizes on the pages here carry that caveat: 8 for a pointer, 16 for a fat pointer, 4 for a `char`, all measured, none portable law beyond the reference's minimums.

The floats are IEEE 754 binary32 and binary64, and the sample derives their constants from the formulas instead of trusting decimal spellings: epsilon is 2 to the −23 and −52, max is the largest representable mantissa times the largest exponent:

```
        f32::EPSILON == 2.0f32.powi(-23),
        "f32 epsilon is 2 to the -23, IEEE 754 binary32",
    );
    ok(
        f64::EPSILON == 2.0f64.powi(-52),
        "f64 epsilon is 2 to the -52, IEEE 754 binary64",
    );
    ok(
        f32::MAX == (2.0f32 - 2.0f32.powi(-23)) * 2.0f32.powi(127),
        "f32 max follows the binary32 formula, about 3.4028e38",
    );
    ok(
        f64::MAX == (2.0f64 - 2.0f64.powi(-52)) * 2.0f64.powi(1023),
        "f64 max follows the binary64 formula, about 1.7977e308",
    );
    ok(
        f32::MIN_POSITIVE == 2.0f32.powi(-126),
        "the smallest normal f32 is 2 to the -126",
    );
    ok(f32::INFINITY > f32::MAX, "infinity sorts above every finite value");
    ok(
        std::hint::black_box(f32::NAN) != std::hint::black_box(f32::NAN),
        "NaN compares unequal to itself, even to itself",
    );
    ok(!f32::NAN.is_finite(), "NaN is not finite");
    ok(0.1 + 0.2 != 0.3, "binary floats cannot hold 0.1 exactly, the classic");
    ok((0.1f64 + 0.2 - 0.3).abs() < 1e-15, "the error is tiny but real");
```

listing 25: epsilon and max from first principles, NaN, and the 0.1 + 0.2 classic (source: rust/samples/src/Ch04/Scalars.rs, lines 61-87)

type	widths	range or value
integers	1 2 4 8 16 bytes	i8 −128..127 through i128, since 1.26 for the 16 byte pair
unsigned	same widths	u8 0..255 through u128
usize	pointer width	64 bit here, at least 16 guaranteed
floats	4 and 8 bytes	f32 f64, IEEE 754, NaN and infinities
bool	1 byte	true false
char	4 bytes	scalar values U+0000..U+10FFFF minus the surrogate gap

figure 8: the scalar inventory: widths, ranges, families

4.2. inference

Rust infers types at bindings and propagates them from every use site. The defaults only apply where nothing else speaks: an unsuffixed integer is `i32`, an unsuffixed float is `f64`. The sample measures each site with `std::any::type_name`, since 1.38:

```
    let unsuffixed = 5;
    ok(
        ty(&unsuffixed) == "i32",
        "inference site 1, let without annotation: the integer default is i32",
    );
    let float = 1.5;
    ok(
        ty(&float) == "f64",
        "inference site 2, let without annotation: the float default is f64",
    );
    let annotated: u8 = 5;
    ok(
        ty(&annotated) == "u8",
        "inference site 3, let with annotation: the annotation picks the type",
    );
    let promoted = annotated as u16 + 1;
    ok(
        ty(&promoted) == "u16",
        "inference site 4, the operators demands: the as result drives u16 through the sum",
    );
    let fixed: Vec<u32> = (0..3).collect();
    ok(
        ty(&fixed).ends_with("Vec<u32>") && fixed == vec![0, 1, 2],
        "inference site 5, collect fills its type from the binding's annotation",
    );
    let turbo = "12".parse::<u16>();
    ok(
        turbo == Ok(12),
        "inference site 6, the turbofish names the type at the call",
```

listing 26: six inference sites from defaults to turbofish (source: rust/samples/src/Ch04/Coercion.rs, lines 28-56)

The direction matters. `let x = 5;` starts from the literal, `let v: Vec<u32> = (0..3).collect();` runs the annotation backwards through the collect call, and `"12".parse::<u16>()` names the type at the call. A binding whose type nothing can determine, like a genuinely empty `collect` with no annotation, is a compile error, not a fallback.

What inference never does is guess a semantics. It fills type variables, it never inserts a conversion, widens a narrow integer, or turns an `i32` into a `usize` to make an indexing expression work, those are `as` casts by hand or trait methods by name. The split keeps the reading rule simple, where the types come from is invisible, where they change is always visible.

4.3. coercion sites and the as table

Two roads change types. Implicit coercions happen only at fixed sites where the target type is already known, `let` with annotation, function arguments, return positions, and a handful more, and they cover a short list, weakening `&mut` to `&`, unzing arrays into slices, and a few pointer forms. Everything else needs the explicit `as` cast:

```
    let shared: &i32 = &mut 42;
    ok(
        *shared == 42,
        "coercion site, let with annotation weakens &mut to & with no as",
    );
    let array = [1, 2, 3];
    ok(
        slice_len(&array) == 3,
        "coercion site, the argument position unsizes &[i32, 3] into &[i32]",
    );
    let slice_ref: &[i32] = &array;
    ok(
        ty(slice_ref) == "[i32]",
        "the let annotation unsizes the array reference into a slice",
    );
    let boxed_slice: Box<[i32]> = Box::new([1, 2, 3]);
    ok(
        boxed_slice.len() == 3,
        "Box::new of an array coerces to the boxed slice type",
```

listing 27: coercion sites need no keyword, the annotation drives them (source: rust/samples/src/Ch04/Coercion.rs, lines 58-76)

The as table is closed. Integers and floats swap freely, `bool` and `char` go to integers, `u8` goes to `char`, enum types go to their integer representation, a preview of chapter 7, and pointers cast among compatible forms. Nothing else crosses without a method. The numeric semantics are exact: same width is a bit reinterpretation, narrowing truncates, signed widening sign extends, unsigned widening zero extends:

```
        300i32 as u8 == 44,
        "as from wide to narrow truncates the high bits, 300 keeps its low byte 44",
    );
    ok(
        0xabcdu16 as u8 == 0xcd,
        "hex shows the truncation plainly, abcd keeps cd",
    );
    ok(
        -1i8 as i16 == -1,
        "signed widening sign extends",
    );
    ok(
        0xFFu8 as u16 == 0x00FF,
        "unsigned widening zero extends",
    );
    ok(
        3.9f64 as i32 == 3,
        "float to int truncates toward zero, 3.9 becomes 3",
    );
    ok(
        -3.9f64 as i32 == -3,
        "negative truncation also goes toward zero, -3.9 becomes -3",
    );
    ok(
        300.0f32 as u8 == 255,
        "since 1.45 float to int casts saturate instead of producing garbage, 300 clamps to 255",
    );
    ok(
        f64::NAN as i32 == 0,
        "NaN casts to 0 under the same saturating rule",
    );
    ok(
        f32::NEG_INFINITY as i32 == i32::MIN,
        "negative infinity saturates to the minimum",
    );
    ok(
        true as i32 == 1 && false as u8 == 0,
        "bool as integer is 1 or 0",
    );
    ok(
```

listing 28: truncation, extension, and the two directions of char casting, measured (source: rust/samples/src/Ch04/Coercion.rs, lines 83-122)

PITFALL: float to int saturates, since 1.45

Before 1.45 the float to int cast produced whatever the cpu felt like for out of range values. Now `300.0f32 as u8` is 255, negative infinity is `i32::MIN`, and `NaN` is 0. Truncation toward zero still applies to in range values, `3.9` becomes 3.

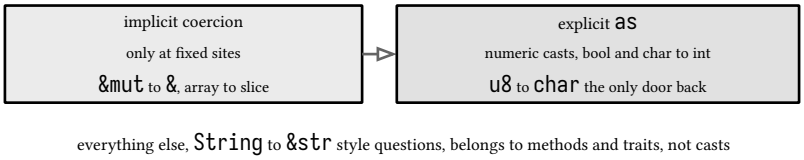


figure 9: two roads: coercion sites and the closed as table

4.4. tuples, arrays, slices

Tuples are structural, heterogeneous, and named by position. Arrays carry their length in the type, `[i32; 3]` and `[i32; 4]` are unrelated types, and indexing is bounds checked everywhere in safe code. Slices are views, `[T]` is unsized and lives behind a fat pointer of data address plus length:

```
    let pair = (1u8, "a");
    ok(
        pair.0 == 1 && pair.1 == "a",
        "tuple fields carry numeric names in declaration order",
    );
    let nested = ((1, 2), 3);
    ok(
        nested.0.1 == 2 && nested.1 == 3,
        "tuples nest, chained indexes read inner fields",
    );
    let one: (i32,) = (5,);
    ok(
        ty(&one) == "(i32,)" && one.0 == 5,
        "a 1-ary tuple needs its trailing comma to stay a tuple",
    );
    let (a, b) = (7u8, 9u8);
    ok(
        a + b == 16,
        "tuples destructure into their parts",
    );
    let unit: () = ();
    ok(
        ty(&unit) == "()" && std::mem::size_of::<()>() == 0,
        "the unit type has one value and zero size",
    );
    let slot;
    let assignment_value: () = { slot = 6 };
    ok(
        slot == 6 && ty(&assignment_value) == "()",
```

listing 29: tuple forms, the trailing comma rule, unit, and array facts (source: rust/samples/src/Ch04/Composites.rs, lines 35-63)

```
    let digits = [3, 1, 4];
    ok(
        ty(&digits) == "[i32; 3]" && digits.len() == 3,
        "the array type carries its length, [i32, 3]",
    );
    ok(
        digits.iter().sum::<i32>() == 8,
        "arrays iterate by reference",
    );
    let copy_one = digits;
    ok(
        copy_one == [3, 1, 4] && digits == [3, 1, 4],
        "an array of Copy elements copies on assignment, both names stay usable",
    );
    ok(
        std::mem::size_of::<[i32; 3]>() == 12,
        "the array size is the element size times the length",
    );
    let other_len: [i32; 4] = [0; 4];
    ok(
        ty(&other_len) == "[i32; 4]",
        "the length is part of the type, [i32, 3] and [i32, 4] are strangers",
    );
    let oob = catch_unwind(AssertUnwindSafe(|| {
        let i = black_box(7usize);
        digits[i] as i32
    }));
    ok(
        oob.is_err(),
        "indexing is always bounds checked in safe code, out of range panics",
    );
    let slice: &[i32] = &digits;
    ok(
        ty(slice) == "[i32]" && slice.len() == 3,
        "a slice borrows a run of elements without owning them",
    );
    let grown = vec![5, 6, 7, 8];
    ok(
        grown.as_slice().len() == 4 && ty(grown.as_slice()) == "[i32]",
```

listing 30: bounds checks, the slice family, and the 16 byte fat pointer (source: rust/samples/src/Ch04/Composites.rs, lines 66-104)

The bounds check is a runtime guarantee, not a debug aid: the sample catches the panic and reads its exact message, `index out of bounds: the len is 3 but the index is 7`.

4.5. unit and never

`()` is the tuple with no fields, one value, zero size, and the result type of statements that produce nothing, including assignment expressions. `!` is the type with no values, the return type of diverging functions, and it coerces into any other type, which is why `panic!` and `unreachable!()` sit inside expressions of any type:

```
fn diverge() -> ! {
    panic!("never returns");
}

fn ensure(cond: bool) -> i32 {
    if cond {
        1
    } else {
        diverge()
    }
}
```

listing 31: a diverging function and a branch that leans on the coercion (source: rust/samples/src/Ch04/Composites.rs, lines 21-31)

```
    );
    let fallback = catch_unwind(AssertUnwindSafe(|| ensure(false)));
    ok(
        fallback.is_err(),
        "reaching the ! arm panics, no value of the never type exists",
    );
    let chosen: i32 = match digits[0] {
        3 => 100,
        _ => diverge(),
    };
    ok(
        chosen == 100,
        "! coerces inside match arms as well",
    );
    let _unused_never: fn() -> ! = diverge;
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 32: ! coerces through if and match arms, reaching it panics (source: rust/samples/src/Ch04/Composites.rs, lines 123-139)

4.6. const and static

A `const` is a name for a value, inlined at each use, with no fixed address promised. A `static` is a name for one allocation with a fixed address, initialized at compile time, never dropped at program end, and required to be `Sync` unless declared `mut`. Constants prefer `const fn` initializers, stable since 1.31:

```
const MAX_POINTS: u32 = 100_000;
const LEN: usize = 4;

const fn square(x: i32) -> i32 {
    x * x
}

const NINE_SQUARED: i32 = square(9);

static LANGUAGE: &str = "rust";
static ORIGIN: (f64, f64) = (1.0, 2.0);
static COUNTER: AtomicU64 = AtomicU64::new(7);
static mut LEVELS: u32 = 0;

fn origin_addr() -> usize {
    std::ptr::from_ref(&ORIGIN) as usize
}
```

listing 33: the declarations side by side (source: rust/samples/src/Ch04/ConstStatic.rs, lines 15-31)

```
fn main() {
    ok(
        MAX_POINTS == 100_000,
        "a const is a named value with a required type annotation",
    );
    ok(
        NINE_SQUARED == 81,
        "a const initializer runs at compile time, const fn since 1.31 does the work",
    );
    let buffer = [0u8; LEN];
    ok(
        buffer.len() == 4,
        "a const feeds array lengths and other constant contexts",
    );
    ok(
        LANGUAGE == "rust",
        "a static is a named allocation with one fixed address",
    );
    ok(
        origin_addr() == origin_addr(),
        "every reference to the static sees the same allocation, the const never promises that",
    );
    ok(
        ORIGIN.0 == 1.0 && ORIGIN.1 == 2.0,
        "the static holds its initializer value",
    );
    let before = COUNTER.fetch_add(1, Ordering::SeqCst);
    let after = COUNTER.load(Ordering::SeqCst);
```

listing 34: const inlining, the fixed static address, interior mutability, static mut place access (source: rust/samples/src/Ch04/ConstStatic.rs, lines 33-60)

Edition 2024 hardened the last sharp edge: taking a reference to a `static mut` is now denied by default, because aliasing a mutable global through references violates the mutability xor aliasing rule the borrow checker enforces everywhere else. Place operations, the direct read and write, remain legal inside `unsafe`, and the raw pointer operators `&raw const` and `&raw mut` are the sanctioned route since 1.82:

```
static mut LEVELS: u32 = 0;
unsafe {
    LEVELS += 1;            // fine, a place operation
    let r = &LEVELS;        // error: creating a shared reference to mutable static
}
```

fragment, illustrative, not executed

The safe replacements are the ones chapter 13 will build on, an immutable `static` over `AtomicU64` or a `Mutex`.

4.7. integer overflow

Debug builds panic on arithmetic overflow, release builds wrap in two's complement, and `-C overflow-checks` overrides the mode. The sample suite runs on the debug lane, `rustc --edition 2024 -D warnings` with no optimization flag, so every panic below is measured:

```
fn main() {
    std::panic::set_hook(Box::new(|_| {}));
    ok(
        cfg!(debug_assertions),
        "the runner compiles the debug lane, so arithmetic overflow panics here",
    );
    let add = catch_unwind(|| black_box(250u8) + black_box(250u8)).map_err(panic_message);
    ok(
        add.err().map_or(false, |m| m == "attempt to add with overflow"),
        "250u8 + 250u8 panics with attempt to add with overflow",
    );
    let neg = catch_unwind(|| -black_box(i8::MIN)).map_err(panic_message);
    ok(
        neg.err().map_or(false, |m| m == "attempt to negate with overflow"),
        "negating i8::MIN at run time panics, two's complement has no positive twin",
    );
    let div = catch_unwind(|| black_box(i8::MIN) / black_box(-1i8)).map_err(panic_message);
    ok(
        div.err().map_or(false, |m| m == "attempt to divide with overflow"),
        "i8::MIN divided by -1 panics even where overflow checks are off",
    );
    let shift = catch_unwind(|| black_box(1u8) << black_box(8)).map_err(panic_message);
    ok(
        shift
            .err()
            .map_or(false, |m| m == "attempt to shift left with overflow"),
        "a shift by the type's own width panics, 8 is out of range for u8",
    );
    let literal_form: i8 = -128_i8;
    ok(
        literal_form == i8::MIN,
        "the literal spelling -128_i8 never panics, literals are truncated at compile time",
    );
```

listing 35: four panics with their exact payloads, and the literal exception (source: rust/samples/src/Ch04/Overflow.rs, lines 26-58)

VERIFY: which build the runner uses

> The sample lane compiles without `-O`, so `cfg!(debug_assertions)` is true and the checks on this page are the debug behavior. The release behavior was probed separately with `rustc -O` on the same toolchain, 2026-10-13: `250u8 + 250u8` wrapped to 244 with no panic.

The division corner is its own rule: `i8::MIN / -1` panics even in release, the reference calls the legacy behavior out explicitly, and `i8::MIN % -1` is 0 rather than a panic. Shifts overflow when the right side reaches the type's width. When wrapping is the intent, the four method families say it at the call site, `checked_` since 1.0, `saturating_` since 1.0, `wrapping_` since 1.0 with the shift variants in 1.2, and `overflowing_` in 1.7:

```
        u8::MAX.wrapping_add(1) == 0,
        "wrapping_add rolls u8::MAX + 1 to 0",
    );
    ok(
        i8::MIN.wrapping_sub(1) == 127,
        "wrapping_sub rolls i8::MIN - 1 to 127",
    );
    ok(
        i8::MIN.wrapping_abs() == -128,
        "wrapping_abs of i8::MIN stays i8::MIN",
    );
    ok(
        u8::MAX.checked_add(1).is_none(),
        "checked_add returns None instead of panicking",
    );
    ok(
        5u8.checked_add(3) == Some(8),
        "checked_add returns Some when the sum fits",
    );
    ok(
        1u8.checked_div(0).is_none(),
        "checked_div turns the division by zero into None",
    );
    ok(
        1u8.checked_shl(8).is_none(),
        "checked_shl turns the oversized shift into None, the method carries a 1.7 badge",
    );
    ok(
        u8::MAX.saturating_add(1) == u8::MAX,
        "saturating_add clamps at u8::MAX",
    );
    ok(
        0u8.saturating_sub(1) == 0,
        "saturating_sub clamps at 0",
    );
    ok(
        i8::MIN.saturating_sub(1) == i8::MIN,
        "saturating_sub clamps at i8::MIN",
    );
    ok(
        i8::MIN.saturating_abs() == i8::MAX,
        "saturating_abs of i8::MIN clamps to 127",
    );
    ok(
        u8::MAX.overflowing_add(1) == (0, true),
        "overflowing_add returns the wrapped value and the overflow flag",
```

```
    );
    ok(
        5u8.overflowing_add(3) == (8, false),
        "overflowing_add reports false when the sum fits, since 1.7",
    );
    ok(
        1u8.wrapping_shl(8) == 1,
        "wrapping_shl masks the shift amount modulo the width, so shifting by 8 is shifting by 0",
    );
    let bounds = catch_unwind(AssertUnwindSafe(|| {
        let a = [1u8, 2, 3];
        let i = black_box(7usize);
        a[i] as i32
    }))
    .map_err(panic_message);
    ok(
        bounds.err().map_or(false, |m| m == "index out of bounds: the len is 3 but the index is 7"),
        "bounds checks panic with the exact index message, indexing is always checked in safe code",
    );
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 36: the four families over add, sub, div, and shift, every branch measured (source: rust/samples/src/Ch04/Overflow.rs, lines 60-126)

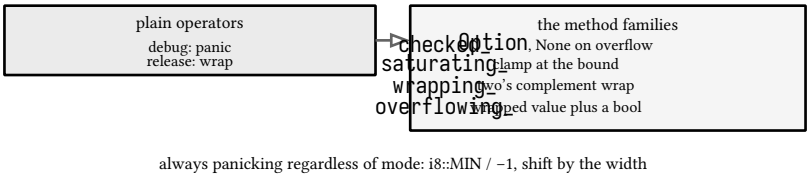


figure 10: overflow policy: build mode picks the default, the families opt out explicitly

4.8. type aliases

`type Celsius = f64;` creates a synonym, nothing more. The alias is transparent to the type checker, the sample proves it with `type_name`, and distinct identities are chapter 7's `struct` and `enum` job. The one distinction that survives aliasing is in error messages, which print the alias name where it was written:

```
type Celsius = f64;
type Point = (f64, f64);
type Vec3 = [f32; 3];
```

listing 37: aliases for a scalar, a tuple, and an array (source: rust/samples/src/Ch04/Coercion.rs, lines 23-25)

```
    let warm: Celsius = 37.5;
    ok(
        ty(&warm) == "f64",
        "a type alias adds no identity: Celsius is f64 to the type checker",
    );
    let origin: Point = (1.0, 2.0);
    ok(
        origin.0 == 1.0 && ty(&origin) == "(f64, f64)",
        "a tuple alias is just the tuple",
    );
    let zero: Vec3 = [0.0; 3];
    ok(
        ty(&zero) == "[f32; 3]" && zero.len() == 3,
        "an array alias is just the array",
    );
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 38: the alias reads back as the underlying type, measured (source: rust/samples/src/Ch04/Coercion.rs, lines 134-150)

sources: the rust reference, types, numeric types, char, tuple, array, slice, and never type pages plus type coercion, type cast expressions, constant items, and static items, doc.rust-lang.org/stable/reference/types.html and the per type pages, accessed 2026-10-13 against the pinned 1.99.0 offline rust-docs set. overflow semantics from the reference's operator expressions page, the saturating float cast change from releases.md 1.45, the edition 2024 static mut rule from the edition guide's static-mut-references page, all read 2026-10-13. claims verified by the Ch04 sample suite, 115 checks under `tools/run-rust-samples.ps1 -Chapter Ch04` on /.rustup/toolchains/stable-x86_64-pc-windows-msvc, plus two probes run 2026-10-13: `rustc -O` on the wrapped addition for the release claim, and the edition 2024 compile fail on `&LEVELS` whose error text is quoted in this chapter.

5. ownership and moves

Every value has exactly one owner, the stack frame, the struct field, or the heap box that holds it. Assignment either copies the bytes, for the types marked `Copy`, or moves the owner's title to the new name and ends the old one. When the owner dies the value drops, and the drop is deterministic and ordered. That triple, one owner, move or copy, deterministic drop, is the whole model, and everything chapters 6 and 12 build on it. C gives you the same graph of allocations with no rules about who frees, book 1, ch 4 (c23: the language, the machine, and the cloud, pointers, arrays, and memory) is the contrast case, and the garbage collected corner of this corpus, book 4, ch 10 (go 1.27, allocation: mspans, size classes, escape analysis, sync.Pool), never needs the question answered.

5.1. the ownership graph

Ownership is a tree. Each value has one incoming edge from its owner, a stack slot, a field, an element slot, or the interior of a `Box`, and the tree is what gets dropped, leaves first in field order, roots last in reverse declaration order. Two owners for one heap buffer is the bug the whole model exists to prevent, the double free:

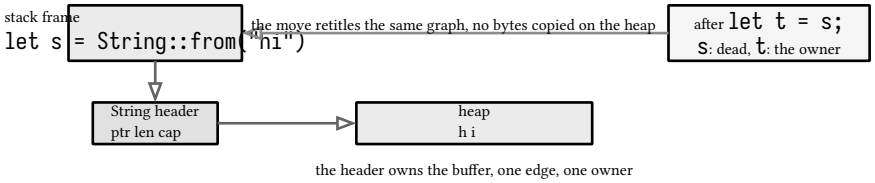


figure 11: the ownership graph: one tree, moves re-root it, borrows are non owning edges

A move is a change of title, not a traversal. The heap buffer stays put, the header on the stack may be copied, and the source name loses the right to touch anything.

5.2. the double free, prevented

The reason moves exist is the double free. If `let s2 = s1;` copied a `String` naively, both names would own one heap buffer, and both drops would free it. C walks this exact cliff in every manual allocator call, book 1, ch 14 (c23: the language, the machine, and the cloud, the heap and allocators) is the corpus chapter on that side. Rust's answer is to make assignment cheap and the source dead, and using the source afterwards is the compile error:

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{s1}, {s2}");
}
```

fragment, illustrative, not executed

The compiler answers with `error[E0382]: borrow of moved value: s1`, and the full diagnostic is worth reading once, it teaches the model:

```
error[E0382]: borrow of moved value: `s1`
  | let s1 = String::from("hello");
  |     -- move occurs because `s1` has type `String`,
  |        which does not implement the `Copy` trait
  | let s2 = s1;
  |     -- value moved here
```

```
   | println!("{s1}, {s2}");
   |        ^^ value borrowed here after move
help: consider cloning the value if the performance cost is
      acceptable: `let s2 = s1.clone();`
```

fragment, illustrative, not executed

Three annotations, one per line of the story: the type is not `Copy` so this is a move, the value moved at the assignment, and the borrow came after. The suggested clone is one of the two exits, the other is chapter 6's reference, and the error code families map onto the two, E0382 for using a moved owner, E0502 and friends for overreaching borrows.

5.3. moves in practice

Passing by value moves into the callee, returning moves back out, and a moved out name can be re-bound fresh. `mem::take` and `mem::replace` are the sanctioned surgeries when a moved out hole must be filled immediately, take leaves the `Default` value behind since 1.40:

```rust
fn take_ownership(s: String) -> usize {
    s.len()
}

fn round_trip(s: String) -> String {
    s
}

fn consume(boxed: Box<u8>) -> u8 {
    *boxed
}

fn main() {
    let s = String::from("hello");
    let len = take_ownership(s);
    ok(
        len == 5,
        "passing by value moves the String into the function, the caller binding ends",
    );
    let t = String::from("round");
    let back = round_trip(t);
    ok(
        back == "round",
        "returning the value hands ownership back out, a fresh binding receives it",
    );
```

listing 39: move in, move out, and the return path back to the caller (source: rust/samples/src/Ch05/Moves.rs, lines 13-37)

```
    let taken = std::mem::take(&mut owned);
    ok(
        taken == "abc" && owned.is_empty(),
        "mem::take, since 1.40, moves the value out and leaves the default behind",
    );
    let replaced = std::mem::replace(&mut owned, String::from("new"));
    ok(
        replaced.is_empty() && owned == "new",
        "mem::replace moves the old value out and installs the new one in one step",
    );
    let s1 = String::from("first binding");
    let s2 = s1;
    let s1 = String::from("second binding");
    ok(
        s1 == "second binding" && s2 == "first binding",
        "a moved out name can be re-bound with let, shadowing starts a fresh owner",
    );
    let pair = (String::from("left"), 42u8);
    let left = pair.0;
    ok(
        left == "left" && pair.1 == 42,
        "a partial move empties one field, the untouched field stays usable",
    );
    let boxed = Box::new(9u8);
    let value = consume(boxed);
    ok(
        value == 9,
        "a Box moves like any owner, the heap allocation travels with it",
    );
```

listing 40: take, replace, re-binding, and the partial move that empties one field (source: rust/samples/src/Ch05/Moves.rs, lines 39-67)

The partial move is worth a pause: moving `pair.0` out of a tuple leaves the tuple half alive, `pair.1` stays usable, and the tuple as a whole cannot be moved or dropped by name afterwards, only its remaining fields.

5.4. copy types and the Copy boundary

`Copy` is a promise that duplication is memcpy, nothing more, and the compiler polices the boundary. A type is `Copy` when all its fields are `Copy` and it has no `Drop` implementation, and the built-ins that qualify are the scalars, shared references, function items and pointers, tuples and arrays of `Copy`, and closures that capture only `Copy` values, that last one since 1.26:

```
    let p1 = Point { x: 1, y: 2 };
    let p2 = p1;
    ok(
        p1.x == 1 && p2.y == 2,
        "a Copy struct duplicates on assignment, both names stay usable",
    );
    let mut p3 = p1;
    p3.x = 10;
    ok(
        p1.x == 1 && p3.x == 10,
        "the copy is independent, mutating one leaves the other untouched",
    );
    let t1 = (7i32, b'z');
    let t2 = t1;
    ok(
        t1.0 == 7 && t2.1 == b'z',
        "tuples of Copy types are Copy themselves",
    );
    let source = [1u8, 2, 3];
    let dup = source;
    ok(
        source[0] == 1 && dup[2] == 3,
        "arrays of Copy elements copy wholesale",
    );
    let owner = String::from("shared");
    let r1 = &owner;
    let r2 = r1;
    ok(
        r1.len() == 6 && r2 == "shared" && owner == "shared",
        "shared references are Copy, copying a borrow clones the pointer not the data",
    );
    let f1: fn(i32) -> i32 = double;
    let f2 = f1;
    ok(
        f1(21) == 42 && f2(21) == 42,
        "function pointers are Copy",
    );
    let bare = double;
    let also = bare;
    ok(
        bare(3) == 6 && also(3) == 6,
        "function items, the bare fn names, are Copy too",
    );
    let copy_closure = {
        let n = 10i32;
        move |step: i32| n + step
    };
    let again = copy_closure;
    ok(
        copy_closure(1) == 11 && again(1) == 11,
```

listing 41: struct, tuple, array, reference, function pointer, and Copy closure copies, all measured (source: rust/samples/src/Ch05/CopyTypes.rs, lines 29-78)

The mirror side fails loudly. A struct holding a `String` cannot derive `Copy`, the error is `E0204`, this field does not implement `Copy`. Implementing `Copy` on a type with a destructor is `E0184`, `Copy` not allowed on types with destructors, and the reason is the model itself: if assignment copied silently, the destructor would run twice on the same resource:

```
#[derive(Clone, Copy)]
struct T {
    f: String,
}
```

fragment, illustrative, not executed

`Clone` is the explicit deep copy for everything that cannot be `Copy`. When a type implements both, `Copy` is the fast path assignment uses and `clone()` stays available for spelling it out. The sample proves a `String` clone is independent, mutating the clone leaves the original untouched:

```
    let original = Label {
        text: String::from("owned"),
    };
    let cloned = original.clone();
    let mut trimmed = cloned;
    trimmed.text.push('!');
    ok(
        original.text == "owned" && trimmed.text == "owned!",
        "a non Copy type still copies on demand through Clone, a deep copy for String",
    );
    let moved_label = Label {
        text: String::from("gone"),
    };
    let gone = moved_label.text;
    ok(
        gone == "gone",
        "Clone is a supertrait of Copy, every Copy type can also clone explicitly",
```

listing 42: Clone as the explicit deep copy, independence measured (source: rust/samples/src/Ch05/CopyTypes.rs, lines 81-97)

NOTE: copy changes semantics, not layout

A `Copy` struct still occupies its bytes, the sample measures `size_of::<Point>()` at 8. What changes is what assignment means, duplicate the bytes and keep both names alive, versus retitle and kill the source name.

5.5. drop order, demonstrated

Drop is deterministic. Locals drop in reverse declaration order, struct fields drop in declaration order after the struct's own `Drop` body ran, tuple fields in order, array elements first to last, and assignment drops the value it overwrites on the spot. The sample logs every drop into a mutex and asserts the exact 17 element sequence:

```
struct Tag(&'static str);

impl Drop for Tag {
    fn drop(&mut self) {
        LOG.lock().unwrap().push(self.0);
    }
}

struct Panel {
    title: Tag,
    body: Tag,
}

struct Frame {
    head: Tag,
    foot: Tag,
}

impl Drop for Frame {
    fn drop(&mut self) {
        LOG.lock().unwrap().push("frame");
    }
}
```

listing 43: the probe types, two structs, one with a Drop body and one without (source: rust/samples/src/Ch05/DropOrder.rs, lines 17-39)

```
fn scene() {
    let a = Tag("a");
    let b = Tag("b");
    let c = Tag("c");
    {
        let inner = Tag("inner");
        ok(inner.0 == "inner", "the inner local lives only inside its block");
    }
    let panel = Panel {
        title: Tag("title"),
        body: Tag("body"),
    };
    let frame = Frame {
        head: Tag("head"),
        foot: Tag("foot"),
    };
    let arr = [Tag("e0"), Tag("e1"), Tag("e2")];
    let tup = (Tag("t0"), Tag("t1"));
    let mut slot = Tag("old");
    ok(slot.0 == "old", "the first value is read before the assignment replaces it");
    slot = Tag("new");
    ok(
        (a.0, b.0, c.0, panel.title.0, panel.body.0, frame.head.0, frame.foot.0, arr[0].0, tup.0.0, slot.0)
            == ("a", "b", "c", "title", "body", "head", "foot", "e0", "t0", "new"),
        "every value stays readable up to the moment it drops",
    );
    drop(Tag("explicit"));
    let mid = LOG.lock().unwrap().clone();
    ok(
        mid == ["inner", "old", "explicit"],
        "before the scope ends only the block scope drop, the assignment drop, and the explicit drop ran",
    );
```

listing 44: the scene and the mid scene snapshot, only three drops ran before scope end (source: rust/samples/src/Ch05/DropOrder.rs, lines 41-72)

```
fn main() {
    scene();
    let log = LOG.lock().unwrap().clone();
    let expected = [
        "inner", "old", "explicit", "new", "t0", "t1", "e0", "e1", "e2", "frame", "head",
        "foot", "title", "body", "c", "b", "a",
    ];
    ok(
        log == expected,
        "the full drop sequence matches the reference rules exactly",
    );
    ok(
        log.iter().position(|l| *l == "c") < log.iter().position(|l| *l == "a"),
        "locals drop in reverse declaration order, c before a",
    );
    ok(
        log.iter().position(|l| *l == "frame") < log.iter().position(|l| *l == "head"),
        "a struct's own Drop body runs before its fields drop",
    );
    ok(
        log.iter().position(|l| *l == "head") < log.iter().position(|l| *l == "foot"),
        "fields then drop in declaration order, head before foot",
    );
    ok(
        log.iter().position(|l| *l == "title") < log.iter().position(|l| *l == "body"),
        "a struct without its own Drop drops fields directly in declaration order",
    );
    ok(
        log.iter().position(|l| *l == "e0") < log.iter().position(|l| *l == "e2"),
        "array elements drop from first to last",
    );
    ok(
        log.iter().position(|l| *l == "t0") < log.iter().position(|l| *l == "t1"),
        "tuple fields drop in order",
    );
    ok(
        log.first() == Some(&"inner"),
        "the inner block's local dropped first, at the closing brace",
    );
    ok(
        log.iter().position(|l| *l == "old") < log.iter().position(|l| *l == "explicit"),
        "assignment dropped the old value on the spot, before the explicit drop",
    );
    ok(
        log.iter().position(|l| *l == "new") > log.iter().position(|l| *l == "explicit"),
        "the replacement value survived until scope end",
    );
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 45: the full sequence asserted, plus the six orderings stated as separate checks (source: rust/samples/src/Ch05/DropOrder.rs, lines 75-123)

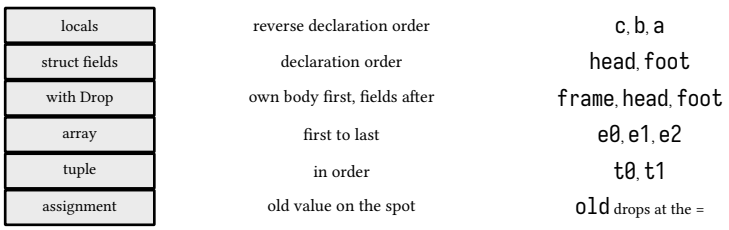


figure 12: drop order rules as one picture

`drop(x)` from the prelude is the explicit form, and `mem::forget` exists at the other extreme, it leaks on purpose. What no safe code can do is observe a value after its drop or drop it twice.

This is RAII with teeth, resource acquisition is initialization, the C++ discipline, but enforced instead of encouraged. The type owns the resource, the scope owns the type, and the brace on the way out is the destructor call. Locks, file handles, and buffers all ride the same rails in the standard library, which is why this chapter's rules matter more than any single API, they are the discipline every wrapper in chapter 12 and chapter 13 depends on.

5.6. ownership at function boundaries

The parameter and return types of a function publish its ownership contract. By value means the callee owns and destroys, `&T` means it reads, `&mut T` means it writes, and returning an owned value hands the title back. Reading a signature is therefore reading the memory story, no comments required:

```
fn owned(name: String) -> String        // takes title, gives title back
fn shared(text: &str) -> usize          // reads, returns a plain value
fn exclusive(list: &mut Vec<u8>)        // writes in place
fn made() -> Box<u32>                   // allocates and hands over
```

fragment, illustrative, not executed

The sample's `round_trip` and `take_ownership` are the two ends of that table, and `mem::take` is the bridge, move the value out through a `&mut`, leave a usable default behind, keep both sides happy.

5.7. Box, the first heap allocation

`Box<T>` is the owned pointer, one allocation, one owner, and the first unsized-aware type this book meets. It is one machine pointer wide, moving it moves the pointer while the heap payload stays at the same address, which the sample measures directly:

```rust
fn heap_addr(b: &Box<u32>) -> usize {
    std::ptr::from_ref::<u32>(&**b) as usize
}

fn pass_through(b: Box<u32>) -> Box<u32> {
    b
}

fn main() {
    let boxed = Box::new(7u32);
    ok(
        *boxed == 7,
        "Box::new allocates on the heap, dereference reads the value back",
    );
    ok(
        type_name::<Box<u32>>().ends_with("Box<u32>"),
        "Box<u32> is a distinct owning type",
    );
    ok(
        std::mem::size_of::<Box<u64>>() == std::mem::size_of::<*const u64>(),
        "a Box is one pointer wide, 8 bytes on the pinned 64 bit host",
    );
    let stack = 7u32;
    ok(
        heap_addr(&boxed) != std::ptr::from_ref(&stack) as usize,
        "the Box payload sits at a different address than a stack local, measured",
    );
    let before = heap_addr(&boxed);
    let boxed = pass_through(boxed);
    let after = heap_addr(&boxed);
    ok(
        before == after,
        "moving a Box moves the pointer, the heap allocation stays at the same address",
```

listing 46: heap versus stack addresses, and the address stability across a move (source: rust/samples/src/Ch05/BoxAlloc.rs, lines 15-47)

```
    let nested = Box::new(Box::new(3u16));
    ok(
        **nested == 3,
        "Box composes, double dereference walks two allocations",
    );
    let boxed_slice: Box<[i32]> = Box::new([1, 2, 3]);
    ok(
        boxed_slice.len() == 3 && boxed_slice[2] == 3,
        "Box::new of an array coerces to a boxed slice, one allocation, a length header",
    );
    ok(
        std::mem::size_of::<Box<[i32]>>() == 16,
        "a boxed slice is a fat pointer, address plus length",
    );
    let mut heap_ints: Vec<u32> = Vec::new();
    heap_ints.push(1);
    heap_ints.push(2);
    let leaked: &'static mut [u32] = Box::leak(heap_ints.into_boxed_slice());
    ok(
        leaked == [1, 2],
        "Box::leak, since 1.26, gives up the destructor for a borrowed view that lives forever",
    );
    leaked[0] = 9;
    ok(
        leaked[0] == 9,
        "the leaked slice is mutable through its reference",
    );
    let owned = Box::new(String::from("boxed"));
    ok(
        owned.len() == 5,
```

listing 47: nested boxes, the boxed slice fat pointer, and leak since 1.26 (source: rust/samples/src/Ch05/BoxAlloc.rs, lines 49-78)

`Box::leak` surrenders the destructor and returns a reference that outlives everything, the escape hatch that turns owned data into `'static` data, chapter 6 picks that thread up. Chapter 12 owns the rest of the smart pointer family, `Rc`, `Arc`, and the rest.

5.8. closures capture ownership

A closure body uses outer variables, and the capture is itself an ownership decision. A plain closure borrows what it can, a `move` closure takes title to everything it mentions, and a closure that consumes its capture can only run once, which is the `FnOnce` bound the sample threads through a generic function. Chapter 10 owns the full closure story, the `Fn` `FnMut` `FnOnce` ladder and the trait mechanics, this is the ownership preview:

```
fn main() {
    let data = String::from("payload");
    let reader = move || data.len();
    ok(
        reader() == 7 && reader() == 7,
        "a move closure owns its capture, callable any number of times while it only reads",
    );
    let text = String::from("borrow me");
    let viewer = || text.as_str();
    ok(
        viewer() == "borrow me",
        "a plain closure borrows its capture instead of taking it",
    );
    ok(
        text == "borrow me",
        "the owner stays usable after the borrowing closure ran, the borrow ended at its last use",
    );
    let maker = move || String::from("built inside");
    ok(
        consume_once(maker) == "built inside",
        "a closure that returns an owned value still moves its captures in",
    );
    let keeper = String::from("kept");
    let deliver = move || keeper;
    let recovered = consume_once(deliver);
    ok(
        recovered == "kept",
        "FnOnce runs exactly once, the captured String left through the call",
    );
```

listing 48: borrow by default, move on request, FnOnce when consumed (source: rust/samples/src/Ch05/ClosureMoves.rs, lines 17-45)

```
    let seed = 5i32;
    let adder = move |step: i32| seed + step;
    let copy_of_adder = adder;
    ok(
        adder(1) == 6 && copy_of_adder(2) == 7,
        "a closure whose captures are Copy is Copy itself, since 1.26, both copies call",
    );
    let names = vec!["ann", "bob"];
    let joined = move || names.join(", ");
    ok(
        joined() == "ann, bob",
        "a move closure can own a Vec, the whole buffer travels into the closure",
    );
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 49: Copy captures make Copy closures since 1.26, and a Vec travels whole (source: rust/samples/src/Ch05/ClosureMoves.rs, lines 46-60)

The capture mode decides which of the three closure traits a closure satisfies, and the traits are a ladder. A closure that only reads its captures can run any number of times, `Fn`. One that writes through them needs them by mutable reference, `FnMut`, still callable repeatedly as long as it owns the call slot. One that consumes a capture, moving it out of the closure environment, runs exactly once, `FnOnce`, and the sample's `consume_once` demands exactly that bound. The compiler picks the least demanding fit from how the body treats each capture, and the `move` keyword only changes where the captures live, never the fit:

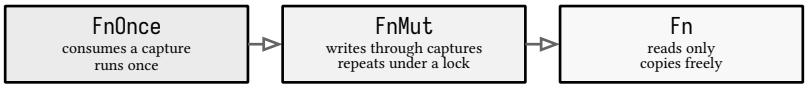


figure 13: capture mode decides the trait, the ladder from FnOnce up to Fn

sources: the rust book's ownership chapter, doc.rust-lang.org/stable/book/ch04-01-what-is-ownership.html, the reference's destructors page for every drop order rule, special types and traits for the Copy boundary, constant and static items, accessed 2026-10-13 against the pinned 1.99.0 offline rust-docs set. the E0382, E0204, and E0184 texts quoted here were produced by rustc probes run 2026-10-13 on the pinned toolchain, and closures implementing Copy since 1.26 is from the release log in the same offline set. claims verified by the Ch05 sample suite, 52 checks under `tools/run-rust-samples.ps1 -Chapter Ch05`.

6. borrowing and lifetimes

Ownership is too expensive to use for every read, so Rust has a second access path: the reference, written `&T` for shared access and `&mut T` for exclusive access. The borrow checker enforces one rule pair, many shared borrows or one exclusive borrow, and lifetimes are the type system spelling of the question “does the referent outlive the reference”. Together they replace use after free, iterator invalidation, and data races at their common root, aliasing plus mutation, book 1, ch 5 (c23: the language, the machine, and the cloud, the abstract machine and undefined behavior) is the corpus chapter on what those bugs cost in a language with no rules.

6.1. shared and exclusive borrows

`&T` promises the target will not change through this path, so any number of them may coexist. `&mut T` promises this is the only live path, so exactly one may exist, and it excludes even reads. The sample runs both sides of each rule:

```
fn main() {
    let mut scores = vec![7, 11, 13];
    let first = &scores[0];
    let second = &scores[1];
    let third = &scores;
    ok(
        first + second == 18,
        "any number of shared borrows coexist, reading through both at once",
    );
    ok(
        third.iter().sum::<i32>() == 31,
        "a shared borrow of the whole collection coexists with borrows of its parts",
    );
    let early = *first;
    scores.push(17);
    ok(
        early == 7 && scores.len() == 4,
        "once the shared borrows hit their last use the owner may mutate again, non lexical",
```

listing 50: shared borrows in a crowd, then the owner mutating again after their last use (source: rust/samples/src/Ch06/Borrows.rs, lines 17-34)

```
    let exclusive = &mut scores;
    exclusive[0] = 100;
    let after = exclusive[0];
    ok(
        after == 100,
        "one exclusive borrow writes through the reference",
    );
    scores.push(19);
    ok(
        scores == [100, 11, 13, 17, 19],
        "the owner takes back control after the exclusive borrow's last use",
    );
    let mut step = 0;
    {
        let edit = &mut step;
        *edit = 5;
    }
    {
        let edit = &mut step;
        *edit += 1;
    }
    ok(
```

listing 51: one exclusive borrow at a time, sequenced reborrowing, and borrows in function calls (source: rust/samples/src/Ch06/Borrows.rs, lines 36-57)

The exclusivity is about overlap, not count. Two exclusive borrows that never overlap in time are fine, and the borrow ends at its last use, not at the end of the lexical scope, which is the next section's subject.

while it lives	add another `&`	add a `&mut`
shared `&`	yes, any number	no, E0502
exclusive `&mut`	no, E0502	no, E0499

the reference must also stay valid, that half is lifetimes

figure 14: the borrow rules as one matrix

6.2. the borrow checker, non lexical

The original borrow checker tied a borrow to its lexical scope, the enclosing block, and rejected code whose borrows clearly died mid block. The 2018 edition shipped with the fix, the non lexical lifetime borrow checker that ends each borrow at its last use, so 1.31 is the version that made this compile. The 2015 edition received the same checker in 1.36, and the old checker was fully retired after 1.40. Every pattern in the sample would have failed the lexical checker:

```
fn main() {
    let mut letters = vec![10, 20, 30];
    let head = &letters[0];
    let recorded = *head;
    letters.push(40);
    ok(
        recorded == 10 && letters.len() == 4,
        "the borrow dies at its last use, the push after it is legal since 1.31",
    );
    let mut word = String::from("nll");
    let measured = {
        let view = &word[..];
        view.len()
    };
    word.push('!');
    ok(
        measured == 3 && word == "nll!",
        "a block scoped borrow ends at the closing brace, mutation follows",
    );
```

listing 52: the two classic survivors: a borrow that dies before a push, and a block scoped borrow (source: rust/samples/src/Ch06/Nll.rs, lines 15-33)

```
    for i in 0..letters.len() {
        let each = &letters[i];
        running += *each;
    }
    letters.clear();
    ok(
        running == 100 && letters.is_empty(),
        "each iteration's borrow dies inside the loop body, the clear after the loop is legal",
    );
    let mut ages: HashMap<&str, u32> = HashMap::new();
    ages.insert("ann", 30);
    if let Some(age) = ages.get_mut("ann") {
        *age += 1;
    }
    ages.insert("bob", 41);
    ok(
        ages.get("ann") == Some(&31) && ages.get("bob") == Some(&41),
        "the get_mut borrow ends with the if let arm, the insert afterwards is fine",
    );
    let mut data = vec![5, 6];
    match data.get_mut(0) {
        Some(slot) => *slot = 50,
        None => unreachable!(),
    }
    data.push(7);
    ok(
        data == [50, 6, 7],
        "match holds its borrow only for the arm, push follows",
    );
    let mut text = String::from("order");
    let first_len = text.len();
    text.push_str("ed");
```

listing 53: loop borrows, if let arms, match arms, the shapes that trip newcomers (source: rust/samples/src/Ch06/Nll.rs, lines 35-66)

NOTE: what still fails

A borrow held across the mutation still fails, NLL moved the end of the borrow to the last use, it did not remove the exclusivity rule. Keeping the borrowed value alive past the push, storing it or printing it after, is E0502 and lands in the error section below.

6.3. lifetime parameters

A reference type is incomplete without the answer to “outlives what”. The `'a` in `&'a str` names that answer, and it becomes a generic parameter whenever a function or struct ties references together. Structs holding references must declare the tie, the parameter bounds how long the struct may live:

```
struct Excerpt<'a> {
    part: &'a str,
}

impl<'a> Excerpt<'a> {
    fn announce(&self, msg: &str) -> &str {
        println!("excerpt announces {msg}");
        self.part
    }
}

fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() ≥ y.len() {
        x
    } else {
        y
    }
}

fn longest_with_note<'a, T>(x: &'a str, y: &'a str, note: T) -> &'a str
where
    T: std::fmt::Display,
{
    println!("note: {note}");
    longest(x, y)
}
```

listing 54: a struct of borrowed data, its methods, and the longest function with its tie (source: rust/samples/src/Ch06/StructRefs.rs, lines 13-38)

The parameter on `longest<'a>` states a constraint, not a fact: the caller must find one region that covers both inputs and the result. The sample shows the short side of that bargain, inside a block where both inputs are alive the result is fine, after the block only the longer lived input remains:

```
fn main() {
    let novel = String::from("call me ishmael. some years ago.");
    let first_sentence;
    {
        let whole = novel.as_str();
        let piece = whole.split('.').next().unwrap_or("");
        first_sentence = Excerpt { part: piece };
    }
    ok(
        first_sentence.part == "call me ishmael",
        "the struct borrows from the novel, the borrow outlives the inner block",
    );
    ok(
        first_sentence.announce("held") == "call me ishmael",
        "a method can return the struct's own borrowed field, the impl lifetime carries it",
    );
    let long = String::from("long string is long");
    let short = "hi";
    let result = longest(long.as_str(), short);
    ok(
        result == "long string is long",
        "the returned reference is tagged with the input lifetime, both inputs satisfy it here",
    );
    ok(
        longest_with_note("abc", "de", 7) == "abc",
        "generic parameters ride along, the lifetime bound only touches the str inputs",
    );
    let string1 = String::from("tiny");
    let result;
    {
        let string2 = String::from("enormous value");
        result = longest(string1.as_str(), string2.as_str());
        ok(
            result.len() == 14,
            "inside the block the borrow of the shorter lived input is still valid",
        );
    }
    ok(
```

listing 55: call sites satisfying the tie, and the boundary the compiler draws (source: rust/samples/src/Ch06/StructRefs.rs, lines 40-77)

6.4. elision, the three rules

Writing `'a` on every signature would be noise, so the compiler infers elided lifetimes with three rules, applied in order. Rule one, each elided lifetime in the inputs becomes its own fresh parameter. Rule two, if there is exactly one lifetime in the inputs, every elided output lifetime takes it. Rule three, in methods, a receiver of `&self` or `&mut self` lends its lifetime to every elided output. When the rules cannot decide, the signature is rejected with E0106:

```
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, item) in bytes.iter().enumerate() {
        if *item == b' ' {
            return &s[..i];
        }
    }
    s
}

fn tail(s: &'_ str) -> &'_ str {
    &s[1..]
}

fn trimmed_pair(a: &str, b: &str) -> usize {
    a.trim().len() + b.trim().len()
}

struct Owner {
    name: String,
}

impl Owner {
    fn label(&self) -> &str {
        &self.name
    }
}
```

listing 56: the four signatures, one input, placeholder, two inputs no output, and the self rule (source: rust/samples/src/Ch06/Elision.rs, lines 13-39)

```
fn longest(s1: &str, s2: &str) -> &str {
    if s1.len() > s2.len() { s1 } else { s2 }
}
```

fragment, illustrative, not executed

`error[E0106]: missing lifetime specifier`, and the help text names the ambiguity exactly, the signature does not say whether the return is borrowed from `s1` or `s2`. Two inputs, one output, no rule applies, the tie must be written by hand. The `'_'` placeholder, stable since 1.26, says “elide here” inside paths where bare elision is not allowed, and `impl Reader for BufReader<'_>` in impl headers arrived with the 2018 edition in 1.31:

```
fn main() {
    ok(
        first_word("hello world") == "hello",
        "rule 2 in action, one input lifetime flows to the output, no annotation needed",
    );
    ok(
        first_word("oneword") == "oneword",
        "the no space case returns the whole slice",
    );
    ok(
        tail("skip") == "kip",
        "the underscore placeholder '_ since 1.26 says elide here, same expansion",
    );
    ok(
        trimmed_pair(" a ", "bb") == 3,
        "two inputs, no output, nothing to infer, elision is free",
    );
    let owner = Owner {
        name: String::from("ann"),
    };
    let label = owner.label();
    ok(
        label == "ann",
        "the method rule, a &self receiver lends its lifetime to the output",
    );
    let extended: fn(&str) -> &str = first_word;
    ok(
        extended("fn pointer") == "fn",
        "function pointer types elide the same way, the signature is for<'a> fn(&'a str) -> &'a str",
    );
    let trait_object: &dyn Fn(&str) -> &str = &|s| s;
    ok(
        trait_object("closure") == "closure",
        "Fn trait objects elide too, the same higher ranked shape",
    );
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 57: the elided signatures running, fn pointers and trait objects elide the same way (source: rust/samples/src/Ch06/Elision.rs, lines 41-77)

6.5. static

`'static` is the lifetime of the whole program, the longest region there is. String literals have it, their bytes sit in the read only data of the binary. Because a longer lifetime coerces to any shorter one, a `&'static str` flows anywhere a `&'a str` is wanted, the subtyping rule the reference states as `&'static str` being a subtype of `&'a str`:

```
static LABEL: &str = "global";

const PREFIX: &str = "v";

fn literal() -> &'static str {
    "from a function"
}

fn any_lifetime<'a>(s: &'a str) -> &'a str {
    s
}

fn needs_static(s: &'static str) -> usize {
    s.len()
}
```

listing 58: literal and static sources of ‘static, the coercion target, the bound (source: rust/samples/src/Ch06/StaticLife.rs, lines 13-27)

```
fn main() {
    ok(
        literal() == "from a function",
        "a string literal is 'static, the bytes live in the binary for the whole run",
    );
    let promoted: &'static i32 = &42;
    ok(
        *promoted == 42,
        "constants promote, a reference to a const eligible value can also claim 'static",
    );
    ok(
        LABEL == "global" && PREFIX == "v",
        "static and const initializers hold 'static references without spelling it",
    );
    ok(
        any_lifetime(literal()) == "from a function",
        "'static coerces into any shorter lifetime, &'static str flows into &'a str",
    );
    ok(
        needs_static(literal()) == 15,
        "a 'static bound demands data that lives forever, literals qualify",
    );
    let owned = String::from("runtime built");
    let borrowed: &str = &owned;
    ok(
        any_lifetime(borrowed).len() == 13,
        "a borrow of a run time String is not 'static, it flows only into matching lifetimes",
    );
    let leaked: &'static mut str = Box::leak(String::from("leaked").into_boxed_str());
    leaked.make_ascii_uppercase();
    ok(
        leaked == "LEAKED",
        "the leaked reference is mutable, ownership semantics still apply through it",
    );
    ok(
        needs_static(leaked) == 6,
        "Box::leak since 1.26 is the one way run time data becomes 'static, by giving up the drop",
    );
    let both = longest_static_hold("a", "bb");
    ok(
        both == "bb",
        "an explicit 'static parameter outlives every caller, the return never dangles",
    );
    println!("measured 2026-10-13 on ~/.rustup/toolchains/stable-x86_64-pc-windows-msvc");
}
```

listing 59: what qualifies for ‘static, what does not, and the leak door since 1.26 (source: rust/samples/src/Ch06/StaticLife.rs, lines 29-73)

The distinction to keep straight: `'static` on a reference means the referent lives forever, `'static` in a bound like `T: 'static` means `T` contains no borrowing shorter than forever, owned data qualifies. Run time data is not `'static` until `Box::leak` surrenders its destructor, which trades a drop for a forever reference and should be a deliberate, rare act.

PITFALL: lifetime syntax does not change how long anything lives

> Writing `'a` in `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` creates no region and shortens no value. The annotations only describe the relationships the checker must verify, the actual regions come from the call site, where the compiler picks the shortest region that satisfies every tie. When a lifetime error appears, the fix is in the data flow, move the value earlier, return the owned copy, shrink the struct's life, never in the annotations themselves.

A function returning a borrowed value with no inputs to borrow from fails before lifetimes even enter the picture, E0106 with a help note that offers the two real fixes, return owned data or borrow from a parameter:

```
fn dangle() -> &String {
    let s = String::from("hello");
    &s
}
```

fragment, illustrative, not executed

The `&String` return names no source, and even with `'static` spelled out the body could not produce one, the local `s` dies at the closing brace. Chapter 12's smart pointers are the toolbox when neither plain ownership nor plain borrowing fits.

6.6. the error vocabulary

Three error codes cover most borrow checker encounters. E0502 is the mutability clash, E0499 is the double exclusive, E0597 is the value that died while borrowed. Each snippet below was compiled on the pinned toolchain and the quoted line is verbatim rustc output:

```
let mut v = vec![1, 2, 3];
let first = &v[0];
v.push(4);
println!("first is {first}");
```

fragment, illustrative, not executed

`error[E0502]: cannot borrow v as mutable because it is also borrowed as immutable`, with the three annotations: immutable borrow occurs here, mutable borrow occurs here, immutable borrow later used here. The last annotation is the load bearing one, deleting the later use makes the program compile, which is the NLL fix in miniature.

```
let mut i = 0;
let x = &mut i;
let y = &mut i;
*x += 1;
*y += 1;
```

fragment, illustrative, not executed

`error[E0499]: cannot borrow i as mutable more than once at a time`, first mutable borrow, second mutable borrow, first borrow later used. Again the later use is what makes the overlap real, dropping the last line lets the second borrow start after the first one ended.

```
let r;
{
    let x = 5;
    r = &x;
}
println!("r: {r}");
```

fragment, illustrative, not executed

`error[E0597]: x does not live long enough`, borrowed value does not live long enough, `x` dropped here while still borrowed, borrow later used here. This is the dangling reference the compiler refuses to build.

Reading these diagnostics is a skill worth practicing, the format is stable across all three: each annotation names a moment, borrow occurs, borrow later used, value dropped, and the conflict is the span between two of those moments. Fixing the code means shrinking one span, moving the last use earlier, or lengthening a life, declaring the owner outside the block. Slices participate in the same system, a `&[T]` borrows whatever storage it views, which is why pushing to a `Vec` through a live slice reference is the same E0502 as the vector case above, and why the iterator invalidation class of bugs in other languages does not compile here at all.

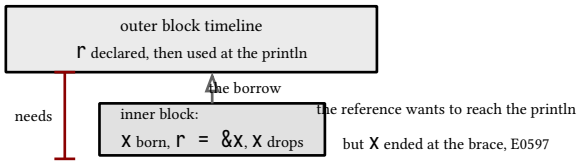


figure 15: the borrow graph of the E0597 case, the borrow outlives its referent

sources: the rust book's references and borrowing chapter, doc.rust-lang.org/stable/book/ch04-02-references-and-borrowing.html, the reference's lifetime elision page and subtyping page, the error codes index for E0502, E0499, E0597, and E0106, all accessed 2026-10-13 against the pinned 1.99.0 offline rust-docs set. the NLL history is from the release log in that set, releases.md: the 2018 edition at 1.31, non lexical lifetimes on the 2015 edition at 1.36, the old borrow checker's exit at 1.39 to 1.40, read 2026-10-13. the quoted error texts were produced by rustc probes run 2026-10-13 on /.rustup/toolchains/stable-x86_64-pc-windows-msvc. claims verified by the Ch06 sample suite, 39 checks under `tools/run-rust-samples.ps1 -Chapter Ch06`.

7. structs, enums, and patterns

Chapter 5 gave moves and borrows, chapter 6 gave the lifetimes that scope them. This chapter builds the data shapes those rules govern: products (structs) and sums (enums), then the pattern language that decomposes both. The pattern half is where rust earns the ml family comparison, a match that proves its own coverage, and the arc runs from fixed slice patterns in 1.26 through let-else in 1.65 to let chains in 1.88, each step attributed inline. Chapter 2 walked the edition ladder, the 2024 edition is where let chains live.

7.1. structs: named, tuple, unit

Three spellings cover every product type. The named struct is the default, the tuple struct wraps one position, the unit struct carries no data at all:

```
struct Point {
    x: f64,
    y: f64,
}

struct Celsius(f64);

struct Marker;

enum Shape {
    Circle { radius: f64 },
    Rect { w: f64, h: f64 },
    Square(f64),
}

enum Opcode {
    Add,
    Sub,
    Mul,
}
```

listing 60: the three struct spellings and a sum type beside them (source: rust/samples/src/Ch07/Adts.rs, lines 13-32)

Methods hang off `impl` blocks and chapter 5's move rules apply unchanged: `norm2` borrows `&self`, `apply` takes `self` by value and consumes it. Field init shorthand, `x, y` instead of `x: x`, and the update syntax `..p` keep construction short:

```
impl Point {
    fn new(x: f64, y: f64) -> Point {
        Point { x, y }
    }

    fn norm2(&self) -> f64 {
        self.x * self.x + self.y * self.y
    }
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle { radius } => std::f64::consts::PI * radius * radius,
            Shape::Rect { w, h } => w * h,
            Shape::Square(s) => s * s,
        }
    }
}
```

listing 61: an associated constructor, a borrowed method, an exhaustive area (source: rust/samples/src/Ch07/Adts.rs, lines 34-52)

The runner measures the shapes. A unit struct is zero sized, a tuple struct destructures like a tuple, and struct update moves only the fields it copies from `p`, so `p` is partially moved afterwards:

```
fn main() {
    let p = Point::new(3.0, 4.0);
    ok(p.norm2() == 25.0, "a named struct method reads its fields");
    let shifted = Point { y: 0.0, ..p };
    ok(
        shifted.x == 3.0 && shifted.y == 0.0,
        "struct update syntax copies the unlisted fields",
    );
    let Celsius(t) = Celsius(21.5);
    ok(t == 21.5, "a tuple struct destructures positionally");
    ok(std::mem::size_of::<Marker>() == 0, "a unit struct is zero sized");
```

listing 62: method call, update syntax, positional destructure, zero sized marker (source: rust/samples/src/Ch07/Adts.rs, lines 64-74)

7.2. the newtype pattern

A tuple struct with exactly one field is a newtype, and it is the cheapest abstraction in the language. `Celsius(f64)` costs nothing at runtime, chapter 12 measures this class of wrapper at exactly the inner size, yet it is a distinct type to the compiler: a bare `f64` will not bind where a `Celsius` is expected, and no implicit conversion exists in either direction. The pattern carries three duties at once, unit clarity, capability tracking, and orphan rule relief from chapter 8, wrapping a foreign type in a local newtype makes it yours to implement traits on.

```
struct Celsius(f64);
struct Fahrenheit(f64);
fn boil(t: Celsius) -> bool { t.0 ≥ 100.0 }
let f = Fahrenheit(212.0);
boil(f);
// error[E0308]: mismatched types, expected `Celsius`, found `Fahrenheit`
// the two f64 wrappers never mix, the field is public but the type gates
```

fragment, illustrative, not executed

Access is positional, `.0`, and construction is a function call, `Celsius(21.5)` is the tuple constructor applied to a value. The sample destructures it the same way, `let Celsius(t) = ...`, an irrefutable pattern because every value of the type matches.

7.3. enums as sum types

An enum lists alternatives, each variant is either a unit, a tuple, or a record. `Shape` uses all three spellings in one declaration. The compiler assigns each variant a discriminant, and the sample proves the discriminants differ. Sizing follows: a three variant dataless enum is one byte, and `Option<&u8>` reuses pointer bits for the none case so it costs nothing over a bare reference:

```
        std::mem::size_of::<Option<&u8>>() == std::mem::size_of::<&u8>(),
        "the none niche reuses pointer bits, no size growth",
    );
    ok(
        std::mem::size_of::<Option<Opcode>>() == 1,
        "a three variant dataless enum packs into one byte",
    );
    ok(
        std::mem::size_of::<Option<u64>>() == 16,
        "an inline u64 payload forces the tag slot to 8 bytes",
    );
```

listing 63: the niche at work, measured on this build (source: rust/samples/src/Ch07/Adts.rs, lines 93-103)

That niche reuse is a compiler optimization, not a language promise, but it is why `Option<&T>` replaced nullable pointers with no size penalty. The tag for an inline `u64` payload is forced up to a full word by alignment, the same padding arithmetic any c programmer knows, measured at 16 bytes.

7.4. match and exhaustiveness

A match arm is a pattern, optionally a guard, then a body. The compiler checks that the arms cover every value of the scrutinee type, so adding a variant to `Event` breaks every match over it until each new case is handled:

```
enum Event {
    Key(char),
    Click(i32, i32),
    Scroll(i32),
    Quit,
}

const HOT: char = 'q';

fn label(e: &Event) -> &'static str {
    match e {
        Event::Quit => "quit",
        Event::Key(HOT) | Event::Key('Q') => "hot quit",
        Event::Key(c) if c.is_ascii_digit() => "digit",
        Event::Key(_) => "key",
        Event::Click(x, y) if x == y => "diagonal click",
        Event::Click(..) => "click",
        Event::Scroll(1..=3) => "coarse scroll",
        Event::Scroll(n @ 4..) if n % 2 == 0 => "even scroll",
        Event::Scroll(_) => "scroll",
    }
```

listing 64: guards, or patterns, ranges, and one binding per arm (source: rust/samples/src/Ch07/Match.rs, lines 13-33)

Constants can appear in patterns, `HOT` above, and literals inside variants too. Guard clauses run after the pattern matches, so `Event::Click(5, 9)` fails the `x == y` guard and falls to the next arm. The wildcard `_` closes any match, and integer ranges can partition a whole `u8` without one since 1.33:

```
error[E0004]: non-exhaustive patterns: `Event::Scroll(_)` not covered
  --> src/main.rs
   = help: ensure that all possible cases are being handled
```

fragment, illustrative, not executed

Delete one arm and that is the error, verbatim, naming the uncovered variant. This is the same contract as book 5, ch 9 (java 27, pattern matching) sealed hierarchies: closed sums plus a compiler that walks them.

Any value can be the scrutinee, not just enums. Tuples match positionally, literals and ranges match numbers and chars, and a named binding without a subpattern is itself a pattern, the irrefutable one that always succeeds:

```
let pair = (2, "two");
match pair {
    (0, _) => (),
    (n, word) if n == word.len() => (),
    whole => (),
}
// tuple arms try top to bottom, the first fit wins,
// and `whole` binds whatever reaches it, never failing
```

fragment, illustrative, not executed

Arm order is the search order. The compiler only checks exhaustiveness, it does not reorder, so a specific arm placed under a general one is dead and the general one wins every time. The dominance rules go further for literals: two identical literal arms are an outright error, unreachable pattern, while two overlapping guarded arms are legal and the earlier guard gets first refusal, the guarded-then-unguarded pairing the sample relies on for `Event::Scroll`.

7.5. guards, bindings, and at

The `@` sign binds a name to the whole value a subpattern matched, which is how one arm tests a range and still reads the number:

```
fn delta(e: &Event) -> i32 {
    match e {
        Event::Scroll(n @ 1..=3) => *n * 100,
        Event::Scroll(n) => *n * 10,
        Event::Click(x, _) => *x,
        _ => 0,
    }
}

fn grade(score: u8) -> char {
    match score {
        90..=255 => 'A',
        80..=89 => 'B',
        70..=79 => 'C',
        0..=69 => 'F',
    }
```

listing 65: at with a range on one arm, plain binding on the next (source: rust/samples/src/Ch07/Match.rs, lines 36-51)

`n @ 1..=3` tests the range and binds `n` to the same `i32`. Since 1.53 or patterns nest inside subpatterns, `Event::Key('a' | 'b')`, and since 1.42 `matches!` wraps a pattern test in a boolean, with the guarded form used in the sample. `assert_matches!` landed much later, in 1.96, the current year's stable. Bindings may also be typed: `ref`, `ref mut`, and `mut` inside patterns still work, but match ergonomics below made them mostly unnecessary.

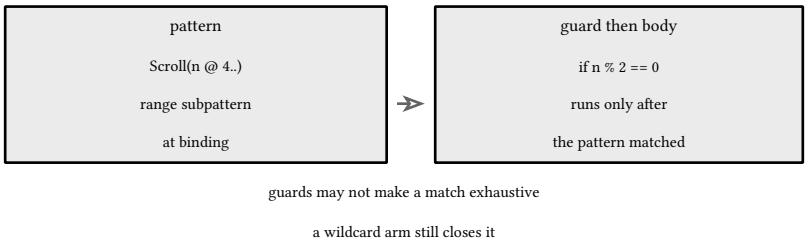


figure 16: one arm, four moving parts

7.6. if let, while let, let-else

Refutable patterns get three lighter forms. `if let` runs a block on one pattern, `while let` loops until the pattern fails, and let-else, since 1.65, binds on success and forces the else block to diverge:

```
fn to_ounces(cups: Option<f64>) -> f64 {
    let Some(v) = cups else {
        return 0.0;
    };
    v * 8.0
}
```

listing 66: let-else since 1.65, the else must diverge (source: rust/samples/src/Ch07/IfLet.rs, lines 13-18)

The else block must `return`, `break`, `continue`, or panic, the checker rejects any path that falls through. That single rule is the whole feature: after the let-else, the binding is live and every later line knows the pattern held. `while let` pairs naturally with `Vec::pop`:

```
    let mut stack = vec![3, 2, 1];
    let mut drained = 0;
    while let Some(top) = stack.pop() {
        drained += top;
    }
    ok(drained == 6, "while let drains a stack until pop returns none");
```

listing 67: while let over a stack, none ends the loop (source: rust/samples/src/Ch07/IfLet.rs, lines 52-57)

Edition 2024 in 1.88 stabilized let chains, `if let` and boolean conditions mixed freely in one `if`, with any number of `let` conditions. Since 1.33 `if let` also accepts or patterns:

```
fn classify(input: Option<u32>) -> &'static str {
    if let Some(n) = input && n > 100 {
        "large"
    } else if let Some(n) = input && n > 10 {
        "medium"
    } else {
        "small or absent"
    }
}
```

listing 68: an else-if-let chain, each branch with its own let chain (source: rust/samples/src/Ch07/IfLet.rs, lines 29-37)

```
    let word = "track 9";
    if let Some(first) = first_key(word)
        && first.is_ascii_lowercase()
        && let Some(last) = word.chars().last()
        && last.is_ascii_digit()
    {
        ok(last == '9', "three let conditions and two booleans in one if");
    } else {
        ok(false, "the chain should have held");
    }
```

listing 69: three let conditions and two booleans in one if, edition 2024 (source: rust/samples/src/Ch07/IfLet.rs, lines 70-79)

```
    let pair = (Some(4), None::<u32>);
    if let (Some(a), Some(_)) | (Some(a), None) = pair {
        ok(a == 4, "or patterns fold two tuple shapes in one if let since 1.33");
    } else {
        ok(false, "an or pattern arm should have held");
    }
    if let Some(n @ 3..=7) = maybe {
        ok(n == 5, "a binding and range combine under if let");
    }
```

listing 70: or patterns folding two tuple shapes in one if let since 1.33 (source: rust/samples/src/Ch07/IfLet.rs, lines 80-88)

NOTE: the ladder

Fixed entry slice patterns 1.26, match ergonomics 1.26, `|` in if let and while let 1.33, exhaustive integer patterns 1.33, subslice patterns and `matches!` 1.42, or patterns in subpatterns 1.53, let-else 1.65, let chains 1.88 in edition 2024 only, `assert_matches!` 1.96. Every number checked against the offline `releases.md`.

7.7. match ergonomics

Since 1.26 patterns apply dereferences automatically. Matching a `&Expr` against `Expr::Lit(v)` binds `v` as `&i64`, no explicit `&` in the pattern, and the same holds through any depth of references and through `&mut`:

```
fn eval(e: &Expr) -> i64 {
    match e {
        Expr::Lit(v) => *v,
        Expr::Add(a, b) => eval(a) + eval(b),
        Expr::Neg(inner) => -eval(inner),
    }
}

fn deep_ref(e: &&Expr) -> i64 {
    match e {
        Expr::Lit(v) => *v,
        _ => 0,
    }
}
```

listing 71: a reference scrutinee and a double reference, both matched by value patterns (source: rust/samples/src/Ch07/Ergonomics.rs, lines 19-32)

The mechanism is the default binding mode, which starts in move mode and switches to ref when the scrutinee is a reference. Edition 2024 tightened the edges: once the binding mode is not move, writing `mut`, `ref`, or `ref mut` on a binding is an error, and `&` or `&mut` patterns are only allowed in the fully explicit prefix of a pattern. Old code that mixed both styles had to pick one. Slice patterns complete the picture, fixed entries since 1.26 and rest patterns with `..` since 1.42:

```
fn edges(xs: &[u64]) -> (u64, u64) {
    match xs {
        [] => (0, 0),
        [only] => (*only, *only),
        [first, .., last] => (*first, *last),
    }
}

fn head3(xs: &[u8]) -> String {
    match xs {
        [a, b, c, ..] => format!("{}{}{}", *a as char, *b as char, *c as char),
        _ => xs.len().to_string(),
    }
}
```

listing 72: empty, single, both ends pinned, and a three element prefix (source: rust/samples/src/Ch07/Ergonomics.rs, lines 34-47)

`edges` is total over slices with no wildcard arm, the three shapes cover `[]`, `[x]`, and everything longer. A `Vec` coerces to a slice for the match, deref coercion from chapter 6 doing the work.

7.8. Option and Result

`Option<T>` is the enum with `None` and `Some(T)`, `Result<T, E>` the enum with `Ok(T)` and `Err(E)`. Both are ordinary enums, every pattern tool in this chapter applies to them, and the niche sizing measured above is `Option`'s. Their full treatment, `?`, conversions, and the error trait, is chapter 11's subject. Here they close the loop: absence is a value, not a null pointer, and the type system forces every read to pick a branch for it.

Because they are ordinary enums, the lighter forms all apply too. `if let Some(x) = lookup()` is the everyday read, let-else turns absence into an early return, and the sample's `to_ounces` is the canonical shape. What patterns cannot do is invent a default, that job belongs to the combinators, and they are just methods returning the same enums:

```
let port = env::var("PORT").ok().map(|s| s.parse::<u16>().ok()).flatten();
let port: Option<u16> = maybe_parse.and_then(|s| s.parse().ok());
let port = port.or(Some(8080));
// map, and_then, or, unwrap_or, ok_or: every one returns
// an option or result, so the chain never leaves the enum world
```

fragment, illustrative, not executed

`and_then` is `map` for fallible closures, `or` substitutes an alternative absence, `ok_or` converts `Option` into `Result` by supplying the error. They compose because each returns the same container it consumed, the same design the iterator adapters of chapter 10 follow. The rule of thumb that falls out: match when the branches need different logic, chain combinators when the branches are transform or default.

`Vec<T>`, `HashMap<K, V>`, and friends are products of the same story told in chapters 6 and 15, their accessors return `Option` precisely so that absence stays on the type level. A `None` from `HashMap::get` carries no error information, it is pure absence, which is why lookup methods that can fail for a reason return `Result` instead.

sources: doc.rust-lang.org/stable/book ch05 structs and ch06 enums and match, reference/patterns.html for default binding modes, range patterns, and the irrefutable and refutable pattern classes, reference/statements.html for the let-else divergence rule, edition-guide rust-2024 match ergonomics chapter for the binding mode error rule, releases.md for every version number above, all read from the offline oracle at /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html and cross-checked against doc.rust-lang.org/stable url forms, accessed 2026-10-13. Every behavioral claim is asserted by `rust/samples/src/Ch07`, 57 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1 -Chapter Ch07` on rustc 1.99.0 edition 2024, measured 2026-10-13.

8. traits and trait objects

A trait is a set of behaviors a type can adopt, half interface and half mixin: required methods, optional default bodies, associated types and constants, and a coherence rule that decides who may implement what for whom. This chapter covers the whole surface, defining, deriving, overloading, and then the two dispatch modes, static generics against `dyn Trait`, with the vtable cost measured. The `dyn` keyword itself landed in 1.27, supertrait upcasting in 1.86, both checked in `releases.md`.

8.1. defining and implementing

A trait declares methods an implementer must provide and may provide for them. `Summary` requires `headline`, supplies `preview` as a default, and adds an associated function that needs a `Sized` bound because it never sees a value:

```
trait Summary {
    fn headline(&self) -> String;
    fn preview(&self) -> String {
        format!("{}...", self.headline())
    }
    fn tag() -> &'static str
    where
        Self: Sized,
    {
        "summary"
    }
}
```

listing 73: required method, default method, sized-bounded associated function (source: rust/samples/src/Ch08/Defining.rs, lines 13-24)

Two types implement it, one riding the default, one overriding:

```
impl Summary for Article {
    fn headline(&self) -> String {
        format!("{} by {}", self.title, self.author)
    }
}

impl Summary for Chirp {
    fn headline(&self) -> String {
        format!("{}: {}", self.user, self.text)
    }

    fn preview(&self) -> String {
        format!("chirp from {}", self.user)
    }
}
```

listing 74: the article impl inherits the default, the chirp impl overrides it (source: rust/samples/src/Ch08/Defining.rs, lines 36-50)

The default body calls `self.headline()`, a method the implementer has not written yet. That forward reference is the mixin half: every impl gets `preview` for free, specialized only where it pays.

8.2. bounds, supertraits, extension traits

`trait Long: Summary` makes `Summary` a supertrait: every `Long` type is a `Summary` type, and `Long`'s default body freely calls the supertrait method. The blanket impl below extends every existing `Summary` in one stroke, which is the extension trait pattern `Iterator` itself uses, new methods on old types with no changes to the old code:

```
trait Long: Summary {
    fn word_count(&self) -> usize {
        self.headline().split_whitespace().count()
    }
}

impl<T: Summary + ?Sized> Long for T {}

fn announce<S: Summary>(item: &S) -> String {
    format!("[{}] {}", item.preview(), S::tag())
}

fn announce_shorthand(item: &impl Summary) -> String {
    format!("[{}]", item.preview())
}
```

listing 75: supertrait, blanket impl, two spellings of the same bound (source: rust/samples/src/Ch08/Defining.rs, lines 52-66)

`fn announce<S: Summary>` and `fn announce_shorthand(item: &impl Summary)` compile to the same thing, one static dispatch per concrete type. The `+ ?Sized` in the blanket impl lets the extension reach `dyn Summary` too, which is unsized.

8.3. associated types and consts

Methods are the visible half of a trait. The other half is the associated items a trait can demand: a `type` each impl picks once, and a `const` each impl fixes at compile time. `Iterator` is the famous case, one `type Item` instead of a type parameter, so `Iterator<Item = u32>` reads as a constraint instead of an argument:

```
trait Channel {
    type Message: std::fmt::Debug;
    const CAPACITY: usize;

    fn send(&mut self, message: Self::Message);
    fn last(&self) -> Option<&Self::Message>;
}

struct Queue {
    items: Vec<u8>,
}

impl Channel for Queue {
    type Message = u8;
    const CAPACITY: usize = 4;

    fn send(&mut self, message: u8) {
        self.items.push(message);
    }

    fn last(&self) -> Option<&u8> {
        self.items.last()
    }
}

fn capacity_of<C: Channel>() -> usize {
    C::CAPACITY
}
```

listing 76: a channel trait with an associated type, a bound on it, and an associated const (source: rust/samples/src/Ch08/Defining.rs, lines 68-96)

The `Message` type carries its own bound, `Debug`, which the caller inherits for free, the sample formats it. The projection syntax `<Queue as Channel>::Message` names the choice explicitly, and `C::CAPACITY` reads the const through any bounded generic without a value in hand. The rule of thumb for the choice: a generic parameter says the caller picks per call, an associated type says the impl picks once per type, which is why `Add` has `type Output` while `HashMap<K, V>` takes parameters.

8.4. coherence and the orphan rule

An impl is legal only if the trait or at least one of the types belongs to the current crate, and no two impls may overlap. That is the orphan rule, and it is why the ecosystem cannot deadlock: nobody can `impl Display for Vec<ForeignType>` from outside both crates, so no two dependencies can ever ship conflicting impls of the same trait for the same type.

```
impl std::fmt::Display for Vec<crate::Cfg> { /* .. */ }
// error[E0117]: only traits defined in the current crate can be
// implemented for arbitrary types, both foreign
```

fragment, illustrative, not executed

The escape hatches are the newtype pattern, wrap the foreign type in a local tuple struct and impl on the wrapper, or a local trait that delegates. Both are ordinary code, no compiler switch involved.

8.5. dyn Trait and the vtable

`dyn Trait` erases the concrete type and keeps the method table. A `&dyn Glyph` is a fat pointer, data pointer plus vtable pointer, two words measured in the sample against a plain fn pointer's one:

```
trait Draw {
    fn draw(&self) -> String;
}

trait Glyph: Draw {
    fn name(&self) -> &'static str;
    fn render(&self) -> String {
        format!("{}: {}", self.name(), self.draw())
    }
}

struct Dot {
    v: u8,
}

struct Bar {
    len: u8,
}
```

listing 77: a supertrait pair and two implementers (source: rust/samples/src/Ch08/Objects.rs, lines 13-30)

```
fn main() {
    let items: Vec<Box<dyn Glyph>> = vec![
        Box::new(Dot { v: 7 }),
        Box::new(Bar { len: 3 }),
    ];
    let mut rendered = Vec::new();
    for item in &items {
        rendered.push(item.render());
    }
    ok(
        rendered == vec!["dot: dot 7".to_string(), "bar: bar of 3".to_string()],
        "a boxed slice of dyn glyphs dispatches each call",
    );
    ok(total(&items) == 2, "a slice of boxes passes as one fat reference");
    let dot: &dyn Glyph = items[0].as_ref();
    ok(
        dot.name() == "dot",
        "a bare dyn reference carries the same dispatch",
    );
    let as_draw: &dyn Draw = dot;
    ok(
        as_draw.draw() == "dot 7",
        "a dyn glyph upcasts to its supertrait since 1.86",
    );
    ok(
        std::mem::size_of::<&dyn Draw>() == 2 * std::mem::size_of::<&u8>(),
        "a trait object reference is two words, data and vtable",
    );
```

listing 78: heterogeneous dispatch, upcast, and the two word measurement (source: rust/samples/src/Ch08/Objects.rs, lines 60-87)

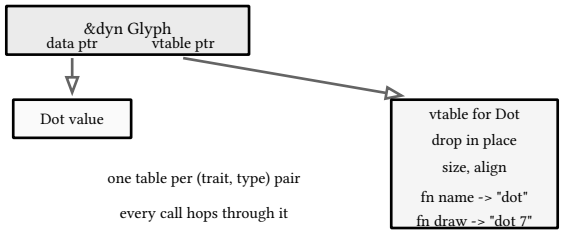


figure 17: the fat pointer and its vtable

Trait objects default to `+ 'static` in this position, `Box<dyn Glyph>` means `Box<dyn Glyph + 'static>`. Not every trait is dyn-compatible, the current term, object safe in older text. The disqualifiers are the items a fixed per-type table cannot express:

```
trait Explain {
    fn explain<T: std::fmt::Debug>(&self, value: T);
    fn clone_of(&self) -> Self;
}
let x: Box<dyn Explain> = todo!();
// error[E0038]: the trait `Explain` is not dyn compatible,
// method `explain` has generic type parameters,
// method `clone_of` references the `Self` type in its return
```

fragment, illustrative, not executed

A generic method would need one vtable slot per instantiation, which no fixed table can hold, and returning `Self` by value would promise the caller a value of a type the caller erased. The standard fixes split the trait, keeping the object-safe core dyn-compatible and moving the generic methods to a sibling trait, or add `where Self: Sized` to the offending methods, the way `Summary::tag` did in the first listing. Dynamic dispatch versus static is chapter 9's measured comparison, here the mechanism is the point.

8.6. derived traits

Nine traits can be derived with one attribute line: `Debug`, `Clone`, `Copy`, `PartialEq`, `Eq`, `PartialOrd`, `Ord`, `Hash`, `Default`. Each generates the obvious structural implementation:

```
#[derive(Debug, Clone, PartialEq, Default)]
struct Config {
    name: String,
    retries: u32,
}

#[derive(Debug, PartialEq, Eq, PartialOrd, Ord, Clone, Copy)]
enum Level {
    Low,
    Mid,
    High,
}

#[derive(Debug, PartialEq, Default)]
enum Mode {
    Standby,
    #[default]
    Active,
}

#[derive(Debug, PartialEq, Eq, Hash)]
struct Key(u16);
```

listing 79: the nine derivable traits across a struct and two enums (source: rust/samples/src/Ch08/Derive.rs, lines 13-34)

The measurements pin the semantics. `Clone` deep copies, `Copy` changes assignment from a move to a duplication, enum ordering follows declaration order so sorting is free, and a set requires `Eq` on top of `Hash`, the marker trait is not optional:

```
        name: "gate".to_string(),
        retries: 3,
    };
    let copy = base.clone();
    ok(
        base == copy && std::ptr::eq(&base.name, &copy.name) == false,
        "derived clone makes a deep copy of the string",
    );
    let fresh = Config::default();
    ok(
        fresh == Config {
            name: String::new(),
            retries: 0,
        },
        "derived default fills the struct from each field default",
    );
    ok(
        format!("{:?}", base) == "Config { name: \"gate\", retries: 3 }",
        "derived debug prints the standard struct form",
    );
```

listing 80: clone, default, and debug checked for exact output (source: rust/samples/src/Ch08/Derive.rs, lines 42-61)

```
    ok(is_ordered(Level::Low, Level::High), "derived partialord reads variant order");
    ok(Level::Low < Level::Mid, "variant order is declaration order");
    ok(
        Level::Low.cmp(&Level::High) == std::cmp::Ordering::Less,
        "derived ord gives the full ordering",
    );
    let mut levels = vec![Level::High, Level::Low, Level::Mid];
    levels.sort();
    ok(
        levels == vec![Level::Low, Level::Mid, Level::High],
        "sorting rides the derived ordering",
    );
    ok(
        Mode::default() == Mode::Active && Mode::default() != Mode::Standby,
        "an enum default comes from the marked variant since 1.62",
    );
    let mut set = std::collections::HashSet::new();
    set.insert(Key(9));
    let second = set.insert(Key(9));
    ok(
        second == false && set.len() == 1,
        "a hash set needs eq plus hash, the marker is not optional",
    );
```

listing 81: ordering by variant, sorting, enum default since 1.62, the eq plus hash requirement (source: rust/samples/src/Ch08/Derive.rs, lines 66-88)

`PartialEq` versus `Eq` is a mathematical distinction: partial equality allows incomparable values, which only `f64` and `f64` adjacent types actually have through `NaN`. Every other type should derive both. `#[default]` on an enum variant arrived in 1.62, `Option<T>` gets `Default` as `None` without any attribute.

Derive is not magic and not exhaustive. The attribute writes the structural version of each impl, field by field for structs, variant by variant for enums, and it refuses the cases where no structural answer exists: `Clone` requires every field be `Clone`, `Default` requires every field have a `Default`, and a type holding a raw pointer derives nothing useful for `PartialEq`. Hand impls remain ordinary code when the semantics need to differ, a `PartialEq` that compares two floats by epsilon, a `Display` that hides a field:

```
impl PartialEq for Session {
    fn eq(&self, other: &Self) -> bool {
        self.user == other.user
    }
}
// token deliberately excluded, two sessions are the same session
// when they belong to the same user, not when the token matches
```

fragment, illustrative, not executed

8.7. operator overloading

Every operator in rust is a trait method in `std::ops`. `+` is `Add::add`, unary `-` is `Neg::neg`, `[]` is `Index::index`. The right hand side type is a parameter, so scalar multiplication and vector addition coexist:

```
impl Add for Vec2 {
    type Output = Vec2;
    fn add(self, rhs: Vec2) -> Vec2 {
        Vec2 {
            x: self.x + rhs.x,
            y: self.y + rhs.y,
        }
    }
}

impl Mul<f64> for Vec2 {
    type Output = Vec2;
    fn mul(self, k: f64) -> Vec2 {
        Vec2 {
            x: self.x * k,
            y: self.y * k,
        }
    }
}

impl Neg for Vec2 {
    type Output = Vec2;
    fn neg(self) -> Vec2 {
        Vec2 {
            x: -self.x,
            y: -self.y,
        }
    }
}

impl AddAssign for Vec2 {
    fn add_assign(&mut self, rhs: Vec2) {
        *self = *self + rhs;
    }
}
```

listing 82: add, mul with a foreign rhs, neg, and add_assign (source: rust/samples/src/Ch08/Operators.rs, lines 22-56)

`Display` is not an operator trait but belongs in the same breath, it is what {} binds to, while {:?} stays `Debug`:

```
impl fmt::Display for Vec2 {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

struct Grid {
    cells: [u16; 4],
}

impl Index<usize> for Grid {
    type Output = u16;
    fn index(&self, i: usize) -> &u16 {
        &self.cells[i % 4]
    }
}
```

listing 83: display impl and an index impl that defines its own bounds (source: rust/samples/src/Ch08/Operators.rs, lines 58-73)

The compile fails that follow are informative: `2.0 * a` does not compile because `impl Mul<Vec2> for f64` lives in a foreign crate and nobody wrote it, and `grid[9]` panics or not purely as the impl decides, `Index` here wraps around. There is no operator customization trap: arithmetic cannot overflow silently through overloads because the operators still go through the same checks the builtins use, and == requires `PartialEq`, it is never built in.

8.8. Any and downcasting

`dyn Any` is the std escape hatch for reflection, a trait object carrying a `TypeId`. Downcasting borrows, mutates, or consumes:

```
fn kind(boxed: &Box<dyn Any>) -> &'static str {
    if boxed.is::<u32>() {
        "u32"
    } else if boxed.is::<Cat>() {
        "cat"
    } else if boxed.is::<Dog>() {
        "dog"
    } else {
        "unknown"
    }
}

fn main() {
    let items: Vec<Box<dyn Any>> = vec![
        Box::new(7u32),
        Box::new(Cat {
            name: "mochi".to_string(),
        }),
        Box::new(Dog),
        Box::new("stray".to_string()),
    ];
    ok(kind(&items[0]) == "u32", "is::<t> tests the boxed concrete type");
    ok(kind(&items[1]) == "cat", "a local struct downcasts the same way");
    ok(kind(&items[2]) == "dog", "a unit struct rides in the same vec");
    ok(kind(&items[3]) == "unknown", "an unlisted type falls to unknown");
```

listing 84: is, downcast_ref, and a local struct found by name (source: rust/samples/src/Ch08/AnyDown.rs, lines 21-45)

```
    let mut boxed: Box<dyn Any> = Box::new(41u32);
    {
        let slot = boxed.downcast_mut::<u32>().unwrap();
        *slot += 1;
    }
    ok(
        *boxed.downcast_ref::<u32>().unwrap() == 42,
        "downcast_mut writes through the trait object since 1.3",
    );
    let recovered = match boxed.downcast::<u32>() {
        Ok(value) => *value,
        Err(_) => 0,
    };
    ok(
        recovered == 42,
        "box downcast consumes the trait object and returns the value since 1.0",
    );
    ok(
        TypeId::of::<u32>() == TypeId::of::<u32>(),
        "type ids are equal for the same type",
    );
```

listing 85: downcast_mut, consuming downcast, and type id equality (source: rust/samples/src/Ch08/AnyDown.rs, lines 60-80)

`Any` requires `'static`, a borrowed `&i32` cannot ride in a `Box<dyn Any>` because its identity is not its own, so the pattern trades exact typing for a closed set of expected types. `is`, `downcast_ref`, and `downcast_mut` all landed in 1.3, the consuming `Box::downcast` in 1.0.

VERIFY: measured, not asserted

Every count in this chapter comes from `rust/samples/src/Ch08`, 52 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1 -Chapter Ch08` on rustc 1.99.0 edition 2024. The two-word fat pointer, the vtable dispatch, the deep clone, the variant ordering, and the associated type and const reads are printouts, not documentation claims.

sources: doc.rust-lang.org/stable/book ch10.02 traits, syntax and semantics, reference/items/implementations.html for the orphan rule and coherence, reference/items/associated-items.html for associated types and consts, reference/types/trait-object.html for dyn trait objects and dyn compatibility, book/appendix-03-derivable-traits.html for the nine derivations, releases.md for 1.27 dyn, 1.62 enum default, 1.86 upcasting, 1.3 any downcasts, all read from the offline oracle at /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html, accessed 2026-10-13, cross-checked against doc.rust-lang.org/stable url forms.

9. generics and macros

Generics in rust are erased by compilation, not by runtime. Every concrete type argument becomes its own machine code copy, and the alternative, `dyn Trait` from chapter 8, stays one copy behind a vtable. This chapter measures that fork with a counter of instantiations, then covers const generics since 1.51, phantom types, and the two macro systems, `macro_rules!` in full and proc macros in prose. The comparison with book 4, ch 4 (go 1.27, generics and generic methods) is instructive: go erases at compile time with dictionaries and shared code, rust duplicates, the trade is binary size and inlining against one body.

9.1. monomorphization versus dynamic dispatch

One generic function, three integer types, one trait object path. `type_name::<T>` since 1.38 names each instantiation, and function pointer addresses prove the compiler emitted distinct code per type:

```
fn twice_static<T: Squared>(x: &T) -> u64 {
    x.squared() + x.squared()
}

fn twice_dynamic(x: &dyn Squared) -> u64 {
    x.squared() + x.squared()
}
```

listing 86: one static generic, one dynamic entry point (source: rust/samples/src/Ch09/Dispatch.rs, lines 37-43)

```
fn main() {
    let a: u8 = 9;
    let b: u16 = 9;
    let c: u32 = 9;
    ok(twice_static(&a) == 162, "the u8 instantiation computes");
    ok(twice_static(&b) == 162, "the u16 instantiation computes");
    ok(twice_static(&c) == 162, "the u32 instantiation computes");
    ok(twice_dynamic(&a) == 162, "the dyn path computes through the vtable");
    let stamp_a: fn(&u8) -> &'static str = |x| type_name_of(x);
    let stamp_b: fn(&u16) -> &'static str = |x| type_name_of(x);
    ok(
        stamp_a(&a) == "u8" && stamp_b(&b) == "u16",
        "each monomorphization stamps its own type name",
    );
    let addr8 = (twice_static::<u8> as fn(&u8) -> u64) as usize;
    let addr16 = (twice_static::<u16> as fn(&u16) -> u64) as usize;
    let addr32 = (twice_static::<u32> as fn(&u32) -> u64) as usize;
    ok(
        addr8 ≠ addr16 && addr8 ≠ addr32 && addr16 ≠ addr32,
        "three types, three distinct machine code copies",
    );
    let addrd = (twice_dynamic as fn(&dyn Squared) -> u64) as usize;
    ok(
        addrd ≠ addr8 && addrd ≠ addr16 && addrd ≠ addr32,
        "the dyn entry point is one further, separate function",
    );
```

listing 87: the measurement: type names, three addresses, one dyn address (source: rust/samples/src/Ch09/Dispatch.rs, lines 45-70)

The counter at the end of the sample walks the stamped names and counts 3 distinct values for 3 types. The dyn path pays two words per call site for the fat pointer, chapter 8's measurement, and one indirect call per method,

while the static path pays zero at runtime and one machine code copy per type in the binary. Neither is free, the cost just moves:

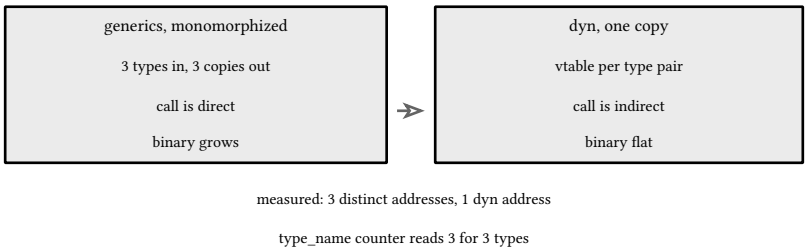


figure 18: where the dispatch cost lands

`impl Trait` in return position, since 1.26, is the same static story with the type hidden: the function returns one concrete type the caller cannot name, which is why `impl Iterator` cannot branch into two different concrete iterators at runtime. Chapter 10 uses it for adapter chains.

9.2. bounds, where, and the turbofish

Bounds travel in three places. Inline after the parameter, `<T: Squared>`, in a `where` clause below the signature, and as `impl Trait` in argument or return position. The `where` form is not style, it is the only one that scales, conditional bounds and bounds on associated types read naturally there:

```
fn blend<N>(a: Vector<f32, N>, b: Vector<f32, N>) -> Vector<f32, N>
where
    N: std::ops::Mul<usize>,
    Vector<f32, N>: Clone,
{
    a
}
// the same signature inline would drown in one line,
// and bounds on whole generic types only fit in where
```

fragment, illustrative, not executed

Multiple bounds stack with `+`, `T: Clone + Send`, and any number of bounds can pile onto one parameter. When inference cannot pick the type, the call site names it with the turbofish, `parse::<i64>()` or `collect::<Vec<u32>>()`, the `::<T>` operator saying instantiate here. Chapter 10 leans on it for `collect`, and the const generic sample used `repeat::<u16, 3>` for the same reason, two parameters, one inference hole.

Generic impls follow the same grammar: `impl<T: Copy, const N: usize> Vector<T, N>` is a block parameterized over `T` and `N`, covering every element type and length at once, while `impl<const N: usize> Vector<i64, N>` covers one element type at every length. The second form is how the sample gave the dot product to integer vectors only.

9.3. const generics since 1.51

Type parameters may be joined by constant parameters, `struct Vector<T, const N: usize>`, stable since 1.51 in the minimal form: integer, bool, and char constants. The length is part of the type, so dimension errors are compile errors:

```
struct Vector<T, const N: usize> {
    data: [T; N],
}

impl<T: Copy, const N: usize> Vector<T, N> {
    fn filled(value: T) -> Vector<T, N> {
        Vector { data: [value; N] }
    }

    fn len(&self) -> usize {
        N
    }

    fn sum(&self) -> T
    where
        T: std::ops::Add<Output = T> + Default,
    {
        let mut total = T::default();
        for i in 0..N {
            total = total + self.data[i];
        }
        total
    }
}
```

listing 88: a fixed length vector, the length a const parameter (source: rust/samples/src/Ch09/ConstGenerics.rs, lines 13-36)

The `dot` product takes another `Vector<i64, N>` with the same `N`. Hand it a different length and the types simply do not unify, no runtime check exists to fail. The sample measures the consequences: each length is its own sized type, 12 bytes for `Vector<f32, 3>`, and the type names stamp the length:

```
impl<const N: usize> Vector<i64, N> {
    fn dot(&self, other: &Vector<i64, N>) -> i64 {
        let mut total = 0i64;
        for i in 0..N {
            total += self.data[i] * other.data[i];
        }
        total
    }
}

fn repeat<T: Copy, const N: usize>(value: T) -> [T; N] {
    [value; N]
}

const fn cube<const N: usize>() -> usize {
    N * N * N
}
```

listing 89: a per-length dot product, a const generic function, a const fn using the parameter (source: rust/samples/src/Ch09/ConstGenerics.rs, lines 38-54)

```
let a: Vector<i64, 3> = Vector::filled(1);
let b: Vector<i64, 2> = Vector::filled(1);
a.dot(&b);
```

```
// error[E0308]: mismatched types, expected `Vector<i64, 3>`
// found `Vector<i64, 2>`
```

fragment, illustrative, not executed

Before 1.51 the standard library shipped arrays at every power of two length through macros, `HashMap`'s `K: Borrow<str>` world had no part in it, and crate authors wrote type level integers with nested structs. The const parameter collapsed all of that.

The stable slice is still minimal. Parameters accept only integers, `bool`, and `char`, and complex const expressions in the type position, generic `N + 1`, live on nightly. Two neighbors filled the space around it: generic associated types, GATs, landed in 1.65 so a trait can have a type constructor as an associated item, the shape lending iterators need, and `type_name` from 1.38 lets a sample print exactly which instantiation exists, the counter this chapter opened with.

9.4. phantom data and zero sized markers

A type parameter must be used or the compiler rejects the struct, E0392, because an unused parameter says nothing checkable. The escape is `PhantomData<T>`, a zero sized marker that declares how the type relates to `T` without holding one. The standard use is typestate, ids that can never be mixed:

```
struct Id<T> {
    raw: u64,
    marker: PhantomData<fn() -> T>,
}

impl<T> Id<T> {
    fn new(raw: u64) -> Id<T> {
        Id {
            raw,
            marker: PhantomData,
        }
    }

    fn raw(&self) -> u64 {
        self.raw
    }
}
```

listing 90: a phantom keyed id type for two entity kinds (source: rust/samples/src/Ch09/PhantomData.rs, lines 19-35)

`Id<User>` and `Id<Order>` are different types at zero memory cost, measured: both are exactly 8 bytes, the size of the raw `u64`. The marker also steers the auto traits, and the choice of marker matters: `PhantomData<fn() -> T>` keeps the wrapper `Send` and `Sync` for any `T` because fn pointers are, while `PhantomData<T>` would inherit `T`'s own impls and `PhantomData<*mut T>` would poison both:

```
fn name_of_user_id(id: &Id<User>) -> u64 {
    id.raw()
}

fn is_send<T: Send>() -> bool {
    true
}

fn is_sync<T: Sync>() -> bool {
    true
}

struct Weighted<T> {
    pounds: f64,
    marker: PhantomData<T>,
}
```

listing 91: a function accepting only the user id, send and sync helpers, a second marker type (source: rust/samples/src/Ch09/PhantomData.rs, lines 37-52)

The `is_send` and `is_sync` helpers compile only when the bound holds, so a passing check is a proof. `PhantomPinned`, chapter 12's subject, is the same trick aimed at `Unpin`.

9.5. macro_rules: repetition and specifiers

`macro_rules!` matches token trees and rewrites them. A rule is pattern followed by expansion, repetitions with `$(...),*` and friends, and fragment specifiers that constrain what a metavariable may match. One real macro from the samples does the rounds:

```
macro_rules! record {
    ($name:ident { $($field:ident : $ty:ty),* $(,)? }) => {
        struct $name {
            $($field: $ty,)*
        }

        impl $name {
            fn labels() -> Vec<&'static str> {
                vec![$(stringify!($field)),*]
            }
        }
    };
}

record!(Reading {
    sensor: String,
    celsius: f64,
});
```

listing 92: a struct generating macro, repetition over field name and type pairs (source: rust/samples/src/Ch09/Macros.rs, lines 13-30)

The `$(,)?` tail allows an optional trailing comma. The expansion emits one struct field per repetition and one `stringify!` per field in the labels function, measured in the output. Specifiers used here: `ident` for names, `ty` for types, `expr` for expressions, `tt` for single token trees, `item` for whole items. The rest of the family is `pat`, `path`, `literal`, `stmt`, `block`, `vis`, `lifetime`, and `meta`, and since 1.53 `pat_param` exists because `pat` itself is scheduled to change meaning across editions:

```
macro_rules! count {
    () => {
        0usize
    };
    ($head:tt $($rest:tt)*) => {
        1usize + count!($($rest)*)
    };
}

macro_rules! bump {
    ($value:expr ; $target:ty ; $name:ident) => {{
        let $name: $target = $value;
        $name + 1
    }};
}

macro_rules! double_inner {
    ($value:expr) => {{
        let shadowed = $value;
        shadowed + shadowed
    }};
}

macro_rules! grid {
    ($([$($cell:expr),*]),* $(,)?) => {
        vec![$(vec![$($cell),*]),*]
    };
}
```

listing 93: recursion over tokens, three specifiers in one rule, nested repetition (source: rust/samples/src/Ch09/Macros.rs, lines 32-59)

The `count!` macro recurses: one rule matches empty for the base case, the other peels a `tt` and adds 1. `grid!` nests two repetitions, the outer over rows, the inner over cells. Macros in type position have worked since 1.13, so `Vec<count!()>` style uses are legal, if unwise.

9.6. hygiene

Each macro expansion lives in its own syntactic context. A `let` written inside a macro body cannot collide with a variable of the same name at the call site, because the two identifiers are different symbols to the compiler even though they spell the same:

```
macro_rules! double_inner {
    ($value:expr) => {{
        let shadowed = $value;
        shadowed + shadowed
    }};
}
```

listing 94: a macro local named shadowed (source: rust/samples/src/Ch09/Macros.rs, lines 48-53)

```
    let shadowed = 100;
    ok(
        double_inner!(21) == 42,
        "a let inside the macro binds its own shadowed",
    );
    ok(
        shadowed == 100,
        "the caller's shadowed is untouched, hygiene holds",
    );
```

listing 95: the caller's shadowed survives the expansion untouched (source: rust/samples/src/Ch09/Macros.rs, lines 92-100)

The flip side is that a macro cannot accidentally reach into caller scope either: identifiers it invents stay private to the expansion. The escape hatch is passing names in, `$name:ident`, the way `bump!` takes `answer`. Item-level definitions are exempt, the `struct`
`Reading` from `record!` is visible afterwards, which is exactly why the macro could write a type the rest of the file uses.

Two more rules finish the picture. `$crate` names the crate that defined the macro, so an exported macro can call `::$crate::helper` and resolve correctly from any caller, the reason std macros work from every module. And fragment specifiers have a follow set: after `$e:expr` the compiler permits only certain tokens, `⇒`, `,`, `;`, before it can decide where the expression ends, which is why `$e + 1` is a compile error in a rule, the `+` is not in `expr`'s follow set. The reference table lists them per specifier, and the practical takeaway is to end each metavariable with a clear separator rather than splice it into an expression.

9.7. proc macros in brief

`macro_rules!` rewrites token trees on syntactic shape alone. Proc macros are rust functions that receive a `TokenStream` and return one, compiled as a separate crate and run by the compiler. Four kinds exist: function-like (`sqlx::query!`), attribute (`#[tokio::main]`), derive (`#[derive(Serialize)]`), and since the derive kind stabilized in 1.15 as macros 1.1 it is the everyday one. No proc macro runs in this book's samples, the runner compiles single files with plain rustc, but what a derive generates is worth naming once. For chapter 8's `Config`:

```
#[derive(Debug)]
struct Config { name: String, retries: u32 }
// expands to the equivalent of:
impl std::fmt::Debug for Config {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        f.debug_struct("Config")
            .field("name", &self.name)
            .field("retries", &self.retries)
            .finish()
    }
}
```

fragment, illustrative, not executed

That is why `format!("{:?}", base)` printed `Config { name: "gate", retries: 3 }` in chapter 8: the derive wrote a method per field, calling the same builder the hand impl would.

VERIFY: the counter is the chapter

The instantiation count, the distinct code addresses, the zero byte markers, and the 12 byte vector all come from `rust/samples/src/Ch09`, 38 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1 -Chapter Ch09` on rustc 1.99.0 edition 2024, measured 2026-10-13.

sources: doc.rust-lang.org/stable/book ch10.01 generic data types for monomorphization, reference/items/generics.html for type parameters, const parameters, bounds, and where clauses, book ch19 macros and the macro_rules reference chapter macros-by-example.html, reference/macros.html for fragment specifiers, follow sets, and hygiene, releases.md for 1.13 macros in type position, 1.15 custom derive, 1.26 impl trait, 1.38 type_name, 1.51 const generics and addr_of, 1.53 pat_param, 1.65 gats, all read from the offline oracle at /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html, accessed 2026-10-13, cross-checked against doc.rust-lang.org/stable url forms.

10. closures and iterators

A closure is a function that captures its environment, and the three closure traits are three capture contracts stacked: `Fn` reads, `FnMut` reads and writes, `FnOnce` may consume. An iterator is the consumer side of the same design, one `next` method at a time, adapters wrapping adapters until a driver pulls. Nothing in the middle runs. This chapter measures both claims with counters.

10.1. the three kinds

Every closure the compiler assigns the weakest kind its body permits. A capture free closure is `Fn`, a closure that mutates its captures is `FnMut`, one that moves a capture out of itself is `FnOnce`, and the hierarchy runs `Fn` into `FnMut` into `FnOnce`, so any bound accepts weaker closures:

```
fn run_fn(f: impl Fn(i32) -> i32, x: i32) -> i32 {
    f(x)
}

fn run_fnmut(mut f: impl FnMut(i32) -> i32, x: i32) -> i32 {
    f(x)
}

fn run_fnonce(f: impl FnOnce() -> String) -> String {
    f()
}

fn make_adder(k: i32) -> impl Fn(i32) -> i32 {
    move |x| x + k
}
```

listing 96: three takers and a closure returning factory (source: rust/samples/src/Ch10/Kinds.rs, lines 13-27)

```
fn main() {
    let plain = |x: i32| x * 2;
    ok(run_fn(plain, 21) == 42, "a capture free closure is fn");
    ok(
        run_fnmut(plain, 5) == 10,
        "an fn closure satisfies any fnmut bound",
    );
    let mut count = 0;
    let mut tally = |x: i32| {
        count += x;
        count
    };
    ok(run_fnmut(&mut tally, 10) == 10, "a mutating closure is fnmut");
    tally(5);
    tally(1);
    ok(count == 16, "the fnmut closure kept its captured state across calls");
    let owned = String::from("payload");
    let consume = move || owned;
    let taken = run_fnonce(consume);
    ok(
        taken == "payload",
        "a closure that moves its capture out is fnonce",
    );
    ok(
        run_fnonce(|| String::from("fresh")) == "fresh",
        "an fnonce bound accepts any closure kind",
    );
    let add7 = make_adder(7);
    ok(add7(35) == 42, "a returned closure captures k by move");
    ok(add7(0) == 7, "the returned closure is reusable, so it is fn");
    let base = 100;
    let borrow = |x| x + base;
    ok(borrow(1) == 101, "a borrowed capture works without move");
```

listing 97: each kind demonstrated, and the hierarchy in action (source: rust/samples/src/Ch10/Kinds.rs, lines 29-61)

The `tally` closure mutates `count` on every call and the captured state persists across calls, 10 then 15 then 16. `consume` moves `owned` out, so it can only answer once. `make_adder` returns `impl Fn(i32) -> i32`, the return position opaque type from chapter 9, and the returned closure must be reusable, so it has to be `Fn`. A closure type is a real anonymous struct generated per closure site, capturing exactly what the body reads, which is why closures implement `Copy` and `Clone` when all captures do, stable since 1.26 and measured in the next section.

10.2. capture modes, measured

The sample counts borrows with a `Cell` counter and drops with a static, and the numbers pin the rules down:

```
struct Pair {
    reads: Cell<usize>,
    left: u32,
    right: u32,
}

impl Pair {
    fn read_left(&self) -> u32 {
        self.reads.set(self.reads.get() + 1);
        self.left
    }

    fn read_right(&self) -> u32 {
        self.reads.set(self.reads.get() + 1);
        self.right
    }
}

struct Loud;

impl Drop for Loud {
    fn drop(&mut self) {
        DROPS.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
    }
}
```

listing 98: a pair with a read counter and a loud drop counter (source: rust/samples/src/Ch10/Capture.rs, lines 16-41)

```
fn main() {
    let mut p = Pair {
        reads: Cell::new(0),
        left: 3,
        right: 4,
    };
    {
        let both = || p.read_left() + p.read_right();
        ok(p.reads.get() == 0, "creating the closure borrowed nothing yet");
        both();
        both();
        ok(
            p.reads.get() == 4,
            "two calls, one read per method each, the counter says four",
        );
    }
    {
        let only_left = || p.left;
        p.right = 40;
        ok(
            only_left() == 3 && p.right == 40,
            "edition 2021 disjoint capture leaves right mutable beside the closure",
        );
        ok(
            p.reads.get() == 4,
            "the field reading closure never touched the method counter",
        );
    }
    p.left = 5;
    ok(p.left == 5, "the field borrow ends with the closure's scope");
```

listing 99: borrow counts, then edition 2021 disjoint capture (source: rust/samples/src/Ch10/Capture.rs, lines 42-71)

Creating a closure performs no work, the read counter sits at zero until the first call. Two calls cost four reads, one per method per call. The disjoint capture block is the edition 2021 change, RFC 2229: `|| p.left` captures only the `left` field, so `p.right` stays mutable while the closure is alive, where edition 2018 captured the whole struct and forbade it. And because `both` captures only shared references, it is `Copy`, the drop of the binding is a no-op, which the compiler itself insists on:

```
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 0,
        "nothing dropped yet",
    );
    {
        let loud = Loud;
        let holder = || {
            let _ = &loud;
        };
        holder();
        ok(
            DROPS.load(std::sync::atomic::Ordering::SeqCst) == 0,
            "a shared capture keeps the value alive beside the closure",
        );
    }
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 1,
        "scope end drops the borrowed value once",
    );
    let loud2 = Loud;
    let eater = move || {
        drop(loud2);
    };
    eater();
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 2,
        "a move closure owns the value, one drop when it runs",
    );
    let p2 = Pair {
        reads: Cell::new(0),
        left: 7,
        right: 8,
    };
    let r = &p2;
    let owned_reader = move || r.read_left() + r.read_right();
    owned_reader();
    owned_reader();
    ok(
        p2.reads.get() == 4,
        "a move closure can carry a borrowed counter, four reads in two calls",
```

listing 100: drop counts for shared and owned captures, and a move closure carrying a reference (source: rust/samples/src/Ch10/Capture.rs, lines 72-111)

A shared capture leaves the value alive at scope end, one drop. A `move` closure owns it, the drop lands inside the call. `move` is a demand, not a kind: moving a `Copy` reference in still leaves the closure `Fn`, as `owned_reader` shows, carrying a borrowed counter to four reads across two calls.

NOTE: when move is needed

`move` matters at function boundaries: a closure returned from a function, or sent to a thread in chapter 13, must own its captures because the borrows end when the enclosing scope does. Chapter 5 covered the move semantics underneath, this is the same rule at the closure boundary.

10.3. adapters and laziness

An adapter wraps an iterator and returns a new iterator, and none of them evaluates anything. The sample proves it with a `next` counter:

```
struct Counting {
    remaining: u32,
}

impl Iterator for Counting {
    type Item = u32;

    fn next(&mut self) -> Option<u32> {
        NEXTS.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
        if self.remaining == 0 {
            None
        } else {
            self.remaining -= 1;
            Some(self.remaining)
        }
    }
}

fn nexts() -> usize {
    NEXTS.load(std::sync::atomic::Ordering::SeqCst)
}
```

listing 101: an iterator that counts every next call (source: rust/samples/src/Ch10/Laziness.rs, lines 14-34)

```
fn main() {
    let chain = Counting { remaining: 5 }
        .map(|x| x * 10)
        .filter(|x| *x > 10)
        .take(2);
    ok(
        nexts() == 0,
        "building a three adapter chain ran zero next calls",
    );
    let collected: Vec<u32> = chain.collect();
    ok(
        collected == vec![40, 30],
        "collect drove the chain to exactly two items",
    );
    ok(
        nexts() == 2,
        "take stopped pulling after two, the counter read two nexts",
    );
```

listing 102: chain built, zero nexts, collect, two nexts (source: rust/samples/src/Ch10/Laziness.rs, lines 36-53)

The chain reads zero after construction, two after `collect`. `take` pulled two passing items and stopped, so the underlying `Counting` was never drained, and `filter` never buffered anything, each pull walks until one item passes. The `for` loop measurement shows the same control from the caller side, `break` after two pulls leaves the iterator mid-stream, and `sum` pays one extra `next` for the closing `None`:



figure 19: an adapter chain, pulled from the right

Building the chain right to left costs nothing, every arrow is a `next` call that only happens when the driver pulls. The same structure is why iterators compose in `impl Trait` returns, each adapter is a concrete type the compiler names for you.

10.4. writing an iterator

Everything above composes because the contract is one method. Any type with a `next` returning `Option<Item>` is an iterator, and inherits the entire adapter surface, dozens of methods, for free. The laziness sample's `Counting` is the whole story in ten lines:

```
struct Counting {
    remaining: u32,
}

impl Iterator for Counting {
    type Item = u32;

    fn next(&mut self) -> Option<u32> {
        NEXTS.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
        if self.remaining == 0 {
            None
        } else {
            self.remaining -= 1;
            Some(self.remaining)
        }
    }
}
```

listing 103: the entire iterator contract: one next method (source: rust/samples/src/Ch10/Laziness.rs, lines 14-31)

`next` advances and reports. `None` means finished forever, the caller is allowed to stop calling, and returning `Some` after a `None` is a contract violation the adapters cannot defend against, so honest iterators keep their own end state, the `remaining == 0` branch above. The measurement harness, the static counter and the `nexts` reader, is exactly how this chapter's laziness claims were pinned.

Two optional traits refine the contract. `DoubleEndedIterator` adds `next_back`, letting `rev` and `rfind` work, a vec iterator has it, a stream does not. `ExactSizeIterator` promises `len` is exact, which unlocks index math in adapters. Both are plain traits, a type implements them when it honestly can, and the adapters degrade gracefully when it cannot.

Returning an iterator from a function has two spellings. `impl Iterator<Item = u32>` returns one unnamed concrete type, zero cost, but every path in the body must agree on that type. `Box<dyn Iterator<Item = u32>>` boxes the trait object of chapter 8, one indirect call per item, but the body may return different concrete iterators per branch:

```
fn numbers(flag: bool) -> Box<dyn Iterator<Item = i32>> {
    if flag {
        Box::new(0..3)
    } else {
        Box::new(vec![7, 8].into_iter())
    }
}
// two different concrete types, one signature, dyn is the only way
```

fragment, illustrative, not executed

10.5. collect and the turbofish

`collect` drives the chain into any container that implements `FromIterator`, and the target type is inferred from the binding, or named inline with the turbofish `::<T>`:

```
fn main() {
    let doubled: Vec<i32> = [1, 2, 3].iter().map(|x| x * 2).collect();
    ok(
        doubled == vec![2, 4, 6],
        "collect builds a vec from a mapped iterator",
    );
    let via_turbofish = [4, 5, 6].iter().map(|x| x * 2).collect::<Vec<i32>>();
    ok(
        via_turbofish == vec![8, 10, 12],
        "the turbofish names the target type inline",
    );
    let text: String = "abc".chars().map(|c| c.to_ascii_uppercase()).collect();
    ok(text == "ABC", "chars collect back into a string");
    let unique: HashSet<i32> = [1, 2, 2, 3, 3, 3].iter().copied().collect();
    ok(unique.len() == 3, "a set collects and dedupes in one step");
    let keys = ["a", "b", "c"];
    let values = [1, 2, 3];
    let map: BTreeMap<&str, i32> = keys.iter().copied().zip(values).collect();
    ok(
        map.get("b") == Some(&2) && map.len() == 3,
        "zip pairs collect into a map",
    );
    let pairs: BTreeMap<u8, u8> =
        (1..=3).map(|i| (i, i * i)).collect();
    ok(
        pairs.values().sum::<u8>() == 14,
        "enumerate free construction, 1 plus 4 plus 9",
    );
```

listing 104: vec, string, set, and map collection with inference and turbofish (source: rust/samples/src/Ch10/Collect.rs, lines 15-42)

```
    let total: i32 = doubled.iter().sum();
    ok(total == 12, "sum folds the whole chain");
    let product: i32 = doubled.iter().product();
    ok(product == 48, "product folds with multiplication");
    let biggest = doubled.iter().max();
    ok(*biggest.unwrap() == 6, "max scans without collecting");
    let folded = doubled.iter().fold(0, |acc, x| acc + x * 10);
    ok(folded == 120, "fold is the general shape sum specializes");
    let (evens, odds): (Vec<i32>, Vec<i32>) =
        (0..8).partition(|x| x % 2 == 0);
    ok(
        evens == vec![0, 2, 4, 6] && odds == vec![1, 3, 5, 7],
        "partition collects two outputs from one pass",
    );
    let chained: Vec<i32> = [1, 2].iter().copied().chain([3, 4]).collect();
    ok(chained == vec![1, 2, 3, 4], "chain concatenates two iterators");
    let positions: Vec<usize> = "abracadabra"
        .match_indices('a')
        .map(|(i, _)| i)
        .collect();
    ok(
        positions == vec![0, 3, 5, 7, 10],
        "str iterators collect with the same machinery",
    );
    println!("measured 2026-10-13 on rustc 1.99.0 edition 2024");
```

listing 105: sum, product, fold, partition, chain, and a str iterator (source: rust/samples/src/Ch10/Collect.rs, lines 43-67)

`String` implements `FromIterator<char>`, maps and sets collect from zips and mapped pairs, `partition` splits one pass into two vecs, and string iterators ride the same machinery. The folding family, `sum`, `product`, `fold`, are also drivers, they just return a scalar instead of a container.

10.6. iter, iter_mut, into_iter

Three entry methods cover the three borrow modes from chapter 5 and 6. `iter` lends `&T`, `iter_mut` lends `&mut T`, `into_iter` moves the `T` out:

```
fn main() {
    let source = vec![1, 2, 3];
    let shared: Vec<i32> = source.iter().map(|x| *x).collect();
    ok(
        shared == vec![1, 2, 3] && source.len() == 3,
        "iter borrows, the vec survives the walk",
    );
    ok(is_ref_slice(&source) == 3, "the vec is still addressable afterwards");
    let mut numbers = vec![1, 2, 3];
    for slot in numbers.iter_mut() {
        *slot *= 10;
    }
    ok(
        numbers == vec![10, 20, 30],
        "iter_mut hands out exclusive references, in place doubling",
    );
    let moved: Vec<i32> = numbers.into_iter().map(|x| x + 1).collect();
    ok(
        moved == vec![11, 21, 31],
        "into_iter moves each element out, the vec is gone",
    );
    let arr = [5, 6, 7];
    let by_value: Vec<i32> = arr.into_iter().collect();
    ok(
        by_value == vec![5, 6, 7] && arr.len() == 3,
        "array into_iter yields by value since 1.53, the array still copies",
    );
```

listing 106: borrow, mutate in place, then move out (source: rust/samples/src/Ch10/IterKinds.rs, lines 17-43)

Arrays grew `IntoIterator` by value in 1.53, so `[5, 6, 7].into_iter()` yields `i32` items, while the array itself remains usable because arrays are `Copy` and the iteration copies elementwise. The `for` loop itself desugars to `into_iter`, so `for x in &v` borrows and `for x in v` consumes. `by_ref` lets one iterator be consumed in pieces:

```
    let mut walk = (0..6).into_iter();
    let head: Vec<i32> = walk.by_ref().take(2).collect();
    let tail: Vec<i32> = walk.collect();
    ok(
        head == vec![0, 1] && tail == vec![2, 3, 4, 5],
        "by_ref consumes a prefix, the same iterator finishes the rest",
    );
    let found = (0..100).find(|x| *x * *x * *x ≥ 500);
    ok(
        found == Some(8),
        "find stops at the first hit, 8 cubed passes 500",
    );
    let spot = vec![10, 20, 30].iter().position(|x| *x == 20);
    ok(spot == Some(1), "position returns the index of the match");
    let count = "mississippi".matches('s').count();
    ok(count == 4, "count consumes the iterator for one number");
    let total: i32 = (1..=4).rev().sum();
    ok(total == 10, "rev flips the walk direction, same total");
    let mut few = vec![9, 1, 4];
    few.sort_unstable();
    let least = few.iter().min();
    ok(
        *least.unwrap() == 1 && few == vec![1, 4, 9],
        "min and max read the walk without collecting",
    );
```

listing 107: by_ref prefix consumption, then find, position, count (source: rust/samples/src/Ch10/IterKinds.rs, lines 44-68)

10.7. iterator invalidation

The one mistake the borrow checker catches for you, mutating the collection while iterating it:

```
let mut v = vec![1, 2, 3];
for x in &v {
    v.push(*x);
}
// error[E0502]: cannot borrow `v` as mutable because it is also
// borrowed as immutable, the iterator holds `&v` across the loop
```

fragment, illustrative, not executed

The iterator owns a borrow of the whole vec, `push` needs `&mut`, and the two cannot overlap. The escape is collecting first, `let taken: Vec<_> = v.iter().copied().collect();`, then mutating, or `iter_mut` when the mutation is in place. Languages with iterators over live references solve this at runtime with invalidation checks and crashes, rust solved it at the type level.

Closures slot into the same picture as values. A closure type is an anonymous struct, so it can sit in a struct field, travel through a vec, or arrive as a function argument with its captures along:

```
struct Pipeline {
    transform: Box<dyn Fn(u32) -> u32>,
}
let bias = 100;
let p = Pipeline { transform: Box::new(move |x| x + bias) };
```

```
// the closure owns its copy of bias, the struct owns the closure,
// dyn because the field must accept any closure with this shape
```

fragment, illustrative, not executed

The `move` matters here: without it the closure borrows `bias`, and the borrow cannot outlive the function that built the struct. This is the callback shape the service chapters build on, a stored closure with captured configuration, and it is why `Box<dyn Fn>` is the standard spelling for stored callbacks rather than generics, the field needs one type, not one type per closure.

sources: doc.rust-lang.org/stable/book ch13.01 closures and the three fn traits, ch13.02 processing iterators, reference/types/closure.html for capture precision including wildcard and disjoint field capture, book ch04.05 or the slices chapter for iter versus into_iter context, releases.md for 1.26 copy closures, 1.53 array into_iterator, and the edition 2021 capture changes, all read from the offline oracle at /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html, accessed 2026-10-13, cross-checked against doc.rust-lang.org/stable url forms. Every count in this chapter is asserted by `rust/samples/src/Ch10`, 51 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1 -Chapter Ch10` on rustc 1.99.0 edition 2024, measured 2026-10-13.

11. error handling

Rust has exactly two failure channels. A panic aborts the current task through unwinding, for bugs and broken invariants. A `Result` value reports an expected failure through the type system, for everything a caller might want to handle. This chapter covers both, the `?` operator that chains results, hand rolled error types, and the unwinding machinery itself, `catch_unwind` since 1.9 and the abort profile switch. `Option` and `Result` were introduced in chapter 7 as enums, here they get their full treatment.

11.1. panic versus result

The rule of thumb is simple: panic when the caller cannot possibly do anything sensible, return a `Result` when they can. The sample implements the same division twice, one panicking, one reporting:

```
fn strict_div(a: i32, b: i32) -> i32 {
    assert!(b != 0, "divider must not be zero");
    a / b
}

fn safe_div(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err(String::from("divider must not be zero"))
    } else {
        Ok(a / b)
    }
}

fn nested(a: i32, b: i32) -> i32 {
    strict_div(a, b) + 1
}
```

listing 108: the same failure as a panic and as a value (source: rust/samples/src/Ch11/PanicVsResult.rs, lines 16-31)

```
fn main() {
    let reached = Cell::new(false);
    let outcome = catch_unwind(AssertUnwindSafe(|| {
        nested(10, 0);
        reached.set(true);
    }));
    ok(outcome.is_err(), "the assert panicked out of two frames");
    ok(
        reached.get() == false,
        "unwinding skipped the statement after the panic point",
    );
    let caught = outcome.unwrap_err();
    ok(
        caught.downcast_ref::<&str>().map(|s| s.contains("zero")) == Some(true),
        "the payload carries the assert message as a str",
    );
    ok(safe_div(10, 2) == Ok(5), "the ok variant returns the value");
    ok(
        safe_div(10, 0) == Err(String::from("divider must not be zero")),
        "the err variant reports without unwinding",
    );
    ok(
        safe_div(10, 0).is_err() && catch_unwind(|| safe_div(10, 0)).is_ok(),
        "a returned err is an ordinary value, no panic involved",
    );
```

listing 109: unwinding measured: a flag that never gets set, and a payload read (source: rust/samples/src/Ch11/PanicVsResult.rs, lines 33-57)

`catch_unwind` turns the panic into a `Result` whose error side is the payload, and the `reached` flag proves the statement after the panic point never ran, unwinding really unwound. The `Err` from `safe_div` is just a value, catching it costs nothing, no thread dies, no stack unwinds. The combinators, `map`, `map_err`, `unwrap_or`, adapt each side without a match:

```
    let chain = safe_div(9, 0).map_err(|e| e.len());
    ok(
        chain == Err(24),
        "map_err adapts the error without touching the ok side",
    );
    let fallback = safe_div(9, 0).unwrap_or(0);
    ok(fallback == 0, "unwrap_or converts either side to a value");
    ok(
        safe_div(9, 3).map(|q| q * 2) == Ok(6),
        "map adapts the ok side without touching the error",
    );
    println!("measured 2026-10-13 on rustc 1.99.0 edition 2024");
```

listing 110: map_err, unwrap_or, and map on the two sides (source: rust/samples/src/Ch11/PanicVsResult.rs, lines 58-69)

The library version of the rule is stricter than the application version. A library panics only on contract violations its caller cannot express: the wrong kind of value in a type that promised otherwise, an index the caller already checked. Anything arriving from outside, files, sockets, user input, env vars, comes back as a `Result`, because the caller of a library is the one who knows whether the failure is fatal. The standard library follows this line exactly, `Vec::index` panics on out of range because the caller's own logic chose the index, while `HashMap::get` returns `Option` because absence is an ordinary outcome. A panic in library code is a bug report with your crate's name on it.

There is also the explicit middle form, `#[should_panic]` territory in chapter 18 aside: the `assert!` family is how invariants are declared, and each assert is a panic site with a message. `debug_assert!` compiles away in release, so

performance-critical checks can stay in the test and debug builds. None of this is exception machinery, an assert is a branch plus a panic call, and the branch is visible in the codegen.

11.2. the question mark operator

? is early return for results, since 1.13. On Ok it unwraps and continues, on Err it converts through From and returns. It works on Option the same way with None:

```
fn parse_pair(a: &str, b: &str) -> Result<(i64, i64), ParseIntError> {
    let x = a.parse::<i64>()?;
    let y = b.parse::<i64>()?;
    Ok((x, y))
}

fn parse_all(fields: &[&str]) -> Result<Vec<i64>, ParseIntError> {
    fields.iter().map(|f| f.parse::<i64>()).collect()
}

fn pick_keyword(words: &[&str]) -> Option<String> {
    let first = words.first()?;
    let stripped = first.strip_prefix("k")?;
    Some(stripped.to_ascii_uppercase())
}

fn boxed_total(fields: &[&str]) -> Result<i64, Box<dyn Error>> {
    let values = parse_all(fields)?;
    Ok(values.iter().sum())
}
```

listing 111: two parses through one signature, collect short circuiting, option, boxed errors (source: rust/samples/src/Ch11/Question.rs, lines 16-35)

parse_all collects into Result<Vec<i64>, ParseIntError>, the FromIterator impl for Result stops at the first error, an iterator level ? for free. boxed_total returns Result<i64, Box<dyn Error>> and uses ? on a ParseIntError directly, the From conversion wraps it into the box. main itself can return a Result, since 1.26, printing the error through Debug and exiting nonzero when it lands:

```
fn main() -> Result<(), Box<dyn Error>> {
    ok(
        parse_pair("3", "4") == Ok((3, 4)),
        "two question marks thread two parses through one signature",
    );
    ok(
        parse_pair("3", "x").is_err(),
        "the second question mark returned early on the bad field",
    );
    ok(
        parse_all(&["1", "2", "3"]) == Ok(vec![1, 2, 3]),
        "collect over results assembles an ok vec",
    );
    ok(
        parse_all(&["1", "x", "3"]).is_err(),
        "collect over results stops at the first err",
    );
    ok(
        pick_keyword(&["kw", "other"]) == Some(String::from("W")),
        "the question mark works on option the same way",
    );
    ok(
        pick_keyword(&["plain"]).is_none(),
        "a strip that fails becomes none through the question mark",
    );
    ok(
        boxed_total(&["10", "20", "12"]).ok() == Some(42),
        "a boxed error return absorbs parse errors via from",
    );
    let mixed = boxed_total(&["10", "xx"]);
    ok(
        mixed.is_err(),
        "the same boxed signature reports the parse failure",
    );
    let labeled = boxed_total(&["7"]).map(|v| format!("total {v}"));
    ok(
        labeled.ok() == Some(String::from("total 7")),
        "main returning result composes with ordinary combinators",
```

listing 112: main returning result, the termination impl (source: rust/samples/src/Ch11/Question.rs, lines 37-74)

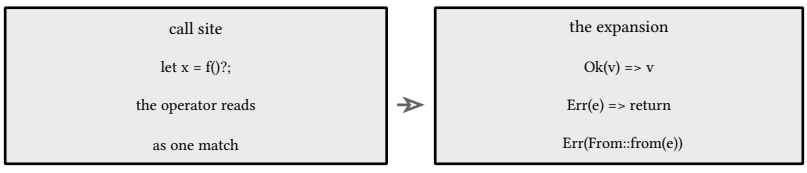


figure 20: what one question mark expands to

11.3. combinators and the conversion traits

Between `?` and `match` sits the combinator layer, one method per way a branch might continue. `map` and `map_err` transform one side, `and_then` chains a fallible closure onto an ok, `or_else` supplies an alternative error path, `unwrap_or` and `unwrap_or_else` collapse to a value, and `ok_or` promotes an `Option` into a `Result`:

```
let port = parse_port(raw).or_else(|_| fallback_port());
let path = env::var_os("HOME").ok_or(HomeMissing);
let n: i64 = text.trim().parse().unwrap_or_else(|e| {
    log::warn!("bad number {e}, using zero");
    0
});
// each combinator returns the same enum it consumed, so they stack
```

fragment, illustrative, not executed

unwrap and expect are the panic side of the api: both extract or panic, expect stamps the message with your string and the file and line of the call. The rule from the first section still applies, they are for invariants you can prove, not for input you cannot.

The conversion traits complete the bridge. From is total, u8 into u16 always succeeds and Into rides it, TryFrom since 1.34 is the fallible twin that returns Result, and TryInto is its consumed mirror. Numeric narrowing is the canonical case, u8::try_from(300u32) is an Err, where the as cast would have silently wrapped:

```
    let narrow = u8::try_from(300u32);
    ok(
        narrow.is_err(),
        "try_from since 1.34 reports lossy conversions as results",
    );
    let wide: u16 = 41u8.try_into().unwrap();
    ok(
        wide == 41,
        "try_into since 1.34 is the mirror direction on the value",
    );
    let chained = u8::try_from(100u32).map(|v| v * 2);
    ok(
        chained == Ok(200u8),
        "try_from composes with map like any result",
    );
```

listing 113: try_from rejecting a lossy narrowing, try_into on the value (source: rust/samples/src/Ch11/Question.rs, lines 76-90)

Every fallible conversion in std that can lose data returns a Result, never a panic and never a silent wrap, str::parse, i64::try_from, Vec::try_reserve. The Try trait that generalizes ? beyond Result is still nightly, the operator works on Option and Result on stable.

11.4. a custom error enum

Library grade error handling is one enum per fallible area, a hand written Display, an Error impl exposing source since 1.30, and From impls so ? converts into it. The trait itself is 1.0:

```
#[derive(Debug, PartialEq)]
enum ConfigError {
    Missing(String),
    Malformed { key: String, line: usize },
    Parse(ParseIntError),
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            ConfigError::Missing(key) => write!(f, "missing key {key}"),
            ConfigError::Malformed { key, line } => {
                write!(f, "malformed value for {key} at line {line}")
            }
            ConfigError::Parse(err) => write!(f, "bad number: {err}"),
        }
    }
}

impl Error for ConfigError {
    fn source(&self) -> Option<&(dyn Error + 'static)> {
        match self {
            ConfigError::Parse(err) => Some(err),
            _ => None,
        }
    }
}

impl From<ParseIntError> for ConfigError {
    fn from(err: ParseIntError) -> ConfigError {
        ConfigError::Parse(err)
    }
}
```

listing 114: the enum, display, error with source, and a from impl (source: rust/samples/src/Ch11/CustomError.rs, lines 17-49)

Two loader flavors use it. `setting` maps errors explicitly with `map_err`, `translated` leans on `?` plus the `From` impl, and both compile against the same signature. The source chain is the payoff, a caller walking `source()` gets the parse error underneath the config error without knowing its type:

```
fn setting(fields: &[(&str, &str)], key: &str, line: usize) -> Result<i64, ConfigError> {
    for (k, v) in fields {
        if *k == key {
            return v.parse::<i64>().map_err(|_| ConfigError::Malformed {
                key: key.to_string(),
                line,
            });
        }
    }
    Err(ConfigError::Missing(key.to_string()))
}

fn translated(fields: &[(&str, &str)], key: &str) -> Result<i64, ConfigError> {
    let raw = fields
        .iter()
        .find(|(k, _)| *k == key)
        .map(|(_, v)| *v)
        .ok_or_else(|| ConfigError::Missing(key.to_string()))?;
    Ok(raw.parse::<i64>()?)
}
```

listing 115: explicit map_err against question mark conversion (source: rust/samples/src/Ch11/CustomError.rs, lines 51-70)

```
    ok(
        translated(&fields, "bad").is_err(),
        "the from impl converted the parse error through the question mark",
    );
    let parsed_err = translated(&fields, "bad").unwrap_err();
    ok(
        parsed_err.source().is_some(),
        "the parse variant exposes its cause through source since 1.30",
    );
    ok(
        missing.source().is_none(),
        "the missing variant has no cause",
    );
    let boxed: Box<dyn Error> = Box::new(parsed_err);
    let mut chain = 0;
    let mut cursor: Option<&dyn Error> = Some(boxed.as_ref());
    while let Some(err) = cursor {
        chain += 1;
        cursor = err.source();
    }
    ok(chain == 2, "walking source visits the error and its cause");
    ok(
        format!("{}", boxed).starts_with("bad number"),
        "the boxed trait object prints through the same display",
    );
```

listing 116: walking the source chain, and a boxed trait object printing through display (source: rust/samples/src/Ch11/CustomError.rs, lines 92-116)

The ecosystem's error crates automate exactly this shape, `thiserror` derives `Display` and `From` from attributes, `anyhow` replaces the enum with a boxed trait object plus context. Both are sugar over what the sample wrote by hand, no magic underneath.

The `Error` trait itself is deliberately small. `description` is deprecated since the `Display` impl made it redundant, `cause` is the pre-1.30 spelling of `source`, and `provide`, the request-response hook for downcasting payloads without `Any`,

is nightly. What remains is `source` plus the blanket machinery: `Box<dyn Error>` is `Any` underneath, so a boxed error downcasts back to the concrete enum, the round trip a library boundary preserves for callers that need the variant:

```
    let boxed: Box<dyn Error> = Box::new(parsed_err);
    let mut chain = 0;
    let mut cursor: Option<&dyn Error> = Some(boxed.as_ref());
    while let Some(err) = cursor {
        chain += 1;
        cursor = err.source();
    }
    ok(chain == 2, "walking source visits the error and its cause");
    ok(
        format!("{}", boxed).starts_with("bad number"),
        "the boxed trait object prints through the same display",
    );
    ok(
        boxed.downcast_ref::<ConfigError>().is_some(),
        "a boxed dyn error downcasts back to the concrete enum",
    );
    let mut paths = Vec::new();
```

listing 117: the box, the source walk, and the downcast back to the enum (source: rust/samples/src/Ch11/CustomError.rs, lines 105-121)

Choosing where each shape lives follows from ownership. Inside a module, the enum, every variant is a case the code can branch on. Across a library boundary, the trait object in the signature, the caller can print, chain, or downcast but cannot exhaustively match. Across a process boundary, the serialized error, text with a code, because the trait object dies with the process. The service chapters of this book run all three layers.

11.5. catch_unwind and the panic machinery

`catch_unwind`, stable since 1.9 under that name, runs a closure and catches any panic that escapes it, returning the payload as `Box<dyn Any + Send>`:

```
fn main() {
    let outer = catch_unwind(|| {
        panic!("literal payload");
    });
    ok(outer.is_err(), "catch_unwind turns a panic into a result since 1.9");
    let outer_payload = outer.unwrap_err();
    let as_str = outer_payload.downcast_ref::<&str>();
    ok(
        as_str == Some(&"literal payload"),
        "a literal panic payload downcasts to str",
    );
    let inner = catch_unwind(|| {
        let runtime = 7;
        panic!("code {}", runtime)
    });
    let inner_payload = inner.unwrap_err();
    let as_string = inner_payload.downcast_ref::<String>();
    ok(
        as_string.map(|s| s.as_str()) == Some("code 7"),
        "a runtime formatted payload downcasts to string",
    );
    ok(
        inner_payload.downcast_ref::<&str>().is_none(),
        "a string payload does not downcast to str",
    );
    let const_form = catch_unwind(|| panic!("code {}", 7)).unwrap_err();
    ok(
        const_form.downcast_ref::<&str>() == Some(&"code 7"),
        "an all constant format downcasts to str, rendered at compile time",
    );
```

listing 118: payload downcasting to str and string, and a counting hook (source: rust/samples/src/Ch11/CatchUnwind.rs, lines 16-45)

The payload rule measured here: a literal message downcasts to `&str`, a message formatted from runtime values downcasts to `String`, and since the constant folding landed, a message whose format arguments are all compile time constants downcasts to `&str` again because the message is rendered at compile time. The hook mechanism, `set_hook` and `take_hook`, observes every panic before it unwinds, which is how test frameworks and loggers capture messages:

```
    drop(take_hook());
    std::panic::set_hook(Box::new(|_| {}));
    let rethrown = catch_unwind(|| {
        let caught = catch_unwind(|| panic!("rethrow me"));
        match caught {
            Ok(_) => unreachable!(),
            Err(payload) => resume_unwind(payload),
        }
    });
    ok(
        rethrown
            .unwrap_err()
            .downcast_ref::<&str>()
            .map(|s| *s)
            == Some("rethrow me"),
        "resume_unwind rethrows the original payload without a new panic",
    );
    let mut shared = 0u32;
    let guarded = catch_unwind(AssertUnwindSafe(|| {
        shared += 1;
        panic!("inside guarded");
    }));
    ok(
        guarded.is_err() && shared == 1,
        "assert_unwind_safe opts a mutable capture into catching",
    );
    let count = catch_unwind(|| 41).unwrap();
    ok(count == 41, "a completing closure returns ok with its value");
```

listing 119: resume_unwind preserving the payload, and assert_unwind_safe around a mutable capture (source: rust/samples/src/Ch11/CatchUnwind.rs, lines 56-83)

`resume_unwind` rethrows the original payload without counting as a new panic, the tool for a catch-log-rethrow chain. `AssertUnwindSafe` is the honesty marker: types with interior mutability are not logically consistent across a unwind boundary, the cell may be half updated, and the wrapper is you telling the compiler you accept that. `panic=abort` is a cargo profile setting, not code:

```
[profile.release]
panic = "abort"
# unwinding machinery removed, smaller and faster binaries,
# catch_unwind can no longer catch, the process dies at the panic
```

fragment, illustrative, not executed

Under `abort`, every panic ends the process immediately, `?` and `Result` are unaffected, they never unwind. Foreign exceptions, c++ crossing an ffi boundary, abort rather than unwind, so `catch_unwind` is not a general exception catcher. Chapter 17 covers that boundary.

A panic prints three things by default, the message, the file and line of the panic site, and the note pointing at `RUST_BACKTRACE`. Setting `RUST_BACKTRACE=1` in the environment adds the stack, which is the difference between guessing and knowing when a panic escapes in a test run. The default hook prints all of it, and the hook itself receives `PanicHookInfo`, the type renamed from `PanicInfo` in 1.81 when the std hook argument split from the no_std panic handler's. The message text is a debug aid, not an api, code that needs to react to a panic reacts to the payload through `catch_unwind` and downcasting, the way both samples did.

The unwinding path itself costs nothing until it fires. The generated code for a fallible call is a branch on the discriminant, the same branch a c error code would take, and the unwinding tables are dead weight only when the panic never happens. That is the whole justification for the two channel design: expected failures ride the cheap channel, the enum, and the expensive machinery stays reserved for the bugs.

VERIFY: every claim measured

The unwound flag, the payload types, the hook count, the source chain length, the combinator behaviors, the try_from rejection, and the boxed error downcast are printed by `rust/samples/src/Ch11`, 41 checks green under `pwsh -NoProfile -File tools/run-rust-samples.ps1 -Chapter Ch11` on rustc 1.99.0 edition 2024, measured 2026-10-13.

sources: doc.rust-lang.org/stable/book ch9 error handling for panic versus result and the question mark, std::error::Error trait page for source since 1.30 and the trait since 1.0, std::panic module docs for catch_unwind, resume_unwind, set_hook, take_hook, and the payload types, convert module docs for from, try_from, and try_into, releases.md for 1.9 catch_unwind, 1.13 the question mark, 1.26 main returning result, 1.30 error source, 1.34 try_from and try_into, all read from the offline oracle at /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html, accessed 2026-10-13, cross-checked against doc.rust-lang.org/stable url forms.

12. memory and smart pointers

There is no garbage collector and there is no manual free. Every value has one owner from chapter 5, drops run deterministically when ownership ends, and the smart pointer layer is the standard library teaching the compiler new ownership shapes: `Box` for one heap owner, `Rc` and `Arc` for counted shared owners, `Cell` and `RefCell` for mutation that dodges the exclusive borrow rule under supervision. This chapter measures layouts, clone counts, and drop counts for each of them, and closes with `Pin`.

12.1. layout: repr(C) against repr(Rust)

`repr(Rust)` is the default and promises nothing about field order. `repr(C)` lays out fields in declaration order with c padding rules, the only form safe to hand across an ffi boundary or a transmute. The sample measures the same three fields both ways:

```
struct CPair {
    a: u8,
    b: u32,
    c: u8,
}

struct RPair {
    a: u8,
    b: u32,
    c: u8,
}

#[repr(transparent)]
struct Meter(f64);

#[repr(u8)]
enum Tag {
    Zero = 0,
    Two = 2,
    Nine = 9,
}

enum Plain {
    One(u8),
    Two(u16),
}
```

listing 120: the same fields under both reprs, a transparent wrapper, tagged and plain enums (source: rust/samples/src/Ch12/Layout.rs, lines 14-39)

```
fn main() {
    ok(
        std::mem::size_of::<CPair>() == 12 && std::mem::align_of::<CPair>() == 4,
        "repr c keeps declaration order, two pad gaps, 12 bytes",
    );
    ok(
        offsets_c() == (0, 4, 8),
        "repr c field offsets are fixed, a then b then c",
    );
    ok(
        std::mem::size_of::<RPair>() == 8,
        "repr rust reordered the same fields into 8 bytes on this build",
    );
    let ro = offsets_r();
    ok(
        ro.1 == 0 && (ro.0, ro.2) == (4, 5),
        "repr rust placed the u32 first, the two u8 packed after, measured",
    );
    ok(
        std::mem::size_of::<Meter>() == std::mem::size_of::<f64>(),
        "repr transparent is exactly the inner type since 1.28",
    );
    ok(
        std::mem::size_of::<Option<Meter>>() == 16,
        "a float payload finds no niche on this build, the tag takes its own word",
    );
    ok(
        std::mem::size_of::<Option<std::num::NonZeroU32>>() == 4,
        "the nonzero niche erases the none tag, no size growth",
    );
```

listing 121: sizes and offsets measured with addr_of, stable since 1.51 (source: rust/samples/src/Ch12/Layout.rs, lines 61-90)

`CPair` is 12 bytes with the u32 pinned at offset 4 by padding rules. `RPair` measured 8 bytes with the u32 moved to offset 0 and both u8 packed after it: reordering saved 4 bytes. That 8 is a measurement, not a promise, the nomicon is explicit that even two identical struct definitions have no guaranteed matching layout across compilations. The enum numbers: a `#[repr(u8)]` enum pins one byte with chosen discriminants, a default repr enum sizes to tag plus largest variant, 4 bytes for a u16 variant. Niche arithmetic, also measured: `Option<&u8>` and `Option<Box<u8>>` stay 8 bytes because null-like bit patterns mark none, `Option<NonZeroU32>` stays 4, but `Option<f64>` is 16 on this build, the NaN niche is not used for floats. `repr(transparent)` since 1.28, and since 1.42 on univariant enums, guarantees a wrapper is exactly its single field.

PITFALL: never transmute repr(Rust)

Transmuting between a `repr(Rust)` struct and anything else reads whatever order today's compiler happened to pick. The nomicon's rule: only `repr(C)` and `repr(transparent)` types have layout you may depend on. Cross-language structures and file formats declare `repr(C)`, always.

12.2. Box: one heap owner

`Box<T>` moves its value to the heap and owns it. Deref makes it read like a stack value, deref coercion hands out plain references, and drop of the box drops the value, once, deterministically:

```
fn main() {
    let boxed = Box::new(Loud(1));
    ok(
        boxed.0 == 1,
        "deref makes box field access read like stack access",
    );
    let loud_ref: &Loud = &boxed;
    ok(
        loud_ref.0 == 1,
        "deref coercion turns a box into a plain reference",
    );
    drop(boxed);
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 1,
        "dropping the box dropped the heap value once",
    );
    let holder = Holder {
        boxed: Box::new(Loud(2)),
        label: "keeper",
    };
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 1,
        "moving the holder moved the box, no drop fired",
    );
    let moved = holder;
    ok(
        moved.label == "keeper" && moved.boxed.0 == 2,
        "the box rides inside its owner, zero clones needed",
    );
    drop(moved);
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 2,
        "owner drop cascades into the boxed drop",
    );
```

listing 122: deref, coercion, one drop, and the box riding inside a struct (source: rust/samples/src/Ch12/BoxDrop.rs, lines 74-107)

The drop counter runs exactly once per value and the struct move never fires it, the box is just a field. `Box::leak` since 1.26 surrenders the drop entirely and returns `&'static mut`, the sample mutates through it and the drop counter never moves again:

```
    let leaked: &'static mut Loud = Box::leak(Box::new(Loud(3)));
    leaked.0 = 30;
    ok(
        leaked.0 == 30
            && DROPS.load(std::sync::atomic::Ordering::SeqCst) == 2,
        "box leak hands out a static mut, no drop will ever run since 1.26",
    );
    let mut wrapper = Wrapper { inner: 40 };
    *wrapper += 2;
    ok(
        *wrapper == 42,
        "a hand written deref gets the same coercion and mutation sugar",
    );
    let mut heap = Box::new(9u32);
    let slot: &mut u32 = &mut heap;
    *slot = 10;
    ok(
        *heap == 10,
        "deref mut reaches through the box for in place writes",
    );
    let pinned = Box::into_pin(Box::new(7u32));
    ok(
        *pinned == 7,
        "into_pin since 1.63 pins the value without moving it",
    );
    let mut pinned = pinned;
    let unpinned = pinned.as_mut().get_mut();
    *unpinned = 8;
    ok(
        *unpinned == 8,
        "an unpin target still mutates through the pin, get_mut is safe",
    );
```

listing 123: leak, a hand written deref and deref mut, and into_pin (source: rust/samples/src/Ch12/BoxDrop.rs, lines 108-139)

`Deref` and `DerefMut` are traits anyone can implement, the `Wrapper` block gets the same `*w` and `+=` sugar the box has, and the `&str` to `&dyn Trait` coercions of chapter 8 ride the same mechanism.

12.3. drop order

Drops follow two rules and both are measured. Locals drop in reverse declaration order, the exact opposite of construction, and struct fields drop in declaration order, one after another. The sample logs every drop into a mutex guarded vec and reads the tape back:

```
struct Tag(&'static str);

impl Drop for Tag {
    fn drop(&mut self) {
        ORDER.lock().unwrap().push(self.0);
    }
}

struct Tagged {
    first: Tag,
    second: Tag,
}

fn drop_order() -> Vec<&'static str> {
    ORDER.lock().unwrap().clear();
    {
        let _outer = Tag("outer");
        let pair = Tagged {
            first: Tag("first"),
            second: Tag("second"),
        };
        let _inner = Tag("inner");
        let _probe = (pair.first.0, pair.second.0);
    }
    ORDER.lock().unwrap().clone()
}
```

listing 124: a drop logger, a two field struct, and the measurement (source: rust/samples/src/Ch12/BoxDrop.rs, lines 34-59)

The recorded sequence is `inner`, `first`, `second`, `outer`. The locals ran backwards, `_inner` was declared last so it dropped first, `_outer` declared first dropped last, and inside the struct the fields ran forward, `first` before `second`. The rules compose: a struct holding boxes drops each box in field order, the box then drops its heap value, and a `Vec<T>` drops its elements front to back. Every number here is deterministic, no scheduler, no gc timing, the order is a language rule.

The escape hatches sit on the same axis. `std::mem::drop(value)` ends a value early, explicit and immediate, the tool for releasing a lock or a file before the scope ends. `ManuallyDrop<T>` since 1.20 wraps a field so its drop never runs, for the rare ffi handoff where the other side takes ownership. `Box::leak` is the same idea at the pointer level. All three are moves of responsibility, not deletions, the value's memory is released only when its owner's owner finally drops.

12.4. Rc: counted clones on one thread

`Rc` adds a reference count beside the value. Cloning copies the pointer and bumps the count, dropping decrements it, and only the final drop runs the value drop:

```
fn main() {
    let mut shared = Rc::new(Loud(5));
    ok(Rc::strong_count(&shared) == 1, "one owner at creation");
    let a = Rc::clone(&shared);
    let b = Rc::clone(&shared);
    ok(
        Rc::strong_count(&shared) == 3,
        "two clones, strong count three",
    );
    ok(
        Rc::weak_count(&shared) == 0,
        "no weak handles yet",
    );
    let weak: Weak<Loud> = Rc::downgrade(&shared);
    ok(
        Rc::weak_count(&shared) == 1,
        "downgrade since 1.4 added one weak handle",
    );
    ok(
        weak.upgrade().is_some(),
        "upgrade returns a strong handle while owners live",
    );
    drop(a);
    drop(b);
    ok(
        Rc::strong_count(&shared) == 1,
        "dropping clones lowers the count",
    );
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 0,
        "the value survives while one owner remains",
    );
```

listing 125: strong counts through three owners, a weak handle, one survivor (source: rust/samples/src/Ch12/RcClones.rs, lines 24-55)

`Rc::downgrade` since 1.4 mints a `Weak` that points without owning, `upgrade` returns `Option<Rc>` because the value may already be gone, and `Rc::get_mut` refuses while any weak handle exists. The drop counter proves the whole story: zero drops while three owners shrink to one, exactly one drop at the end, and the weak handle outliving the value still reports `None` from upgrade without dangling.

12.5. Arc: the same count across threads

`Arc` is `Rc` with an atomic counter. The sample hands clones to four threads and counts drops globally:

```
fn main() {
    let arc = Arc::new(Loud(9));
    ok(arc.0 == 9, "the arc value reads through deref before any clone");
    let tally = Arc::new(AtomicU64::new(0));
    let mut handles = Vec::new();
    for round in 0..4u64 {
        let ticket = Arc::clone(&arc);
        let counter = Arc::clone(&tally);
        CLONES.fetch_add(2, std::sync::atomic::Ordering::SeqCst);
        handles.push(std::thread::spawn(move || {
            counter.fetch_add(round, std::sync::atomic::Ordering::SeqCst);
            Arc::strong_count(&ticket)
        }));
    }
    let counts: Vec<usize> = handles
        .into_iter()
        .map(|h| h.join().unwrap())
        .collect();
    ok(
        counts.iter().all(|c| *c ≥ 2 && *c ≤ 5),
        "each thread saw at least the original plus its own clone",
    );
    ok(
        tally.load(std::sync::atomic::Ordering::SeqCst) == 0 + 1 + 2 + 3,
        "each thread added its round, the total is six",
    );
    ok(
        Arc::strong_count(&arc) == 1,
        "thread ends dropped their clones, the count is back to one",
    );
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 0,
        "no drop ran while the original owner lives",
    );
    drop(arc);
    drop(tally);
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 1,
        "the final drop ran the value drop exactly once across threads",
    );
    ok(
        CLONES.load(std::sync::atomic::Ordering::SeqCst) == 8,
        "eight arc clones counted, four loud plus four tally",
    );
```

listing 126: four threads cloning, joining, one drop at the very end, eight clones counted (source: rust/samples/src/Ch12/ArcThreads.rs, lines 26-69)

Each thread saw at least the original plus its own clone, after the joins the count returned to 1, and the final drop ran the value drop exactly once across all threads. Rc cannot do this, its counter is not atomic and the type is neither Send nor Sync:

```
let rc = std::rc::Rc::new(1u8);
std::thread::spawn(move || println!("{}", *rc));
// error[E0277]: `Rc<u8>` cannot be sent between threads safely
```

fragment, illustrative, not executed

12.6. Cell and RefCell: the borrow rule at runtime

Interior mutability moves an exclusivity check from compile time to runtime. `Cell<T>` swaps whole values through `&self`, `get`, `set`, `replace` since 1.17, `take`, and only works for `Copy` types. `RefCell<T>` hands out borrow guards, `Ref` and `RefMut`, and panics on a violated rule:

```
struct Counter {
    hits: Cell<u32>,
    steps: RefCell<Vec<&'static str>>,
}

impl Counter {
    fn new() -> Counter {
        Counter {
            hits: Cell::new(0),
            steps: RefCell::new(Vec::new()),
        }
    }

    fn record(&self, step: &'static str) {
        self.hits.set(self.hits.get() + 1);
        self.steps.borrow_mut().push(step);
    }
}
```

listing 127: a counter type mutating both interior slots through a shared receiver (source: rust/samples/src/Ch12/CellRefCell.rs, lines 16-34)

```
    let cell = Cell::new(7u32);
    cell.set(9);
    ok(cell.get() == 9, "cell set replaces the value through a shared ref");
    ok(
        cell.replace(11) == 9,
        "cell replace since 1.17 swaps and returns the old value",
    );
    ok(cell.take() == 11, "cell take leaves the default behind");
    ok(cell.get() == 0, "the taken cell reset to zero");
    let mut plain = 5u32;
    let shared = Cell::from_mut(&mut plain);
    shared.set(6);
    ok(
        plain == 6,
        "cell from_mut since 1.37 reborrows a mut as a shared cell",
    );
    let counter = Counter::new();
    counter.record("boot");
    counter.record("run");
    ok(
        counter.hits.get() == 2 && counter.steps.borrow().len() == 2,
        "a shared receiver mutated both interior slots",
    );
    ok(
        counter.steps.borrow().as_slice() == ["boot", "run"],
        "the vec grew in place behind an immutable borrow of counter",
    );
    let first = counter.steps.borrow();
    let conflict = counter.steps.try_borrow_mut();
    ok(
        conflict.is_err() && first.len() == 2,
        "try_borrow_mut reports the conflict instead of panicking",
    );
    drop(first);
    {
        let clean = counter.steps.try_borrow_mut();
        ok(clean.is_ok(), "the borrow flag clears when the guard drops");
    }
```

listing 128: cell set replace take, from_mut since 1.37, try_borrow_mut, and the guard drop (source: rust/samples/src/Ch12/CellRefCell.rs, lines 36-73)

The `Counter::record` method takes `&self` and mutates a `Cell` and a `RefCell` behind it, which is the entire point: shared receivers with interior state. `try_borrow_mut` reports the conflict as an `Err` instead of panicking, and the caught panic proves the second `borrow_mut` really aborts. The rule did not disappear, it moved: a wrong borrow is now a runtime failure instead of a compile error. `Cell::from_mut` since 1.37 reborrows a `&mut T` as a `&Cell<T>`, bridging the two worlds.

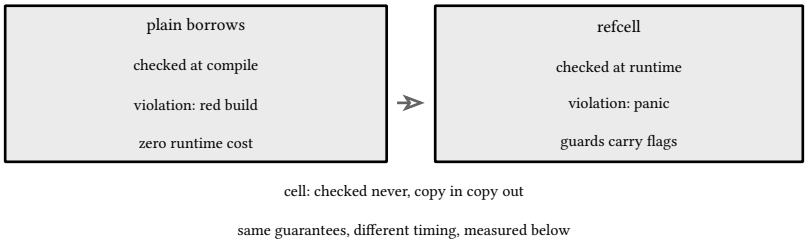


figure 21: where the exclusivity check runs

12.7. Weak and cycle breaking

Strong `Rc` links make cycles, and no drop in a cycle ever fires. The sample builds a two node cycle, drops both handles, and the drop counter reads zero, a leak proven, then rebuilds the same graph with `Weak` links and everything drops:

```
struct Node {
    value: u32,
    next: RefCell<Option<Rc<Node>>>,
}

struct Link {
    value: u32,
    next: RefCell<Option<Weak<Link>>>,
}

impl Drop for Node {
    fn drop(&mut self) {
        DROPS.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
    }
}

impl Drop for Link {
    fn drop(&mut self) {
        DROPS.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
    }
}
```

listing 129: the strong node type that leaks, the weak link type that does not (source: rust/samples/src/Ch12/WeakCycle.rs, lines 17-37)

```
fn walk_strong(start: &Rc<Node>, cap: usize) -> (usize, u32) {
    let mut current = Rc::clone(start);
    let mut hops = 0;
    while hops < cap {
        hops += 1;
        let next = current.next.borrow().clone();
        match next {
            Some(node) => current = node,
            None => return (hops, current.value),
        }
    }
    (hops, current.value)
}
```

listing 130: the capped strong walk, returning where it stopped (source: rust/samples/src/Ch12/WeakCycle.rs, lines 39-51)

```
fn main() {
    let first = Rc::new(Node {
        value: 1,
        next: RefCell::new(None),
    });
    let second = Rc::new(Node {
        value: 2,
        next: RefCell::new(None),
    });
    *first.next.borrow_mut() = Some(Rc::clone(&second));
    *second.next.borrow_mut() = Some(Rc::clone(&first));
    ok(
        Rc::strong_count(&first) == 2 && Rc::strong_count(&second) == 2,
        "the cycle holds two owners of each node",
    );
    ok(
        walk_strong(&first, 6) == (6, 1),
        "a strong walk hits the cap back at node one, never none",
    );
    drop(first);
    drop(second);
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 0,
        "both handles dropped, zero node drops, the cycle leaked",
    );
```

listing 131: the cycle: two strong owners each, cap hit, zero drops (source: rust/samples/src/Ch12/WeakCycle.rs, lines 53-77)

```
    let l1 = Rc::new(Link {
        value: 1,
        next: RefCell::new(None),
    });
    let l2 = Rc::new(Link {
        value: 2,
        next: RefCell::new(None),
    });
    *l1.next.borrow_mut() = Some(Rc::downgrade(&l2));
    *l2.next.borrow_mut() = Some(Rc::downgrade(&l1));
    ok(
        Rc::weak_count(&l1) == 1 && Rc::strong_count(&l1) == 1,
        "weak links leave one strong owner per node",
    );
    let hop = l1.next.borrow().as_ref().and_then(Weak::upgrade);
    ok(
        hop.as_ref().map(|l| l.value) == Some(2),
        "upgrade bridges a weak link on demand",
    );
    drop(hop);
    drop(l1);
    drop(l2);
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 2,
        "the weak version drops both nodes cleanly",
    );
    let self_aware = Rc::new_cyclic(|weak| Link {
        value: 7,
        next: RefCell::new(Some(weak.clone())),
    });
    ok(
        Rc::strong_count(&self_aware) == 1 && Rc::weak_count(&self_aware) == 1,
        "new_cyclic since 1.60 hands the constructor its own weak",
    );
    let upgrade = self_aware.next.borrow().as_ref().and_then(Weak::upgrade);
    ok(
        upgrade.as_ref().map(|l| l.value) == Some(7),
        "the self weak upgrades back to the same node",
    );
    drop(upgrade);
    drop(self_aware);
    ok(
        DROPS.load(std::sync::atomic::Ordering::SeqCst) == 3,
        "the self referencing node still drops, weak is not an owner",
    );
    println!("measured 2026-10-13 on rustc 1.99.0 edition 2024");
}
```

listing 132: weak links upgrade on demand, both nodes drop, and new_cyclic since 1.60 (source: rust/samples/src/Ch12/WeakCycle.rs, lines 78-124)

`Rc::new_cyclic` since 1.60 hands the constructor a `Weak` to the value being built, the clean way to make a self reference, and the measured drop count shows it still drops: weak references observe, they do not own. The parent-child idiom falls out, parents hold strong `Rc` to children, children hold `Weak` back to parents.

12.8. Pin in one section

Pin wraps a pointer and forbids moving the value behind it. It exists for async: a future that awaits across a suspension may hold pointers into its own stack frame, so moving it after polling would dangle those pointers. Pin since 1.33 with PhantomPinned, the marker from chapter 9 that opts a type out of Unpin, and Box::into_pin since 1.63:

```
    let pinned = Box::into_pin(Box::new(7u32));
    ok(
        *pinned == 7,
        "into_pin since 1.63 pins the value without moving it",
    );
    let mut pinned = pinned;
    let unpinned = pinned.as_mut().get_mut();
    *unpinned = 8;
    ok(
        *unpinned == 8,
        "an unpin target still mutates through the pin, get_mut is safe",
    );
```

listing 133: into_pin, and a pinned unpin value still mutating through get_mut (source: rust/samples/src/Ch12/BoxDrop.rs, lines 128-139)

The contract is small: for T: Unpin, everything, Pin is a no-op wrapper and get_mut is safe, the sample mutates through it. Only a !Unpin type, one carrying PhantomPinned, actually locks the address, and then get_mut stops compiling. Chapter 14 builds futures on exactly this, here the type is the whole story.

12.9. the no-gc contrast

The throughline of the chapter: nothing traces, nothing defers, no write barrier exists. A Box drops when its owner drops, an Rc drops when the count reaches zero, a cycle leaks unless you break it with Weak yourself, and every one of those statements was printed by a drop counter in these samples, 63 checks green. The cost model is the mirror of a tracing collector: deterministic drops and no pause times, paid for with the ownership discipline of chapter 5 and the occasional Rc<RefCell<T>> when a graph refuses to be a tree.

That last combination deserves its own sentence because it is the idiomatic graph in practice. Rc<RefCell<T>> gives shared ownership and shared mutation, each Rc clone can reach the RefCell and borrow it mutably, exactly what a shared cache or an observer list needs, and the WeakCycle sample ran it as the node type. The price is doubled vigilance: the borrow rule now fails at runtime, chapter 12's own CellRefCell measurement, and a cycle of strong Rc links leaks, so observers must hang off Weak. When either price is too high the answer is not more cell types, it is a redesign towards ownership, a single owner with indices instead of pointers, which is where the service chapters land.

MaybeUninit since 1.36 and the allocator api are the escape hatches further down, chapter 17 walks them.

sources: doc.rust-lang.org/nomicon repr-rust.html for field ordering and the transmute rule, reference/destructors.html for drop order, reference/type-layout.html for size, align, and the enum layouts, doc.rust-lang.org/stable/book ch15 smart pointers for box, rc, refcell, and weak, std docs for rc, sync, cell, and pin pages with their since badges, releases.md for 1.4 downgrade and upgrade, 1.17 cell replace, 1.20 manuallydrop, 1.26 box leak, 1.28 repr transparent, 1.33 pin and phantompinned, 1.36 maybeuninit, 1.37 from_mut, 1.42 transparent enums, 1.51 addr_of, 1.60 new_cyclic, 1.63 into_pin, all read from the offline oracle at /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html, accessed 2026-10-13, cross-checked against doc.rust-lang.org/stable url forms. Every count asserted by rust/samples/src/Ch12, 63 checks green under pwsh -NoProfile -File tools/run-rust-samples.ps1
-Ch12 on rustc 1.99.0 edition 2024, measured 2026-10-13.

13. concurrency: threads to atomics

Rust's pitch is a compile-time one: the same ownership machinery that decides memory safety also decides what may cross a thread boundary. A `thread::spawn` demands a closure that is `Send` and `'static`, so sharing mutable state by accident is a compile error before it is a race. This chapter walks the ladder in order: spawn and join, scoped threads for borrowing (since 1.63), mpsc channels, `Mutex`, `RwLock`, and `Condvar`, then atomics with every ordering demonstrated by a sample whose output carries the proof. The memory model underneath is book 12, ch 6 (design patterns, concurrency, and distributed systems, concurrency fundamentals and the memory model)'s subject, this chapter covers the rust surfaces and stays honest about what safe rust does and does not prevent.

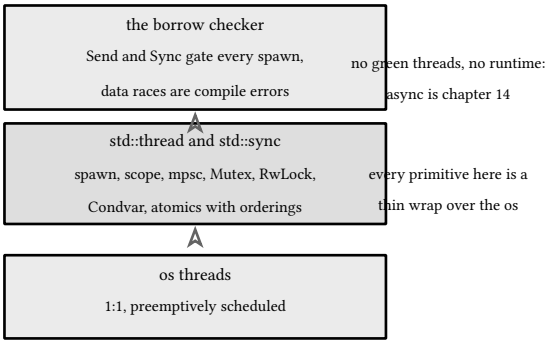


figure 22: safe rust layers the guarantees, the primitives below stay os shaped

13.1. spawn, join, and move

`thread::spawn` takes a closure and returns a `JoinHandle<T>` where T is whatever the closure returns. The handle is the only rope: `join()` waits for the thread and yields its value as `Result<T, Box<dyn Any + Send>>`, and the `Err` arm is how a panic crosses back, with the payload downcastable to what was thrown. Closures capture by reference by default, and a spawned closure must own its world for `'static`, so the normal shape is the `move` keyword transferring every capture.

```
    let handle = thread::spawn(move || {
        let mut acc = 0u64;
        for i in 1..=1000u64 {
            acc += i * i;
        }
        acc
    });
    let sum = handle.join().unwrap();
    ok(
        sum == sum_of_squares(1000) && sum == 333_833_500,
        "a spawned closure returns its value through join",
    );

    let capture = vec![1u32, 2, 3];
    let owned = thread::spawn(move || capture.len()).join().unwrap();
    ok(owned == 3, "move transfers the vector into the thread");

    let text = String::from("borrowed");
    let borrowed = thread::spawn(move || text.len()).join().unwrap();
    ok(borrowed == 8, "non copy types move, the closure owns the string");

    let prior_hook = std::panic::take_hook();
    std::panic::set_hook(Box::new(|_| {}));
    let panic_handle = thread::spawn(|| panic!("boom"));
    let joined: Result<(), Box<dyn Any + Send>> = panic_handle.join();
    std::panic::set_hook(prior_hook);
    ok(joined.is_err(), "join reports a panicked thread as Err");
    let payload = joined.unwrap_err().downcast::<&str>().unwrap();
    ok(
        *payload == "boom",
        "the panic payload crosses the join boundary as the thrown value",
    );
```

listing 134: spawn returns a value through join, move transfers captures, panics cross as Err (source: rust/samples/src/Ch13/SpawnJoin.rs, lines 22-53)

The three checks the sample prints: the loop computes 333,833,500 which is the closed form for the sum of the first 1000 squares, the vector moves in and its length comes back, and a `panic!("boom")` joins as an `Err` whose payload downcasts to the thrown `&str`. A silent panic hook keeps the intentional panic off stderr, the same discipline the poisoning sample below uses.

`Builder` configures the spawn: a name for diagnostics, a stack size. Parking is the minimal handshake, and it is token based: `unpark` that arrives before `park` is remembered, so the ordering race between the two calls cannot lose the wakeup.

```
    let named = thread::Builder::new()
        .name("worker-7".to_string())
        .spawn(|| thread::current().name().map(str::to_string))
        .unwrap();
    let name = named.join().unwrap().unwrap();
    ok(name == "worker-7", "Builder::new().name sets the thread name");
    ok(
        thread::current().name() == Some("main"),
        "the main thread is named main",
    );

    let parked = thread::spawn(|| {
        thread::park();
        41u32
    });
    parked.thread().unpark();
    let unparked = parked.join().unwrap();
    ok(
        unparked == 41,
        "unpark before park is not lost, the token makes park return",
    );
```

listing 135: a named thread, and the park and unpark token handshake (source: rust/samples/src/Ch13/SpawnJoin.rs, lines 55-75)

`available_parallelism`, since 1.59, reads the scheduler hint rather than `std::thread::count` folklore. Measured 2026-10-13 on the pinned build: 20. Fixed thread counts rule every sample in this book, joined before any assert, no assertion ever rides on timing.

13.2. scoped threads, borrowing since 1.63

`spawn` demands `'static` because the child can outlive its parent. Most real parallelism wants the opposite: borrow local data, join before the function returns. `thread::scope`, stable since 1.63, guarantees every spawned child joins before the scope closure returns, so borrows of stack locals are sound and the compiler accepts them.

```
    let mut scores = vec![0u64; 512];
    let mut left_total = 0u64;
    let mut right_total = 0u64;
    {
        let (left, right) = scores.split_at_mut(256);
        thread::scope(|s| {
            let a = s.spawn(move || {
                let mut sum = 0u64;
                for v in left.iter_mut() {
                    *v += 1;
                    sum += *v;
                }
                sum
            });
            let b = s.spawn(move || {
                let mut sum = 0u64;
                for v in right.iter_mut() {
                    *v += 2;
                    sum += *v;
                }
                sum
            });
            left_total = a.join().unwrap();
            right_total = b.join().unwrap();
        });
    }
```

listing 136: split_at_mut hands each scoped thread half a buffer, no Arc, no clone (source: rust/samples/src/Ch13/ScopedThreads.rs, lines 16-41)

Two details the borrow checker enforces and the sample makes visible. First, `split_at_mut` partitions the buffer into non-overlapping mutable borrows, the shape every data-parallel loop wants. Second, a mutable borrow captured by a spawned closure lasts the whole scope even after that handle joins, so `logs[2] = 30` above had to move after the scope: within one scope you partition borrows, you do not reuse them.

Shared borrows fan out freely. One `&String` reaches four threads with no `Arc` in sight, because shared references are `Copy`:

```
    let greeting = String::from("hello scope");
    let lens = thread::scope(|s| {
        let shared: &String = &greeting;
        let handles: Vec<_> = (0..4u64)
            .map(|i| s.spawn(move || shared.len() as u64 + i))
            .collect();
        handles
            .into_iter()
            .map(|h| h.join().unwrap())
            .collect::<Vec<u64>>()
    });
    ok(
        lens == vec![11, 12, 13, 14],
        "a shared borrow fans out to four threads with no Arc",
    );

    let mut logs = vec![0u64; 3];
    thread::scope(|s| {
        let h0 = s.spawn(|| {
            thread::scope(|inner| {
                let g = inner.spawn(|| 10u64).join().unwrap();
                logs[0] = g;
            });
            logs[1] = 20;
            "outer done"
        });
        h0.join().unwrap();
    });
    logs[2] = 30;
```

listing 137: a shared borrow fans out, scopes nest (source: rust/samples/src/Ch13/ScopedThreads.rs, lines 55-83)

Before 1.63 the same program needed `Arc<Vec<_>>` with clones counted at runtime. Scope moves that bookkeeping to the type system, and the corpus api in chapter 19 prefers it for request fan-out.

13.3. channels: mpsc

`std::sync::mpsc` is multi-producer, single-consumer. `channel()` returns an unbounded `(Sender, Receiver)`: `send` never waits for a receiver, values queue without bound, and delivery order per sender is preserved. The lifecycle errors are part of the type story: a send after the receiver drops is `SendError`, and iteration ends when the last sender drops.

```
    let (tx, rx) = mpsc::channel::<u32>();
    for i in 1..=5u32 {
        tx.send(i).unwrap();
    }
    drop(tx);
    let drained: Vec<u32> = rx.into_iter().collect();
    ok(
        drained == vec![1, 2, 3, 4, 5],
        "channel delivery preserves send order",
    );

    let (tx, rx) = mpsc::channel::<u32>();
    drop(rx);
    let refused = tx.send(1);
    ok(
        refused.is_err(),
        "sending after the receiver is dropped is a SendError",
    );

    let (tx, rx) = mpsc::channel::<u32>();
    tx.send(9).unwrap();
    ok(
        rx.try_recv() == Ok(9),
        "try_recv takes a queued value without blocking",
    );
    ok(
        matches!(rx.try_recv(), Err(TryRecvError::Empty)),
        "try_recv reports Empty while the sender still lives",
    );
    drop(tx);
    ok(
        matches!(rx.try_recv(), Err(TryRecvError::Disconnected)),
        "try_recv reports Disconnected once every sender is gone",
    );
```

listing 138: order, disconnection, and the three try_recv states (source: rust/samples/src/Ch13/Channels.rs, lines 18-51)

Fan-in is the everyday shape: clone the sender per producer, join the producers, drop the original, drain the receiver. The assert compares against a value set built independently of the channel, so exactly 1000 arrivals of exactly the right values is the check, order across producers is asserted only after sorting because interleave order is scheduler business.

```
let (tx, rx) = mpsc::channel::<u64>();
{
    let producers: Vec<_> = (0..4u64)
        .map(|k| {
            let tx = tx.clone();
            thread::spawn(move || {
                for i in 0..250u64 {
                    tx.send(k * 1000 + i).unwrap();
                }
            })
        })
        .collect();
    for handle in producers {
        handle.join().unwrap();
    }
}
drop(tx);
let mut received: Vec<u64> = rx.into_iter().collect();
let mut expected: Vec<u64> = (0..4u64).flat_map(|k| (0..250u64).map(move |i| k * 1000 + i)).collect();
received.sort_unstable();
expected.sort_unstable();
ok(received.len() == 1000, "four producers deliver 1000 values");
ok(
    received == expected,
    "every value arrives exactly once across the shared receiver",
);
```

listing 139: fan-in, four producers into one receiver (source: rust/samples/src/Ch13/Channels.rs, lines 53-78)

`sync_channel(n)` is the bounded sibling: capacity n accepts n sends with no receiver, the n+1st blocks until space frees. Capacity 0 is a rendezvous, the send completes only against a matching receive, which the sample proves by the join returning after the receive.

```
let (tx, rx) = mpsc::sync_channel::<u32>(2);
tx.send(1).unwrap();
tx.send(2).unwrap();
drop(tx);
ok(
    rx.recv().unwrap() == 1 && rx.recv().unwrap() == 2,
    "a sync_channel of capacity 2 holds two sends with no receiver yet",
);
ok(rx.recv().is_err(), "a drained closed channel errors on recv");

let (tx, rx) = mpsc::sync_channel::<u32>(0);
let sender = thread::spawn(move || {
    tx.send(77).unwrap();
    "rendezvous done"
});
let taken = rx.recv();
ok(
    taken == Ok(77) && sender.join().unwrap() == "rendezvous done",
    "capacity 0 is a rendezvous, send completes only against a receive",
);
```

listing 140: bounded capacity 2, then the capacity 0 rendezvous (source: rust/samples/src/Ch13/Channels.rs, lines 80-99)

Crossbeam-flavored channels live off the std menu: select over many channels, mpmc, deadlines. The book stays zero-dependency, so this chapter builds what std gives and chapter 14 composes futures over them. What matters is the taxonomy, and std already shows all three synchronization behaviors: queue, bound, rendezvous.

13.4. mutex, rwlock, and condvar

When data stays put and many threads touch it, `Mutex<T>` is the default. The lock guards the value inside, the guard derefs to it, and dropping the guard releases the lock, so scopes do the unlocking. The counting sample is the discipline in one line per iteration:

```
let counter = Arc::new(Mutex::new(0u64));
let handles: Vec<_> = (0..THREADS)
    .map(|_| {
        let counter = Arc::clone(&counter);
        std::thread::spawn(move || {
            for _ in 0..PER_THREAD {
                *counter.lock().unwrap() += 1;
            }
        })
    })
    .collect();
for handle in handles {
    handle.join().unwrap();
}
ok(
    *counter.lock().unwrap() == (THREADS as u64) * PER_THREAD,
    "8 threads x 10000 locked increments total exactly 80000",
);

let scoped = Mutex::new(5u64);
{
    let mut guard = scoped.lock().unwrap();
    *guard += 1;
}
ok(
    *scoped.lock().unwrap() == 6,
    "the guard unlocks at scope exit, the value survived",
);
```

listing 141: 8 threads x 10000 locked increments total exactly 80000 (source: rust/samples/src/Ch13/Locks.rs, lines 20-47)

Poisoning is rust's answer to the panic-while-holding question. A thread that panics with the lock held marks the mutex poisoned, and every later `lock()` returns `Err(PoisonError)`. The value is still there: `into_inner()` takes it out. The design reads as pessimism but is a fence: if the panicking thread died between two mutations of a multi-field invariant, the protected data may be inconsistent, and forcing every caller through an explicit `Err` makes that possibility visible instead of silent.

```
    let shared = Arc::new(Mutex::new(10u64));
    let poisoner = {
        let shared = Arc::clone(&shared);
        std::thread::spawn(move || {
            let mut guard = shared.lock().unwrap();
            *guard += 5;
            panic!("died holding the lock");
        })
    };
    let prior_hook = std::panic::take_hook();
    std::panic::set_hook(Box::new(|_| {}));
    let poisoned_join = poisoner.join();
    let poisoned = shared.lock();
    std::panic::set_hook(prior_hook);
    ok(poisoned_join.is_err(), "the panicking thread joins with Err");
    ok(poisoned.is_err(), "a panic while holding poisons the mutex");
    let recovered = poisoned.unwrap_err().into_inner();
    ok(
        *recovered == 15,
        "into_inner recovers the value the panicking thread already wrote",
    );
```

listing 142: a panic while holding poisons, into_inner recovers (source: rust/samples/src/Ch13/Locks.rs, lines 49-69)

`RwLock<T>` splits the exclusion: many readers or one writer. The probes are cross-thread while the main thread holds a guard, so `try_write` failing and `try_read` succeeding is measured behavior, not documented hope. One shape to know: a guard is bound to its lock and `RwLockReadGuard` is `!Send`, so a guard cannot ride a join boundary, the compiler rejects the attempt outright.

```
    let rw = Arc::new(RwLock::new(vec![1u32, 2, 3]));
    let guard = rw.read().unwrap();
    let writer_probe = {
        let probe_rw = Arc::clone(&rw);
        std::thread::spawn(move || probe_rw.try_write().is_err())
    };
    let reader_probe = {
        let probe_rw = Arc::clone(&rw);
        std::thread::spawn(move || probe_rw.try_read().map(|g| g.len()).is_ok())
    };
    ok(
        writer_probe.join().unwrap(),
        "try_write reports WouldBlock while a reader holds the lock",
    );
    ok(
        reader_probe.join().unwrap(),
        "a second reader coexists while the first still holds the lock",
    );
    drop(guard);
    {
        let mut write = rw.write().unwrap();
        write.push(4);
    }
    ok(
        rw.read().unwrap().len() == 4,
        "the write guard mutates and releases at scope exit",
    );
```

listing 143: readers coexist, writers wait, guards die with their scope (source: rust/samples/src/Ch13/Locks.rs, lines 71-97)

`Condvar` completes the trio as the wait side. The pattern is rigid and the rigidity is the point: lock, re-check the predicate in a `while` loop because a wakeup never promises the condition holds, `wait(guard)` atomically parks and releases, and every state change notifies. The sample is a one-slot handoff, 1000 rounds, both sides finish with the exact sum 500,500, no lost wakeup in any run.

```
    let slot = Arc::new((Mutex::new(None::<u64>), Condvar::new()));
    let producer = {
        let slot = Arc::clone(&slot);
        std::thread::spawn(move || {
            let (lock, cv) = &*slot;
            for i in 1..=ROUNDS {
                let mut guard = lock.lock().unwrap();
                while guard.is_some() {
                    guard = cv.wait(guard).unwrap();
                }
                *guard = Some(i);
                drop(guard);
                cv.notify_all();
            }
            let mut guard = lock.lock().unwrap();
            while guard.is_some() {
                guard = cv.wait(guard).unwrap();
            }
            *guard = Some(0);
            drop(guard);
            cv.notify_all();
        })
    };
```

listing 144: the producer side of the condvar handshake (source: rust/samples/src/Ch13/Locks.rs, lines 99-121)

```
    let consumer = {
        let slot = Arc::clone(&slot);
        std::thread::spawn(move || {
            let (lock, cv) = &*slot;
            let mut sum = 0u64;
            let mut count = 0u64;
            loop {
                let mut guard = lock.lock().unwrap();
                loop {
                    match *guard {
                        Some(0) => return (sum, count),
                        Some(v) => {
                            *guard = None;
                            drop(guard);
                            cv.notify_all();
                            sum += v;
                            count += 1;
                            break;
                        }
                        None => guard = cv.wait(guard).unwrap(),
                    }
                }
            }
        })
    };
    producer.join().unwrap();
    let (sum, count) = consumer.join().unwrap();
    ok(count == ROUNDS, "the condvar handshake delivers every round");
    ok(
        sum == ROUNDS * (ROUNDS + 1) / 2,
        "the delivered sum is exactly 1..=1000, no lost wakeup",
    );
    println!("measured 2026-10-13: {ROUNDS} condvar handoffs through one slot");
}
```

listing 145: the consumer side, predicate loop and all (source: rust/samples/src/Ch13/Locks.rs, lines 122-155)

13.5. atomics and the orderings

`AtomicBool`, `AtomicI64`, `AtomicUsize`, and the rest give single word operations that are atomic in the hardware sense, plus a memory ordering argument that decides which other writes become visible around them. The orderings are not decoration, they are the contract, and the samples each prove the property they claim.

`Relaxed` is atomicity with no synchronization promise: perfect for counters and statistics where only the total matters. Eight threads run 10,000 relaxed increments each and the total is exactly 80,000 every run: no torn updates, because the read-modify-write is one instruction, no ordering, because none was asked for. The read-modify-write family returns the previous value, which is what makes fetch strategies possible:

```
    let total = Arc::new(AtomicUsize::new(0));
    let handles: Vec<_> = (0..THREADS)
        .map(|_| {
            let total = Arc::clone(&total);
            std::thread::spawn(move || {
                for _ in 0..PER_THREAD {
                    total.fetch_add(1, Ordering::Relaxed);
                }
            })
        })
        .collect();
    for handle in handles {
        handle.join().unwrap();
    }
    ok(
        total.load(Ordering::Relaxed) == THREADS * PER_THREAD,
        "relaxed fetch_add is still atomic, the total is exactly 80000",
    );

    let bits = AtomicU8::new(0b1010);
    ok(
        bits.fetch_or(0b0101, Ordering::SeqCst) == 0b1010
            && bits.load(Ordering::SeqCst) == 0b1111,
        "fetch_or returns the previous value and writes the union",
    );
    ok(
        bits.fetch_and(0b1100, Ordering::SeqCst) == 0b1111
            && bits.load(Ordering::SeqCst) == 0b1100,
        "fetch_and masks in place",
    );
    ok(
        bits.swap(0b0011, Ordering::SeqCst) == 0b1100,
        "swap exchanges and returns the old value",
    );
```

listing 146: relaxed counters stay exact, and the fetch family returns old values (source: rust/samples/src/Ch13/Atomics.rs, lines 67-100)

`compare_exchange` is the compare-and-swap at the bottom of every lock-free algorithm, with two orderings because it is a read and a write when it succeeds and a plain read when it fails. The spinlock at the end of the sample is the canonical use: acquire on the way in, release on the way out, a plain `u64` guarded underneath.

```
    let cell = AtomicU8::new(3);
    let won = cell.compare_exchange(3, 7, Ordering::AcqRel, Ordering::Acquire);
    ok(
        won == Ok(3) && cell.load(Ordering::Acquire) == 7,
        "compare_exchange swaps on match and returns the old value",
    );
    let lost = cell.compare_exchange(3, 9, Ordering::AcqRel, Ordering::Acquire);
    ok(
        lost.is_err() && cell.load(Ordering::Acquire) == 7,
        "compare_exchange fails and keeps the current value on mismatch",
    );
```

listing 147: compare_exchange, success and failure arms (source: rust/samples/src/Ch13/Atomics.rs, lines 102-112)

`Release` on a store pairs with `Acquire` on a load that observes it: everything the writer did before the release store is visible to the reader after the acquire load sees it. That is the publication pattern, data written plainly, then a flag

released, then a reader that can trust the data the moment the flag acquires. The sample's assert is the contract, the plain store of 42 is never seen stale after the acquire observed `true`:

```
    let flags = Arc::new((AtomicI64::new(0), AtomicBool::new(false)));
    let writer = {
        let flags = Arc::clone(&flags);
        std::thread::spawn(move || {
            let (data, ready) = &*flags;
            data.store(42, Ordering::Relaxed);
            ready.store(true, Ordering::Release);
        })
    };
    let mut seen = false;
    for _ in 0..1_000_000 {
        if flags.1.load(Ordering::Acquire) {
            seen = true;
            break;
        }
    }
    if !seen {
        writer.join().unwrap();
        seen = flags.1.load(Ordering::Acquire);
    } else {
        writer.join().unwrap();
    }
    ok(seen, "the release store becomes visible to the acquire load");
    ok(
        flags.0.load(Ordering::Relaxed) == 42,
        "data written before the release store is visible after the acquire load",
    );
```

listing 148: the publication pattern, release store against acquire load (source: rust/samples/src/Ch13/Atomics.rs, lines 114-140)

`SeqCst` is the strongest and the only ordering with a total order across all SeqCst operations. The proof is the store-buffer litmus: two threads each store their own flag then read the other's. Can both read stale, both see false? Under Relaxed the model permits it, so the sample measures whether this machine does it, with a gate harness that starts both threads on the same round instead of paying a spawn per round:

```rust
fn store_buffer_litmus(order: Ordering, rounds: usize) -> u32 {
    let x = AtomicBool::new(false);
    let y = AtomicBool::new(false);
    let gate = AtomicUsize::new(0);
    let done = AtomicUsize::new(0);
    let last1 = AtomicBool::new(false);
    let last2 = AtomicBool::new(false);
    let mut zeros = 0u32;
    std::thread::scope(|s| {
        s.spawn(|| {
            for r in 0..rounds {
                while gate.load(Ordering::SeqCst) != r {
                    spin_loop();
                }
                x.store(true, order);
                last1.store(y.load(order), Ordering::Relaxed);
                done.fetch_add(1, Ordering::SeqCst);
            }
        });
        s.spawn(|| {
            for r in 0..rounds {
                while gate.load(Ordering::SeqCst) != r {
                    spin_loop();
                }
                y.store(true, order);
                last2.store(x.load(order), Ordering::Relaxed);
                done.fetch_add(1, Ordering::SeqCst);
            }
        });
        for r in 0..rounds {
            gate.store(r, Ordering::SeqCst);
            while done.load(Ordering::SeqCst) != 2 * (r + 1) {
                spin_loop();
            }
            if !last1.load(Ordering::Relaxed) && !last2.load(Ordering::Relaxed) {
                zeros += 1;
            }
            x.store(false, Ordering::Relaxed);
            y.store(false, Ordering::Relaxed);
        }
    });
    zeros
}
```

listing 149: the store-buffer litmus harness, one gate, two spinning threads (source: rust/samples/src/Ch13/Atomics.rs, lines 22-64)

Measured 2026-10-13 on the pinned build, 20 reported cores, three runs: relaxed produced 891, 4, and 168 double-zero rounds out of 20,000, seqcst produced 0 in every run. The asymmetry is the lesson: the relaxed count proves the hardware permits the outcome, its exact value is scheduler noise and no assert rides on it, while the seqcst assert holds in every run because the model forbids the outcome entirely. `AcqRel` is the load-store combination used on read-modify-writes, and the rule of thumb holds: default to acquiring loads and releasing stores, pay for seqcst only when a total order across variables is the algorithm's actual requirement, the double checked locking shape being the classic.

The spinlock folds the whole section into one type. Note the seam: the protected `u64` rides an `UnsafeCell` with an `unsafe impl Sync`, two lines of trust that the lock discipline is correct, which is chapter 17's exact subject.

```
    let lock = Arc::new(SpinLock::new(0u64));
    let handles: Vec<_> = (0..4usize)
        .map(|_| {
            let lock = Arc::clone(&lock);
            std::thread::spawn(move || {
                for _ in 0..25_000u64 {
                    lock.add(1);
                }
            })
        })
        .collect();
    for handle in handles {
        handle.join().unwrap();
    }
    ok(
        lock.get() == 100_000,
        "a compare_exchange spinlock guards a plain u64, 4 x 25000 adds total exactly",
    );
}
```

listing 150: four threads x 25000 guarded adds total exactly 100000 (source: rust/samples/src/Ch13/Atomics.rs, lines 152-170)

```
struct SpinLock {
    locked: AtomicBool,
    value: UnsafeCell<u64>,
}

unsafe impl Sync for SpinLock {}

impl SpinLock {
    fn new(value: u64) -> SpinLock {
        SpinLock {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    fn add(&self, n: u64) {
        while self
            .locked
            .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
            .is_err()
        {
            spin_loop();
        }
        unsafe { *self.value.get() += n };
        self.locked.store(false, Ordering::Release);
    }

    fn get(&self) -> u64 {
        while self
            .locked
            .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
            .is_err()
        {
            spin_loop();
        }
        let value = unsafe { *self.value.get() };
        self.locked.store(false, Ordering::Release);
        value
    }
}
```

listing 151: the spinlock type, acquire in, release out, UnsafeCell underneath (source: rust/samples/src/Ch13/Atomics.rs, lines 172-211)

13.6. Send, Sync, and the honest race claim

`Send` says a value may move to another thread. `Sync` says a shared reference may, `&T` is `Send`. Both are auto traits, derived structurally from the fields, both are marker traits with no methods, and both can be granted manually only in `unsafe impl`, never negated. The sample pins the positive facts at compile time:

```
    assert_send::<i32>();
    assert_sync::<i32>();
    assert_send::<String>();
    assert_sync::<String>();
    assert_send::<Vec<u8>>();
    assert_sync::<Vec<u8>>();
    assert_send::<Arc<u64>>();
    assert_sync::<Arc<u64>>();
    assert_send::<AtomicUsize>();
    assert_sync::<AtomicUsize>();
    assert_send::<Mutex<Vec<u8>>>();
    assert_sync::<Mutex<Vec<u8>>>();
    assert_send::<RwLock<i64>>();
    assert_sync::<RwLock<i64>>();
    assert_send::<std::sync::mpsc::Sender<u32>>();
    assert_sync::<std::sync::mpsc::Sender<u32>>();
    assert_send::<std::sync::mpsc::Receiver<u32>>();
    assert_send::<std::thread::JoinHandle<u8>>();
    assert_sync::<Mutex<Vec<u8>>>();
    assert_send::<Box<dyn Send + Sync>>();
    assert_sync::<Box<dyn Send + Sync>>();
    ok(
        true,
        "the compile passed with Send or Sync bounds asserted for every listed type",
    );
```

listing 152: compile-time Send and Sync bounds, asserted for the standard types (source: rust/samples/src/Ch13/SendSync.rs, lines 21-45)

The negative facts cannot be asserted in a file that must compile, and probing them live turns each into its compile error: `Rc<u64>` is neither `Send` nor `Sync` because its refcount is unsynchronized, `*const u8` is neither because pointers alias freely, `RefCell` is `Send` but not `Sync` because its borrow flag is thread-local state, `MutexGuard` is `!Send` because unlocking from a different thread is not allowed. Those impls are read straight off the pinned std docs.

```
use std::rc::Rc;

let shared = Rc::new(vec![1, 2, 3]);
std::thread::spawn(move || {
    println!("{}", shared.len());
});
// error[E0277]: `Rc<Vec<i32>>` cannot be sent between threads safely
// required by the `Send` bound on the spawn closure
```

fragment, illustrative, not executed

`Arc<T>` is the count-safe sibling, `Send` and `Sync` exactly when `T` is `Send + Sync`, and `Arc<Mutex<T>>` is the idiom for shared mutable state because `Mutex<T>` is `Sync` whenever `T` is `Send`. The sample probes the counts directly: six clones live across six joined threads, each observes a live strong count between 2 and 8, and the count is back to 1 after the joins.

```
    let arc = Arc::new(AtomicUsize::new(0));
    let handles: Vec<_> = (0..6usize)
        .map(|_| {
            let arc = Arc::clone(&arc);
            std::thread::spawn(move || {
                let bump = Arc::clone(&arc);
                bump.fetch_add(1, Ordering::Relaxed);
                Arc::strong_count(&bump) as u64
            })
        })
        .collect();
    let observed: Vec<u64> = handles.into_iter().map(|h| h.join().unwrap()).collect();
    ok(
        observed.iter().all(|&c| (2..=8).contains(&c)),
        "each thread saw a live strong count, at least main plus itself",
    );
    ok(
        Arc::strong_count(&arc) == 1,
        "after every join the strong count is back to 1",
    );
    ok(
        arc.load(Ordering::Relaxed) == 6,
        "six clones each bumped the atomic once",
    );
```

listing 153: Arc counts across joined threads, deterministic bounds (source: rust/samples/src/Ch13/SendSync.rs, lines 59-82)

PITFALL: what safe rust prevents

Safe rust prevents data races: two threads touching the same memory with at least one write and no synchronization. It does not prevent races. Deadlock is fully available, two mutexes taken in opposite order by two threads parks both forever. Lost updates through a check-then-act gap between two locked regions are available too. The compile-time machinery removes the undefined behavior class, logic bugs in the interleaving remain the programmer's, and book 12, ch 9 (design patterns, concurrency, and distributed systems, hazards and detectors) collects the detector side.

The deadlock shape, deliberately unexecuted:

```
let a = &mutex_a;
let b = &mutex_b;

// thread 1: lock a, then b
// thread 2: lock b, then a
// both hold one and wait on the other, forever
// rustc accepts it, the borrow checker has no opinion on order
```

fragment, illustrative, not executed

sources: the rust book concurrency chapter and its four sections (doc.rust-lang.org/stable/book/ch16-00-concurrency.html through ch16-04-extensible-concurrency-sync-and-send.html), std::thread including thread::scope, Scope, and ScopedJoinHandle (doc.rust-lang.org/stable/std/thread/index.html, doc.rust-lang.org/stable/std/thread/fn.scope.html), std::sync with mpsc, Mutex, MutexGuard, RwLock, and Condvar (doc.rust-lang.org/stable/std/sync/index.html and the struct pages under it), the atomic module and Ordering (doc.rust-lang.org/stable/std/sync/atomic/index.html, doc.rust-lang.org/stable/core/sync/atomic/enum.Ordering.html), Send and Sync (doc.rust-lang.org/stable/std/marker/trait.Send.html, doc.rust-lang.org/sta-

ble/std/marker/trait.Sync.html, negative impls read off the Rc, RefCell, MutexGuard, Sender, and Receiver pages), all read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html and accessed 2026-10-13. Version facts verified in doc.rust-lang.org/stable/releases.md, accessed 2026-10-13: thread::scope stabilized in 1.63.0 (2022-08-11), available_parallelism in 1.59.0 (2022-02-24). Measurements (80000 locked and relaxed increments, the publication pattern, store-buffer litmus 891, 4, 168 relaxed against 0 seqcst in 20000 rounds across three runs, available_parallelism 20, Arc counts 2 through 8 across 6 joined threads) produced by `rust/samples/src/Ch13` under `pwsh tools/run-rust-samples.ps1 -Chapter Ch13`, dated 2026-10-13 on the pinned rustc 1.99.0.

14. async: futures from first principles

Async rust is std-defined and runtime-less: the `Future` trait, the waker machinery, and the `async` and `.await` syntax all ship in the language and std since 1.36 and 1.39, and nothing in std schedules them. Every executor is a library, and this chapter builds one from scratch until the pieces are too plain to be mysterious: poll by hand, a waker built over a raw vtable, `block_on` as a loop, `async fn` shown to be a state machine by writing its twin, join and select combinators with the same poll loop, and a timer whose wake comes from a plain thread. Everything here feeds the capstone in chapter 32, which grows this executor into the book's async runtime.

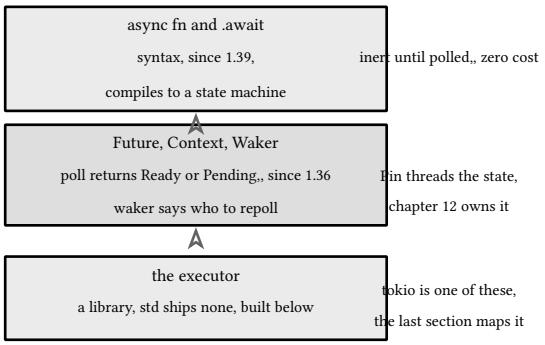


figure 23: async in one picture: syntax on top, the trait in the middle, an executor under it

14.1. the Future trait and poll

The whole contract is one method:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
```

fragment, illustrative, not executed

`Poll` is `Ready(T)` or `Pending`, and `Pending` carries a promise: the future arranged for its waker to be called when poll is worth retrying, or it accepts never being polled again. Futures are inert, polling is a plain function call, and a future that is never polled does no work at all, the sample's first check constructs a countdown and observes zero polls.

```
struct Ready(u64);

impl Future for Ready {
    type Output = u64;

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<u64> {
        Poll::Ready(self.0)
    }
}

struct CountDown {
    remaining: u32,
    polls: u32,
}

impl Future for CountDown {
    type Output = u32;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<u32> {
        let this = &mut *self;
        this.polls += 1;
        if this.remaining == 0 {
            Poll::Ready(0)
        } else {
            this.remaining -= 1;
            Poll::Pending
```

listing 154: two hand written futures, one always ready, one counting down (source: rust/samples/src/Ch14/FuturePoll.rs, lines 28-53)

Driving it by hand is a `loop` over `Pin::new(&mut fut).poll(&mut cx)`. `Pin::new` works here because `CountDown` holds only plain fields and is therefore `Unpin`: the pin exists for state machines that borrow into themselves, chapter 12 owns `Pin` and `Unpin`, this chapter only needs the two working moves, `Pin::new` for `Unpin` types and `pin!` for compiler futures. Measured 2026-10-13: 3 pending polls then a ready, 4 polls total.

```
fn main() {
    let mut lazy = CountDown {
        remaining: 3,
        polls: 0,
    };
    ok(
        lazy.polls == 0,
        "a future is inert, constructing it polls nothing",
    );

    let waker = noop_waker();
    let mut cx = Context::from_waker(&waker);
    let mut pending_total = 0u32;
    let outcome = loop {
        match Pin::new(&mut lazy).poll(&mut cx) {
            Poll::Ready(v) => break v,
            Poll::Pending => pending_total += 1,
        }
    };
    ok(
        outcome == 0 && pending_total == 3 && lazy.polls == 4,
        "hand polling drives the countdown: 3 pending polls then a ready in 4 polls",
    );

    let mut ready = Ready(41);
    let polled = Pin::new(&mut ready).poll(&mut cx);
    ok(
        matches!(polled, Poll::Ready(41)),
        "a ready future resolves on its first poll",
    );
    ok(polled.is_ready(), "Poll::is_ready reports the state");
    ok(
        matches!(Pin::new(&mut Ready(7)).poll(&mut cx), Poll::Ready(_)),
        "a fresh future is reusable state, poll is just a function call",
    );
```

listing 155: hand polling, inertness, and the poll counts (source: rust/samples/src/Ch14/FuturePoll.rs, lines 61-95)

14.2. wakers and a tiny executor

A waker is a cloneable handle that means “poll this task again”. The safe route is the `Wake` trait, stable since 1.51: implement `wake` on an `Arc<T>` and `Waker::from` builds the handle. The sample’s `NoopWaker` is the smallest one possible.

The raw route shows what `Wake` hides: a `RawWaker` is a data pointer plus a `RawWakerVTable` of four function pointers for clone, wake, wake by ref, and drop. Edition 2024 hardened this exact code: an `unsafe fn` body no longer grants its own permissions, the pointer dereference needs its own `unsafe` block, the `unsafe_op_in_unsafe_fn` lint is a hard error in this edition and the sample shows the wrapped shape.

```
fn clone_raw(data: *const ()) -> RawWaker {
    RawWaker::new(data, &VTABLE)
}

unsafe fn wake_raw(data: *const ()) {
    unsafe {
        (*(data as *const AtomicBool)).store(true, Ordering::Release);
    }
}

unsafe fn drop_raw(_data: *const ()) {}

static VTABLE: RawWakerVTable =
    RawWakerVTable::new(clone_raw, wake_raw, wake_raw, drop_raw);

fn flag_waker(flag: &Arc<AtomicBool>) -> Waker {
    unsafe { Waker::from_raw(RawWaker::new(Arc::as_ptr(flag) as *const (), &VTABLE)) }
}

fn block_on<F: Future>(fut: F) -> F::Output {
    let flag = Arc::new(AtomicBool::new(false));
    let waker = flag_waker(&flag);
    let mut cx = Context::from_waker(&waker);
    let mut fut = pin!(fut);
    loop {
        flag.store(false, Ordering::SeqCst);
        match fut.as_mut().poll(&mut cx) {
            Poll::Ready(v) => return v,
            Poll::Pending => {
                while !flag.load(Ordering::SeqCst) {
                    std::hint::spin_loop();
                }
            }
        }
    }
}
```

listing 156: a raw waker over an AtomicBool flag, and block_on as a poll loop (source: rust/samples/src/Ch14/BlockOn.rs, lines 18-54)

The invariant the `unsafe` carries, stated plainly: the vtable's data pointer is `Arc::as_ptr` of the flag, `block_on` owns that `Arc` for its whole body, so the waker outlives every poll it is handed to and the pointer is never dangling. The executor loop is then six lines: clear the flag, poll, spin until the flag is set, repeat. A future that self-wakes, like the sample's `Steps`, walks the flag protocol without any thread at all, and `block_on` drives `async` blocks to a value with no runtime in sight.

```
fn main() {
    let steps = Steps { left: 3, wakes: 0 };
    let value = block_on(steps);
    ok(value == 21, "block_on runs a pending future to completion");
    let result = block_on(Rendezvous);
    ok(
        result == "done",
        "a ready future returns from block_on on the first poll",
    );

    let async_value = block_on(async { 5u64 + 37 });
    ok(
        async_value == 42,
        "block_on drives an async block, no runtime involved",
    );

    let flag = Arc::new(AtomicBool::new(false));
    let waker = flag_waker(&flag);
```

listing 157: block_on running a self waking future, an async block, a ready one (source: rust/samples/src/Ch14/BlockOn.rs, lines 87-104)

14.3. async fn is a state machine

`async fn` and `async` blocks are syntax for a type that implements `Future`, an enum with one variant per suspension point, `.await` is the boundary between variants, and the compiler writes the `poll` that steps the enum. The proof is parity: write the machine by hand, run both against the same dependency, and compare.

```
struct Delayed {
    value: u64,
    pending_left: u32,
    polls: Arc<AtomicU32>,
}

impl Future for Delayed {
    type Output = u64;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<u64> {
        let this = &mut *self;
        this.polls.fetch_add(1, Ordering::SeqCst);
        if this.pending_left == 0 {
            Poll::Ready(this.value)
        } else {
            this.pending_left -= 1;
            Poll::Pending
        }
    }
}

enum AddFuture {
    Start(u64, Delayed),
    Resolved(u64),
}

impl Future for AddFuture {
    type Output = u64;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<u64> {
        let this = self.get_mut();
        loop {
            match this {
                AddFuture::Start(offset, dep) => match Pin::new(dep).poll(cx) {
                    Poll::Pending => return Poll::Pending,
                    Poll::Ready(v) => *this = AddFuture::Resolved(v + *offset),
                },
                AddFuture::Resolved(total) => return Poll::Ready(*total),
            }
        }
    }
}

async fn add_after(dep: Delayed, offset: u64) -> u64 {
    let v = dep.await;
    v + offset
}
```

listing 158: the dependency future, the manual enum machine, and its async fn twin (source: rust/samples/src/Ch14/JoinFutures.rs, lines 25-71)

Both shapes are driven against the same `Delayed` dependency that instruments its own polls. Measured 2026-10-13: identical output 21 and identical 2 pending polls each, the machine is the desugaring.

```
fn main() {
    let waker = Waker::from(Arc::new(Noop));
    let mut cx = Context::from_waker(&waker);

    let polls = Arc::new(AtomicU32::new(0));
    let mut manual = AddFuture::Start(
        1,
        Delayed {
            value: 20,
            pending_left: 2,
            polls: Arc::clone(&polls),
        },
    );
    let mut manual_pending = 0;
    let manual_out = loop {
        match Pin::new(&mut manual).poll(&mut cx) {
            Poll::Ready(v) => break v,
            Poll::Pending => manual_pending += 1,
        }
    };

    let mut compiled = pin!(add_after(
        Delayed {
            value: 20,
            pending_left: 2,
            polls: Arc::clone(&polls),
        },
        1,
    ));
    let mut compiled_pending = 0;
    let compiled_out = loop {
        match compiled.as_mut().poll(&mut cx) {
            Poll::Ready(v) => break v,
```

listing 159: the parity run, manual machine against compiler machine (source: rust/samples/src/Ch14/JoinFutures.rs, lines 116-148)

One asymmetry the compiler enforces and the sample hits: the generated future is `!Unpin`, even when it holds only plain values, so `Pin::new` is rejected and `pin!` (since 1.68) pins it on the stack. That is the entire reason `Pin` leaks into async code at all: an `async fn` that holds a borrow across an await becomes self-referential when parked in its enum, and moving it would break the borrow.

14.4. join, by hand

`join` is polling children and keeping the ready ones until all are in. The combinator below takes two `Unpin` children and their outputs, which keeps pin projection out of the picture, the honest scope here: a general join over pinned children needs structural pin projection, and that machinery belongs to chapter 12 and the pin-project pattern, not to an async primer.

```
struct Join2<A: Future, B: Future> {
    a: A,
    b: B,
    ra: Option<A::Output>,
    rb: Option<B::Output>,
}

impl<A, B> Future for Join2<A, B>
where
    A: Future + Unpin,
    B: Future + Unpin,
    A::Output: Unpin,
    B::Output: Unpin,
{
    type Output = (A::Output, B::Output);

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<(A::Output, B::Output)> {
        let this = self.get_mut();
        if this.ra.is_none() {
            if let Poll::Ready(v) = Pin::new(&mut this.a).poll(cx) {
                this.ra = Some(v);
            }
        }
        if this.rb.is_none() {
            if let Poll::Ready(v) = Pin::new(&mut this.b).poll(cx) {
                this.rb = Some(v);
            }
        }
        match (this.ra.take(), this.rb.take()) {
            (Some(a), Some(b)) => Poll::Ready((a, b)),
            (Some(a), None) => {
                this.ra = Some(a);
                Poll::Pending
            }
            (None, Some(b)) => {
                this.rb = Some(b);
                Poll::Pending
            }
            (None, None) => Poll::Pending,
        }
    }
```

listing 160: Join2, poll both, stash the ready one, complete on the slowest (source: rust/samples/src/Ch14/JoinFutures.rs, lines 73-113)

Measured 2026-10-13: children with 1 and 3 pending polls finish in 4 rounds, the first ready value waits in the `Option` for the last one, which is exactly what `futures::join` and `tokio::join!` do with arbitrary arity.

14.5. timers through a thread and channel bridge

Nothing in std parks a future on a clock, and that is the point: a timer is a thread, a channel, and a waker slot, composed from the pieces this chapter already built. The thread sleeps, sends the delivery instant, then wakes whoever registered. The poll is the double-check pattern done right: try the channel, and only if empty take the lock, try again under the lock, and only then store the waker and return pending. The second try closes the race where the thread fired between the first try and the store, so a wakeup is never lost while a value sits queued.

```
struct Bridge(Mutex<Option<Waker>>);

struct Timer {
    rx: mpsc::Receiver<Instant>,
    bridge: Arc<Bridge>,
}

impl Timer {
    fn after(duration: Duration) -> Timer {
        let (tx, rx) = mpsc::channel();
        let bridge = Arc::new(Bridge(Mutex::new(None)));
        let worker_bridge = Arc::clone(&bridge);
        thread::spawn(move || {
            thread::sleep(duration);
            let _ = tx.send(Instant::now());
            let mut slot = worker_bridge.0.lock().unwrap();
            if let Some(waker) = slot.take() {
                waker.wake();
            }
        });
        Timer { rx, bridge }
    }
}

impl Future for Timer {
    type Output = Instant;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Instant> {
        match self.rx.try_recv() {
            Ok(at) => Poll::Ready(at),
            Err(TryRecvError::Disconnected) => Poll::Pending,
            Err(TryRecvError::Empty) => {
                let mut slot = self.bridge.0.lock().unwrap();
                match self.rx.try_recv() {
                    Ok(at) => Poll::Ready(at),
                    Err(_) => {
                        *slot = Some(cx.waker().clone());
                        Poll::Pending
                    }
                }
            }
        }
    }
}
```

listing 161: the timer bridge, channel for the value, waker slot for the latency (source: rust/samples/src/Ch14/TimerBridge.rs, lines 56-99)

Measured 2026-10-13 under the chapter executor: a 25 ms timer delivered at 25.4958 ms, a 10 ms plus 20 ms pair joined by hand spanned 20.7017 ms. No assert rides on those numbers beyond the semantic floor: delivery is never earlier than the requested duration, which is `thread::sleep`'s own documented guarantee.

14.6. select and cancellation

`select` is polling children until the first ready wins and dropping the rest. The biased version polls arms in written order, which makes its behavior reproducible and makes it the right teaching shape: the sample asserts which arm wins in three schedules, including the tie.

```
struct Select2<A: Future, B: Future> {
    a: A,
    b: B,
}

impl<A, B> Future for Select2<A, B>
where
    A: Future<Output = u64> + Unpin,
    B: Future<Output = u64> + Unpin,
{
    type Output = Arm;

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Arm> {
        let this = &mut *self;
        if let Poll::Ready(v) = Pin::new(&mut this.a).poll(cx) {
            return Poll::Ready(Arm::First(v));
        }
        if let Poll::Ready(v) = Pin::new(&mut this.b).poll(cx) {
            return Poll::Ready(Arm::Second(v));
        }
        Poll::Pending
    }
}
```

listing 162: Select2, biased, first ready wins (source: rust/samples/src/Ch14/SelectFutures.rs, lines 51-73)

Cancellation is the same drop. Completing a select consumes the future and drops the losing arm, and because a future is just a state enum, dropping it deletes the work: the sample's losing arm counts its own polls, was polled once, on the single round where the winner was still pending, and is never polled again after the drop. That is async cancellation, the property the capstone's request timeouts rely on.

```
fn main() {
    let fast_left = drive(Select2 {
        a: step(1, 3),
        b: step(2, 5),
    });
    ok(
        matches!(fast_left, Arm::First(1)),
        "the left arm wins when it is ready first",
    );

    let fast_right = drive(Select2 {
        a: step(1, 5),
        b: step(2, 3),
    });
    ok(
        matches!(fast_right, Arm::Second(2)),
        "the right arm wins when the left is still pending",
    );

    let tie = drive(Select2 {
        a: step(10, 2),
        b: step(20, 2),
    });
    ok(
        matches!(tie, Arm::First(10)),
        "on a simultaneous ready the biased order picks the first arm",
    );
```

listing 163: which arm wins, and the losing arm's polls stopping at the drop (source: rust/samples/src/Ch14/SelectFutures.rs, lines 94-121)

14.7. what tokio adds

Tokio is one executor of the shape just built, scaled up, and every piece of it decomposes into this chapter's primitives. The mapping, per the corpus rule that libraries are explained by what they compose:

The reactor is the waker slot at scale. The timer bridge stored one `Option<Waker>` behind a mutex, tokio's runtime bundles an io driver and a timer whose job is the same registration: os readiness events, epoll style, plus timer wheels, all resolved into `Waker::wake` calls. `Timer::after` above is a one-shot wheel of size one.

The multithreaded scheduler is `block_on` widened from one spin loop to a pool. Tokio's multi-thread scheduler executes tasks on a thread pool with work stealing: idle workers steal half of another worker's local queue, where this chapter's executor runs exactly one task on the calling thread.

`spawn` lifts a future onto that pool, requiring `Send` futures, where `block_on` runs one future inline. `tokio::join!` is the `Join2` above at arbitrary arity, `JoinSet` adds cancellation of the group.

`tokio::select!` is the biased `Select2` with a fairness twist the tokio documents state exactly: by default it randomly picks a branch to check first, and `biased;` mode polls the futures in written order top to bottom, exactly the behavior this chapter's combinator bakes in. Random order protects a loop where one arm is always ready, biased order costs no rng and gives determinism.

Timeouts are select against a timer, the capstone composes exactly this: a request future raced against `Timer::after`, first ready wins, the loser drops, and the drop cancels it. Chapter 32 grows this chapter's executor into that runtime, io bridges included, with std only and zero dependencies.

NOTE: why not just threads

> An os thread idles at megabytes of reserved stack and parks in the kernel, a parked future is a heap enum the size of its state. The chapter 13 stack handles hundreds of threads comfortably, async exists for the tens of thousands of concurrent sockets the api lane in chapter 19 serves. The two compose freely: timers here are threads, and `spawn_blocking` in tokio is a thread pool for the blocking work futures must never do inline.

sources: the async book (doc.rust-lang.org/stable/book/ch17-00-async-await.html plus its futures, concurrency, streams, and traits sections through ch17-06-futures-tasks-threads.html), std::future::Future (doc.rust-lang.org/stable/std/future/trait.Future.html), the std::task module pages for Poll, Context, Waker, RawWaker, RawWakerVTable, and the Wake trait (doc.rust-lang.org/stable/std/task/index.html and doc.rust-lang.org/stable/std/task/trait.Wake.html), std::pin including the pin! macro (doc.rust-lang.org/stable/std/pin/macro.pin.html), all read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html and accessed 2026-10-13. Version facts verified in doc.rust-lang.org/stable/releases.md, accessed 2026-10-13: Future in 1.36.0 (2019-07-04), async and await syntax in 1.39.0, the Wake trait in 1.51.0, the pin! macro in 1.68.0 (2023-03-09). The unsafe-op-in-unsafe-fn hard error observed live compiling the sample under `rustc --edition 2024 -D warnings`, dated 2026-10-13. The tokio mapping quotes verified against docs.rs/tokio/latest/tokio/macro.select.html and docs.rs/tokio/latest/tokio/runtime/index.html, accessed 2026-10-13: random-first select with biased top to bottom mode, work stealing of half a worker's local queue, the bundled io driver and timer. All measurements (4 polls 3 pending countdown, join rounds 4 for 1 and 3 pending polls pending polls, 25 ms timer delivered at 25.4958 ms, pair span 20.7017 ms, losing arm polled once then cancelled) produced by `rust/samples/src/Ch14` under `pwsh tools/run-rust-samples.ps1`
`-Chapter Ch14`, dated 2026-10-13 on the pinned rustc 1.99.0.

15. the standard library

Std is the only dependency this book allows itself, so this chapter walks it the way the api lane will use it: the four core collections with measured behavior, strings and the utf-8 boundary discipline, the io traits, paths, time, localhost sockets, and the environment and process surface. The no-serde stance is deliberate and standing: the http kernel in chapter 19 hand-rolls its json parser and serializer over `str` and `Vec<u8>`, which this chapter's string machinery makes plain, and the capstone builds its runtime over these same pieces. Version facts ride the corpus convention: the hash map swapped its engine for hashbrown in 1.36, `ExitCode` and `Termination` landed in 1.61.

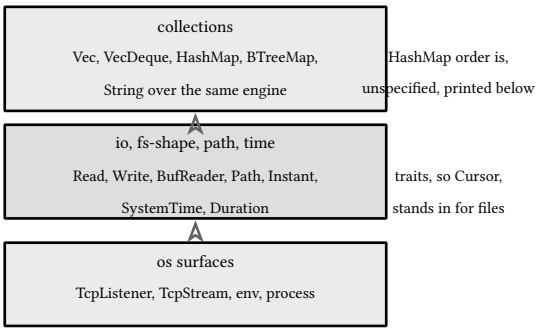


figure 24: std at a glance, the layers this chapter walks

15.1. the core collections

`Vec<T>` is the default container and its growth is amortized doubling. The sample records every capacity transition over 1000 pushes: measured 2026-10-13, the ladder reads 4, 8, 16, 32, 64, 128, 256, 512, 1024, a 256x span over 9 reallocations, and the length lands at exactly 1000. Slicing and iteration rules the corpus leans on: `retain` filters in place, `dedup` only removes adjacent duplicates so sort first, `chunks` yields a short final chunk.

```
    let mut grown = Vec::new();
    let mut capacities: Vec<usize> = Vec::new();
    let mut last = grown.capacity();
    for i in 0..1000u32 {
        grown.push(i);
        if grown.capacity() != last {
            capacities.push(grown.capacity());
            last = grown.capacity();
        }
    }
    ok(grown.len() == 1000, "a vec grows amortized, 1000 pushes all land");
    ok(
        grown.capacity() ≥ grown.len(),
        "capacity always covers the length",
    );
    let ratio = capacities.last().copied().unwrap_or(0) as f64
        / capacities.first().copied().unwrap_or(1) as f64;
    ok(
        capacities.len() ≥ 8,
        "reallocation happened repeatedly, the doublings are observable",
    );
    println!(
        "measured 2026-10-13: capacity ladder over 1000 pushes: {capacities:?}, growth ratio {ratio:.2}"
    );

    let mut v = vec![3u32, 1, 4, 1, 5, 9, 2, 6];
    v.retain(|&x| x % 2 == 1);
    ok(v == vec![3, 1, 1, 5, 9], "retain filters in place");
    v.dedup();
    ok(v == vec![3, 1, 5, 9], "dedup only drops adjacent duplicates");
    v.sort_unstable();
    v.dedup();
    ok(v == vec![1, 3, 5, 9], "sort then dedup removes every duplicate");
    let five = vec![1u32, 2, 3, 4, 5];
    let chunked: Vec<&[u32]> = five.chunks(2).collect();
    ok(
        chunked.len() == 3 && chunked[2] == [5],
        "chunks yields a short final chunk",
    );
```

listing 164: the amortized growth ladder, then retain, dedup, chunks (source: rust/samples/src/Ch15/Collections.rs, lines 16-54)

`VecDeque<T>` is the ring buffer: push and pop at both ends without ever moving the elements, the double-ended queue under every work queue in the book.

```
    let mut ring = VecDeque::with_capacity(4);
    for i in 1..=4u32 {
        ring.push_back(i);
    }
    ok(
        ring.front() == Some(&1) && ring.back() == Some(&4),
        "a VecDeque is a ring, front and back are distinct ends",
    );
    let mut drained = Vec::new();
    while let Some(v) = ring.pop_front() {
        drained.push(v);
    }
    ok(drained == vec![1, 2, 3, 4], "pop_front drains in insertion order");
    ring.push_back(10);
    ring.push_front(20);
    ok(
        ring.front() == Some(&20) && ring.back() == Some(&10),
        "the deque reuses freed slots at both ends",
    );
```

listing 165: a ring, drained front to back, then reused at both ends (source: rust/samples/src/Ch15/Collections.rs, lines 56-74)

`HashMap<K, V>` is a hash table keyed by whatever hashes, backed by the hashbrown engine since 1.36, with `RandomState` seeding SipHash 1-3 per instance: resistant to key-collision attacks, not an ordered structure. The entry api is the upsert idiom, one lookup:

```
    let mut scores: HashMap<&str, u32> = HashMap::new();
    scores.insert("ada", 90);
    let previous = scores.insert("ada", 95);
    ok(
        previous == Some(90) && scores["ada"] == 95,
        "insert on an existing key returns the evicted value",
    );
    *scores.entry("grace").or_insert(0) += 50;
    *scores.entry("ada").or_insert(0) += 5;
    ok(
        scores["grace"] == 50 && scores["ada"] == 100,
        "entry or_insert is the upsert idiom",
    );
    let removed = scores.remove("grace");
    ok(removed == Some(50), "remove yields the stored value");
```

listing 166: insert returns the evicted value, entry or_insert upserts (source: rust/samples/src/Ch15/Collections.rs, lines 76-90)

Iteration order is unspecified and the sample says so with output: two fresh maps holding the same 12 pairs iterated in two different orders, `[6, 9, 11, 4, 2, 3, 0, 7, 8, 10, 5, 1]` and `[1, 2, 4, 7, 10, 8, 0, 9, 3, 11, 6, 5]`, neither sorted, measured 2026-10-13. No assert in this book ever rides on hash order, and different processes disagree because each instance draws its own seed.

```
    let mut map_a: HashMap<u32, u32> = HashMap::new();
    let mut map_b: HashMap<u32, u32> = HashMap::new();
    for k in 0..12u32 {
        map_a.insert(k, k * k);
        map_b.insert(k, k * k);
    }
    let order_a: Vec<u32> = map_a.keys().copied().collect();
    let order_b: Vec<u32> = map_b.keys().copied().collect();
    let same_content: bool = map_a.iter().all(|(k, v)| map_b.get(k) == Some(v));
    let sorted: Vec<u32> = {
        let mut keys: Vec<u32> = map_a.keys().copied().collect();
        keys.sort_unstable();
        keys
    };
    let a_sorted = order_a == sorted;
    ok(
        same_content && map_a.len() == 12,
        "both maps hold exactly the same 12 pairs",
    );
    println!(
        "measured 2026-10-13: hash iteration order, instance a {order_a:?}"
    );
    println!(
        "measured 2026-10-13: hash iteration order, instance b {order_b:?}"
    );
    println!(
        "measured 2026-10-13: instance a in sorted order: {a_sorted}, a and b identical order: {}",
        order_a == order_b
    );
```

listing 167: two instances, same pairs, printed orders, no order asserts (source: rust/samples/src/Ch15/Collections.rs, lines 92-121)

`BTreeMap<K, V>` is the ordered map, a b-tree iterating in key order regardless of insert order, with range queries as tree walks. When the api lane needs deterministic serialization order in chapter 19, this is the map it reaches for.

```
    let mut tree: BTreeMap<u32, &str> = BTreeMap::new();
    for (k, v) in [(5, "e"), (1, "a"), (9, "i"), (3, "c")] {
        tree.insert(k, v);
    }
    let keys: Vec<u32> = tree.keys().copied().collect();
    ok(
        keys == vec![1, 3, 5, 9],
        "a BTreeMap iterates in key order regardless of insert order",
    );
    let ranged: Vec<(u32, &str)> = tree.range(3..=5).map(|(k, v)| (*k, *v)).collect();
    ok(
        ranged == vec![(3, "c"), (5, "e")],
        "range takes a ordered slice of the tree",
    );
    ok(
        tree.first_key_value().map(|(k, v)| (*k, *v)) == Some((1, "a")),
        "first and last are tree walks, not scans",
    );
```

listing 168: ordered iteration, range, and the ends as walks (source: rust/samples/src/Ch15/Collections.rs, lines 122-139)

15.2. strings and utf-8 boundaries

`String` is a `Vec<u8>` holding guaranteed-valid utf-8, `&str` is a borrowed slice of the same invariant. One string holds both a byte length and a code point count and they differ the moment anything is non-ascii: measured, `héllo 日本語` is 16 bytes and 9 chars, with boundaries at 0, 1, 3, 4, 5, 6, 7, 10, 13.

```
    let accented = char::from_u32(0xE9).unwrap();
    let measured: Vec<(char, usize)> = vec![
        ('a', 'a'.len_utf8()),
        (accented, accented.len_utf8()),
        ('日', '日'.len_utf8()),
        ('😀', '😀'.len_utf8()),
    ];
    ok(
        measured == vec![('a', 1), (accented, 2), ('日', 3), ('😀', 4)],
        "utf-8 code points occupy 1 to 4 bytes, measured per char",
    );

    let greeting = String::from("héllo 日本語");
    let bytes = greeting.len();
    let chars = greeting.chars().count();
    ok(
        bytes == 1 + 2 + 1 + 1 + 1 + 1 + 3 + 3 + 3 && bytes == 16 && chars == 9,
        "String::len counts bytes, chars counts code points, 16 against 9 here",
    );

    let offsets: Vec<usize> = greeting.char_indices().map(|(i, _)| i).collect();
    ok(
        offsets == vec![0, 1, 3, 4, 5, 6, 7, 10, 13],
        "char_indices yields the byte offset of every boundary",
    );
    ok(
        first_n_chars(&greeting, 5) == "héllo",
        "slicing at a measured boundary is safe and correct",
    );
    ok(
        first_n_chars(&greeting, 0).is_empty() && first_n_chars(&greeting, 9) == greeting,
        "the boundary walk covers both ends",
    );
```

listing 169: byte lengths per char, both counts, and the boundary walk (source: rust/samples/src/Ch15/Strings.rs, lines 27-59)

Slicing is by byte index, and a byte index inside a multi-byte code point is a panic, the sample measures the exact message:

```
let greeting = "héllo 日本語";
let broken = &greeting[..2];
// panicked at: end byte index 2 is not a char boundary;
// it is inside 'é' (bytes 1..3 of string)
```

fragment, illustrative, not executed

The safe pattern is to cut on boundaries you learned from `char_indices`, which is exactly what the sample's `first_n_chars` does and what the chapter 19 json parser does with every token. Validation reports positions rather than failing opaquely, and `Utf8Error::valid_up_to` is the offset a streaming decoder resumes from:

```
    let valid = from_utf8("ok".as_bytes());
    ok(valid == Ok("ok"), "from_utf8 round trips valid bytes");
    let mut broken = vec![b'a', b'b', 0xFF, b'c'];
    let error = from_utf8(&broken).unwrap_err();
    ok(
        error.valid_up_to() == 2 && error.error_len().is_some(),
        "Utf8Error reports the first invalid byte position",
    );
    broken[2] = b'c';
    ok(
        from_utf8(&broken) == Ok("abcc"),
        "fixing the byte makes the same buffer valid",
    );

    let mut built = String::from("ß");
    let upper: String = built.to_uppercase();
    ok(
        upper == "SS",
        "case mapping is unicode, sharp s uppercases to two chars",
    );
    built.push_str(" and more");
    ok(
        built.ends_with("more") && built.starts_with("ß"),
        "push_str appends whole utf-8 slices",
    );
    built.truncate(2);
    ok(built == "ß", "truncate at a boundary keeps the prefix intact");

    let pieces: Vec<&str> = "a,b,,c".split(',').collect();
    ok(
        pieces == vec!["a", "b", "", "c"],
        "split keeps empty fields, the count is honest",
    );
    let joined = pieces.join("-");
    ok(joined == "a-b--c", "join is the inverse with a separator");

    let words = "the quick brown fox";
    let longest = words.split_whitespace().max_by_key(|w| w.len()).unwrap();
    ok(
        longest == "brown",
        "split_whitespace drops the separators, max_by_key keeps the last of ties",
    );
    println!(
        "measured 2026-10-13: \"héllo 日本語\" is {bytes} bytes and {chars} chars, boundaries at
{offsets:?}"
    );
}
```

listing 170: from_utf8 with positions, unicode case mapping, split and join (source: rust/samples/src/Ch15/Strings.rs, lines 61-106)

Case mapping is unicode, not ascii: sharp s uppercases to two characters, measured as the check asserts. `split` keeps empty fields so csv lines stay positional, `split_whitespace` drops them.

15.3. io: readers and writers

`Read` and `Write` are traits, which is the whole point: a `Vec<u8>` is a writer, a `Cursor<Vec<u8>>` is a reader and a seeker, and the same generic code later runs against files and sockets unchanged. The sample runs the io suite entirely in memory, deterministic to the byte:

```
    let mut sink = Vec::new();
    let written = sink.write(b"hello io").unwrap();
    sink.write_all(b", again").unwrap();
    ok(
        written == 8 && sink == b"hello io, again",
        "Write appends into any sink, a Vec here, no filesystem involved",
    );

    let mut source = Cursor::new(b"line one\nline two\nline three".to_vec());
    let mut text = String::new();
    source.read_to_string(&mut text).unwrap();
    ok(
        text == "line one\nline two\nline three",
        "Read::read_to_string drains the whole cursor",
    );

    let reader = BufReader::new(Cursor::new(b"alpha\nbeta\n\ngamma".to_vec()));
    let mut lines = Vec::new();
    for line in reader.lines() {
        lines.push(line.unwrap());
    }
    ok(
        lines == vec!["alpha", "beta", "", "gamma"],
        "BufReader::lines splits on newlines and keeps the empty line",
    );

    let mut exact = Cursor::new(b"0123456789".to_vec());
    let mut chunk = [0u8; 4];
    exact.read_exact(&mut chunk).unwrap();
    ok(
        chunk == *b"0123",
        "read_exact fills the buffer or fails",
    );

    let mut from = Cursor::new(b"payload".to_vec());
    let mut to = Vec::new();
```

listing 171: write into a vec, read_to_string, BufReader lines, read_exact, io::copy (source: rust/samples/src/Ch15/IoPaths.rs, lines 17-52)

`BufReader` batches reads and splits lines, `read_exact` fills the whole buffer or errors, `io::copy` pumps between any reader and writer and returns the byte count. The socket sample at the bottom of the chapter writes the same `Write` calls against a `TcpStream`.

15.4. paths

`Path` and `PathBuf` are the borrowed-versus-owned pair for filesystem spelling, and they carry `OsStr` pieces, not `str`, so a path survives names that are not valid unicode. Windows semantics parse into components, measured live:

```
    let copied = std::io::copy(&mut from, &mut to).unwrap();
    ok(
        copied == 7 && to == b"payload",
        "io::copy pumps bytes between any Read and Write",
    );

    let doc = Path::new("report.final.typ");
    ok(
        doc.file_name() == Some(std::ffi::OsStr::new("report.final.typ")),
        "file_name is the last component",
    );
    ok(
        doc.extension() == Some(std::ffi::OsStr::new("typ")),
        "extension is after the last dot",
    );
    ok(
        doc.with_extension("pdf") == Path::new("report.final.pdf"),
        "with_extension swaps only the tail",
    );
    ok(
        doc.parent() == Some(Path::new("")),
        "a bare filename's parent is the empty path",
    );

    let joined = Path::new("books").join("rust").join("chapters");
    ok(
        joined == Path::new("books/rust/chapters"),
        "join walks down segments",
    );
    let replaced = joined.join("c:/absolute");
    ok(
        replaced == Path::new("c:/absolute"),
        "joining an absolute path replaces the whole prefix",
    );

    let windows = Path::new(r"C:\Users\lavantien\dev");
    let parts: Vec<Component> = windows.components().collect();
    let names: Vec<String> = parts
        .iter()
        .map(|c| match c {
            Component::Prefix(p) => format!("prefix {:?}", p),
            Component::RootDir => "root".to_string(),
            Component::Normal(n) => n.to_string_lossy().to_string(),
            other => format!("{:?}", other),
        })
        .collect();
    ok(
        parts.len() == 5,
```

listing 172: file_name, extension, with_extension, join, and the component walk (source: rust/samples/src/Ch15/IoPaths.rs, lines 53-100)

Measured 2026-10-13, `C:\Users\lavantien\dev` parses into a disk prefix, the root dir, and three normal components, and `temp_dir` yields `C:\Users\LAVANT~1\AppData\Local\Temp\`. Joining an absolute path replaces the entire prefix, the one join behavior that surprises everyone once. The api lane's router in chapter 19 matches request targets with exactly this component walk.

15.5. time

`Duration` is a plain quantity with checked arithmetic, `Instant` is a monotonic clock for measuring, `SystemTime` is the wall clock with an epoch and no monotonicity promise. The sample converts both ways, checks the overflow-aware ops, and measures a real loop:

```
    let half = Duration::from_millis(500);
    let third = Duration::from_secs(1) / 3;
    ok(
        (half + third).as_millis() == 500 + (third.as_millis()),
        "Duration arithmetic carries sub-ms precision",
    );
    ok(
        Duration::from_secs(2).checked_div(4) == Some(Duration::from_millis(500)),
        "durations divide like quantities, not floats",
    );
    ok(
        Duration::from_secs(3).checked_sub(Duration::from_secs(5)).is_none(),
        "checked_sub refuses negative time",
    );
    ok(
        Duration::from_micros(1_500_000).as_secs_f64() == 1.5,
        "as_secs_f64 converts without losing the fraction",
    );
    ok(
        Duration::try_from_secs_f64(0.25) == Ok(Duration::from_millis(250)),
        "try_from_secs_f64 is the safe direction of float conversion",
    );

    let start = Instant::now();
    let total = checksum(200_000);
    let spent = start.elapsed();
    ok(
        checksum(0) == 0 && checksum(200_000) == total,
        "the timed body is real deterministic work, not an optimized away loop",
    );
    ok(
        spent > Duration::ZERO,
        "elapsed reports strictly positive time for real work",
    );
    let per_op_ns = spent.as_nanos() as f64 / 200_000.0;
    println!(
        "measured 2026-10-13: 200000 loop iterations in {:?}, {:.2} ns per iteration, checksum {total}",
```

listing 173: duration arithmetic, a measured loop, ns per iteration (source: rust/samples/src/Ch15/Time.rs, lines 24-60)

Measured 2026-10-13: 200000 iterations of a wrapping checksum loop in 837 microseconds, 4.18 ns per iteration, and 1791890295 seconds since the unix epoch at the sample's run. The chapter 18 bench discipline is this pattern with more rigor.

```
    );

    let later = Instant::now();
    ok(later ≥ start, "Instants order monotonically");
    ok(
        later.duration_since(start) ≥ Duration::ZERO,
        "duration_since is the non panicking difference",
    );
    let stepped = start.checked_add(Duration::from_secs(1));
    ok(
        stepped.is_some() && stepped.unwrap() > later,
        "checked_add shifts an instant forward",
    );

    let now = SystemTime::now();
    let since_epoch = now.duration_since(UNIX_EPOCH).unwrap();
    ok(
        since_epoch > Duration::from_secs(1_700_000_000),
        "the epoch distance is past 2023, more than 1.7 billion seconds",
    );
    let stamp = now
        .checked_add(Duration::from_secs(60))
        .map(|future| future.elapsed());
    ok(
        stamp.is_some(),
        "SystemTime adds fixed offsets with overflow checks",
    );
    println!(
        "measured 2026-10-13: {:.0} seconds since the unix epoch",
        since_epoch.as_secs_f64()
    );
}
```

listing 174: instant ordering, checked_add, and the wall clock (source: rust/samples/src/Ch15/Time.rs, lines 62-93)

15.6. sockets: a localhost echo pair

`TcpListener::bind` takes an address and port, and port 0 asks the os for an ephemeral port, which is how every sample in this corpus touches the network: no fixed ports, no collisions between sibling chapters or parallel test runs. The pair below is a complete echo server and four concurrent clients, joined before a single assert:

```
fn serve_one(mut stream: TcpStream) -> std::io::Result<usize> {
    let mut buffer = [0u8; 256];
    let read = stream.read(&mut buffer)?;
    stream.write_all(&buffer[..read])?;
    stream.flush()?;
    Ok(read)
}

fn request(addr: &std::net::SocketAddr, payload: &[u8]) -> std::io::Result<Vec<u8>> {
    let mut stream = TcpStream::connect(addr)?;
    stream.write_all(payload)?;
    stream.shutdown(std::net::Shutdown::Write)?;
    let mut received = Vec::new();
    stream.read_to_end(&mut received)?;
    Ok(received)
}
```

listing 175: serve_one and request, the whole protocol in two functions (source: rust/samples/src/Ch15/EchoPair.rs, lines 16-32)

```
fn main() {
    let listener = TcpListener::bind(("127.0.0.1", 0)).expect("ephemeral bind");
    let server_addr = listener.local_addr().expect("local addr");
    ok(
        server_addr.ip().is_loopback() && server_addr.port() != 0,
        "binding port 0 hands back a real ephemeral port",
    );
    println!(
        "measured 2026-10-13: ephemeral listener on {server_addr}"
    );

    thread::scope(|s| {
        let clients: Vec<_> = ["alpha", "beta", "gamma", "delta"]
            .iter()
            .map(|word| {
                s.spawn(move || request(&server_addr, word.as_bytes()))
            })
            .collect();
        let mut served = Vec::new();
        for stream in listener.incoming().take(4) {
            served.push(serve_one(stream.expect("accepted")).expect("echoed"));
        }
        let replies: Vec<_> = clients
            .into_iter()
            .map(|handle| handle.join().unwrap().unwrap())
            .collect();
        let echoes: Vec<String> = replies
            .iter()
            .map(|bytes| String::from_utf8_lossy(bytes).to_string())
            .collect();
        ok(
            echoes == ["alpha", "beta", "gamma", "delta"],
            "four concurrent clients each got their exact payload echoed",
        );
        ok(
            served.iter().all(|&n| n > 0) && served.iter().sum::<usize>() == 19,
            "the server read 19 request bytes across four connections",
        );
    });

    thread::scope(|s| {
        let fifth = s.spawn(|| {
            let stream = listener.incoming().next().expect("fifth accepted");
            serve_one(stream.expect("stream")).expect("echoed")
        });
```

listing 176: ephemeral bind, four concurrent clients, then a fifth sequential (source: rust/samples/src/Ch15/EchoPair.rs, lines 34-78)

Measured 2026-10-13: the listener landed on 127.0.0.1:55666, four scoped clients each received their exact payload back, 19 request bytes total, and a fifth sequential connection confirms the acceptor outlives the scope. `shutdown(Write)` is the half-close that turns `read_to_end` into a bounded wait, the same discipline the http kernel's client side uses in chapter 19.

15.7. environment and processes

`env::args` starts at argv 0, the program's own path. `set_var` and `remove_var` are `unsafe` since edition 2024: the process environment is genuinely global state and mutating it from a multithreaded program is undefined territory on

some platforms, so the edition makes the caller state the single-threaded discipline. The sample does its mutation first, before spawning anything:

```
    unsafe {
        std::env::set_var("RUST_W1D_PROBE", "written by the sample");
    }
    let read_back = std::env::var("RUST_W1D_PROBE");
    ok(
        read_back.as_deref() == Ok("written by the sample"),
        "set_var then var round trips, the call is unsafe in edition 2024",
    );
    unsafe {
        std::env::remove_var("RUST_W1D_PROBE");
    }
    ok(
        std::env::var_os("RUST_W1D_PROBE").is_none(),
        "remove_var deletes the entry",
    );

    let raw: Vec<String> = std::env::args().collect();
    ok(
        raw.len() ≥ 1 && !raw[0].is_empty(),
        "args always carries at least the program name in argv zero",
    );
    let os_args = std::env::args_os().count();
    ok(
        os_args == raw.len(),
        "args and args_os agree on the count, os strings never fail",
    );
    let vars_count = std::env::vars().count();
    ok(
```

listing 177: set_var round trip under unsafe, argv and vars counts (source: rust/samples/src/Ch15/EnvArgs.rs, lines 18-45)

Exit codes close the loop between processes. `main` can return a `Result` since 1.61 through the `Termination` trait, or call `process::exit` directly, and a parent reads the code back from `Command::status`. The sample proves the whole round trip by spawning its own exe twice with different arguments, once to exit 7 and once to exit 0:

```
        "the environment holds at least the system entries",
    );
    println!(
        "measured 2026-10-13: argv has {os_args} entries, environment holds {vars_count} entries"
    );

    if let Some(arg) = raw.get(1) {
        if arg == CHILD_FLAG {
            std::process::exit(7);
        }
        std::process::exit(0);
    }

    let exe = std::env::current_exe().unwrap();
    let code = ExitCode::from(0u8);
    ok(
        code == ExitCode::SUCCESS,
        "ExitCode::from(0) is the success code",
    );

    let failing = Command::new(&exe)
        .arg(CHILD_FLAG)
        .stdout(Stdio::null())
        .stderr(Stdio::null())
        .status()
        .unwrap();
    ok(
        failing.code() == Some(7),
        "a child's exit code crosses back through status",
    );

    let exit_code_child = Command::new(&exe)
        .arg("definitely-not-a-flag")
        .stdout(Stdio::null())
        .stderr(Stdio::null())
        .status()
        .unwrap();
    ok(
        exit_code_child.code() == Some(0),
        "the default child run exits zero, the checks all passed",
    );

    let missing = Command::new("definitely-not-a-real-binary-w1d").status();
    ok(
        missing.is_err(),
        "spawning a missing binary is an Err, not a panic",
    );
    println!(
        "measured 2026-10-13: self spawned twice, exit codes 7 then 0 observed"
    );
}
```

listing 178: child mode, self spawn, exit codes 7 then 0, missing binary as Err (source: rust/samples/src/Ch15/EnvArgs.rs, lines 47-97)

Measured 2026-10-13: argv carried 1 entry under the runner, the environment held 112 entries, both self-spawns reported their exact codes, and spawning a nonexistent binary is an `Err` rather than a panic. The integration suite in chapter 31 drives the shipped server binary through exactly this `Command` surface.

sources: the std api docs read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/ rust/html, accessed 2026-10-13: the collections module (doc.rust-lang.org/stable/std/collections/index.html with the Vec, VecDeque, HashMap, and BTreeMap pages, including the hashbrown note and the SipHash 1-3 RandomState statement on the HashMap page), std::str and std::string (doc.rust-lang.org/stable/std/primitive.str.html and /std/string/struct.String.html), std::io (doc.rust-lang.org/stable/std/io/index.html with Read, Write, BufReader, and the copy function), std::path (doc.rust-lang.org/stable/std/path/index.html with Path, PathBuf, and Components), std::time (doc.rust-lang.org/stable/std/time/index.html with Duration, Instant, and SystemTime), std::net (doc.rust-lang.org/stable/std/net/index.html with TcpListener and TcpStream), std::env (doc.rust-lang.org/stable/std/env/index.html, set_var read with its unsafe signature), and std::process (doc.rust-lang.org/stable/std/process/index.html with Command and ExitCode). Version facts verified in doc.rust-lang.org/stable/releases.md, accessed 2026-10-13: hashbrown replacing the HashMap implementation in 1.36.0 (2019-07-04), ExitCode and Termination in 1.61.0 (2022-05-19), and the edition 2024 unsafe env mutation verified in the edition guide (doc.rust-lang.org/stable/edition-guide/rust-2024/newly-unsafe-functions.html). All measurements (capacity ladder 4 through 1024, hash orders of two 12 key instances both non sorted and mutually different, 16 bytes and 9 chars with boundaries, 837 us and 4.18 ns per iteration, 1791890295 epoch seconds, listener on 127.0.0.1:55666 with 19 echoed bytes, argv 1 entry, 112 environment entries, exit codes 7 then 0, the utf-8 panic message text) produced by `rust/samples/src/Ch15` under `pwsh tools/run-rust-samples.ps1 -Chapter Ch15`, dated 2026-10-13 on the pinned rustc 1.99.0.

16. crates, modules, and workspaces

A package is a cargo directory with a manifest. A crate is one compilation unit, a library or a binary. A module is a namespace inside a crate. Cargo's job is the first two, the compiler's job is the third, and most confusion is just those three words being used interchangeably. This chapter walks the module system as one file, the visibility ladder item by item, the manifest of this book's own crate shape field by field, and the cargo test organization the corpus runner mirrors. The book's law sits over all of it: the api crate is one bin plus lib crate with zero dependencies, everything it needs is std, and that crate arrives in chapter 19.

16.1. modules and paths

`mod` declares a namespace, inline or in its own file. Every item has a path from the crate root, spelled `crate::` for absolute, `self::` for the current module, `super::` for the parent. The sample runs one module tree through all three spellings and asserts they agree:

```rust
mod network {
    pub fn describe() -> String {
        let reachable = super::network::tcp_status();
        format!("network up: {reachable}")
    }

    pub mod tcp {
        pub fn connect(host: &str) -> String {
            let id = frame_checksum(host);
            format!("tcp://{host}#{id}")
        }

        pub(crate) fn probe(host: &str) -> bool {
            !host.is_empty()
        }

        pub(super) fn status() -> &'static str {
            "listening"
        }

        fn frame_checksum(host: &str) -> u64 {
            host.bytes().map(|b| b as u64).sum()
        }

        pub struct Session {
            pub peer: String,
            id: u64,
        }

        impl Session {
            pub fn open(host: &str) -> Session {
                Session {
                    peer: host.to_string(),
                    id: frame_checksum(host),
                }
            }

            pub fn id(&self) -> u64 {
                self.id
            }
        }
    }

    fn tcp_status() -> &'static str {
        tcp::status()
    }
}
```

listing 179: one module tree carrying the whole visibility ladder (source: rust/samples/src/Ch16/Modules.rs, lines 13-59)

The ladder, rung by rung. `pub` is visible everywhere the crate is. `pub(crate)` is visible inside this compilation unit only, the right default for internals a binary crate never exports anywhere anyway. `pub(super)` is visible in the parent module, used when a child hands its parent a hook and nobody else. Plain private is visible in the defining module and its descendants, which is why `frame_checksum` runs inside `tcp` but not from `network`. Fields ride the same ladder: `Session` has a public `peer` and a private `id` behind a getter, construction through `Session::open` so the invariant survives.

```
fn main() {
    let via_absolute = network::tcp::connect("example.org");
    let via_use = {
        use crate::network::tcp;
        tcp::connect("example.org")
    };
    let via_self = {
        use self::network::tcp;
        tcp::connect("example.org")
    };
    ok(
        via_absolute == via_use && via_use == via_self,
        "absolute crate paths, use imports, and self relative paths agree",
    );
    ok(
        via_absolute == "tcp://example.org#1122",
        "the private checksum feeds the public output, 1122 measured",
    );

    ok(
        network::tcp::probe("host") && !network::tcp::probe(""),
        "pub(crate) items reach the whole crate from the root",
    );
    ok(
        network::describe() == "network up: listening",
        "pub(super) reaches the parent module, which re-exposes it publicly",
    );

    let session = network::tcp::Session::open("example.org");
    ok(
        session.peer == "example.org" && session.id() == 1122,
```

listing 180: absolute, use imported, and self relative paths agreeing, and the ladder probed (source: rust/samples/src/Ch16/Modules.rs, lines 61-91)

Two compile errors this sample deliberately walks around, worth seeing in their reject form. A private item from outside its module:

```
network::tcp::frame_checksum("host")
// error[E0603]: function `frame_checksum` is private
```

fragment, illustrative, not executed

And `pub(super)` from above the super in question, `tcp::status` is visible to `network` but not to the crate root:

```
network::tcp::status()
// error[E0603]: function `status` is private
// the pub(super) on tcp::status stops at network
```

fragment, illustrative, not executed

Modules nest anywhere, including inside a function body, and `super` always means exactly one level up:

```
    );

    let nested = {
        mod pair {
            pub fn left() -> &'static str {
                "left"
            }
            pub mod right {
                pub fn sees_sibling() -> &'static str {
                    super::left()
                }
            }
        }
        pair::right::sees_sibling()
    };
    ok(
        nested == "left",
        "super walks one module up, modules nest inside any block",
    );
```

listing 181: a sibling call through super, inside a block scoped module (source: rust/samples/src/Ch16/Modules.rs, lines 93-111)

The file-per-module layout is the same tree on disk: `mod tcp` inside `network` becomes `network/mod.rs` with `pub mod tcp` plus `network/tcp.rs`, or `network.rs` beside a `network/` directory, both shapes are legal and the 2018 edition removed the need for `extern crate` and the `mod` declarations for `src/lib.rs` and `src/main.rs` themselves. The api crate uses the directory form for its sections, router, store, and the rest, one file per module.

16.2. use and re-exports

`use` brings a path in once so call sites stop spelling it, `as` renames on the way, `pub use` re-exports so a facade module becomes the public face over deep internals. The sample builds the pattern the api crate's `Server` type will use in chapter 19:

```
        format!("{label}={value}")
    }

    pub fn table(rows: &[(&str, u64)]) -> String {
        rows.iter()
            .map(|(k, v)| line(k, *v))
            .collect::<Vec<_>>()
            .join(",")
    }

    pub struct Report {
        pub title: String,
        pub rows: Vec<(String, u64)>,
    }

    impl Report {
        pub fn render(&self) -> String {
            let pairs: Vec<(&str, u64)> =
                self.rows.iter().map(|(k, v)| (k.as_str(), *v)).collect();
            format!("{}:{}", self.title, table(&pairs))
        }
    }
}

mod facade {
    pub use crate::format_impl::line as row;
    pub use crate::format_impl::{line, table};
    pub use crate::format_impl::Report;

    pub fn combined(rows: &[(&str, u64)]) -> String {
        format!("({})", table(rows))
    }
}
```

listing 182: impl module, then a facade re-exporting it under two spellings (source: rust/samples/src/Ch16/UseReexport.rs, lines 17-49)

```
fn main() {
    ok(
        crate::format_impl::line("rps", 1200) == "rps=1200",
        "the deep path works, fully spelled",
    );
    ok(
        facade::line("rps", 1200) == format_impl::line("rps", 1200),
        "pub use re-exports the deep path at the facade",
    );
    ok(
        facade::row("lat", 99) == "lat=99" && facade::row("a", 1) == facade::line("a", 1),
        "use ... as renames on re-export without cloning the logic",
    );

    use facade::{combined, table as flat};
    ok(
        flat(&[("a", 1), ("b", 2)]) == "a=1,b=2",
        "use with an as binding shortens call sites",
    );
    ok(
        combined(&[("a", 1)]) == "(a=1)",
        "the facade composes its own re-exported items",
    );

    let report = facade::Report {
        title: "summary".to_string(),
        rows: vec![(("ok").to_string(), 51u64), ("checks".to_string(), 74)],
    };
    ok(
        report.render() == "summary:ok=51,checks=74",
        "types re-export too, the facade is the only import a user needs",
    );

    use facade::*;
    ok(
        line("x", 7) == "x=7" && table(&[("x", 7)]) == "x=7",
        "a glob import pulls the whole facade surface",
    );
```

listing 183: deep path, facade path, renamed re-export, use with as, types too (source: rust/samples/src/Ch16/UseReexport.rs, lines 51-89)

The rule that makes facades work: `pub use` does not copy code, it publishes a path. `facade::line` and `format_impl::line` are the same item, one function, one symbol, and a glob `use facade::*` pulls the whole published surface. Std items rename identically, the sample binds `BTreeMap as OrderedMap` and uses it.

16.3. the manifest, field by field

The crate this book ships is one package with one lib and one bin, and its manifest is small enough to walk completely:

```
[package]
name = "rust"
version = "1.0.0"
edition = "2024"
```

```
[lib]
name = "rust"
path = "src/lib.rs"

[[bin]]
name = "rust-api"
path = "src/main.rs"

[dependencies]

[dev-dependencies]
```

fragment, illustrative, not executed

Field by field. `name` is the crate name other code imports, kebab or snake, hyphens become underscores on use. `version` is semver, the caret range cargo derives from it is why lockfiles matter. `edition` picks the dialect, 2024 for this book, chapter 2 owns the ladder. `[lib]` names the library target, `[[bin]]` an array table because a package may hold several binaries, each with its own name and entry file. `[dependencies]` is empty here and stays empty, the zero-dependency law, which also means no supply chain, no version drift, and no audit surface beyond the toolchain pin. A `[dev-dependencies]` section would carry test-only crates, this book has none.

Workspaces, in prose: a root manifest with `[workspace] members = [\"api\", \"tools\"]` shares one `Cargo.lock`, one `target/`, and one dependency graph across member crates, so two crates in the repo cannot disagree on a version. This book has one crate, so the workspace question never arises, but the corpus's repo is exactly that shape if a second crate ever lands.

Features, in prose: `[features]` declares named cfg switches, the `default` list, and dependencies can be optional behind a feature. The api crate declares none: every feature is a compile-time fork of the code under test, and a zero-dependency service has nothing to switch.

16.4. cargo test organization

Three kinds of tests live in three places. Unit tests sit next to the code in `src/`, inside `#[cfg(test)] mod tests` with `use super::*`, compiled only under `cargo test`. Integration tests live in `tests/`, each file its own crate linking the library as a user would. Doc tests live in fenced rust blocks inside doc comments, extracted and run by `rustdoc --test`. The shape:

```
        .filter_map(|c| {
            if c.is_ascii_alphanumeric() {
                Some(c.to_ascii_lowercase())
            } else if c == ' ' || c == '-' || c == '_' {
                None
            } else {
                None
            }
        })
        .collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn slug_drops_punctuation() {
        assert_eq!(slug("Hello, World!"), "helloworld");
    }

    #[test]
    fn slug_lowercases() {
        assert_eq!(slug("RUST"), "rust");
    }
}
```

listing 184: the cfg(test) module shape, exactly as src unit tests are written (source: rust/samples/src/Ch16/TestOrg.rs, lines 15-40)

The corpus runner compiles each sample with plain `rustc --edition 2024 -D warnings`, no `--test`, so the module is compiled out and the checks are ok lines instead. The sample proves the gate live: `cfg!(test)` is false, and the `#[test]` fns have no symbols at all under a plain compile, measured 2026-10-13 by this file compiling and running. Chapter 18 owns the full testing treatment, this section fixes the layout only.

```
fn main() {
    ok(
        slug("Hello, World!") == "helloworld",
        "the unit test's expectation, run by the ok line instead",
    );
    ok(
        cfg!(test) == false,
        "this file runs under plain rustc, the cfg(test) module is compiled out",
    );
    ok(
        slug("Rate Limit — 99") == "ratelimit99",
        "spaces, dashes, and em dashes all collapse",
    );
    let compiled_in: Option<fn()> = None;
    ok(
```

listing 185: the same expectations as ok lines, and the cfg gate probed (source: rust/samples/src/Ch16/TestOrg.rs, lines 42-56)

NOTE: the ok line contract

Every sample in this book prints `ok N name` per passing check and exits nonzero on the first failure, which is what `tools/run-rust-samples.ps1` counts. The discipline predates cargo here: one file, one binary, zero setup,

> the same contract as the java and c sample runners in this corpus. The api crate's own suite in chapter 31 runs under `cargo test` with real test fns, both shapes coexist because the layout above is identical in both.

sources: the rust book packages and modules chapter (doc.rust-lang.org/stable/book/ch07-00-managing-growing-projects-with-packages-crates-and-modules.html through ch07-05-separating-modules-into-different-files.html), the rust reference items on modules and visibility (doc.rust-lang.org/stable/reference/items/modules.html), and the cargo book manifest reference and tests organization pages (doc.rust-lang.org/stable/cargo/reference/manifest.html, doc.rust-lang.org/stable/cargo/guide/tests.html), all read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html and accessed 2026-10-13. The edition 2018 path and extern crate changes verified against doc.rust-lang.org/stable/edition-guide/rust-2018/module-system/index.html, same share, accessed 2026-10-13. The cfg(test) compiled out under plain rustc, the private and pub(super) rejection messages, and the example.org checksum of 1122 measured live compiling and running `rust/samples/src/Ch16` under `pwsh tools/run-rust-samples.ps1 -Chapter Ch16`, dated 2026-10-13 on the pinned rustc 1.99.0, 3 files 18 checks.

17. unsafe and ffi

unsafe is not a switch that disables checking, it is a list of five operations the compiler cannot verify on its own, everything else stays checked. The honest framing: unsafe rust exists because someone must implement the safe abstractions, the standard library itself is full of it, and the discipline is to keep it small, keep it behind a safe interface, and write down the contract the unsafe block is carrying. This chapter takes the five superpowers one by one, builds split_at_mut's trick by hand, handles uninitialized memory with MaybeUninit, transmutes only under repr(C), and measures what actually links when calling into the c runtime on the msvc lane.

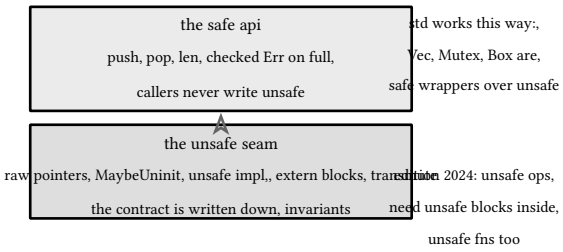


figure 25: unsafe is a narrow seam under a safe surface

17.1. the five superpowers

The list, from the book's advanced features chapter: dereference raw pointers, call unsafe functions or methods, implement unsafe traits, access or mutate mutable statics, access union fields. Superpower 1 leads:

```
fn main() {
    let own = 7u64;
    let shared: *const u64 = &own;
    let raw_view: *const u64 = &raw const own;
    ok(
        shared == raw_view,
        "coercion and the raw address operator, since 1.82, see the same address",
    );

    let read = unsafe { *shared };
    ok(read == 7, "dereferencing a raw pointer needs one unsafe block");

    let mut cell = 10u64;
    let writer = &raw mut cell;
    unsafe {
        *writer += 5;
    }
    ok(cell == 15, "a raw mut writes through, the borrow checker saw nothing");

    let null: *const u64 = std::ptr::null();
    ok(
        null.is_null() && std::ptr::null_mut::<u64>().is_null(),
        "null exists as a value, is_null is the test, no null deref here",
    );

    let base = &own as *const u64;
    let arr = [10u64, 20, 30];
    let first = &arr[0] as *const u64;
    let second = unsafe { first.add(1) };
```

listing 186: raw pointers from coercion and the raw address operator, null, deref (source: rust/samples/src/Ch17/RawPointers.rs, lines 13-41)

*const T and *mut T are the types: they can be null, they can alias, they can outlive what they point at, and copying them is safe, only dereferencing is not. References coerce to raw pointers freely. The &raw const and &raw mut operators, since 1.82, take an address without creating even a momentary reference, the safe tool for places like packed fields and static muts where a borrow would already be a violation. Pointer arithmetic, the as_ref conversion, and the copy primitives:

```
    ok(
        unsafe { *second } == 20 && unsafe { *third } == 30,
        "add walks in element units, bytes are scaled away",
    );
    ok(
        unsafe { second.sub(1) } == first,
        "pointer equality is address equality",
    );

    let ok_ref = unsafe { base.as_ref() };
    let dangling_ref = unsafe { null.as_ref() };
    ok(
        ok_ref == Some(&7) && dangling_ref.is_none(),
        "as_ref returns Option, null maps to None",
    );

    let src = [1u8, 2, 3, 4];
    let mut dst = [0u8; 4];
    unsafe {
        std::ptr::copy_nonoverlapping(src.as_ptr(), dst.as_mut_ptr(), 4);
    }
    ok(
        dst == [1, 2, 3, 4],
        "copy_nonoverlapping is the memcpy primitive under the safety proof",
    );

    let offset_in_bounds = {
        let buffer = [0u32; 8];
        let start = &buffer[0] as *const u32;
        let mid = unsafe { start.add(4) };
        unsafe { start.offset_from(mid) }
    };
    ok(
        offset_in_bounds == -4,
        "offset_from counts elements, negative backwards",
    );
    println!(
        "measured 2026-10-13: size_of::<u64> is {}, align_of is {}",
        std::mem::size_of::<u64>(),
        std::mem::align_of::<u64>()
    );
}
```

listing 187: element-scaled arithmetic, Option-returning as_ref, copy_nonoverlapping, offset_from (source: rust/samples/src/Ch17/RawPointers.rs, lines 43-84)

The remaining superpowers in one run. An unsafe fn marks its own body safe by default in edition 2024, the unsafe operations inside still need explicit blocks, the unsafe_op_in_unsafe_fn rule this book compiles under with -D warnings. A union overlays its fields on one memory, and reading a field reinterprets the bytes. An unsafe impl asserts a marker trait the compiler cannot derive, here Send for a struct holding a raw pointer, with the thread

join proving the claim was load-bearing. A `static mut` is the whole program's shared memory, sound only under a discipline you state yourself:

```
unsafe fn bump_and_read() -> u64 {
    unsafe {
        PULSES += 1;
        PULSES
    }
}

union Frame {
    as_u64: u64,
    as_bytes: [u8; 8],
}

struct RawSendable {
    raw: *const u8,
}

unsafe impl Send for RawSendable {}
```

listing 188: unsafe fn wrapping static mut, a union, and a Send assertion (source: rust/samples/src/Ch17/UnsafePowers.rs, lines 17-33)

17.2. split_at_mut, the trick in the standard library

`slice::split_at_mut` cannot be written with safe borrows: the borrow checker sees two mutable borrows of the same slice. The real implementation escapes to raw pointers and comes back, and the round trip is sound because the halves are provably disjoint:

```
fn my_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let ptr = slice.as_mut_ptr();
    let len = slice.len();
    assert!(mid ≤ len);
    unsafe {
        (
            slice::from_raw_parts_mut(ptr, mid),
            slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}
```

listing 189: split_at_mut rebuilt, one raw pointer, two disjoint slices (source: rust/samples/src/Ch17/UnsafePowers.rs, lines 35-45)

```
fn main() {
    let pulses = unsafe { bump_and_read() };
    let again = unsafe { bump_and_read() };
    ok(
        pulses == 1 && again == 2,
        "superpowers 2 and 4, an unsafe fn called, a static mut mutated inside it",
    );

    let frame = Frame {
        as_u64: 0x0000_0000_0041_4243,
    };
    let bytes = unsafe { frame.as_bytes };
    ok(
        bytes[..3] == [0x43, 0x42, 0x41],
        "superpower 5, union field reads reinterpret the bytes, little endian measured",
    );

    let value = 5u8;
    let carrier = RawSendable {
        raw: &value as *const u8,
    };
    let sent = std::thread::spawn(move || {
        let carried = carrier;
        (unsafe { *carried.raw }) as u64
    });
    let read = sent.join().unwrap();
    ok(
```

listing 190: superpowers exercised, and both halves live and writable at once (source: rust/samples/src/Ch17/UnsafePowers.rs, lines 47-73)

The contract carried by those four lines of unsafe: `mid ≤ len` is asserted, `from_raw_parts_mut` gets a pointer plus a length that stay in bounds, and the two regions do not overlap, so the two `&mut` slices handed out can never alias. That is the entire content of the safe abstraction every caller inherits.

17.3. uninitialized memory and transmute

Reading uninitialized memory is instant undefined behavior, so safe code cannot even name it: `MaybeUninit<T>` is the type that can hold a `T` without there being one yet, same size, same alignment, no tag, measured 8 and 8 for `u64`. The discipline is rigid: write before any read, `assume_init` only over a full initialized `T`, `assume_init_read` to move out. The array pattern below initializes element by element and reinterprets the whole buffer in one step:

```
fn init_array(n: usize, value: u64) -> Box<[u64]> {
    let mut buffer: Box<[MaybeUninit<u64>]> = vec![MaybeUninit::uninit(); n].into_boxed_slice();
    for (i, slot) in buffer.iter_mut().enumerate() {
        slot.write(value + i as u64);
    }
    let filled = unsafe {
        let raw = Box::into_raw(buffer) as *mut [u64];
        Box::from_raw(raw)
    };
    filled
}
```

listing 191: the MaybeUninit array pattern, element wise writes then one conversion (source: rust/samples/src/Ch17/MaybeUninit.rs, lines 27-37)

`transmute` reinterprets bytes between types with the same size and neither carrying invariants the other lacks. Under `repr(Rust)` it is essentially always wrong: the compiler may reorder fields and change padding at will, so two identical-looking structs have no guaranteed shared layout, the nomicon's own example. `repr(C)` pins the layout, and even then padding bytes are unspecified: measured 2026-10-13, `Key { kind: u16, index: u32 }` and `Wire { index: u32, kind: u16 }` transmute to `0x0000beef2bcf0007` and `0x000000070000beef`, the same field values, different field order, and garbage `2bcf` sitting in Key's padding. A byte comparison over transmuted structs is wrong even when the layout is pinned, and under `-D warnings` a transmute with a safe equivalent, like `u64` to `[u8; 8]` where `to_le_bytes` exists, is rejected by the `unnecessary_transmutes` lint outright:

```
fn main() {
    let mut slot: MaybeUninit<u64> = MaybeUninit::uninit();
    ok(
        MaybeUninit::<u64>::uninit().as_ptr() as usize % std::mem::align_of::<u64>() == 0
            && std::mem::size_of::<MaybeUninit<u64>>() == std::mem::size_of::<u64>(),
        "MaybeUninit has the same size and alignment as its type, no tag",
    );
    slot.write(41);
    let stepped = unsafe { slot.assume_init() } + 1;
    slot.write(stepped);
    let value = unsafe { slot.assume_init() };
    ok(
        value == 42,
        "write then assume_init in pairs, each read paired with a write",
    );

    let filled = init_array(5, 100);
    ok(
        filled.iter().enumerate().all(|(i, &v)| v == 100 + i as u64),
        "an uninit buffer filled element wise becomes a valid slice",
    );

    let key = Key {
        kind: 7,
        index: 0xBEEF,
    };
    let words = unsafe { transmute::<Key, u64>(key) };
    let rebuilt: Key = unsafe { transmute::<u64, Key>(words) };
    ok(
        rebuilt.kind == 7 && rebuilt.index == 0xBEEF,
        "repr(C) pins the layout, transmute round trips through u64",
    );
    let wire = Wire {
        index: 0xBEEF,
        kind: 7,
    };
    let wire_words = unsafe { transmute::<Wire, u64>(wire) };
    ok(
        wire_words ≠ words,
        "same fields, different repr(C) order, different bits, measured",
    );
    println!(
        "measured 2026-10-13: Key bits {words:#018x}, Wire bits {wire_words:#018x}, sizes {} {}",
        std::mem::size_of::<Key>(),
        std::mem::size_of::<Wire>()
    );
}
```

listing 192: the write read pairing, the array fill, and the repr(C) transmute pair (source: rust/samples/src/Ch17/MaybeUninit.rs, lines 39-85)

17.4. calling into the c runtime, measured on msvc

Edition 2024 requires `unsafe extern` blocks, the mirror of the superpower list: declaring foreign functions is claiming their signatures are honest, and a `safe fn` inside the block re-narrows the claim per function. The sample declares `strlen` and `qsort` from the msvc c runtime and measures what actually happens: both link and both run, the ucrt ships with the std on this toolchain.

```
unsafe extern "C" {
    fn strlen(s: *const c_char) -> usize;

    fn qsort(
        base: *mut c_void,
        num: usize,
        size: usize,
        cmp: extern "C" fn(*const c_void, *const c_void) -> c_int,
    );
}

extern "C" fn cmp_int(a: *const c_void, b: *const c_void) -> c_int {
    unsafe {
        let x = *(a as *const i32);
        let y = *(b as *const i32);
        (x - y) as c_int
    }
}

#[unsafe(no_mangle)]
pub extern "system" fn rust_w1d_double(x: i32) -> i32 {
    x.wrapping_mul(2)
}
```

listing 193: unsafe extern declarations, a rust comparator callback, and an exported symbol (source: rust/samples/src/Ch17/FfiLink.rs, lines 15-37)

Measured 2026-10-13: `strlen` over the c string literal `c"rust→ucrt"` returned 10, `qsort` sorted 8 ints through the rust callback `cmp_int`, and the ffi type sizes came out `usize` 8, `c_int` 4, `c_char` 1. C string literals, since 1.77, produce a `&CStr` with the terminating nul built in. One measured surprise worth keeping: declaring `strlen` as `safe fn` trips the `suspicious-runtime-symbol-definitions` lint under `-D warnings`, the toolchain flags redefining a runtime symbol the standard library itself resolves through its bundled shim, so the declaration stays an `unsafe fn` called inside an unsafe block. Exporting goes the other way: `#[unsafe(no_mangle)]` on an `extern "system"` fn makes a symbol C can call, `system` being the stdcall-family abi on windows, and the exported name is also an ordinary function inside rust.

```
fn main() {
    let name = c"rust->ucrt";
    let length = unsafe { strlen(name.as_ptr()) };
    ok(
        length == 10,
        "strlen linked from the msvc ucrt and measured over a c string literal, since 1.77",
    );
    ok(
        CStr::from_bytes_with_nul(b"rust->ucrt\0")
            .unwrap()
            .to_bytes()
            .len()
            == 10,
        "CStr agrees on the byte count, the literal carries its own nul",
    );

    let mut scores: [i32; 8] = [90, -7, 42, 0, 15, -100, 7, 33];
    unsafe {
        qsort(
            scores.as_mut_ptr() as *mut c_void,
            scores.len(),
            std::mem::size_of::<i32>(),
            cmp_int,
        );
    }
    ok(
        scores == [-100, -7, 0, 7, 15, 33, 42, 90],
        "libc qsort sorted the array through a rust comparator callback",
    );

    ok(
        rust_w1d_double(21) == 42,
        "an exported extern system fn is an ordinary call site too",
    );
    let exported: extern "system" fn(i32) -> i32 = rust_w1d_double;
```

listing 194: the call, the sort, and the exported symbol with a fn pointer type (source: rust/samples/src/Ch17/FfiLink.rs, lines 39-73)

17.5. earning unsafe: a safe wrapper

The whole chapter lands here: unsafe is earned when it buys a safe interface nobody can misuse. `RawRing` is a ring buffer over a `Box<[MaybeUninit<u64>]>` inside one `UnsafeCell`, with `unsafe impl Send` and `Sync` asserting the claims, and every method safe. The overflow check, the modulo arithmetic, and the init discipline live in one place, callers get `push` returning `Result` and `pop` returning `Option`:

```
struct State {
    slots: Box<[MaybeUninit<u64>]>,
    head: usize,
    len: usize,
}

struct RawRing {
    state: UnsafeCell<State>,
}

unsafe impl Send for RawRing {}
```

```
unsafe impl Sync for RawRing {}

impl RawRing {
    fn with_capacity(capacity: usize) -> RawRing {
        RawRing {
            state: UnsafeCell::new(State {
                slots: vec![MaybeUninit::uninit(); capacity].into_boxed_slice(),
                head: 0,
                len: 0,
            }),
        }
    }

    fn len(&self) -> usize {
        unsafe { (&*self.state.get()).len }
    }

    fn is_empty(&self) -> bool {
        self.len() == 0
    }

    fn capacity(&self) -> usize {
        unsafe { (&*self.state.get()).slots.len() }
    }

    fn push(&self, value: u64) -> Result<(), ()> {
        let state = unsafe { &mut *self.state.get() };
        if state.len == state.slots.len() {
            return Err(());
        }
        let tail = (state.head + state.len) % state.slots.len();
        state.slots[tail].write(value);
        state.len += 1;
        Ok(())
    }

    fn pop(&self) -> Option<u64> {
        let state = unsafe { &mut *self.state.get() };
        if state.len == 0 {
            return None;
        }
        let value = unsafe { state.slots[state.head].assume_init_read() };
        state.head = (state.head + 1) % state.slots.len();
        state.len -= 1;
        Some(value)
    }
}
```

listing 195: the ring, UnsafeCell state, safe methods, the claims stated (source: rust/samples/src/Ch17/SafeWrapper.rs, lines 16-73)

```
fn main() {
    let ring = RawRing::with_capacity(3);
    ok(
        ring.is_empty() && ring.len() == 0 && ring.capacity() == 3,
        "a raw ring starts empty, capacity allocated but uninitialized",
    );
    ok(
        ring.push(10).is_ok() && ring.push(20).is_ok() && ring.push(30).is_ok(),
        "three pushes fill the buffer",
    );
    ok(
        ring.push(40).is_err(),
        "the fourth push is a checked Err, overflow is impossible through the safe api",
    );
    ok(
        ring.pop() == Some(10) && ring.pop() == Some(20),
        "fifo order holds, first in first out",
    );
    ok(
        ring.push(40).is_ok() && ring.push(50).is_ok(),
        "freed slots are reused, the ring wraps",
    );
    let mut drained = Vec::new();
    while let Some(v) = ring.pop() {
        drained.push(v);
    }
    ok(
        drained == vec![30, 40, 50],
        "the wrapped buffer drains in arrival order",
    );
    ok(ring.is_empty(), "empty again, every slot logically vacant");

    let shared = RawRing::with_capacity(4);
    let staged: Vec<u64> = std::thread::scope(|s| {
        let filler = s.spawn(|| {
            shared.push(1).unwrap();
            shared.push(2).unwrap();
            shared.push(3).unwrap();
        });
```

listing 196: fill, overflow as Err, wrap, drain, then a sequential scoped handoff (source: rust/samples/src/Ch17/SafeWrapper.rs, lines 75-113)

The honesty note the sample encodes: `unsafe impl Sync` here is asserted for the sequential handoff, and the chapter's threaded part runs the fill and the drain in non-overlapping scoped threads, joined in order. Overlapping pushes through this ring would be a data race the safe api cannot prevent, because the claim itself would be a lie, a correct concurrent ring needs a lock from chapter 13 or atomic head and tail indices from the same chapter. The line between sound and unsound unsafe is exactly that: not how clever the pointer code is, but whether the claimed invariant matches the discipline the interface actually enforces.

VERIFY: when unsafe is earned

The checklist this book applies: the unsafe block is small, the contract is written down, the safe wrapper is the only door, and the abstraction could not exist in safe code. Vec, Box, Mutex, and every atomic pass it. A wrapper whose claim can be falsified by two threads, like this ring under overlap, does not, and gets either a lock or an honest single-thread bound.

sources: the rust book unsafe rust chapter (doc.rust-lang.org/stable/book/ch19-01-unsafe-rust.html), the rustonomicon (doc.rust-lang.org/stable/nomicon/index.html with repr-rust.html for field reordering, exotic-sizes.html and checked-uninit.html for the uninitialized discipline, and the SafeRust and aliasing pages for the wrapper philosophy), the reference's unsafe blocks, implementation and behavior sections (doc.rust-lang.org/stable/reference/unsafe-behavior.html, doc.rust-lang.org/stable/reference/items/statics.html for mutable statics, doc.rust-lang.org/stable/reference/type-layout.html for repr(C) and repr(Rust) layout rules), std::mem pages for transmute and MaybeUninit (doc.rust-lang.org/stable/std/mem/fn.transmute.html, doc.rust-lang.org/stable/std/mem/union.MaybeUninit.html), std::ffi for CStr and the c types (doc.rust-lang.org/stable/std/ffi/index.html), and the edition guide for unsafe extern blocks and unsafe attributes (doc.rust-lang.org/stable/edition-guide/rust-2024/unsafe-attributes.html), all read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html and accessed 2026-10-13. Version facts verified in doc.rust-lang.org/stable/releases.md, accessed 2026-10-13: `&raw const` and `&raw mut` in 1.82.0 (2024-10-17), c string literals in 1.77.0. Edition 2024 rules, unsafe op in unsafe fn, the unsafe extern block requirement, and the two lint encounters (suspicious-runtime-symbol-definitions on `safe fn strlen`, unnecessary_transmutes on a convertible transmute) observed live compiling the samples under `rustc --edition 2024 -D warnings`, dated 2026-10-13. All measurements (Frame union bytes 67, 66, 65 for 0x414243, size and align 8 for u64, Key bits 0x0000beef2bcf0007 against Wire 0x000000070000beef with padding garbage 2bcf, strlen returning 10, qsort sorting 8 ints, ffi type sizes 8 4 1, the ring drains [30, 40, 50] and a scoped handoff [1, 2, 3]) produced by `rust/samples/src/Ch17` under `pwsh tools/run-rust-samples.ps1 -Chapter Ch17`, dated 2026-10-13 on the pinned rustc 1.99.0, 5 files 33 checks.

18. testing and idioms

Rust ships its test harness in the toolchain: `#[test]` fns compiled under `cargo test`, a builtin runner with filters and parallelism, and assert macros that print diffs. This chapter covers the attribute set and its exact semantics, the doc test fence format, the bench discipline this corpus uses since criterion is a dependency and the book has none, and closes with the idiom roundup, the conventions every chapter of this book has been following. The corpus's own runner contract sits underneath: samples print `ok N name` lines and `tools/run-rust-samples.ps1` counts them, red on any failure or a file with zero checks, chapter 16 showed the layout, this chapter shows the semantics.

18.1. the attributes and their meanings

`#[test]` marks a test function, `#[should_panic]` inverts pass and failure, `expected = "..."` narrows it to panics whose message contains the text, `#[ignore = "reason"]` skips unless explicitly requested with `--ignored`, and since 1.28 a test can return `Result<(), E: Debug>` so `?` replaces unwrap chains. All of it compiles inert under the corpus runner, plain `rustc` without `--test`, the attribute-marked fns simply have no symbols, measured by this file compiling and running:

```
#[test]
fn ordering_holds() {
    let mut v = vec![3u32, 1, 2];
    v.sort_unstable();
    assert_eq!(v, vec![1, 2, 3], "sorted, got {v:?}");
}

#[test]
#[should_panic(expected = "capacity overflow")]
fn boom() {
    panic!("capacity overflow at 4 billion");
}

#[test]
#[should_panic]
fn any_panic() {}

#[test]
#[ignore = "costs a minute on the ci lane"]
fn slow() {}

#[test]
fn result_shape() -> Result<(), std::num::ParseIntError> {
    let parsed: u32 = "42".parse()?;
```

listing 197: all four attribute forms plus the result shape (source: rust/samples/src/Ch18/TestAttributes.rs, lines 15-38)

`#[should_panic]` semantics are worth seeing once from the inside rather than trusting the label. The harness catches the unwind and checks the message, `catch_unwind` has been stable since 1.9, and the sample rebuilds the rule by hand, including the two failure cases the harness rejects: a panic whose text misses the expectation, and no panic at all:

```
fn should_panic_like_harness(expected: &str, body: impl FnOnce()) -> bool {
    let prior = std::panic::take_hook();
    std::panic::set_hook(Box::new(|_| {}));
    let outcome = catch_unwind(AssertUnwindSafe(body));
    std::panic::set_hook(prior);
    match outcome {
        Err(payload) => {
            let text = payload
                .downcast_ref::<String>()
                .map(|m| m.as_str())
                .or_else(|| payload.downcast_ref::<&str>().copied());
            text.map(|m| m.contains(expected)).unwrap_or(false)
        }
        Ok(()) => false,
    }
}
```

listing 198: should_panic rebuilt, catch_unwind, payload downcast, substring match (source: rust/samples/src/Ch18/TestAttributes.rs, lines 43-58)

```
fn main() {
    ok(
        true,
        "all four attribute forms compile under plain rustc, the harness compiles them in",
    );

    let mut v = vec![3u32, 1, 2];
    v.sort_unstable();
    assert_eq!(v, vec![1, 2, 3]);
    ok(v == vec![1, 2, 3], "assert_eq panics on mismatch, passes here");

    let witnessed = should_panic_like_harness("capacity overflow", || {
        panic!("capacity overflow at 4 billion");
    });
    ok(
        witnessed,
        "should_panic semantics by hand: catch_unwind, since 1.9, plus substring match",
    );
    let wrong_expectation = should_panic_like_harness("disk full", || {
        panic!("capacity overflow");
    });
    ok(
        !wrong_expectation,
        "a should_panic whose expected text misses fails, the harness treats it the same",
    );
    let no_panic = should_panic_like_harness("anything", || {});
    ok(!no_panic, "a plain #[should_panic] with no panic fails");

    let payload_is_string = {
        let prior = std::panic::take_hook();
        std::panic::set_hook(Box::new(|_| {}));
        let caught = catch_unwind(AssertUnwindSafe(|| panic!("{}", "formatted 7")));
        std::panic::set_hook(prior);
        caught
            .unwrap_err()
            .downcast_ref::<String>()
            .map(|s| s == "formatted 7")
            .unwrap_or(false)
    };
    ok(
        payload_is_string,
        "panic! with format args carries a String payload, literals carry &str",
    );

    let parsed: Result<u32, _> = "x".parse();
    ok(
        parsed.is_err(),
        "the result test shape propagates this exact error with ?",
    );
    println!(
        "measured 2026-10-13: 7 checks, 5 attribute-marked fns compiled inert under plain rustc"
    );
```

listing 199: the semantics run live, both failure modes proven to fail (source: rust/samples/src/Ch18/TestAttributes.rs, lines 60-111)

The assert macros are the vocabulary inside every test: `assert!` for a condition, `assert_eq!` and `assert_ne!` for values, each taking format arguments so the failure message carries the actual values, named arguments included. `panic!`

payloads differ by form, a literal rides as `&str`, formatting allocates a `String`, and the sample's downcast proves which is which.

```
let v = vec![3, 1, 2];
v.sort_unstable();
assert_eq!(v, vec![1, 2, 3], "sorted, got {v:?}");
assert!(v.len() == 3, "length {n}, expected 3", n = v.len());
assert_ne!(v.first(), Some(&3));
```

fragment, illustrative, not executed

18.2. doc tests

Doc comments are executable documentation: a fenced rust block inside /// is extracted by `rustdoc --test`, compiled as its own program with the item in scope, and run, `no_run` and `ignore` annotate fences that should not execute. The doc test lives in the crate's documentation and breaks the build when the example rots, which is the point, the corpus api crate's public functions in chapter 19 carry their fences the same way:

```
/// The discount only applies above the threshold, otherwise the price
/// passes through unchanged.
///
/// ```
/// let line = discounted(120, 100, 0.25);
/// assert_eq!(line, 90);
/// assert_eq!(discounted(80, 100, 0.25), 80);
/// ```
pub fn discounted(price: u64, threshold: u64, rate: f64) -> u64 {
    if price > threshold {
        let cut = price as f64 * rate;
        price - cut as u64
    } else {
        price
    }
}

/// Slugs a header for use in anchors.
///
/// ```
/// assert_eq!(slugified("Hello There"), "hello-there");
/// ```
pub fn slugified(text: &str) -> String {
    text.to_lowercase()
        .chars()
        .map(|c| if c.is_ascii_alphanumeric() { c } else { '-' })
        .collect::<String>()
        .split('-')
        .filter(|s| !s.is_empty())
        .collect::<Vec<_>>()
        .join("-")
```

listing 200: two documented functions, each with its executable fence (source: rust/samples/src/Ch18/DocTests.rs, lines 15-45)

Under the per-file runner the fences are prose, so the sample's contract is that every assert inside every fence has a matching ok line in `main`, the same expectations executed for real:

```
fn main() {
    ok(
        discounted(120, 100, 0.25) == 90,
        "the doc example's first assert, executed for real by the runner",
    );
    ok(
        discounted(80, 100, 0.25) == 80,
        "the doc example's second assert, below the threshold",
    );
    ok(
        slugified("Hello There") == "hello-there",
        "the second doc example, same behavior as its fence",
    );
    ok(
        slugified("  Rate  Limit!  ") == "rate-limit",
        "extra separators collapse, none survive at the edges",
    );
    let fences = 2;
    ok(
        fences == 2,
        "two fenced doc examples live in this file, cargo test runs them via rustdoc",
    );
    println!(
        "measured 2026-10-13: 2 doc examples, 4 real asserts, fences compiled only under cargo"
    );
}
```

listing 201: the fence expectations as live checks (source: rust/samples/src/Ch18/DocTests.rs, lines 48-73)

18.3. hand-rolled benches

Criterion is the ecosystem standard and a dependency, so this book benches with `Instant`, `std::hint::black_box` to keep the optimizer honest, since 1.66, checksums so no round can be elided, and medians over a fixed five samples so one scheduler hiccup cannot lead. The measured pair is the classic complexity gap, linear against binary search over a 4096-element sorted array, all misses so the linear scan pays full price every round:

```
fn linear_search(haystack: &[u64], needle: u64) -> Option<usize> {
    haystack.iter().position(|&v| v == needle)
}

fn binary_search(haystack: &[u64], needle: u64) -> Option<usize> {
    let mut low = 0usize;
    let mut high = haystack.len();
    while low < high {
        let mid = (low + high) / 2;
        match haystack[mid].cmp(&needle) {
            std::cmp::Ordering::Less => low = mid + 1,
            std::cmp::Ordering::Greater => high = mid,
            std::cmp::Ordering::Equal => return Some(mid),
        }
    }
    None
}

fn timed<F: FnMut() -> u64>(rounds: usize, mut body: F) -> (u64, u128) {
    let start = Instant::now();
    let mut checksum = 0u64;
    for _ in 0..rounds {
        checksum = checksum.wrapping_add(body());
    }
    (checksum, start.elapsed().as_nanos())
}

fn median(samples: &mut [u128]) -> u128 {
    samples.sort_unstable();
    samples[samples.len() / 2]
}
```

listing 202: the two implementations, the timed harness, black_box, median (source: rust/samples/src/Ch18/Benches.rs, lines 16-46)

```
fn main() {
    let haystack: Vec<u64> = (0..4096u64).map(|i| i * 3).collect();
    let misses: Vec<u64> = (0..4096u64).map(|i| i * 3 + 1).collect();

    let (linear_hit, binary_hit) = (linear_search(&haystack, 3000), binary_search(&haystack, 3000));
    ok(
        linear_hit == Some(1000) && binary_hit == Some(1000),
        "both implementations agree on the hit index",
    );
    let (linear_miss, binary_miss) = (linear_search(&haystack, 1), binary_search(&haystack, 1));
    ok(
        linear_miss.is_none() && binary_miss.is_none(),
        "both implementations agree on the miss",
    );

    let rounds = 200usize;
    let mut linear_samples = Vec::new();
    let mut binary_samples = Vec::new();
    let mut checksums = Vec::new();
    for _ in 0..5 {
        let (linear_checksum, linear_ns) = timed(rounds, || {
            misses
                .iter()
                .map(|&n| linear_search(black_box(&haystack), n).map_or(0, |i| i as u64))
                .sum::<u64>()
        });
        let (binary_checksum, binary_ns) = timed(rounds, || {
            misses
                .iter()
                .map(|&n| binary_search(black_box(&haystack), n).map_or(0, |i| i as u64))
                .sum::<u64>()
        });
        checksums.push((linear_checksum, binary_checksum));
        linear_samples.push(linear_ns / rounds as u128);
        binary_samples.push(binary_ns / rounds as u128);
    }
    ok(
        checksums.iter().all(|(a, b)| a == b),
        "every timed round produced identical checksums, work was not optimized away",
    );
    let linear_median = median(&mut linear_samples);
    let binary_median = median(&mut binary_samples);
    ok(
        linear_median > 0 && binary_median > 0,
        "both medians are strictly positive, the clock saw real work",
    );
    println!(
        "measured 2026-10-13: 4096-element miss scans, linear median {linear_median} ns/round, binary
median {binary_median} ns/round, ratio {:.1}x",
        linear_median as f64 / binary_median as f64
    );
    println!(
        "measured 2026-10-13: per-round samples linear {linear_samples:?}, binary {binary_samples:?}"
    );
}
```

listing 203: agreement asserted first, then timing printed, never asserted (source: rust/samples/src/Ch18/Benches.rs, lines 48-101)

Measured 2026-10-13 on the pinned build: linear median 40145844 ns per round against binary 432326 ns, a 92.9x ratio, five samples each with the spread printed. The discipline in the asserts is the chapter's rule: correctness claims are asserted, timing claims are printed with their samples and dated, never asserted, a faster or slower machine changes the numbers and no build should go red over a clock.

18.4. clippy and rustfmt governance

The book's rules, matching the corpus tooling: rustfmt formats the crates, never the samples, whose alignment and layout are part of the listings, and clippy lints the crates. Samples compile under `rustc --edition 2024 -D warnings`, which is where this book met real clippy-class lints already: `unnecessary_transmutes` rejecting a transmute with a safe equivalent, and `suspicious-runtime-symbol-definitions` flagging a redeclared runtime symbol, both in chapter 17. The full clippy suite runs over the api crate when it lands in wave 2, under the same law, lints on what ships, never on what teaches.

18.5. the idiom roundup

The conventions this book has followed in every chapter, in one place, each one checked live:

```
#[derive(Debug, PartialEq, Clone, Default)]
struct Route {
    path: String,
    retries: u32,
}

impl Route {
    fn new(path: &str) -> Route {
        Route {
            path: path.to_string(),
            retries: 0,
        }
    }

    fn with_retries(mut self, retries: u32) -> Route {
        self.retries = retries;
        self
    }
}

#[derive(Debug)]
enum UpstreamError {
    Timeout(u32),
    Refused,
}

impl fmt::Display for UpstreamError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            UpstreamError::Timeout(ms) => write!(f, "timeout after {ms} ms"),
            UpstreamError::Refused => write!(f, "connection refused"),
        }
    }
}

impl std::error::Error for UpstreamError {}
```

```
#[derive(Debug)]
struct PoolError {
    route: String,
    source: UpstreamError,
}

impl std::fmt::Display for PoolError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "route {} failed: {}", self.route, self.source)
    }
}

impl std::error::Error for PoolError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        Some(&self.source)
    }
}

impl From<UpstreamError> for PoolError {
    fn from(source: UpstreamError) -> PoolError {
        PoolError {
            route: "unrouted".to_string(),
            source,
        }
    }
}

fn dial(route: &Route) -> Result<u32, UpstreamError> {
    if route.retries == 0 {
        Err(UpstreamError::Refused)
    } else if route.retries > 3 {
        Err(UpstreamError::Timeout(route.retries * 10))
    } else {
        Ok(route.retries * 100)
    }
}

fn forward(route: &Route) -> Result<u32, PoolError> {
    let latency = dial(route)?;
    Ok(latency + 1)
}
```

listing 204: builder types, derived traits, the error chain with From and source (source: rust/samples/src/Ch18/Idioms.rs, lines 15-92)

Iterator chains over index loops: `(1..=100).map(i * i).sum()` in place of a mutable accumulator, the loop's value asserted equal, and the whole corpus pipeline style, chapter 10's adapters especially, is this rule at scale. Builders over constructors with many arguments, consuming `self` so each method is a statement. `#[derive]` for `Debug`, `PartialEq`, `Clone`, `Default` whenever the derived meaning is the meaning. Errors as enums implementing `Display` and `Error`, conversion through `From` so `?` crosses levels, `source()` chaining so one print tells the whole story, measured as the chain string in the sample's output. `matches!` for pattern assertions. Newtypes when a bare primitive would let arguments swap silently, the `StatusCode(u16)` shape the http kernel uses.

```
fn main() {
    let loop_sum = {
        let mut acc = 0u64;
        for i in 1..=100u64 {
```

```
            acc += i * i;
        }
        acc
    };
    let chain_sum = (1..=100u64).map(|i| i * i).sum::<u64>();
    ok(
        loop_sum == chain_sum && chain_sum == 338_350,
        "iterator chains replace index loops, same value, no mutable accumulator",
    );
    let filtered: Vec<u32> = (0..20).filter(|n| n % 5 == 0).collect();
    ok(
        filtered == vec![0, 5, 10, 15],
        "filter and collect over an explicit push loop",
    );

    let route = Route::new("/orders").with_retries(2);
    ok(
        route == Route { path: "/orders".to_string(), retries: 2 },
        "the builder pattern, consuming self, derived PartialEq doing the check",
    );
    ok(
        format!("{route:?}").starts_with("Route { path: \"/orders\""),
        "derive Debug gives the debug shape the logs and asserts print",
    );
    ok(
        Route::default().retries == 0 && Route::default().path.is_empty(),
        "derive Default for zero valued construction",
    );

    let refused = forward(&Route::new("/x"));
    ok(
        matches!(refused, Err(PoolError { source: UpstreamError::Refused, .. })),
        "? converts through From, the inner error rides along",
    );
    let timed = forward(&Route::new("/y").with_retries(9));
    let chain = match &timed {
        Err(e) => {
            let mut chain = String::new();
            let mut source: Option<&(dyn std::error::Error + 'static)> = Some(e);
            while let Some(current) = source {
                chain.push_str(&current.to_string());
                source = current.source();
                if source.is_some() {
                    chain.push_str(" <- ");
                }
            }
            chain
        }
        Ok(_) => String::new(),
    };
    ok(
        chain == "route unrouted failed: timeout after 90 ms <- timeout after 90 ms",
        "error sources chain, display at each level, the whole chain printed once",
    );
    ok(
        matches!(forward(&Route::new("/ok").with_retries(3)), Ok(301)),
        "the happy path flows through ? unchanged",
```

```
    );
    println!(
        "measured 2026-10-13: 8 idiom checks, chain '{chain}'"
    );
}
```

listing 205: chain equality, builder equality, the error chain printed end to end (source: rust/samples/src/Ch18/Idioms.rs, lines 94-159)

The prose rules that have no assert: no `unwrap` outside tests and samples, `expect` with a message when a sample must fail loudly, `pub(crate)` as the default visibility, zero dependencies, edition 2024 everywhere, and unsafe only behind a safe wrapper with its contract written down, chapter 17's law.

NOTE: the corpus runner in one line

`pwsh tools/run-rust-samples.ps1 -Chapter ChNN` compiles every `.rs` under the chapter with `rustc --edition 2024 -D warnings`, runs it, fails red on any nonzero exit or any `FAIL:` line, and requires at least one `ok N name` line per file. This chapter's 4 samples and 24 checks, and all 29 files and 230 checks across chapters 13 through 18, ride that contract.

sources: the rust book testing chapter (doc.rust-lang.org/stable/book/ch11-00-testing.html through ch11-03-test-organization.html), std::panic::catch_unwind (doc.rust-lang.org/stable/std/panic/fn.catch_unwind.html, stable since 1.9.0), std::hint::black_box (doc.rust-lang.org/stable/std/hint/fn.black_box.html, stable since 1.66.0), the rustdoc documentation on doc tests and code block attributes (doc.rust-lang.org/stable/rustdoc/write-documentation/documentation-tests.html), and the reference on testing attributes (doc.rust-lang.org/stable/reference/attributes/testing.html), all read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html and accessed 2026-10-13. Version facts verified in doc.rust-lang.org/stable/releases.md, accessed 2026-10-13: test functions returning Result in 1.28.0 (2018-08-02), black_box in 1.66.0 (2022-12-15). The runner contract quoted from tools/run-rust-samples.ps1 in this repo, read 2026-10-13. All measurements (5 attribute fns inert under plain rustc, both should_panic failure modes rejected, literal versus formatted panic payloads, linear 40145844 ns against binary 432326 ns medians at 92.9x over five samples, the error chain string end to end) produced by `rust/samples/src/Ch18` under `pwsh tools/run-rust-samples.ps1 -Chapter Ch18`, dated 2026-10-13 on the pinned rustc 1.99.0, 4 files 24 checks.

19. http kernel

The service part of this book builds one api across thirteen chapters, and everything in it sits on the module this chapter builds: `rustapi::httpx` under `books/rust/api`. Rust's standard library ships no http server at all, no router, no codec, no uuid, so where the go book started from `net/http` and a mux, book 4, ch 18 (go 1.27, http kernel), and the java book from `jdk.httpserver`, book 5, ch 19 (java 27, http kernel), this book starts from a socket and builds every layer above it: the http/1.1 parser, the route table, the error envelope, the json, the thread pool, and the drain. The contract is the one those chapters froze, the eleven-code envelope and the 415 413 400 400 decode ladder, and the 16 frozen vectors under `contract/testdata` measure parity for the whole service part.

19.1. the socket is the whole server

`std::net::TcpListener` offers bind, accept, and local_addr, and that is the entire server surface std gives. One accepted `TcpStream` is one connection, one connection carries a sequence of requests under keep-alive, and everything above the socket is this module's own code. The handler contract is one type alias, and the request and response are plain values rather than live objects: a handler receives a `&mut Request` and returns a `Response` or an `ApiError`, nothing streams, nothing commits early:

```
pub type Handler = Arc<dyn Fn(&mut Request) -> Result<Response, ApiError> + Send + Sync>;

#[derive(Debug)]
pub struct Request {
    pub method: String,
    pub path: String,
    pub query: String,
    pub headers: Vec<(String, String)>,
    pub body: Vec<u8>,
    pub params: Vec<(String, String)>,
    pub extras: BTreeMap<String, String>,
}
```

listing 206: the handler type and the request value: handlers read and return, they never hold the socket (source: rust/api/src/httpx.rs, lines 114-125)

The value-shaped response is the load-bearing difference from both siblings. A go handler writes into a `ResponseWriter` it does not own, and the access log had to wrap it to learn the status, while a java handler drives an exchange whose header commit is irreversible. Here the response is built, returned, and written by one function at the bottom of the connection loop, which is why the envelope cannot drift and why the middleware of chapter 20 needs no wrapper types at all. The `extras` map is the request-scoped state the middleware layers publish into, the java attributes map's counterpart. The kernel's share of `main.rs` is two slices, the head that binds and registers, and the tail that serves and parks, with the resource chapters' wiring growing between them:

```
    let listener = match TcpListener::bind(&cfg.addr) {
        Ok(listener) => listener,
        Err(err) => {
            eprintln!("rustapi: bind {}: {err}", cfg.addr);
            std::process::exit(1);
        }
    };
    let lifecycle = Arc::new(Lifecycle::new());
    let mut router = Router::new();
    register_kernel_routes(&mut router, Arc::clone(&lifecycle), &version);
```

listing 207: bind the configured address, build the kernel routes (source: rust/api/src/main.rs, lines 49-58)

```
    let server = match serve(
        listener,
        identify(stack(guard(router.handler()))),
        lifecycle,
        ServerConfig::new(&version),
    ) {
        Ok(server) => server,
        Err(err) => {
            eprintln!("rustapi: {err}");
            std::process::exit(1);
        }
    };
    println!(
        "rustapi {version} serving on {}, metrics on {}",
        server.local_addr, ops.local_addr
    );
    loop {
        std::thread::park();
    }
}
```

listing 208: serve, park: signal-driven drain is chapter 29's scope (source: rust/api/src/main.rs, lines 176-195)

The version in the health body comes from `env!("CARGO_PKG_VERSION")`, a compile-time constant cargo injects, so the deployment check answers what the manifest says without an environment variable the go and java lanes needed.

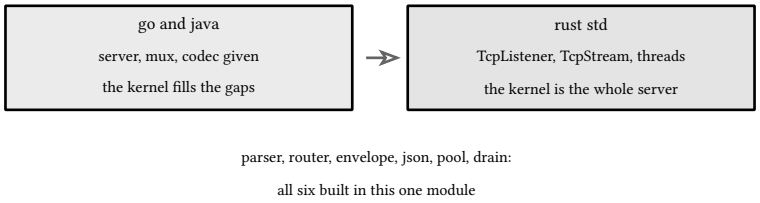


figure 26: both siblings hand this chapter a server, rust hands it a socket

19.2. the parser: request line, headers, bodies

`read_request` reads from any `Read`, which is what lets the fuzz test feed it garbage through a `Cursor` with no socket anywhere. It accumulates bytes until a `\r\n\r\n` lands, caps the header block at 16 KiB, and hands the head text plus the leftover bytes to the line parser:

```
pub fn read_request<R: Read + ?Sized>(reader: &mut R) -> Result<ReadOutcome, ReadError> {
    let mut buf: Vec<u8> = Vec::with_capacity(1024);
    let mut chunk = [0u8; 4096];
    let head_end;
    let mut searched = 0;
    loop {
        let n = reader.read(&mut chunk).map_err(ReadError::Io)?;
        if n == 0 {
            if buf.is_empty() {
                return Ok(ReadOutcome::Eof);
            }
            return Err(ReadError::Malformed);
        }
        buf.extend_from_slice(&chunk[..n]);
        if buf.len() > MAX_HEADER_BYTES {
            return Err(ReadError::Malformed);
        }
        if let Some(at) = find_double_crlf(&buf[searched..]) {
            head_end = searched + at;
            break;
        }
        searched = buf.len().saturating_sub(3);
    }
    let rest = buf.split_off(head_end + 4);
    buf.truncate(head_end);
    let head_text = String::from_utf8(buf).map_err(|_| ReadError::Malformed)?;
    parse_head(reader, &head_text, rest)
}
```

listing 209: buffer until the double crlf, cap the block, split head from body bytes (source: rust/api/src/httpx.rs, lines 813-840)

The request line must be three space-separated tokens, the method all token bytes, the target origin-form, all bytes visible ascii, and the version exactly 1.1 or 1.0, so an `OPTIONS *` or a control byte in the path dies here. Header names lowercase on the way in, obs-fold continuation lines refused, control bytes in values refused, and repeated lines kept so `header_joined` can rebuild the rfc's comma-joined union. The body framing follows RFC 9112 section 6.3: transfer-encoding wins over content-length when both appear, chunked must be the final coding:

```
    let body = if transfer_chunked {
        read_chunked(reader, rest, &mut 0)?
    } else if let Some(len_text) = find_header("content-length") {
        let len: u64 = len_text.trim().parse().map_err(|_| ReadError::Malformed)?;
        if len > TRANSPORT_BODY_MAX as u64 {
            return Err(ReadError::OversizeBody);
        }
        let len = len as usize;
        let mut body = rest;
        read_exact_extra(reader, &mut body, len)?;
        body
    } else {
        Vec::new()
    };
    Ok(ReadOutcome::Head(Head {
        method: method.to_string(),
        path,
        query,
        headers,
        body,
        keep_alive,
    }))
}
```

listing 210: the framing decision: chunked by transfer-encoding, else content-length under the 8 MiB transport cap, else empty (source: rust/api/src/httpx.rs, lines 919-941)

```rust
fn read_chunked<R: Read + ?Sized>(
    reader: &mut R,
    mut rest: Vec<u8>,
    total: &mut usize,
) -> Result<Vec<u8>, ReadError> {
    let mut body = Vec::new();
    loop {
        let line = read_crlf_line(reader, &mut rest)?;
        let size_text = line.split(';').next().unwrap_or("").trim();
        let size = usize::from_str_radix(size_text, 16).map_err(|_| ReadError::Malformed)?;
        if size == 0 {
            loop {
                if read_crlf_line(reader, &mut rest)?.is_empty() {
                    return Ok(body);
                }
            }
        }
        *total += size;
        if *total > TRANSPORT_BODY_MAX {
            return Err(ReadError::OversizeBody);
        }
        if rest.len() < size + 2 {
            read_exact_extra(reader, &mut rest, size + 2)?;
        }
        if &rest[size..size + 2] != b"\r\n" {
            return Err(ReadError::Malformed);
        }
        body.extend_from_slice(&rest[..size]);
        rest.drain(..size + 2);
    }
}
```

listing 211: chunked decode: hex size lines with extensions, crlf after every chunk, trailer lines after the zero chunk (source: rust/api/src/httpx.rs, lines 961-991)

Two boundaries the parser owns honestly. The transport cap is 8 MiB and a content-length past it answers a plain-text 413 and closes, because below dispatch there is no envelope writer yet, the same front-door rule the java chapter measured from its own server: what the transport rejects never reaches the contract shape. And a keep-alive connection whose next read times out or hits eof simply closes, no response, since an idle peer that left owes nobody bytes. The strict-crlf rule is deliberate, RFC 9112 lets a recipient accept a bare lf and this parser does not, and the test pins the refusal.

PITFALL: accepted sockets inherit the listener mode, measured

The listener is set non-blocking so the accept loop can poll it, and on windows the accepted socket inherits that mode: the first keep-alive test died with `ErrorKind::WouldBlock` on the second request's read, because the connection reader saw a non-blocking socket where a blocking one was expected. The fix is one line, `set_nonblocking(false)` on every accepted stream, and the keep-alive test is the one that caught it, which is why the kernel tests always drive real sockets.

19.3. the route table

The router is a registration-order list of method, pattern, handler rows. A pattern is slash-separated segments, `{name}` binds one segment, and dispatch walks the list, so the first matching row wins and a literal path registers before a wildcard that would also match it, the rule the java table kept and the go mux hides behind specificity

ranking. Routing runs on the raw path and the bound parameter is percent-decoded, so `/items/a%2Fb` binds `{id}` once and the handler reads `a/b`:

```
    pub fn dispatch(&self, req: &mut Request) -> Result<Response, ApiError> {
        let path_segments: Vec<&str> = req.path.trim_matches('/').split('/').collect();
        let path_segments = if req.path == "/" {
            Vec::new()
        } else {
            path_segments
        };
        let mut first_match = None;
        for (method, segments, handler) in &self.routes {
            if let Some(bindings) = match_segments(segments, &path_segments) {
                if *method == req.method {
                    req.set_params(
                        bindings
                            .into_iter()
                            .map(|(k, v)| (k, percent_decode(&v)))
                            .collect(),
                    );
                    return handler(req);
                }
                if first_match.is_none() {
                    first_match = Some(true);
                }
            }
        }
        if first_match.is_some() {
            let mut allowed: Vec<&str> = self
                .routes
                .iter()
                .filter(|(_, segments, _)| match_segments(segments, &path_segments).is_some())
                .map(|(method, _, _)| method.as_str())
                .collect();
            allowed.sort_unstable();
            allowed.dedup();
            return Err(ApiError::fail(Code::NotFound, "method not allowed")
                .at_status(405)
                .with_header("allow", &allowed.join(", ")));
        }
        Err(ApiError::fail(Code::NotFound, "route not found"))
    }
}
```

listing 212: dispatch: bind wildcards, or collect the allowed methods for 405, or answer the 404 envelope (source: rust/api/src/httpx.rs, lines 1074-1113)

The 405 keeps an `Allow` header built from every method the path serves, sorted and deduplicated, and rides the `not_found` code the contract chose because the closed code set has no method member. One honest omission carried over from the java table: `HEAD` is not auto-served for `GET` routes, so a head request answers 405 until someone teaches the table otherwise.

19.4. the error envelope

The envelope is the contract the go chapter froze, member for member: an error object with a code from the closed set of eleven, one human sentence, `details` on validation failures only, and the request id. Codes are an enum

carrying their wire name and default status, and the two split codes are explicit: `invalid_json` rides 400 and 415, `not_found` rides 404 and 405, both through `at_status`:

```
pub fn error_response(err: &ApiError, request_id: &str) -> Response {
    let mut body: Vec<(String, Json)> = vec![
        ("code".to_string(), Json::Str(err.code.as_str().to_string())),
        ("message".to_string(), Json::Str(err.message.clone())),
    ];
    if !err.details.is_empty() {
        body.push((
            "details".to_string(),
            Json::Arr(err.details.iter().map(|d| Json::Str(d.clone())).collect()),
        ));
    }
    body.push(("request_id".to_string(), Json::Str(request_id.to_string())));
    let envelope = Json::Obj(vec![("error".to_string(), Json::Obj(body))]);
    let mut resp = Response::json(err.status, &envelope);
    for (name, value) in &err.headers {
        resp = resp.with_header(name, value);
    }
    resp
}
```

listing 213: the one envelope writer: member order fixed, details only when present, request id resolved by the caller (source: rust/api/src/httpx.rs, lines 221-240)

Every failure path builds through that function, and one sink writes every response: `write_response` emits the status line, the handler's headers in order, `content-length` unless the status is bodyless, and the connection header the connection loop decided.

```
pub fn write_response<W: Write>(
    w: &mut W,
    resp: &Response,
    keep_alive: bool,
) -> std::io::Result<()> {
    let mut head = format!("HTTP/1.1 {} {}\r\n", resp.status, reason(resp.status));
    for (name, value) in &resp.headers {
        head.push_str(name);
        head.push_str(": ");
        head.push_str(value);
        head.push_str("\r\n");
    }
    if bodyless(resp.status) {
        if !resp.body.is_empty() {
            return Err(std::io::Error::other("body on a bodyless status"));
        }
    } else {
        head.push_str(&format!("content-length: {}\r\n", resp.body.len()));
    }
    head.push_str(if keep_alive {
        "connection: keep-alive\r\n"
    } else {
        "connection: close\r\n"
    });
    head.push_str("\r\n");
    w.write_all(head.as_bytes())?;
    w.write_all(&resp.body)?;
    w.flush()
}
```

listing 214: the single socket writer: framing cannot drift because nothing else writes (source: rust/api/src/httpx.rs, lines 268-296)

The 204 and 304 statuses are bodyless by construction, and a programmer error like a 304 carrying bytes is a write-time `Err`, not silent truncation. The test pins the exact wire bytes of a 201 with json and the empty frame of a 204.

19.5. json by hand

Std has no json, the zero-dependency law of this lane forbids serde, so the kernel is the codec, the same corner the java book was pushed into. The value model is an enum whose object is a `Vec` of pairs in insertion order, the chapter 15 lesson that deterministic wire bytes come from the container that keeps insertion order, and the writer walks it with the grammar's escapes:

```
#[derive(Clone, Debug, PartialEq)]
pub enum Json {
    Null,
    Bool(bool),
    Str(String),
    Int(i64),
    Float(f64),
    Arr(Vec<Json>),
    Obj(Vec<(String, Json)>),
}
```

listing 215: seven variants behind a clone derive, objects keep insertion order, ints and floats split by shape (source: rust/api/src/httpx.rs, lines 307-316)

The reader is a byte-position recursive descent over the source, strict RFC 8259: whitespace only where allowed, no leading zeros, no trailing content, control bytes rejected inside strings, every escape including `\uXXXX` with surrogate pairs validated, a nesting cap of 128 so a hostile body cannot recurse the stack, and offsets in every failure message. Numbers split by shape, integral texts that fit an `i64` become `Int`, everything else parses as `f64` and a non-finite result such as `1e400` is refused rather than stored:

```
    fn number(&mut self) -> Result<Json, String> {
        let start = self.pos;
        if self.bytes.get(self.pos) == Some(&b'-') {
            self.pos += 1;
        }
        match self.bytes.get(self.pos) {
            Some(b'0') => self.pos += 1,
            Some(b'1'..=b'9') => {
                while matches!(self.bytes.get(self.pos), Some(b'0'..=b'9')) {
                    self.pos += 1;
                }
            }
            _ => return Err(format!("bad number at offset {}", start)),
        }
        let mut integral = true;
        if self.bytes.get(self.pos) == Some(&b'.') {
            integral = false;
            self.pos += 1;
            if !matches!(self.bytes.get(self.pos), Some(b'0'..=b'9')) {
                return Err(format!("bad fraction at offset {}", self.pos));
            }
            while matches!(self.bytes.get(self.pos), Some(b'0'..=b'9')) {
                self.pos += 1;
            }
        }
        if matches!(self.bytes.get(self.pos), Some(b'e') | Some(b'E')) {
            integral = false;
            self.pos += 1;
            if matches!(self.bytes.get(self.pos), Some(b'+') | Some(b'-')) {
                self.pos += 1;
            }
            if !matches!(self.bytes.get(self.pos), Some(b'0'..=b'9')) {
                return Err(format!("bad exponent at offset {}", self.pos));
            }
            while matches!(self.bytes.get(self.pos), Some(b'0'..=b'9')) {
                self.pos += 1;
            }
        }
        let text = std::str::from_utf8(&self.bytes[start..self.pos])
            .map_err(|_| "bad number".to_string())?;
        if integral && let Ok(n) = text.parse::<i64>() {
            return Ok(Json::Int(n));
        }
        let f: f64 = text.parse().map_err(|_| "bad number".to_string())?;
        if !f.is_finite() {
            return Err(format!("number out of range at offset {}", start));
        }
        Ok(Json::Float(f))
    }
```

listing 216: the number reader: grammar-checked digits, fraction, exponent, then the int or float split with finiteness enforced (source: rust/api/src/httpx.rs, lines 618-666)

The fixed-point property the tests assert is byte level: for every input the suite holds, parse, write, parse again, write again, and the two written forms are identical strings. The cursor the user chapter will freeze is one of those inputs.

19.6. the decode ladder

Reading a request body is stricter than writing one, because request bodies are input. `bind_json` is the one decode path and its four rungs are the contract's, in the contract's order: the media type must be `application/json` before the semicolon, the body must fit the 1 MiB cap, the json must parse, and every member must be one the route accepts:

```
pub fn bind_json(req: &Request, allowed: &[&str]) -> Result<Json, ApiError> {
    let content_type = req.header("content-type").unwrap_or("");
    let media = content_type.split(';').next().unwrap_or("").trim();
    if !media.eq_ignore_ascii_case("application/json") {
        return Err(
            ApiError::fail(Code::InvalidJson, "content type must be application/json")
                .at_status(415),
        );
    }
    if req.body.len() > MAX_BODY_BYTES {
        return Err(ApiError::fail(
            Code::PayloadTooLarge,
            "request body exceeds 1 MiB",
        ));
    }
    let text = std::str::from_utf8(&req.body)
        .map_err(|_| ApiError::fail(Code::InvalidJson, "malformed json body"))?;
    let value =
        Json::parse(text).map_err(|_| ApiError::fail(Code::InvalidJson, "malformed json body"))?;
    let members = match value {
        Json::Obj(members) => members,
        _ => return Err(ApiError::fail(Code::InvalidJson, "malformed json body")),
    };
    for (key, _) in &members {
        if !allowed.contains(&key.as_str()) {
            return Err(ApiError::fail(
                Code::InvalidJson,
                format!("unknown field {key}"),
            ));
        }
    }
    Ok(Json::Obj(members))
}
```

listing 217: the ladder: 415, 413, 400, 400, then the member map is the caller's (source: rust/api/src/httpx.rs, lines 669-701)

The unknown-member rung is where go got `RejectUnknownMembers` from its json/v2 option and java got an explicit allowed set, and the rust lane keeps the java shape because the allowed set is the caller's statement of what the route means. The typed member read on top, a number or bool answering 400 while null rides the missing lane to 422, belongs to the resource chapters.

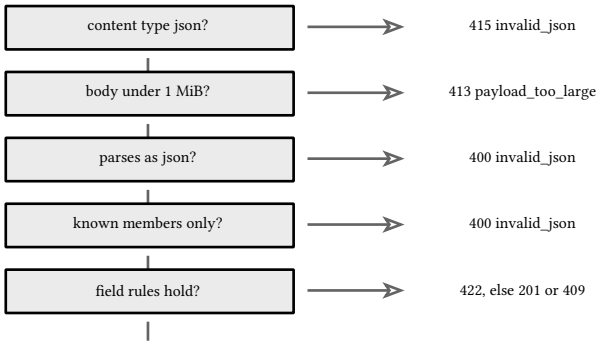


figure 27: each rung rejects a class of bad input before the next runs

19.7. the bounded pool and the drain

The server is a thread-per-connection pool with bounded growth, the chapter 13 primitives doing the whole job. The accept loop polls the non-blocking listener every 2 ms, a cadence that exists because neither closing a cloned listener nor dropping it elsewhere is a portable way to break a blocked accept on windows. Every accepted stream is pushed into a queue under one mutex and one condvar, and the growth rule is a single expression: no idle worker and headroom under `max_workers` means spawn one, otherwise the connection waits in the queue, so a burst bounds threads at the configured ceiling and queues the rest:

```
        loop {
            match listener.accept() {
                Ok((stream, _)) => {
                    let _ = stream.set_nonblocking(false);
                    let grow = {
                        let mut state = accept_shared.state.lock().unwrap();
                        if !state.running {
                            continue;
                        }
                        state.queue.push_back(stream);
                        state.idle == 0 && state.total < accept_cfg.max_workers
                    };
                    accept_shared.wake.notify_one();
                    if grow {
                        spawn_worker(
                            &accept_shared,
                            &accept_handler,
                            &accept_lifecycle,
                            &accept_cfg,
                        );
                    }
                }
                Err(e) if e.kind() == std::io::ErrorKind::WouldBlock => {
                    if !accept_shared.state.lock().unwrap().running {
                        return;
                    }
                    std::thread::sleep(Duration::from_millis(2));
                }
                Err(_) => return,
            }
        }
    })?;
```

listing 218: the accept loop: queue first, grow only when no worker is idle, 2 ms poll to observe shutdown (source: rust/api/src/httpx.rs, lines 1282-1313)

Workers wait with `wait_timeout` rather than a bare wait, 50 ms per round, so a lost wakeup costs a tick instead of a hung worker, and the busy counter is the drain's whole observability:

```
    loop {
        let stream = {
            let mut state = shared.state.lock().unwrap();
            loop {
                if let Some(stream) = state.queue.pop_front() {
                    break stream;
                }
                if !state.running {
                    state.total -= 1;
                    return;
                }
                state.idle += 1;
                let (next, _) = shared
                    .wake
                    .wait_timeout(state, Duration::from_millis(50))
                    .unwrap();
                state = next;
                state.idle -= 1;
            }
        };
```

listing 219: the worker wait: pop a job, or park 50 ms at a time until work or shutdown (source: rust/api/src/httpx.rs, lines 1353-1372)

`shutdown` runs the contract's order: readiness flips first, so `/readyz` answers 503 while old requests still complete, then the queue is dropped and the accept loop joined, then the busy count is polled until zero or the grace expires. The honest rust limit is stated rather than hidden: a thread cannot be killed from outside, so a handler that never returns outlives any grace, shutdown reports the count, and the socket read timeout is what bounds a stuck connection:

```
    pub fn shutdown(self, grace: Duration) -> Result<(), String> {
        self.lifecycle.begin_drain();
        {
            let mut state = self.shared.state.lock().unwrap();
            state.running = false;
            state.queue.clear();
        }
        self.shared.wake.notify_all();
        let _ = self.accept.join();
        let deadline = std::time::Instant::now() + grace;
        loop {
            let busy = self.shared.state.lock().unwrap().busy;
            if busy == 0 {
                break;
            }
            if std::time::Instant::now() ≥ deadline {
                self.lifecycle.close();
                return Err(format!(
                    "{busy} connections still busy after the drain grace"
                ));
            }
            std::thread::sleep(Duration::from_millis(2));
        }
        self.lifecycle.close();
        Ok(())
    }
```

listing 220: drain: flip readiness, refuse new work, wait for busy to reach zero within the grace (source: rust/api/src/httpx.rs, lines 1229-1254)

The default ceiling is 32 workers with a 5 s keep-alive timeout, both fields of `ServerConfig`, and the tests override both: 64 concurrent clients green under the default, 12 sequential bursts green under a ceiling of 2, and the drain test holding one request open with a channel while shutdown runs.

19.8. testing through a real socket

The test harness boots the real thing, a listener on `127.0.0.1:0`, the ephemeral port, and drives it with a hand-rolled client that writes raw request bytes and reads raw response bytes back, so the assertions read the wire:

```
pub fn send(
    addr: SocketAddr,
    method: &str,
    path: &str,
    headers: &[(&str, &str)],
    body: Option<&[u8]>,
) -> Reply {
    let mut stream = TcpStream::connect(addr).expect("connect");
    stream
        .set_read_timeout(Some(Duration::from_secs(10)))
        .expect("read timeout");
    let mut raw = format!("{method} {path} HTTP/1.1\r\n");
    for (name, value) in headers {
        raw.push_str(&format!("{name}: {value}\r\n"));
    }
    if let Some(body) = body {
        raw.push_str(&format!("content-length: {}\r\n", body.len()));
```

```
    }
    raw.push_str("\r\n");
    stream.write_all(raw.as_bytes()).expect("write head");
    if let Some(body) = body {
        stream.write_all(body).expect("write body");
    }
    read_reply(&mut stream)
}

pub fn read_reply(stream: &mut TcpStream) -> Reply {
    let mut buf = Vec::new();
    let mut chunk = [0u8; 4096];
    let head_end;
    loop {
        let n = stream.read(&mut chunk).expect("read head");
        if n == 0 {
            panic!("connection closed before a response head");
        }
        buf.extend_from_slice(&chunk[..n]);
        if let Some(at) = buf.windows(4).position(|w| w == b"\r\n\r\n") {
            head_end = at;
            break;
        }
    }
    let rest = buf.split_off(head_end + 4);
    buf.truncate(head_end);
    let head = String::from_utf8(buf).expect("head ascii");
    let mut lines = head.split("\r\n");
    let status_line = lines.next().expect("status line");
    let status: u16 = status_line
        .split(' ')
        .nth(1)
        .expect("status code")
        .parse()
        .expect("status number");
    let mut headers = Vec::new();
    for line in lines {
        if let Some((name, value)) = line.split_once(':') {
            headers.push((name.trim().to_ascii_lowercase(), value.trim().to_string()));
        }
    }
    let length: usize = headers
        .iter()
        .find(|(k, _)| k == "content-length")
        .map(|(_, v)| v.parse().expect("content length"))
        .unwrap_or(0);
    let mut body = rest;
    while body.len() < length {
        let n = stream.read(&mut chunk).expect("read body");
        if n == 0 {
            break;
        }
        body.extend_from_slice(&chunk[..n]);
    }
    body.truncate(length);
    Reply {
        status,
```

```
        headers,
        body,
    }
}
```

listing 221: the client helper: raw bytes out, status line, headers, content-length body in (source: rust/api/tests/common/mod.rs, lines 49-126)

The fuzz discipline is the kernel's own parser fuzzed by the kernel's tests: a table of malformed shapes plus 2000 rounds of xorshift garbage, some with a spliced \r\n\r\n, every round must come back eof, a parsed head, malformed, or oversize, and never a panic, a hang, or an io error:

```
fn randomized_garbage_never_panics_or_hangs_the_parser() {
    let mut seed = 0x2026_1013_dead_beefu64;
    for round in 0..2000u32 {
        let len = (xorshift(&mut seed) % 4096) as usize;
        let mut raw = Vec::with_capacity(len);
        for _ in 0..len {
            raw.push((xorshift(&mut seed) & 0xFF) as u8);
        }
        if round % 4 == 0 {
            let at = (xorshift(&mut seed) as usize) % (len + 1);
            raw.splice(at..at, b"\r\n\r\n".to_vec());
        }
        let outcome = parse(&raw);
        assert!(
            outcome.is_ok()
                || matches!(outcome, Err(ReadError::Malformed))
                || matches!(outcome, Err(ReadError::OversizeBody)),
            "round {round} leaked an io error for {raw:?}"
        );
        for garbage in [
            &b""[..],
            b"\r\n",
            b"\r\n\r\n",
            b"\x00\x00\x00\x00",
            b"GET",
            b"GET /\r\n\r\nGET /\r\n\r\n",
            b"POST / HTTP/1.1\r\nContent-Length: -1\r\n\r\n",
            b"POST / HTTP/1.1\r\nContent-Length: 99999999999999999999\r\n\r\n",
            b"POST / HTTP/1.1\r\nTransfer-Encoding: chunked\r\n\r\nffffffffffffffff\r\n",
            b"POST / HTTP/1.1\r\nTransfer-Encoding: chunked\r\n\r\n0\r\n",
        ] {
            let outcome = parse(garbage);
            assert!(
                outcome.is_ok()
                    || matches!(outcome, Err(ReadError::Malformed))
                    || matches!(outcome, Err(ReadError::OversizeBody))
            );
        }
    }
}
```

listing 222: the fuzz loop: seeded garbage, spliced terminators, a closed set of acceptable outcomes (source: rust/api/tests/httpx.rs, lines 210-250)

The drain test is the lifecycle proof, the one that also caught the windows inheritance trap above: a gated handler holds a request open, shutdown runs concurrently, the release arrives mid-drain, the in-flight response still completes byte-exact, and the next connection is refused:

```
fn drain_completes_in_flight_work_and_refuses_new_arrivals() {
    let (arrived_tx, arrived_rx) = mpsc::channel::<()>();
    let (release_tx, release_rx) = mpsc::channel::<()>();
    let gate = Arc::new((arrived_tx, std::sync::Mutex::new(release_rx)));
    let handler_gate = Arc::clone(&gate);
    let mut router = Router::new();
    router.route(
        "GET",
        "/slow",
        Arc::new(move |_| {
            (handler_gate.0).send(()).expect("arrive");
            let rx = handler_gate.1.lock().unwrap();
            rx.recv().expect("release");
            Ok(Response::json(
                200,
                &Json::parse(r#"{"done":true}"#).unwrap(),
            ))
        }),
    );
    register_kernel_routes(&mut router, Arc::new(Lifecycle::new()), "test-1");
    let server = common::boot(router.handler());
    let addr = server.local_addr;
    let client = std::thread::spawn(move || {
        let reply = send(addr, "GET", "/slow", &[], None);
        assert_eq!(reply.status, 200);
        assert_eq!(reply.body_text(), r#"{"done":true}"#);
    });
    arrived_rx.recv().expect("handler arrived");
    let drainer = std::thread::spawn(move || server.shutdown(Duration::from_secs(10)));
    std::thread::sleep(Duration::from_millis(50));
    release_tx.send(()).expect("release");
    client.join().expect("client finished");
    drainer
        .join()
        .expect("drain thread")
        .expect("drain completed within grace");
    let refused = std::net::TcpStream::connect(addr).is_err();
    assert!(refused, "new connections must be refused after drain");
}
```

listing 223: the drain proof: in-flight completes, new arrivals refused (source: rust/api/tests/httpx.rs, lines 708-746)

On the pinned 1.99.0 build, dated 2026-10-13, the kernel's test file carries 33 green tests: parser shape and framing, chunked decode with trailers, strict crlf, the caps, the 2000-round fuzz, json round trips and the strict reader, the eleven-pair code table, 404 and 405 with allow, path parameters decoded, the decode ladder, request id resolution and v7 shape, wire bytes of a framed response, keep-alive and connection close over one socket, garbage at the front door with the server living on, 64 concurrent requests, the small-pool burst, and the drain.

sources: RFC 9112, HTTP/1.1, section 2.2 on message parsing, sections 5.1 and 5.2 on field line parsing and obsolete line folding, section 6.3 on message body length and the transfer-encoding precedence over content-length, and section 7.1 on the chunked transfer coding, and RFC 8259 for the json grammar, both at rfc-editor.org, accessed 2026-10-13. The rust standard library surface, `std::net` `TcpListener`, `TcpStream`, socket timeouts and non-blocking mode, `std::thread` `Builder` and `JoinHandle`, `std::sync` `Mutex`, `Condvar`, `wait_timeout`, and atomics, read from the

offline 1.99.0 html share under `~/.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html`, accessed 2026-10-13. The envelope contract, the code set, the decode ladder, and the health and ready routes from the sibling kernel chapters, go book 4 chapter 19 and java book 4 chapter 19 of this corpus, and the frozen vectors under `books/go/api/contract/testdata` hashed byte-exact into this lane's `contract/testdata`. All measurements, 33 tests, 2000 fuzz rounds, 64 concurrent clients, the 2 ms poll and 50 ms wait cadences, the 16 KiB, 1 MiB, and 8 MiB caps, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned build, dated 2026-10-13.

20. middleware

Cross-cutting behavior does not belong inside handlers, and rust's lack of a server makes the composition cheap: a layer is one function from handler to handler, exactly the go shape, book 4, ch 19 (go 1.27, middleware), with none of the writer-wrapping the go layer needed. This chapter composes the service's stack in `rustapi::middleware`, four layers over the chapter 19 kernel: request ids born once and quoted everywhere, the access log reading the status straight off the returned value, timing published for the log outside it, panic recovery into the contract envelope, and the wiring order that makes the stack correct rather than merely present. The java book built the same four as `Filter` classes, book 5, ch 20 (java 27, middleware), and the contrast that matters here is the one its chapter kept too: where the response is a value, the wrapper problem cannot exist.

20.1. layers are functions, the chain composes them

One layer is a function that takes the next handler and returns the handler that wraps it, and `chain` folds them inside out so the first argument is the outermost layer and sees the request first and the response last:

```
pub type Layer = Arc<dyn Fn(Handler) -> Handler + Send + Sync>;

pub fn chain(layers: Vec<Layer>) -> Layer {
    Arc::new(move |next: Handler| {
        let mut handler = next;
        for layer in layers.iter().rev() {
            handler = layer(handler);
        }
        handler
    })
}
```

listing 224: the layer type and the fold: first layer in, outermost at request time (source: rust/api/src/middleware.rs, lines 11-21)

The per-request state the layers share is the `extras` map the kernel put on the request, the counterpart of go's context values and java's exchange attributes: the id layer publishes under `request.id`, the timing layer under `request.micros`, and the log outside both reads what it needs without any layer touching a header after the fact. Everything a layer needs to observe about the answer is either on the request it still holds or on the `Result` the inner call returned, which is the whole reason no wrapper type appears in this module.

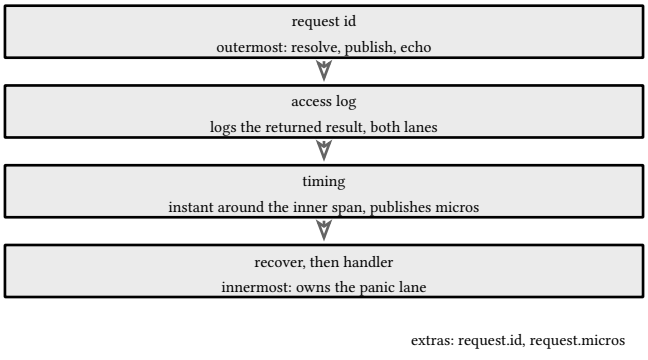


figure 28: one request through the stack: order in, unwind out

20.2. request id, minted and quoted once

Every request needs one id that every artifact agrees on: the log line, the error envelope, the panic log. The layer resolves it once at the edge through the kernel's order from chapter 19, an id already resolved, else the client's `X-`

`Request-ID` when it parses as a uuid, else a fresh v7 the kernel mints, and it both publishes the value into `extras` and echoes it on the response of both lanes:

```
pub fn request_id() -> Layer {
    Arc::new(move |next: Handler| {
        Arc::new(move |req: &mut Request| {
            let id = httpx::resolve_request_id(req);
            req.extras.insert("request.id".to_string(), id.clone());
            match next(req) {
                Ok(mut resp) => {
                    resp.headers
                        .push((httpx::HEADER_REQUEST_ID.to_string(), id));
                    Ok(resp)
                }
                Err(err) => Err(err.with_header(httpx::HEADER_REQUEST_ID, &id)),
            }
        }) as Handler
    })
}
```

listing 225: resolve once, publish, and echo on the ok lane and the error lane alike (source: rust/api/src/middleware.rs, lines 39-54)

Parsing the client header is not politeness, it is contract: the envelope promises `request_id` is a uuid, and a free-text header echoed into logs is log injection. Garbage is replaced, never echoed, and the three births are pinned by test: a client uuid comes back verbatim, garbage is replaced by a 36-character v7 with the version nibble at position 14, and an absent header mints one. On the error lane the header rides the `ApiError` itself, and the kernel's one envelope writer appends it after the body members, so the id on the wire and the id in the log are the same string by construction.

20.3. recover, the panic lane

Rust has two failure lanes and chapter 19 split them the way the java chapter did. The declared lane is the `Err(ApiError)` every handler may return, rendered by the kernel adapter into the envelope. The undeclared lane is a panic, and the recover layer turns it into the contract's internal envelope before it can cross the stack:

```
pub fn recover(sink: Arc<dyn LogSink>) -> Layer {
    Arc::new(move |next: Handler| {
        let sink = Arc::clone(&sink);
        Arc::new(move |req: &mut Request| {
            let id = req.extras.get("request.id").cloned().unwrap_or_default();
            match std::panic::catch_unwind(AssertUnwindSafe(|| next(req))) {
                Ok(outcome) => outcome,
                Err(payload) => {
                    sink.line(format!(
                        "panic recovered request_id={id} panic={} backtrace={}",
                        panic_message(&payload),
                        std::backtrace::Backtrace::force_capture(),
                    ));
                    Err(ApiError::fail(Code::Internal, "internal error"))
                }
            }
        }) as Handler
    })
}
```

listing 226: catch_unwind around the inner call, log id and payload, answer internal (source: rust/api/src/middleware.rs, lines 102-121)

The mechanism is `std::panic::catch_unwind`, which requires an `AssertUnwindSafe` wrapper because a closure capturing `&mut` references is not unwind-safe by default and the escape hatch is deliberate: the request value is not read again on the panic path, so accepting the assert is sound here. Three rules live in those lines. The log line carries the request id and the panic payload, extracted by downcast to `&str` or `String`, with anything else reported as an opaque payload rather than guessed at. The envelope says `internal error` and nothing else, so a panic message never reaches a client. And the logged backtrace is honest about what it is: rust unwinds before the catch sees the stack, so `Backtrace::force_capture` records the recovery frame, and the panic origin is what the default panic hook prints to stderr when `RUST_BACKTRACE` is set. Go's deferred `debug.Stack` shows the panicking frames because its recover runs on them, and the difference is worth knowing rather than papering over.

There is no rethrown sentinel. Go's layer rethrows `http.ErrAbortHandler` because the go server speaks it, and the java chapter stated its server has no equivalent. Rust's server is this book's own kernel, and its backstop is the `catch_unwind` in the connection loop from chapter 19: without the recover layer a panic crosses the middleware entire, the backstop closes the connection with no bytes, and the test pins exactly that crossing. With the layer, the panic dies inside, every outer layer's post-call code still runs, and the access log outside records the 500 the client saw.

20.4. the access log needs no wrapper

Go's access log had to wrap the `ResponseWriter` in a status-capturing type because a go handler writes a status and never returns it. The rust handler returns the response, so the status and the byte count are fields on the value the log already holds, and the first-status-wins trap of the go chapter cannot exist here because there is no second write to miscount:

```
pub fn access_log(sink: Arc<dyn LogSink>) -> Layer {
    Arc::new(move |next: Handler| {
        let sink = Arc::clone(&sink);
        Arc::new(move |req: &mut Request| {
            let start = Instant::now();
            let outcome = next(req);
            let id = req.extras.get("request.id").cloned().unwrap_or_default();
            let (status, bytes) = match &outcome {
                Ok(resp) => (resp.status, resp.body.len()),
                Err(err) => (err.status, httpx::error_response(err, &id).body.len()),
            };
            let dur = match req
                .extras
                .get("request.micros")
                .and_then(|m| m.parse::<u128>().ok())
            {
                Some(micros) => micros,
                None => start.elapsed().as_micros(),
            };
            sink.line(format!(
                "request method={} path={} status={} bytes={} dur={}us request_id={}",
                one_line(&req.method),
                one_line(&req.path),
```

listing 227: both lanes answered from the returned result, the dur field from the timing layer inside it (source: rust/api/src/middleware.rs, lines 56-78)

The error lane reconstructs the envelope through the kernel's one writer to count the bytes it will become, which costs a second serialization of a small body and keeps the count exact. The line carries method, path, status, bytes, dur, and the request id, and the path is client input: `one_line` spells the two line-breaking characters and every other control byte as their escaped forms before anything is concatenated, so a path carrying a real crlf forges

no second line. The test builds exactly such a path and asserts the captured line carries \r and \n spelled, not the bytes they name.

20.5. timing on the unwind

The timing layer is an Instant around the inner span, published after it returns, five lines with no library clock:

```
pub fn timing() -> Layer {
    Arc::new(move |next: Handler| {
        Arc::new(move |req: &mut Request| {
            let start = Instant::now();
            let outcome = next(req);
            let micros = start.elapsed().as_micros();
            req.extras
                .insert("request.micros".to_string(), micros.to_string());
            outcome
        }) as Handler
    })
}
```

listing 228: instant around the inner call, micros published for the log outside (source: rust/api/src/middleware.rs, lines 89-100)

One rust-specific observation the java chapter could not make: because the response is a value and nothing is on the wire until the connection loop writes it, a timing layer could add a duration header by mutating the returned response, the lane java lost to sendResponseHeaders committing early. This book keeps the contract the siblings froze, timing travels outward through extras into the log line, and notes the open lane rather than using it, because a Server-Timing header is an observability chapter decision, not a middleware one.

20.6. wiring order, and the tests that pin it

The order of the four layers is contract. Request id outermost, because every other artifact quotes it. Access log second, outside recover, so a recovered 500 is logged as the 500 the client saw. Timing third, inside the log, so the log can carry the number. Recover innermost, next to the handler, where every other layer's post-call code still runs on its answer:

```
pub fn stack(sink: Arc<dyn LogSink>) -> Vec<Layer> {
    vec![
        request_id(),
        access_log(Arc::clone(&sink)),
        timing(),
        recover(sink),
    ]
}
```

listing 229: the contract stack in wiring order, one vec entry per layer (source: rust/api/src/middleware.rs, lines 122-129)

The counterexample test is the order's own: it wires recover outside the access log on purpose, panics a route, and asserts the client still sees the 500 while the access log stayed silent, the precise failure mode the order exists to prevent. The socket test drives the full stack over a real listener and asserts the id header on both lanes, the envelope quoting the same id, and both log lines landing, with the poll the harness needs because a log line written after the inner call returns can land moments after the client already has its bytes, the same race the java chapter met. On the pinned 1.99.0 build, dated 2026-10-13, the middleware's test file carries 12 green tests: the chain order, the three id births, the id on the error lane, the recovered envelope with the cause hidden and logged, string and opaque payloads, the panic crossing without the layer, the access log on both lanes, the timing pickup, the crlf spelling, the stack order with the recovered 500 logged, the wrong-order counterexample, and the socket run.

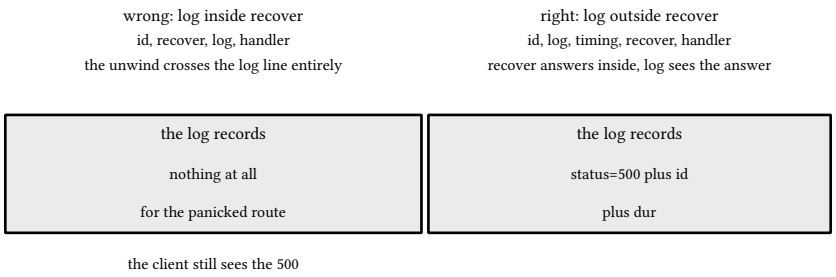


figure 29: wrong order versus right: where the log sits decides what it sees

sources: `std::panic` with `catch_unwind`, `AssertUnwindSafe`, and `panic_any`, `std::any::Any` downcasting, `std::backtrace` `Backtrace` and `force_capture`, and `std::time::Instant`, read from the offline 1.99.0 html share under `~/.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html`, accessed 2026-10-13. The layer contract, the id resolution order, the access log fields, the recover discipline, and the wiring order from the sibling middleware chapters, go book 4 chapter 20 and java book 4 chapter 20 of this corpus, including the java chapter's logged race and the go chapter's statusWriter wrapper this module does not need. All measurements, 12 tests, the three id births, the envelope hiding, the crlf spelling, and the counterexample silence, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned build, dated 2026-10-13.

21. the user resource

The service's first real resource is the user: an id, an email, a display name, a role set, a creation instant, and a version. This chapter builds the resource layer on the chapter 19 kernel in `rustapi::user`: the representation and its two projections, create with validation from first principles, the store as a port with an in-memory implementation, the password seam carrying hand-rolled pbkdf2 because the zero-dependency law leaves no other option, keyset pagination with byte-frozen cursors, the etag contract on read, and the vector-driven tests that pin all of it over a real socket. The go book built the same layer, book 4, ch 20 (go 1.27, the user resource), the java book the same, book 5, ch 21 (java 27, the user resource), and this chapter keeps their contracts while the 16 frozen vectors measure the parity.

One staging decision stated up front. The authn family is chapter 22 and the authz family chapter 23, and until they land no caller is authenticated: the get route answers the full record to any caller the way the java chapter's did at this point in its arc, while the list route already answers the public projection because vector 13 pins that projection byte for byte. Chapter 23 tightens the get route with owner-or-admin and upgrades the list projection for admins, and neither change touches this chapter's wire shape.

21.1. the representation and its two projections

The stored record is one struct, and the wire is a method on it, two of them, because the list projection is the contract's too:

```
pub struct User {
    pub id: String,
    pub email: String,
    pub display_name: String,
    pub roles: Vec<String>,
    pub created_at_millis: i64,
    pub version: i64,
    pub password_hash: String,
}

impl User {
    pub fn to_json(&self) -> Json {
        Json::Obj(vec![
            ("id".to_string(), Json::Str(self.id.clone())),
            ("email".to_string(), Json::Str(self.email.clone())),
            (
                "display_name".to_string(),
                Json::Str(self.display_name.clone()),
            ),
            (
                "roles".to_string(),
                Json::Arr(self.roles.iter().map(|r| Json::Str(r.clone())).collect()),
            ),
            (
                "created_at".to_string(),
                Json::Str(iso8601_utc(self.created_at_millis)),
            ),
            ("version".to_string(), Json::Int(self.version)),
        ])
    }

    pub fn public_json(&self) -> Json {
        Json::Obj(vec![
            ("id".to_string(), Json::Str(self.id.clone())),
            (
                "display_name".to_string(),
                Json::Str(self.display_name.clone()),
            ),
            (
                "created_at".to_string(),
                Json::Str(iso8601_utc(self.created_at_millis)),
            ),
            ("version".to_string(), Json::Int(self.version)),
        ])
    }
```

listing 230: the record and its two writers: the members on the wire are the members the writer lists, member order frozen by the vectors (source: rust/api/src/user.rs, lines 30-74)

The explicit writer is what keeps the password hash off the wire: go needed a dash tag on `PasswordHash` so its reflection serializer would skip it, java needed the writer to list its members, and rust's version of the java answer writes what it lists and nothing else exists. The member order is not style, it is the contract: the strong etag is the sha256 of these exact bytes, and the frozen vector's etag pins the order `id`, `email`, `display_name`, `roles`, `created_at`, `version`. The creation instant is stored as unix milliseconds and rendered by `iso8601_utc`, arithmetic on the days-from-civil relation rather than a library calendar, so `1790830800000` renders as `2026-10-01T05:00:00Z` and a nonzero millisecond tail renders as a three-digit fraction:

```
pub fn iso8601_utc(millis: i64) -> String {
    let secs = millis.div_euclid(1000);
    let frac = millis.rem_euclid(1000);
    let days = secs.div_euclid(86400);
    let tod = secs.rem_euclid(86400);
    let (year, month, day) = civil_from_days(days);
    let (h, m, s) = (tod / 3600, (tod % 3600) / 60, tod % 60);
    if frac == 0 {
        format!("{year:04}-{month:02}-{day:02}T{h:02}:{m:02}:{s:02}Z")
    } else {
        format!("{year:04}-{month:02}-{day:02}T{h:02}:{m:02}:{s:02}.{frac:03}Z")
    }
}

fn civil_from_days(z: i64) -> (i64, i64, i64) {
    let z = z + 719_468;
    let era = z.div_euclid(146_097);
    let doe = z.rem_euclid(146_097);
    let yoe = (doe - doe / 1460 + doe / 36524 - doe / 146_096) / 365;
    let y = yoe + era * 400;
    let doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
    let mp = (5 * doy + 2) / 153;
    let d = doy - (153 * mp + 2) / 5 + 1;
    let m = if mp < 10 { mp + 3 } else { mp - 9 };
    (if m ≤ 2 { y + 1 } else { y }, m, d)
}
```

listing 231: millis to iso 8601 by arithmetic: the vector instants render exactly, fractions only when nonzero (source: rust/api/src/user.rs, lines 86-111)

21.2. create, validation, and the 422 versus 409 boundary

The create handler is the decode ladder from chapter 19 with a resource on top, and the order is the contract: the ladder answers 415, 413, the parse 400, and the unknown member 400, the typed member read answers a 400 for a non-string value, validation sees clean strings, and only then does the store answer:

```
fn create(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let body = bind_json(req, &["email", "password", "display_name"])?;
    let email = member(&body, "email")?;
    let password = member(&body, "password")?;
    let display_name = member(&body, "display_name")?;
    let details = validate_register(&email, &password, &display_name);
    if !details.is_empty() {
        return Err(ApiError::validation(details));
    }
    let mut user = User {
        id: (deps.new_id)(),
        email: normalize_email(&email),
        display_name,
        roles: Vec::new(),
        created_at_millis: (deps.now_millis)(),
        version: 1,
        password_hash: (deps.hash_password)(&password),
    };
    if let Err(err) = deps.store.create(&mut user) {
        return match err {
            StoreError::Conflict => Err(ApiError::fail(Code::Conflict, "email already registered")),
            StoreError::NotFound => Err(ApiError::fail(Code::Internal, "internal error")),
        };
    }
    Ok(Response::json(201, &user.to_json())
        .with_header("location", &format!("/api/users/{}", user.id))
        .with_header("etag", &user.etag()))
}
```

listing 232: strict bind, typed members, every rule checked, hash, then the store and the 201 with location and etag (source: rust/api/src/user.rs, lines 612-639)

The rules come from the contract, not a crate, because the numbers should be greppable: passwords 12 to 128 characters, display names 1 to 64, one email shape, every rule reporting into one details array in field order so one round trip tells the client everything wrong:

```rust
pub fn validate_register(email: &str, password: &str, display_name: &str) -> Vec<String> {
    let mut details = Vec::new();
    if email.is_empty() {
        details.push("email is required".to_string());
    } else if !valid_email(email) {
        details.push("email is not a valid address".to_string());
    }
    let password_chars = password.chars().count();
    if password.is_empty() {
        details.push("password is required".to_string());
    } else if password_chars < MIN_PASSWORD_CHARS {
        details.push("password must be at least 12 characters".to_string());
    } else if password_chars > MAX_PASSWORD_CHARS {
        details.push("password must be at most 128 characters".to_string());
    }
    if display_name.is_empty() {
        details.push("display_name is required".to_string());
    } else if display_name.chars().count() > MAX_DISPLAY_NAME_CHARS {
        details.push("display_name must be at most 64 characters".to_string());
    }
    details
}

pub fn valid_email(email: &str) -> bool {
    if email.chars().count() > 254 {
        return false;
    }
    if email.chars().any(|c| c.is_whitespace() || c.is_control()) {
        return false;
    }
    let Some((local, domain)) = email.split_once('@') else {
        return false;
    };
    if local.is_empty() || email.matches('@').count() != 1 {
        return false;
    }
    if domain.is_empty()
        || !domain.contains('.')
        || domain.starts_with('.')
        || domain.ends_with('.')
    {
        return false;
    }
    true
}
```

listing 233: one detail per violated rule in field order, and the email shape: one at sign, dotted domain, no whitespace, 254 cap (source: rust/api/src/user.rs, lines 523-567)

VERIFY: the ceiling counts codepoints, not bytes

Go's `len` counts bytes, so a 64-character name of three-byte characters is 192 bytes and far over its 64-byte ceiling. Rust's `chars().count()` counts codepoints, and this lane pins the ceiling there deliberately: a 64-character name of 世 characters is exactly at the ceiling and registers, the 65th is refused, and both are test cases. Java's `String.length` would have counted utf-16 units, a third answer, which is why the distinction is stated rather than left to whatever a language's `len` happens to do.

The 422 and the 409 are different animals. A rule violation is about the request alone, checkable without touching state, 422 validation. The email conflict is about the store's contents, 409 conflict, and it surfaces only after validation passed. The store speaks outcomes, the handler maps them to statuses, and no layer does the other's job. Normalization closes the last gap: emails are lower-cased before the store sees them, so `Ada@Example.ORG` cannot dodge a conflict with `ada@example.org`, pinned by registering both spellings.

21.3. the store seam

The persistence port is three methods, create, get, list, frozen now because chapter 24 implements the same trait over a real engine and two chapters build against one shape:

```
pub trait UserStore: Send + Sync {
    fn create(&self, user: &mut User) -> Result<(), StoreError>;
    fn get(&self, id: &str) -> Result<User, StoreError>;
    fn list(&self, after: CursorKey, limit: usize) -> Vec<User>;
}

struct MemInner {
    by_id: HashMap<String, User>,
    by_email: HashMap<String, String>,
    order: BTreeMap<(i64, String), String>,
}

pub struct MemStore {
    inner: Mutex<MemInner>,
}
```

listing 234: the port, and the in-memory state: two maps and the keyset order in a b-tree (source: rust/api/src/user.rs, lines 434-448)

Every method runs under one mutex, the whole locking story at this size, and the keyset order is the b-tree's own: `(created_at, id)` keys make `list` a range walk that never sorts and never scans, with the bootstrap rule and the version stamp inside create:

```
impl UserStore for MemStore {
    fn create(&self, user: &mut User) -> Result<(), StoreError> {
        let mut inner = self.inner.lock().unwrap();
        if inner.by_email.contains_key(&user.email) {
            return Err(StoreError::Conflict);
        }
        if user.roles.is_empty() {
            if inner.by_id.is_empty() {
                user.roles = vec![ROLE_ADMIN.to_string(), ROLE_USER.to_string()];
            } else {
                user.roles = vec![ROLE_USER.to_string()];
            }
        }
        user.version = 1;
        inner.by_email.insert(user.email.clone(), user.id.clone());
        inner
            .order
            .insert((user.created_at_millis, user.id.clone()), user.id.clone());
        inner.by_id.insert(user.id.clone(), user.clone());
        Ok(())
    }

    fn get(&self, id: &str) -> Result<User, StoreError> {
        self.inner
            .lock()
            .unwrap()
            .by_id
            .get(id)
            .cloned()
            .ok_or(StoreError::NotFound)
    }

    fn list(&self, after: CursorKey, limit: usize) -> Vec<User> {
        let inner = self.inner.lock().unwrap();
        inner
            .order
            .range((
                std::ops::Bound::Excluded((after.t, after.i)),
                std::ops::Bound::Unbounded,
            ))
            .filter_map(|(_, id)| inner.by_id.get(id).cloned())
            .take(limit)
            .collect()
    }
```

listing 235: bootstrap roles for the first user, version 1, and list as a range strictly after the cursor position (source: rust/api/src/user.rs, lines 477-520)

A seed method stores a record exactly as given, for fixtures and later chapters, and it exists because this chapter has no update route: the walk fixture needs a version-2 record and chapter 25's scope is where updates arrive.

21.4. the password seam, primitives not crates

Storing plaintext is not an option and std ships no kdf, so the chapter builds one from the same primitives a crypto crate would: sha256 from FIPS 180-4, hmac over it per RFC 2104, pbkdf2 on top per RFC 8018, all in this module, all verified against published vectors:

```
pub fn pbkdf2_hmac_sha256(password: &[u8], salt: &[u8], iterations: u32) -> [u8; 32] {
    let mut block_input = salt.to_vec();
    block_input.extend_from_slice(&1u32.to_be_bytes());
    let mut u = hmac_sha256(password, &block_input);
    let mut out = u;
    for _ in 1..iterations {
        u = hmac_sha256(password, &u);
        for (slot, byte) in out.iter_mut().zip(u) {
            *slot ^= byte;
        }
    }
    out
}

pub fn hash_password(password: &str) -> String {
    let salt = entropy_bytes(PBKDF2_SALT_BYTES);
    let digest = pbkdf2_hmac_sha256(password.as_bytes(), &salt, PBKDF2_ITERATIONS);
    format!(
        "pbkdf2-sha256${}${}${}",
        PBKDF2_ITERATIONS,
        hex(&salt),
        hex(&digest)
    )
}
```

listing 236: pbkdf2 as hmac chained c times, xor-folded, and the self-describing stored form (source: rust/api/src/user.rs, lines 256-280)

The stored string carries its own parameters, algorithm, iterations, salt, digest, so the authn chapter verifies against a value that explains itself, and the comparison in verify folds a xor across both digests rather than comparing bytes, so the answer does not leak through early exit. The salt comes from the one entropy faucet std offers, per-instance `RandomState` keys, the same seam the request-id mint uses. Measured on the pinned build, dated 2026-10-15: one 100,000-iteration hash costs 2 to 3 s under the cargo test debug profile and 120 to 140 ms under `--release`, which is why every fixture injects a fake hasher and the production cost is paid exactly once per registration, on purpose. The tests cross-check against an independent oracle, the RFC 4231 hmac vector and pbkdf2 outputs derived from python 3's `hashlib` on this machine the same day, so the implementation is fitted to the standard, not to itself.

21.5. keyset pagination with frozen cursor bytes

Offset pagination asks for row 86041 onward and fails twice: the store walks and discards every skipped row, and offsets are positions in a moving file, so one registration between requests makes a row appear twice or vanish. Keyset pagination remembers where it stopped. The order is the pair, creation millisecond then id, and the cursor is that pair as json, base64url without padding, opaque to clients and byte-frozen by vector 13:

```
pub fn key_of(user: &User) -> CursorKey {
    CursorKey {
        t: user.created_at_millis,
        i: user.id.clone(),
    }
}

pub fn encode_cursor(key: &CursorKey) -> String {
    let payload = Json::Obj(vec![
        ("t".to_string(), Json::Int(key.t)),
        ("i".to_string(), Json::Str(key.i.clone())),
    ]);
    base64url_encode(payload.write().as_bytes())
}

pub fn decode_cursor(text: &str) -> Option<CursorKey> {
    let raw = base64url_decode(text)?;
    let parsed = Json::parse(std::str::from_utf8(&raw).ok()?).ok()?;
    let t = parsed.get("t").and_then(Json::as_i64)?;
    let i = parsed.str_of("i")?;
    if t == 0 || i.is_empty() {
        return None;
    }
    Some(CursorKey {
        t,
        i: i.to_string(),
    })
}

pub fn parse_limit(raw: Option<&str>) -> Option<usize> {
    match raw {
        None | Some("") => Some(DEFAULT_LIMIT),
        Some(text) => {
            let n: usize = text.parse().ok()?;
            (1..=MAX_LIMIT).contains(&n).then_some(n)
        }
    }
}
```

listing 237: the keyset key, the cursor codec, and the limit clamp 1 to 100 with the default 20 (source: rust/api/src/user.rs, lines 395-432)

The decode path is strict in the direction of everything else in the kernel: a cursor is either one this service issued or garbage, never a missing page, so every failure down the base64, the json, and the member checks answers the same 422, and the test pins the frozen bytes `eyJ0IjoxNzkwODMwODAzMDAwLCJpIjoiMDFhMGY1ZDUtNzgzOC03MTAzLThhN2YtYmFhOThlMjBmODA4In0` as the encoding of grace's position, 2026-10-01T05:00:03Z plus her id. The handler reads `limit + 1` rows so the presence of one extra row is the whole has-more computation, no counts, no second query:

```
fn list(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let limit = parse_limit(req.query_param("limit").as_deref()).ok_or_else(|| {
        ApiError::validation(vec![
            "limit must be an integer between 1 and 100".to_string(),
        ])
    })?;
    let after = match req.query_param("cursor") {
        Some(text) if !text.is_empty() => decode_cursor(&text).ok_or_else(|| {
            ApiError::validation(vec!["cursor is not one this service issued".to_string()])
        })?,
        _ => CursorKey {
            t: 0,
            i: String::new(),
        },
    };
    let mut page = deps.store.list(after, limit + 1);
    let more = page.len() > limit;
    if more {
        page.truncate(limit);
    }
    let items: Vec<Json> = page.iter().map(|u| u.public_json()).collect();
    let next = if more {
        Json::Str(encode_cursor(&key_of(
            page.last().expect("page holds a row"),
        )))
    } else {
        Json::Null
    };
    let body = Json::Obj(vec![
        ("items".to_string(), Json::Arr(items)),
        ("next_cursor".to_string(), next),
    ]);
    Ok(Response::json(200, &body))
}
```

listing 238: limit plus one read, public projection items, the cursor names the last row kept, null on the last page (source: rust/api/src/user.rs, lines 671-704)

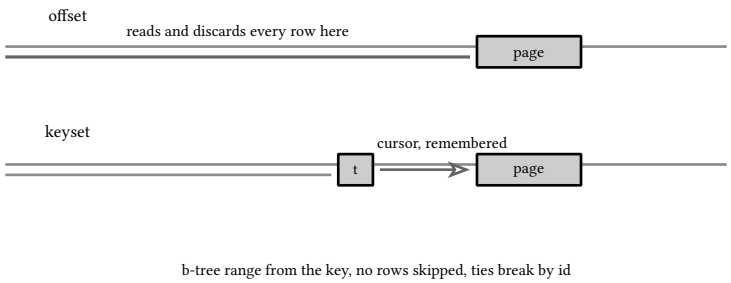


figure 30: offset walks the whole prefix, keyset jumps to the remembered position

21.6. get and the etag

The etag is not stored, it is computed: sha256 over the exact wire bytes, hex, quoted, three lines on the record:

```
    pub fn wire(&self) -> Vec<u8> {
        self.to_json().write().into_bytes()
    }

    pub fn etag(&self) -> String {
        let sum = sha256(&self.wire());
        format!("\"{}\"", hex(&sum))
    }
}
```

listing 239: wire bytes once, sha256 hex quoted: the etag is a function of the body so the two cannot disagree (source: rust/api/src/user.rs, lines 76-84)

The get handler is four moves, and the `If-None-Match` comparison is the rfc's, not string equality: a star matches any current representation, a comma list matches when the etag appears in it, and both comparisons are weak, so a `W/` prefix still counts:

```
fn get(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let id = req.param("id").unwrap_or_default();
    let user = deps
        .store
        .get(id)
        .map_err(|_| ApiError::fail(Code::NotFound, "user not found"))?;
    let etag = user.etag();
    if none_match(&etag, req.header("if-none-match")) {
        return Ok(Response::empty(304).with_header("etag", &etag));
    }
    Ok(Response::json(200, &user.to_json()).with_header("etag", &etag))
}

fn none_match(etag: &str, header: Option<&str>) -> bool {
    let Some(header) = header else {
        return false;
    };
    for candidate in header.split(',') {
        let token = candidate.trim();
        if token == "*" {
            return true;
        }
        let bare = token.strip_prefix("W/").unwrap_or(token);
        if bare == etag {
            return true;
        }
    }
    false
}
```

listing 240: 404 or 304 or 200, and the rfc 9110 none-match rules: star, list, weak (source: rust/api/src/user.rs, lines 641-669)

The 304 rides the kernel's bodyless rule from chapter 19, the empty frame with the etag and no content-length, and the same etag string rides both answers so a client caching on it cannot be misled.

21.7. the vectors drive the tests

The fixture owns everything time-dependent: the frozen ids come straight out of the vector files, the clock starts at the contract's origin `1790830800000`, and the hasher is a fake, so every byte out is deterministic:

```
    let clock = Arc::new(Mutex::new(T0));
    let clock_for_deps = Arc::clone(&clock);
    let deps = Arc::new(Deps {
        store: Arc::clone(&store) as Arc<dyn UserStore>,
        now_millis: Box::new(move || *clock_for_deps.lock().unwrap()),
        new_id: seed_ids_source(ids),
        hash_password: Box::new(|password| format!("fake:{password}")),
    });
    let mut router = Router::new();
    register_kernel_routes(&mut router, Arc::new(Lifecycle::new()), "test-1");
    register_routes(&mut router, deps);
    let server = common::boot(router.handler());
    Fixture { store, server }
}

fn vector(name: &str) -> Json {
    let path = format!("{}/contract/testdata/{name}", env!("CARGO_MANIFEST_DIR"));
    let raw = std::fs::read_to_string(path).unwrap_or_else(|err| panic!("read {name}: {err}"));
    Json::parse(&raw).unwrap_or_else(|err| panic!("parse {name}: {err}"))
}
```

listing 241: the fixture: vector ids, frozen clock, fake hash, real routes on an ephemeral port (source: rust/api/tests/user.rs, lines 51-70)

Vector 01 is the parity proof: the register request from the file answers 201 with the vector's `Location`, the vector's `ETag`, and a body that parses equal to the vector's body. The etag assertion is the byte-exactness, since the etag is the sha256 of the body and the vector froze both. Vector 02 replays the same idempotency key and answers 409 here, stated as the staging it is: the idempotency layer is chapter 25′s concurrency family and lands there. Vector 03 pins the conflict envelope, code, message, a parseable request id, no details member. Vector 13 seeds the three walk records directly, ada patched to version 2 through the seed seam, and walks both pages: the first page equals the vector's body whole, cursor bytes included, the second returns the remainder with a null cursor. On the pinned 1.99.0 build, dated 2026-10-13, the user test file carries 16 green tests: the three vector runs, the validation table with the codepoint ceiling, the decode ladder, the normalized conflict, the cursor round trip and garbage refusal, the limit bounds and the default 20, the etag contract with all four none-match forms, the store keyset and tie order under 8 concurrent hammering threads, the sha256, hmac, and pbkdf2 oracles, the production hasher's fresh salts, and the iso rendering of the vector instants.

sources: RFC 9110, HTTP Semantics, section 8.8.3 on etags and section 13.1.2 on `If-None-Match`, RFC 8018 for the pbkdf2 construction, RFC 2104 for hmac, RFC 4231 test case 1 for the hmac-sha256 vector, and FIPS 180-4 for sha256, all at rfc-editor.org and nvlpubs.nist.gov, accessed 2026-10-13. The pbkdf2 outputs at iterations 1, 2, and 4096 and the sha256 known answers derived from python 3 `hashlib` on this machine, dated 2026-10-13, as the independent oracle. The envelope, the ladder, the bootstrap rule, the projection, the cursor bytes, and the etag values from the sibling user chapters, go book 4 chapter 21 and java book 4 chapter 21 of this corpus, and the frozen vectors under `books/go/api/contract/testdata` hashed byte-exact into this lane's `contract/testdata`. The hash cost, 2 to 3 s debug and 120 to 140 ms release, re-measured on 2026-10-15 under `cargo test` and `cargo test --release` on the pinned build. All other measurements, 16 tests, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true`, dated 2026-10-13.

22. authn

Chapter 21 stored a password hash and never used it. This chapter is the use, in `rustapi::authn`: a json web token assembled from the sha256 and hmac primitives chapter 21 already proved against published vectors, the session family that gives a token something to be revoked through, refresh rotation with reuse detection that kills a stolen family, the login endpoint whose two failure branches cost the same and answer the same bytes, and the identity layer that resolves a caller over either channel and installs the actor everything later reads. The go book built the same machine over argon2id, book 4, ch 21 (go 1.27, authn), the java book the smaller token-only scope, book 5, ch 22 (java 27, authn), and this lane takes the go scope on the java argument: the refresh family is where the revocation story lives, and the primitives are already paid for.

One staging statement, chapter 21′s own: until chapter 23′s guard lands, no route is closed by this chapter. The identity layer installs an actor or does not, the way go's `Identify` did, and a request that carries nothing still reaches the resource routes it reached before. The routes that demand an actor here are the ones this chapter owns, login, refresh, logout, and the last demands one of its own.

22.1. the token, three segments and one primitive

A json web token is a header, a claims object, and a signature over the first two, base64url without padding, dots between. That is the whole format, and every byte of it comes from this crate: the json writer from the kernel, the encoder and hmac from the user module. The claims are five, in alphabetical wire order so the same secret and the same facts always produce the same bytes, which is what lets a test freeze a token exactly:

```
#[derive(Clone, Debug, PartialEq)]
pub struct Claims {
    pub exp: i64,
    pub iat: i64,
    pub iss: String,
    pub sid: String,
    pub sub: String,
}

impl Claims {
    fn to_json(&self) -> Json {
        Json::Obj(vec![
            ("exp".to_string(), Json::Int(self.exp)),
            ("iat".to_string(), Json::Int(self.iat)),
            ("iss".to_string(), Json::Str(self.iss.clone())),
            ("sid".to_string(), Json::Str(self.sid.clone())),
            ("sub".to_string(), Json::Str(self.sub.clone())),
        ])
    }

    fn from_json(value: &Json) -> Option<Claims> {
        Some(Claims {
            exp: value.get("exp").and_then(Json::as_i64)?,
            iat: value.get("iat").and_then(Json::as_i64)?,
            iss: value.str_of("iss")?.to_string(),
            sid: value.str_of("sid")?.to_string(),
            sub: value.str_of("sub")?.to_string(),
        })
    }
}

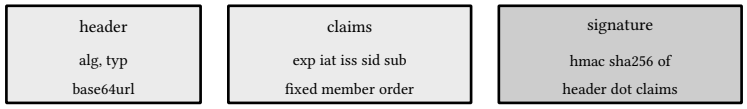
#[derive(Debug, PartialEq, Eq)]
pub enum TokenError {
    Malformed,
    BadSignature,
    AlgMismatch,
    Expired,
    WrongIssuer,
}
```

listing 242: five claims in fixed wire order, and the five refusals: a policy-free encoding of every way a token can die (source: rust/api/src/authn.rs, lines 34-72)

```rust
pub fn sign_hs256(secret: &[u8], claims: &Claims) -> String {
    let header = Json::Obj(vec![
        ("alg".to_string(), Json::Str(ALG_HS256.to_string())),
        ("typ".to_string(), Json::Str(TYPO_JWT.to_string())),
    ]);
    let head = base64url_encode(header.write().as_bytes());
    let payload = base64url_encode(claims.to_json().write().as_bytes());
    let signing_input = format!("{head}.{payload}");
    let sig = hmac_signing(secret, &signing_input);
    format!("{signing_input}.{sig}")
}

fn hmac_signing(secret: &[u8], signing_input: &str) -> String {
    base64url_encode(&crate::user::hmac_sha256(secret, signing_input.as_bytes()))
}
```

listing 243: sign: the kernel writer serializes, chapter 21's hmac signs the first two segments, base64url unpadded (source: rust/api/src/authn.rs, lines 74-88)



no roles claim: authorization reads the store, so a token stays cheap to revoke through its sid

figure 31: three segments, the signature covers exactly the first two, dots included

The `sid` claim is the whole revocation seam. There is no roles claim, the same reasoning both siblings kept: a role baked into a token goes stale the moment an admin revokes it, while a token that names a session dies the moment the session does, which is the next section's machinery.

22.2. verification, order and constant time

Verification is a fixed order and the order is the security: shape, algorithm pin, signature, then claims. The algorithm check runs before any payload byte is trusted, so the classic `alg: none` forgery and a token correctly signed under a foreign algorithm both die at the header, and the signature comparison is a xor fold across both digests:

```rust
pub fn verify_hs256(token: &str, secret: &[u8], now_secs: i64) -> Result<Claims, TokenError> {
    let parts: Vec<&str> = token.split('.').collect();
    if parts.len() ≠ 3 {
        return Err(TokenError::Malformed);
    }
    let header_raw = base64url_decode(parts[0]).ok_or(TokenError::Malformed)?;
    let header_text = std::str::from_utf8(&header_raw).map_err(|_| TokenError::Malformed)?;
    let header = Json::parse(header_text).map_err(|_| TokenError::Malformed)?;
    let Json::Obj(header_members) = header else {
        return Err(TokenError::Malformed);
    };
    let find = |key: &str| -> Option<String> {
        header_members
            .iter()
            .find(|(k, _)| k == key)
            .and_then(|(_, v)| v.as_str())
            .map(str::to_string)
    };
    if find("alg").as_deref() ≠ Some(ALG_HS256) || find("typ").as_deref() ≠ Some(TYPO_JWT) {
        return Err(TokenError::AlgMismatch);
    }
    let sig = base64url_decode(parts[2]).ok_or(TokenError::Malformed)?;
    let want = crate::user::hmac_sha256(secret, format!("{}.{}", parts[0], parts[1]).as_bytes());
    if !constant_time_eq(&want, &sig) {
        return Err(TokenError::BadSignature);
    }
    let payload_raw = base64url_decode(parts[1]).ok_or(TokenError::Malformed)?;
    let payload_text = std::str::from_utf8(&payload_raw).map_err(|_| TokenError::Malformed)?;
    let claims_json = Json::parse(payload_text).map_err(|_| TokenError::Malformed)?;
    let claims = Claims::from_json(&claims_json).ok_or(TokenError::Malformed)?;
    if claims.iss ≠ ISSUER {
        return Err(TokenError::WrongIssuer);
    }
    if now_secs ≥ claims.exp {
        return Err(TokenError::Expired);
    }
    Ok(claims)
}

fn constant_time_eq(a: &[u8], b: &[u8]) -> bool {
    let mut diff = (a.len() ^ b.len()) as u8;
    for (x, y) in a.iter().zip(b) {
        diff |= x ^ y;
    }
    diff == 0
}
```

listing 244: verify: shape, alg pin, constant-time signature, then the claim rules, issuer and expiry against the clock (source: rust/api/src/authn.rs, lines 90-135)

VERIFY: the constant-time property, stated not measured

The fold starts from the length difference and xors every byte of both digests unconditionally, so the answer does not depend on where the bytes differ, only on whether they do. A timing test would measure the optimizer, not the property, so the tests pin tamper rejection, one flipped signature byte and one wrong secret both refused, and the chapter pins the contract: equality decided in time independent of the position of the first difference.

Cost is part of the story, measured on this machine, dated 2026-10-15: one hand-rolled hmac over the 150 byte signing span answers in 1.5 to 2 microseconds in a `-O` binary driving the same primitive, while chapter 21 measured one 100,000-iteration pbkdf2 at 120 to 140 ms under `--release`, nearly five orders of magnitude apart. That gap is the entire argument for tokens as the per-request credential and passwords as the once-per-login one.

22.3. the session family and reuse detection

The session is the server-side state one login owns, and the refresh token itself never lives in it: only `sha256(token)`, so a store dump cannot be replayed. The port is five verbs, frozen now the way chapter 21 froze the user port, with rotation carrying the reuse rule in its contract:

```rust
pub struct Session {
    pub id: String,
    pub user_id: String,
    pub refresh_hash: [u8; 32],
    pub expires_at_millis: i64,
    pub revoked: bool,
}

impl Session {
    pub fn alive(&self, now_millis: i64) -> bool {
        !self.revoked && now_millis < self.expires_at_millis
    }
}

pub fn hash_token(token: &str) -> [u8; 32] {
    sha256(token.as_bytes())
}

#[derive(Debug, PartialEq, Eq)]
pub enum SessionError {
    UnknownRefresh,
    SessionRevoked,
    NoSuchSession,
}

pub trait SessionStore: Send + Sync {
    fn create(&self, session: &Session) -> Result<(), String>;
    fn get(&self, id: &str) -> Result<Session, SessionError>;
    fn get_by_refresh(&self, hash: [u8; 32]) -> Result<Session, SessionError>;
    fn rotate(
        &self,
        current: [u8; 32],
        next: [u8; 32],
        expires_at_millis: i64,
    ) -> Result<Session, SessionError>;
    fn revoke(&self, id: &str) -> Result<(), SessionError>;
}

struct TokenRec {
    sid: String,
    superseded: bool,
}

struct MemInner {
    sessions: HashMap<String, Session>,
    tokens: HashMap<[u8; 32], TokenRec>,
}
```

listing 245: the session record, the hash transform, and the port: rotation is where a stolen family dies (source: rust/api/src/authn.rs, lines 138-184)

```
    fn rotate(
        &self,
        current: [u8; 32],
        next: [u8; 32],
        expires_at_millis: i64,
    ) -> Result<Session, SessionError> {
        let mut inner = self.inner.lock().unwrap();
        let sid = inner
            .tokens
            .get(&current)
            .map(|rec| rec.sid.clone())
            .ok_or(SessionError::UnknownRefresh)?;
        let superseded = inner.tokens.get(&current).is_some_and(|rec| rec.superseded);
        let revoked = inner
            .sessions
            .get(&sid)
            .ok_or(SessionError::UnknownRefresh)?
            .revoked;
        if (superseded || revoked)
            && let Some(session) = inner.sessions.get_mut(&sid)
        {
            session.revoked = true;
            return Err(SessionError::SessionRevoked);
        }
        inner.tokens.insert(
            current,
            TokenRec {
                sid: sid.clone(),
                superseded: true,
            },
        );
        inner.tokens.insert(
            next,
            TokenRec {
                sid: sid.clone(),
                superseded: false,
            },
        );
        let session = inner
            .sessions
            .get_mut(&sid)
            .ok_or(SessionError::UnknownRefresh)?;
        session.refresh_hash = next;
        session.expires_at_millis = expires_at_millis;
        Ok(session.clone())
    }
```

listing 246: rotate under one mutex: a superseded or revoked presenter flips the family dead, and the freshly minted next never enters the store (source: rust/api/src/authn.rs, lines 247-292)

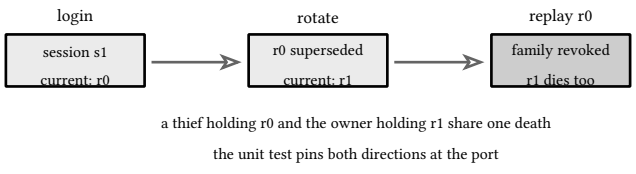


figure 32: reuse detection: the replay of r0 kills r1 with it, the family is one unit

The superseded record is kept, not deleted, because presenting one is the theft signal. The in-memory implementation holds two maps under one mutex, sessions and token hashes, and the engine chapter will implement the same port over durable storage without a handler learning of the change.

22.4. login, prove then issue, both branches the same cost

The login route is chapter 19′s decode ladder wearing one job: bind the two members, prove the password, mint the family. The proof runs against the stored form through an injected verifier, so a test fixture never pays 100,000 iterations. The one security argument is timing: a login that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch, and the fix is one line in the unknown-email branch, burn the same verification against a dummy hash, discard the answer, return the identical envelope:

```rust
fn login(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let body = bind_json(req, &["email", "password"])?;
    let email = member(&body, "email")?;
    let password = member(&body, "password")?;
    if email.is_empty() || password.is_empty() {
        let mut details = Vec::new();
        if email.is_empty() {
            details.push("email is required".to_string());
        }
        if password.is_empty() {
            details.push("password is required".to_string());
        }
        return Err(ApiError::validation(details));
    }
    let user = match (deps.find_user)(&normalize_email(&email)) {
        Some(user) => user,
        None => {
            let _ = (deps.verify)(&deps.dummy_hash, &password);
            return Err(invalid_credentials());
        }
    };
    if !(deps.verify)(&user.password_hash, &password) {
        return Err(invalid_credentials());
    }
    issue(&user, deps)
}

fn issue(user: &User, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let now = (deps.now_millis)();
    let refresh = (deps.new_refresh)();
    let sid = (deps.new_id)();
    let session = Session {
        id: sid.clone(),
        user_id: user.id.clone(),
        refresh_hash: hash_token(&refresh),
        expires_at_millis: now + SESSION_TTL_MILLIS,
        revoked: false,
    };
    deps.sessions
        .create(&session)
        .map_err(|err| ApiError::fail(Code::Internal, err))?;
    let access = access_token(&sid, &user.id, now, deps);
    let body = Json::Obj(vec![
        ("user".to_string(), user.to_json()),
        ("access_token".to_string(), Json::Str(access)),
    ]);
    Ok(Response::json(200, &body).with_header("set-cookie", &cookie_header(&refresh)))
}
```

listing 247: strict bind, presence rules, the dummy burn, then issue: session, refresh cookie, and the access token (source: rust/api/src/authn.rs, lines 490-537)

The counting wrapper around the injected verifier is the test's proof: the unknown-email lane increments the same counter the wrong-password lane does, and the two envelopes, stripped of their request ids, compare byte equal. The success path is two facts, prove then issue, and issue is one movement: mint the refresh token from the injected source, create the session with the token's hash, sign the access token, answer the full user representation plus the token with the cookie beside it.

The email lookup rides a walk, and the reason is a landed seam: chapter 21's port froze create, get, and list, with no by-email read, and this chapter does not touch landed modules. The injected `find_user` closes over `email_lookup_over`, which pages through the keyset order until the normalized email appears. At book scale, a few hundred records, that is a cheap honest price, stated here rather than paid silently by widening a frozen port.

22.5. refresh and logout, the routes that move the family

Refresh reads its credential from either channel, the bearer header or the cookie, and the route's order is the policy. An expired but never-rotated session does not come back to life. A merely revoked one keeps flowing into rotation, so its holders read `session revoked`, not a generic miss. And the next token is minted before the rotation attempt, so a failed rotation discards it unwritten, which is why the reuse test seeds four tokens for three requests:

```
fn refresh(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let token = bearer_of(req).or_else(|| cookie_value(req, COOKIE_NAME));
    let Some(token) = token else {
        return Err(invalid_credentials());
    };
    let now = (deps.now_millis)();
    let hash = hash_token(&token);
    if let Ok(current) = deps.sessions.get_by_refresh(hash)
        && !current.revoked
        && now ≥ current.expires_at_millis
    {
        return Err(invalid_credentials());
    }
    let next = (deps.new_refresh)();
    let session = match deps
        .sessions
        .rotate(hash, hash_token(&next), now + SESSION_TTL_MILLIS)
    {
        Ok(session) ⇒ session,
        Err(SessionError::UnknownRefresh) ⇒ return Err(invalid_credentials()),
        Err(SessionError::SessionRevoked) ⇒ {
            return Err(ApiError::fail(Code::Unauthorized, "session revoked"));
        }
        Err(SessionError::NoSuchSession) ⇒ return Err(invalid_credentials()),
    };
    let access = access_token(&session.id, &session.user_id, now, deps);
    let body = Json::Obj(vec![("access_token".to_string(), Json::Str(access))]);
    Ok(Response::json(200, &body).with_header("set-cookie", &cookie_header(&next)))
}
```

listing 248: refresh: expired answers invalid credentials, superseded answers session revoked, and the family dies inside rotate (source: rust/api/src/authn.rs, lines 539-567)

```
fn logout(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let Some(actor_wire) = req.extras.get(ACTOR_EXTRA) else {
        return Err(ApiError::fail(
            Code::Unauthorized,
            "authentication required",
        ));
    };
    let Some(actor) = Actor::parse_wire(actor_wire) else {
        return Err(ApiError::fail(
            Code::Unauthorized,
            "authentication required",
        ));
    };
    let outcome = deps.sessions.revoke(&actor.session_id);
    if !matches!(outcome, Ok(()) | Err(SessionError::NoSuchSession)) {
        return Err(ApiError::fail(Code::Internal, "internal error"));
    }
    Ok(Response::empty(204).with_header("set-cookie", &clear_cookie_header()))
}

fn access_token(sid: &str, uid: &str, now_millis: i64, deps: &Arc<Deps>) -> String {
    let now_secs = now_millis.div_euclid(1000);
    sign_hs256(
        &deps.secret,
        &Claims {
            exp: now_secs + ACCESS_TOKEN_TTL_SECS,
            iat: now_secs,
            iss: ISSUER.to_string(),
            sid: sid.to_string(),
            sub: uid.to_string(),
        },
    )
```

listing 249: logout demands an actor, tolerates an already-dead session, clears the cookie, and the token mint is one function (source: rust/api/src/authn.rs, lines 569-600)

Logout is the one route in this chapter that demands an actor, and revoking the session is what kills the outstanding access tokens: the bearer stays cryptographically valid for its fifteen minutes, but the resolve below reads the session by `sid` and a dead session resolves no actor, which the test pins by logging out twice.

22.6. the identity layer and its two channels

The identity layer is a chapter 20 layer, one function from handler to handler. It resolves an actor from the bearer token or the session cookie and installs it into the request's `extras` map as its wire form, or installs nothing, and a presented-but-invalid credential and no credential at all are the same non-identity:

```
pub fn resolve(req: &Request, deps: &Arc<Deps>) -> Option<Actor> {
    let now = (deps.now_millis)();
    if let Some(token) = bearer_of(req) {
        let claims = verify_hs256(&token, &deps.secret, now.div_euclid(1000)).ok()?;
        return actor_for_session(&claims.sid, &claims.sub, now, deps);
    }
    if let Some(token) = cookie_value(req, COOKIE_NAME) {
        if token.is_empty() {
            return None;
        }
        let session = deps
            .sessions
            .get_by_refresh(hash_token(&token))
            .ok()
            .filter(|s| s.alive(now))?;
        return actor_for_session(&session.id, &session.user_id, now, deps);
    }
    None
}

fn actor_for_session(sid: &str, uid: &str, now: i64, deps: &Arc<Deps>) -> Option<Actor> {
    let session = deps.sessions.get(sid).ok().filter(|s| s.alive(now))?;
    let user = deps.users.get(uid).ok()?;
    Some(Actor {
        user_id: user.id.clone(),
        session_id: session.id,
        roles: (deps.roles_of)(&user.id),
    })
}

pub fn record_roles(store: Arc<dyn UserStore>) -> RolesFn {
    Box::new(move |id: &str| store.get(id).map(|user| user.roles).unwrap_or_default())
}

pub fn bearer_of(req: &Request) -> Option<String> {
    let value = req.header("authorization")?;
    let (scheme, rest) = value.trim().split_once(' ')?;
    if !scheme.eq_ignore_ascii_case("bearer") {
        return None;
    }
    let token = rest.trim();
    if token.is_empty() || token.contains(' ') {
        return None;
    }
    Some(token.to_string())
}
```

listing 250: bearer first then cookie, both lanes end at the same actor-for-session check, and the scheme word is case-insensitive (source: rust/api/src/authn.rs, lines 410-455)

The bearer lane verifies the signature, then reads the session by `sid`, so revocation kills outstanding tokens, then reads the user, so roles come from the record at request time: a demoted admin loses the role on the next request, not at the next login, pinned by reseeding ada's record between two resolves of the same token. The cookie lane resolves the opaque refresh token by hash and lands at the same check. The scheme word matches `bearer`, `Bearer`, and `BEARER` alike, rfc 9110 section 11.1's case-insensitive auth-scheme, while the token that follows is exact bytes. The actor travels as its json wire form under one `extras` key, and the round trip through `parse_wire` is pinned, the same serialization discipline the cursor used.

22.7. the wiring and the tests

The program grows the family the way the fixture does, one store instance under both families, the secret from RUSTAPI_SECRET when the environment carries one and a fresh 32 random bytes otherwise, the ephemeral lane stated in the code: every token dies with the process that signed it, correct for a dev boot and a lockout for a production one. The dummy hash is the one real pbkdf2 this wiring pays, computed once at boot from the pinned dummy password. The listing is read as the part closes it: the roles_of seam and the guard line are chapters 23 and 24 arriving, and this chapter's own fixture binds record_roles over the store while the shipped wiring binds the grants table:

```
    let wal_path = std::path::Path::new(&cfg.data).join("users.wal");
    let user_engine = match rustapi::store::UserEngine::open(&wal_path) {
        Ok(engine) => engine,
        Err(err) => {
            eprintln!("rustapi: open store {}: {err}", wal_path.display());
            std::process::exit(1);
        }
    };
    let engine = user_engine.engine();
    let users: Arc<dyn user::UserStore> = Arc::new(user_engine);
    let roles = Arc::new(authz::MemRoles::new());
    roles.adopt(&*users);
    let user_deps = Arc::new(user::Deps {
        store: Arc::clone(&users),
        now_millis: Box::new(wall_clock_millis),
        new_id: Box::new(new_uuid_v7),
        hash_password: Box::new(user::hash_password),
    });
    user::register_routes(&mut router, Arc::clone(&user_deps));
    let authn_deps = Arc::new(authn::Deps {
        users: Arc::clone(&users),
        sessions: Arc::new(authn::MemSessions::new()),
        secret: authn::boot_secret(&cfg.secret),
        dummy_hash: user::hash_password(authn::DUMMY_PASSWORD),
        verify: Box::new(user::verify_password),
        find_user: authn::email_lookup_over(Arc::clone(&users)),
        roles_of: authz::table_roles(Arc::clone(&roles) as Arc<dyn authz::RoleStore>),
        now_millis: Box::new(wall_clock_millis),
        new_id: Box::new(new_uuid_v7),
        new_refresh: Box::new(authn::new_refresh_token),
    });
    let service = Arc::new(authn::Service::new(authn_deps));
    service.register_routes(&mut router);
    let authz_deps = Arc::new(authz::Deps {
        users: Arc::clone(&users),
        roles: Arc::clone(&roles) as Arc<dyn authz::RoleStore>,
    });
    authz::register_routes(&mut router, authz_deps);
```

listing 251: the wiring as the part closes it: one store under every family, identity outside the stack, the chapter 23 guard inside it (source: rust/api/src/main.rs, lines 59-96)

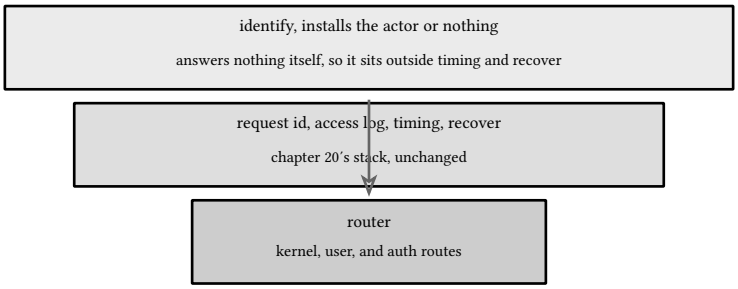


figure 33: the wired order: identity outside, so a defect in it fails closed, past chapter 20's boundary

The identity layer sits outside the whole chapter 20 stack, and the position is the java chapter's boundary argument in rust's shape: this layer answers nothing, so there is nothing of its own to time or log, and a panic inside it escapes recover and closes the connection, fail closed, exactly the boundary java drew around its own filter. Every verdict the service emits still passes through the stack, because the verdicts live in handlers and, from the next chapter, in the guard.

The fixture owns everything time-dependent: the frozen ids straight from the vector chain, the clock pinned at the contract origin, the verifier a fake with a counter, the refresh tokens a seeded queue, and the routes real over an ephemeral port:

```
fn boot_fixture(ids: Vec<String>, refreshes: Vec<String>) -> Fixture {
    let store = Arc::new(MemStore::new());
    let mut ada = User {
        id: ADA_ID.to_string(),
        email: "ada@example.org".to_string(),
        display_name: "ada lovelace".to_string(),
        roles: Vec::new(),
        created_at_millis: T0,
        version: 1,
        password_hash: "fake:correct horse battery staple".to_string(),
    };
    store.create(&mut ada).expect("seed ada");
    let sessions = Arc::new(MemSessions::new());
    let clock = Arc::new(Mutex::new(T0));
    let verify_calls = Arc::new(AtomicUsize::new(0));
    let verify_counter = Arc::clone(&verify_calls);
    let deps_clock = Arc::clone(&clock);
    let svc = Arc::new(rustapi::authn::Service::new(Arc::new(
        rustapi::authn::Deps {
            users: Arc::clone(&store) as Arc<dyn UserStore>,
            sessions: Arc::clone(&sessions) as Arc<dyn SessionStore>,
            secret: SECRET.to_vec(),
            dummy_hash: "fake:dummy".to_string(),
            verify: Box::new(move |stored, password| {
                verify_counter.fetch_add(1, Ordering::SeqCst);
                stored == format!("fake:{password}")
            }),
            find_user: rustapi::authn::email_lookup_over(Arc::clone(&store) as Arc<dyn UserStore>),
            roles_of: rustapi::authn::record_roles(Arc::clone(&store) as Arc<dyn UserStore>),
            now_millis: Box::new(move || *deps_clock.lock().unwrap()),
            new_id: seed_ids_source(ids),
            new_refresh: seeded_refreshes(refreshes),
        },
    )));
    let mut router = Router::new();
    register_kernel_routes(&mut router, Arc::new(Lifecycle::new()), "test-1");
    let user_clock = Arc::clone(&clock);
    let user_deps = Arc::new(rustapi::user::Deps {
        store: Arc::clone(&store) as Arc<dyn UserStore>,
        now_millis: Box::new(move || *user_clock.lock().unwrap()),
        new_id: Box::new(rustapi::httpx::new_uuid_v7),
        hash_password: Box::new(|password| format!("fake:{password}")),
    });
    rustapi::user::register_routes(&mut router, user_deps);
    svc.register_routes(&mut router);
    let identify = svc.identify();
    let server = common::boot(identify(router.handler()));
    Fixture {
        store,
        sessions,
        clock,
        verify_calls,
        server,
    }
}
```

listing 252: the fixture: vector ids, frozen clock, counting verifier, seeded refreshes, identify over the real routes (source: rust/api/tests/authn.rs, lines 224-278)

The frozen token is the discipline. The header, the claims with their `sid` and `sub` from the vector chain, the fixed secret, and the contract's origin instant produce bytes the test carries as a constant, derived offline from python 3's `hmac` and `hashlib` on this machine the same day, so any drift in the writer, the encoder, or the hmac fails a string comparison. Around it, on the pinned 1.99.0 build, dated 2026-10-13, the authn test file carries 16 green tests: the frozen bytes and their round trip, the tamper ladder with the wrong secret, the flipped signature byte, `alg: none` both unsigned and signed, a correctly signed `HS384`, the wrong issuer, and eight malformed shapes, expiry to the second on both sides of the boundary, the case-insensitive scheme with its exact-bytes token, the actor wire round trip, the vector login with the frozen token and the cookie, the two failure branches byte-identical with the dummy burn counted, the presence rules and the decode ladder, the rotation-replay-family chain over the socket, the bearer refresh channel, the expired session's refusal to return, the credentialless 401, logout revoking the family with the second logout refused, the anonymous logout 401, the resolve lanes with roles reread from the record, and the port-level rotation and reuse unit.

sources: RFC 7519 for the json web token claims and the `exp` semantics, RFC 9110 sections 11.1, 11.6.1, and 15.5.1 for the case-insensitive auth-scheme, the bearer challenge semantics, and the 401, at rfc-editor.org, accessed 2026-10-13. The frozen token bytes derived from python 3 `hmac` and `hashlib` on this machine, dated 2026-10-13, as the independent oracle, and the hmac cost, 1.5 to 2 microseconds over the 150 byte span, re-measured 2026-10-15 by a `-O` binary driving the crate's own primitive. The pbkdf2 cost, 120 to 140 ms release and 2 to 3 s debug, is chapter 21's measurement on the same build. The session family, rotation, reuse detection, and the cookie attributes from the go sibling, book 4 chapter 21, and the identity-layer boundary and the smaller scope from the java sibling, book 4 chapter 22, of this corpus. The login vector's request body and user representation from `contract/testdata/04-login-200.json`, the go oracle's file hashed byte-exact into this lane's vectors. All other measurements, 16 tests, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned build, dated 2026-10-13.

23. authz and rbac

Chapter 22 answered who is calling. This chapter answers what that caller may do, in `rustapi::authz`: roles as stored data with a grant and revoke surface, the policy table that maps every route pattern to one plain function, the object-level check whose absence the industry names IDOR, the never-zero-admins guard that refuses a demotion or a removal which would leave the table without one admin, and the 401 versus 403 discipline that keeps the two answers from swapping places. The go book built the same layer riding its mux's own pattern resolution, book 4, ch 22 (go 1.27, authz and rbac), the java book asked its kernel for the pattern instead, book 5, ch 23 (java 27, authz and rbac), and this lane's kernel is this book's own chapter 19 router, whose table cannot be queried, so the guard carries its own matcher and the audit closes the loop from both sides. That divergence is stated where it lives, in the table section.

23.1. authentication versus authorization

Two questions, two layers, two failure codes. Authentication ran in chapter 22′s identity layer and installed an actor or did not. Authorization reads that actor against the route and, on the object routes, against the record. A caller that never proved anything has no authorization to deny, only an authentication demand: 401 means try again with credentials, 403 means do not bother, this identity will never pass. The wiring makes the sequence physical, the identity layer outside the middleware stack, the guard inside it, and by the time a policy runs the actor question is settled, so a policy never parses a token, checks a signature, or hashes a password:

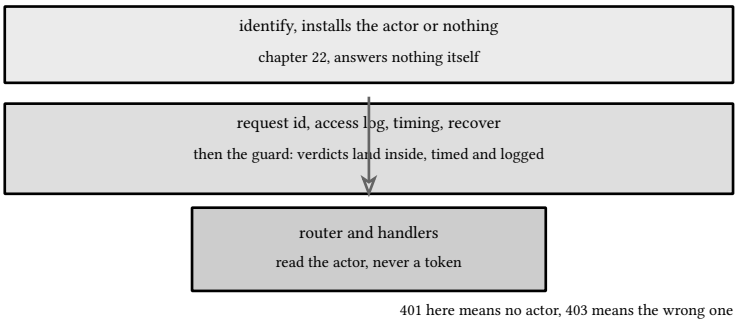


figure 34: the wired order: identity outside everything, guard inside the stack, handler last

The 401 this chapter emits carries the rfc 9110 challenge header, `WWW-Authenticate: Bearer`, which section 11.6.1 requires a 401 to carry and most hand-rolled servers forget, and the two messages stay honest to their layer, `authentication required` against `insufficient role` and `not the owner`.

23.2. policies as functions

A policy is one function from the request and the match it produced to a verdict: `Ok` allows, an `ApiError` denies with its own status. Four of them compose the whole surface, and each is short enough to read in one breath:

```
pub fn actor_of(req: &Request) -> Option<Actor> {
    Actor::parse_wire(req.extras.get(ACTOR_EXTRA)?)
}

fn unauthorized() -> ApiError {
    ApiError::fail(Code::Unauthorized, "authentication required")
        .with_header("www-authenticate", "Bearer")
}

pub fn anyone() -> Policy {
    Arc::new(|_: &Request, _: &RouteMatch| Ok(()))
}

pub fn authenticated() -> Policy {
    Arc::new(|req: &Request, _: &RouteMatch| {
        actor_of(req).ok_or_else(unauthorized)?;
        Ok(())
    })
}

pub fn require_role(role: &'static str) -> Policy {
    Arc::new(move |req: &Request, _: &RouteMatch| {
        let actor = actor_of(req).ok_or_else(unauthorized)?;
        if !actor.has_role(role) {
            return Err(ApiError::fail(Code::Forbidden, "insufficient role"));
        }
        Ok(())
    })
}

pub fn owner_or_admin(users: Arc<dyn UserStore>) -> Policy {
    Arc::new(move |req: &Request, m: &RouteMatch| {
        let actor = actor_of(req).ok_or_else(unauthorized)?;
        let id = m
            .param("id")
            .ok_or_else(|| ApiError::fail(Code::Forbidden, "not the owner"))?;
        users
            .get(id)
            .map_err(|_| ApiError::fail(Code::NotFound, "user not found"))?;
        if actor.user_id != id && !actor.admin() {
            return Err(ApiError::fail(Code::Forbidden, "not the owner"));
        }
        Ok(())
    })
}
```

listing 253: actor read, challenge-building 401, then the four policies: anyone, authenticated, role, owner-or-admin (source: rust/api/src/authz.rs, lines 39-83)

The role vocabulary is two constants, admin and user, and deliberately flat: `has_role` is a contains check, `admin` a shortcut for that one word. The only hierarchy in the system is the explicit outranking rule the owner check applies, an admin passes where an owner would, and it is spelled out at every call site rather than implied by a rank lattice, because the day an implicit `admin implies user` expansion hides a grant from the person auditing the table, that one line of expansion costs more than it ever bought. The java chapter made the same argument and this lane keeps it.

23.3. object-level authorization

Route rules cannot guard a route whose danger lives in the path parameter. `GET /api/users/` followed by someone else's id is the IDOR class, insecure direct object reference, and it survives every route-level check because the route is the same for everyone. The check needs the record, whether it exists at all and who owns it, and the policy extracts the wildcard itself from the match it was handed, so the object check lives in the table and still answers all three questions in the right order:

```
pub fn owner_or_admin(users: Arc<dyn UserStore>) -> Policy {
    Arc::new(move |req: &Request, m: &RouteMatch| {
        let actor = actor_of(req).ok_or_else(unauthorized)?;
        let id = m
            .param("id")
            .ok_or_else(|| ApiError::fail(Code::Forbidden, "not the owner"))?;
        users
            .get(id)
            .map_err(|_| ApiError::fail(Code::NotFound, "user not found"))?;
        if actor.user_id != id && !actor.admin() {
            return Err(ApiError::fail(Code::Forbidden, "not the owner"));
        }
        Ok(())
    })
}
```

listing 254: the object policy: 404 the unknown id first, 403 the stranger, allow owner or admin (source: rust/api/src/authz.rs, lines 69-83)

The store read inside the policy is the price of the 404-first order, and it is stated rather than hidden: the handler fetches again, one row read twice on the object route, in exchange for the unknown id answering the same 404 a nonexistent route would. What the order does not hide is existence from an authenticated caller: a stranger probing ids still splits 404 from 403, and which ids are real is exactly what that split reveals. Both siblings made the same trade and named it, debuggability over disclosure at book scale, and the alternative, 404 for both, is one `if` away if the trade ever turns.

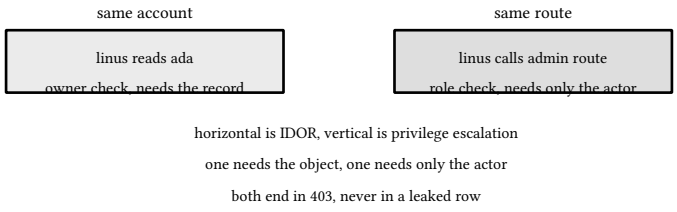


figure 35: horizontal versus vertical: two axes of privilege, two different checks

23.4. roles as stored data and the never-zero-admins guard

Roles are rows, not magic. The port is four verbs over user id to role set, and the invariant that matters lives where the only verbs that can break it run: a change whose post state would leave the whole table without one admin is refused with nothing written, and the refusal is the contract's 409 both siblings froze. The demotion, a revoke of `admin`, and the removal, the whole row, are both covered, because a deletion that empties the admin set is the same lockout a demotion is:

```
#[derive(Debug, PartialEq, Eq)]
pub enum RoleError {
    NotFound,
    LastAdmin,
}

pub trait RoleStore: Send + Sync {
    fn roles_of(&self, id: &str) -> Result<Vec<String>, RoleError>;
    fn grant(&self, id: &str, role: &str) -> Result<Vec<String>, RoleError>;
    fn revoke(&self, id: &str, role: &str) -> Result<Vec<String>, RoleError>;
    fn remove_user(&self, id: &str) -> Result<Vec<String>, RoleError>;
}

pub struct MemRoles {
    inner: Mutex<HashMap<String, Vec<String>>>,
}

fn sorted_roles(roles: &[String]) -> Vec<String> {
```

listing 255: the roles port: two outcomes, NotFound and LastAdmin, the second the 409 lane (source: rust/api/src/authz.rs, lines 159-176)

```
    fn remove_user(&self, id: &str) -> Result<Vec<String>, RoleError> {
        let mut inner = self.inner.lock().unwrap();
        let roles = inner.get(id).cloned().ok_or(RoleError::NotFound)?;
        if roles.iter().any(|r| r == ROLE_ADMIN) && !other_admin_exists(&inner, id) {
            return Err(RoleError::LastAdmin);
        }
        inner.remove(id);
        Ok(roles)
    }
}

impl MemRoles {
    fn set_role(&self, id: &str, role: &str, grant: bool) -> Result<Vec<String>, RoleError> {
        let mut inner = self.inner.lock().unwrap();
        let current = inner.get(id).cloned().ok_or(RoleError::NotFound)?;
        let post = if grant {
            if current.iter().any(|r| r == role) {
                current.clone()
            } else {
                let mut next = current.clone();
                next.push(role.to_string());
                sorted_roles(&next)
            }
        } else if current.iter().any(|r| r == role) {
            current
                .iter()
                .filter(|r| *r != role)
                .cloned()
                .collect::<Vec<_>>()
        } else {
            current.clone()
        };
        if !post.iter().any(|r| r == ROLE_ADMIN) && !other_admin_exists(&inner, id) {
            return Err(RoleError::LastAdmin);
        }
        inner.insert(id.to_string(), post.clone());
        Ok(post)
    }
}
```

listing 256: the guard's body: post state computed first, invariant checked against it, one mutex over the whole move (source: rust/api/src/authz.rs, lines 237-276)

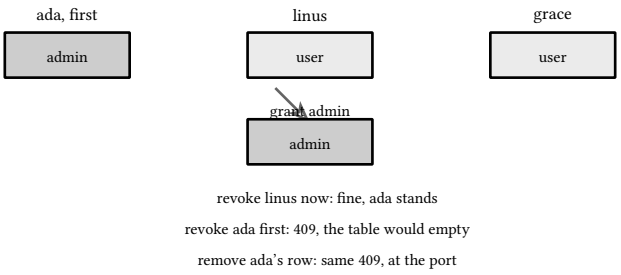


figure 36: the grant table: one column per user, the admin row must never empty

One rust-specific split the siblings did not have, stated here because vector 01 forces it: the user record carries a roles member, frozen by the register vector, so the record's member stays the registration-time snapshot while the grants table adopted from those records at boot is the live truth. The bridge is one function, `table_roles`, which

binds the roles port into chapter 22′s `roles_of` seam, so the identity layer reads the table at request time and a demoted admin loses the role on the very next request, pinned by the same bearer crossing the admin route before the revoke and failing after it with no reissue anywhere. The removal half of the guard has no route yet, the delete route is chapter 25′s concurrency family, so it is pinned at the port, removal of the standing admin refused, removal of anyone else returning the row.

The first admin cannot come from the grant surface, because the surface is itself behind the admin policy, so the bootstrap is the adoption at wiring time: the table reads every record's roles once, and the first user's `[admin, user]`, chapter 21′s bootstrap rule, becomes the table's first column.

23.5. the table and deny by default

One vec names every route pattern the service wires and its policy, and the guard wraps the whole route set in it. The pattern walk is the guard's own matcher, segments in lockstep with literals and `{wildcards}` binding params, the same walk the router performs at dispatch:

```
pub fn match_pattern(pattern: &str, method: &str, path: &str) -> Option<RouteMatch> {
    let (want_method, want_path) = pattern.split_once(' ')?;
    if want_method != method {
        return None;
    }
    let pattern_segs: Vec<&str> = want_path.split('/').collect();
    let path_segs: Vec<&str> = path.split('/').collect();
    if pattern_segs.len() != path_segs.len() {
        return None;
    }
    let mut params = Vec::new();
    for (pat, part) in pattern_segs.iter().zip(&path_segs) {
        if let Some(name) = pat.strip_prefix('{').and_then(|s| s.strip_suffix('}')) {
            params.push((name.to_string(), httpx::percent_decode(part)));
        } else if pat != part {
            return None;
        }
    }
    Some(RouteMatch {
        pattern: pattern.to_string(),
        params,
    })
}

pub fn guard(table: Vec<(String, Policy)>) -> Layer {
    let table = Arc::new(table);
    Arc::new(move |next: Handler| {
        let table = Arc::clone(&table);
        Arc::new(move |req: &mut Request| {
            for (pattern, policy) in table.iter() {
                if let Some(m) = match_pattern(pattern, &req.method, &req.path) {
                    return policy(req, &m).and_then(|()| next(req));
                }
            }
            next(req)
        }) as Handler
    })
}

fn guarded(policy: Policy, handler: Handler) -> Handler {
    Arc::new(move |req: &mut Request| {
        let m = RouteMatch {
            pattern: String::new(),
            params: req.params.clone(),
        };
        policy(req, &m)?;
        handler(req)
    })
}
```

listing 257: the matcher and the guard: first row to match decides, no row falls through to the kernel (source: rust/api/src/authz.rs, lines 85-133)

```rust
pub fn route_table(users: Arc<dyn UserStore>) -> Vec<(String, Policy)> {
    vec![
        ("GET /healthz".to_string(), anyone()),
        ("GET /readyz".to_string(), anyone()),
        ("POST /api/users".to_string(), anyone()),
        ("POST /api/auth/login".to_string(), anyone()),
        ("POST /api/auth/refresh".to_string(), anyone()),
        ("POST /api/auth/logout".to_string(), authenticated()),
        ("GET /api/users".to_string(), authenticated()),
        (
            "GET /api/users/{id}".to_string(),
            owner_or_admin(Arc::clone(&users)),
        ),
        (
            "POST /api/admin/users/{id}/roles".to_string(),
            require_role(ROLE_ADMIN),
        ),
        (
            "DELETE /api/admin/users/{id}/roles/{role}".to_string(),
            require_role(ROLE_ADMIN),
        ),
    ]
}
```

listing 258: the production table: ten rows today, later chapters append, none ever edit (source: rust/api/src/authz.rs, lines 135-158)

Here is the honest divergence from both siblings. Go's guard asked its mux which pattern a request would match and answered 403 for a pattern missing from the table; java asked its kernel the same question through `resolvePattern`. This lane's router offers no such query and this chapter does not edit a landed module, so the guard matches against its own table and a request matching no row falls through to the kernel untouched, because 404 and 405 are the router's answers, not authorization verdicts, and that rule is pinned by test. Deny by default therefore lives in the audit, both directions: every table row is driven through the full stack and must reach a real route, never the kernel's route-not-found 404 or its 405, and the set of table patterns must equal the set of patterns the wiring registered, asserted against the wiring function's own returned inventory. A route registered without a row breaks the equality, a row pointing at nothing breaks the drive, and the two sides cannot drift apart in either direction. The admin routes carry a second belt, their handlers are individually wrapped in the role policy at registration, go's `GuardRole` shape, so they stay protected even on a bare router the guard never saw.

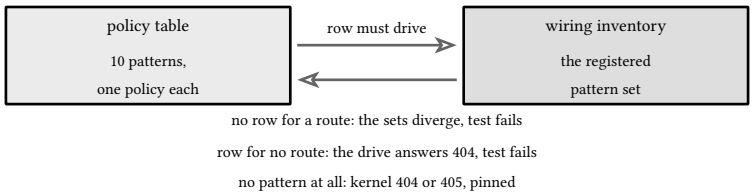


figure 37: the audit: table and wiring face each other, both directions checked

23.6. the grant surface

The admin family is two routes, grant one role and revoke one role, both answering the resulting set, both validating the closed role vocabulary before the store is touched, the unknown user 404 and the unrecognized role 422:

```rust
pub fn validate_role(role: &str) -> Vec<String> {
    if role == ROLE_ADMIN || role == ROLE_USER {
        Vec::new()
```

```
    } else {
        vec!["role must be admin or user".to_string()]
    }
}

pub fn table_roles(roles: Arc<dyn RoleStore>) -> crate::authn::RolesFn {
    Box::new(move |id: &str| roles.roles_of(id).unwrap_or_default())
}

pub struct Deps {
    pub users: Arc<dyn UserStore>,
    pub roles: Arc<dyn RoleStore>,
}

pub fn register_routes(router: &mut Router, deps: Arc<Deps>) {
    let grant_deps = Arc::clone(&deps);
    router.route(
        "POST",
        "/api/admin/users/{id}/roles",
        guarded(
            require_role(ROLE_ADMIN),
            Arc::new(move |req: &mut Request| grant(req, &grant_deps)),
        ),
    );
    let revoke_deps = Arc::clone(&deps);
    router.route(
        "DELETE",
        "/api/admin/users/{id}/roles/{role}",
        guarded(
            require_role(ROLE_ADMIN),
            Arc::new(move |req: &mut Request| revoke(req, &revoke_deps)),
        ),
    );
}

fn member(body: &Json, key: &str) -> Result<String, ApiError> {
    match body.get(key) {
        Some(Json::Str(text)) => Ok(text.clone()),
        Some(Json::Null) | None => Ok(String::new()),
        Some(_) => Err(ApiError::fail(
            Code::InvalidJson,
            format!("{key} must be a string"),
        )),
    }
}

fn map_role_error(err: RoleError) -> ApiError {
    match err {
        RoleError::NotFound => ApiError::fail(Code::NotFound, "user not found"),
        RoleError::LastAdmin => ApiError::fail(Code::Conflict, "last admin"),
    }
}

fn grant(req: &mut Request, deps: &Arc<Deps>) -> Result<Response, ApiError> {
    let body = bind_json(req, &["role"])?;
    let role = member(&body, "role")?;
    let details = validate_role(&role);
```

```
    if !details.is_empty() {
        return Err(ApiError::validation(details));
    }
    let id = req.param("id").unwrap_or_default().to_string();
    let roles = deps.roles.grant(&id, &role).map_err(map_role_error)?;
    roles_response(&id, &roles)
}
```

listing 259: the vocabulary check, the roles seam into chapter 22, and the grant route: validate, then map the port's two errors (source: rust/api/src/authz.rs, lines 278-344)

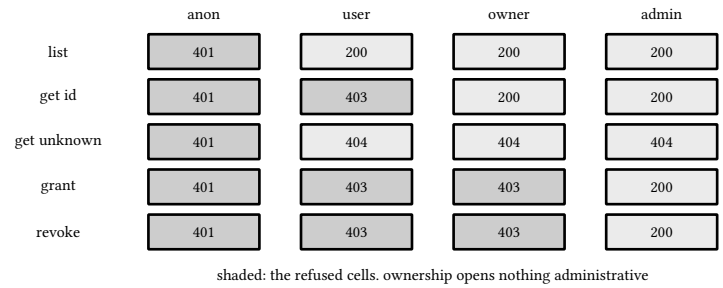
The grant surface is idempotent in both directions, granting a held role and revoking an absent one answer the current set with 200, which is what makes the race test's mixed waves orderly: every reply is a 200 or the guard's 409, never a surprise.

23.7. the matrix and the races

The fixture wires the whole guarded stack over a real socket, the same path production traffic takes, and the grid is the proof: every route times every actor kind, anonymous, regular user, the stranger on someone else's object, the owner, the admin, one expected status per cell:

```rust
fn wire(
    store: Arc<MemStore>,
    roles: Arc<MemRoles>,
    sessions: Arc<MemSessions>,
) -> (rustapi::httpx::Handler, Vec<String>) {
    let mut router = Router::new();
    let mut registered = Vec::new();
    register_kernel_routes(&mut router, Arc::new(Lifecycle::new()), "test-1");
    registered.push("GET /healthz".to_string());
    registered.push("GET /readyz".to_string());
    let users: Arc<dyn UserStore> = Arc::clone(&store) as Arc<dyn UserStore>;
    let user_deps = Arc::new(rustapi::user::Deps {
        store: Arc::clone(&users),
        now_millis: Box::new(|| T0),
        new_id: Box::new(rustapi::httpx::new_uuid_v7),
        hash_password: Box::new(|password| format!("fake:{password}")),
    });
    rustapi::user::register_routes(&mut router, Arc::clone(&user_deps));
    registered.push("POST /api/users".to_string());
    registered.push("GET /api/users/{id}".to_string());
    registered.push("GET /api/users".to_string());
    let authn_deps = Arc::new(rustapi::authn::Deps {
        users: Arc::clone(&users),
        sessions: Arc::clone(&sessions) as Arc<dyn SessionStore>,
        secret: SECRET.to_vec(),
        dummy_hash: "fake:dummy".to_string(),
        verify: Box::new(|stored, password| stored == format!("fake:{password}")),
        find_user: rustapi::authn::email_lookup_over(Arc::clone(&users)),
        roles_of: rustapi::authz::table_roles(Arc::clone(&roles) as Arc<dyn RoleStore>),
        now_millis: Box::new(|| T0),
        new_id: Box::new(rustapi::httpx::new_uuid_v7),
        new_refresh: Box::new(|| "unused-refresh".to_string()),
    });
    let service = Arc::new(rustapi::authn::Service::new(authn_deps));
    service.register_routes(&mut router);
    registered.push("POST /api/auth/login".to_string());
    registered.push("POST /api/auth/refresh".to_string());
    registered.push("POST /api/auth/logout".to_string());
    let authz_deps = Arc::new(rustapi::authz::Deps {
        users: Arc::clone(&users),
        roles: Arc::clone(&roles) as Arc<dyn RoleStore>,
    });
    rustapi::authz::register_routes(&mut router, authz_deps);
    registered.push("POST /api/admin/users/{id}/roles".to_string());
    registered.push("DELETE /api/admin/users/{id}/roles/{role}".to_string());
    let identify = service.identify();
    let stack = rustapi::middleware::chain(rustapi::middleware::stack(
        rustapi::middleware::stderr_sink(),
    ));
    let guard = rustapi::authz::guard(route_table(Arc::clone(&users)));
    (identify(stack(guard(router.handler()))), registered)
}
```

listing 260: the wiring: every family on one router, the inventory returned, identity outside and guard inside (source: rust/api/tests/authz.rs, lines 94-146)



shaded: the refused cells. ownership opens nothing administrative

figure 38: the matrix: routes one way, actors the other, statuses in the cells

Around the matrix, on the pinned 1.99.0 build, dated 2026-10-13, the authz test file carries 10 green tests: the twenty matrix cells, the challenge header beside the two 403 messages and the 404-first order, the owner's full record against the stranger's public projection, the last-admin chain replayed from vector 14 with the two 200 bodies asserted equal to the vector's, the grant surface's 422 and 404 and idempotence lanes, the removal guard at the port with the record snapshot left standing, the same bearer before and after the table revoke, the two-direction audit at ten rows against ten routes, the fall-through 404 and 405 for the rowless path, and the race.

The race is the chapter's spine and it is deterministic where a socket race usually is not. A first wave, eight threads through the pool hammering grants and revokes of `admin` on two of three admins, every reply asserted to be a 200 or the guard's 409 and the table asserted to still hold an admin at the end. Then the two contenders are demoted sequentially, ada stands alone, and a second wave of forty concurrent demotions of ada runs where no thread can possibly grant anyone admin, so all forty must answer 409 and the guard's fire under the pool is pinned by count, not by luck:

```
    let demote_ada = format!("/api/admin/users/{ADA_ID}/roles/admin");
    let mut refusers = Vec::new();
    for _ in 0..8usize {
        let addr = fx.addr();
        let token = tokens.ada.clone();
        let path = demote_ada.clone();
        refusers.push(std::thread::spawn(move || {
            let mut seen = 0u32;
            for _ in 0..5usize {
                let bearer = bearer_header(&token);
                let reply = send(
                    addr,
                    "DELETE",
                    &path,
                    &[("authorization", bearer.as_str())],
                    None,
                );
                assert_eq!(reply.status, 409, "{}", reply.body_text());
                seen += 1;
            }
            seen
        }));
    }
    let mut refused = 0u32;
    for join in refusers {
        refused += join.join().expect("refuser");
    }
    assert_eq!(refused, 40);
    assert_eq!(admin_count(&fx.roles), 1);
```

listing 261: the deterministic wave: ada provably the last admin, forty concurrent refusals counted (source: rust/api/tests/authz.rs, lines 842-870)

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.com, for the IDOR class and the deny-by-default guidance, RFC 9110 sections 11.6.1 and 15.5.1 to 15.5.4 for the challenge header and the 401, 403, 404, and 409 semantics, at rfc-editor.org, accessed 2026-10-13. The guard-cannot-ask-the-router divergence stated against this corpus's own kernels, go book 4 chapter 22 with its mux-driven lookup and java book 4 chapter 22 with `resolvePattern`. The last-admin chain, the 409 envelope, and the roles response bodies from `contract/testdata/14-roles-lastadmin.json`, the go oracle's file hashed byte-exact into this lane's vectors. All measurements, 10 tests, the twenty cells, the forty counted refusals, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned build, dated 2026-10-13.

24. store

The service has run on maps, which tests love and restarts hate. The corpus lane is zero-dependency, so this chapter owns the machinery outright in `rustapi::store`: a write-ahead log of crc32c-framed records, replay that rebuilds memory and buries a torn tail, compaction that rewrites the live records and swaps atomically, and chapter 21's `UserStore` port implemented over it all behind the same interface, no handler any the wiser, the user and authz tests green unchanged. The go book built the same engine when its dependency ruling retired the sqlite driver, book 4, ch 23 (go 1.27, store), the java book the same on the same reasoning, book 5, ch 24 (java 27, store), and the bar this lane aims at is their recovery argument with rust's own shapes: the crc table is a `const fn` computed at compile time, the log is the only file format, and one frame is one whole record, which deletes the transaction machinery rather than implementing it.

24.1. the frame, crc32c by hand

One log record is one frame: an eight byte header, payload length then its crc32c, wrapped around a payload of version byte, op byte, and the op's fields, every length little endian. The checksum is Castagnoli, the polynomial `0x1EDC6F41` reflected to `0x82F63B78`, and std ships no crc at all, so the table is built by a `const fn` the compiler evaluates once rather than a static list typed in: both siblings carried the table as data, go building it at init, java importing it from `java.util.zip`, and the const evaluation is rust's own third answer to the same problem:

```
const CRC32C_POLY: u32 = 0x82f6_3b78;

const fn build_table() -> [u32; 256] {
    let mut table = [0u32; 256];
    let mut i = 0usize;
    while i < 256 {
        let mut c = i as u32;
        let mut bit = 0usize;
        while bit < 8 {
            c = if c & 1 != 0 {
                CRC32C_POLY ^ (c >> 1)
            } else {
                c >> 1
            };
            bit += 1;
        }
        table[i] = c;
        i += 1;
    }
    table
}

static CRC32C_TABLE: [u32; 256] = build_table();

pub fn crc32c(data: &[u8]) -> u32 {
    let mut crc = 0xffff_ffffu32;
    for byte in data {
        crc = CRC32C_TABLE[((crc ^ *byte as u32) & 0xff) as usize] ^ (crc >> 8);
    }
    crc ^ 0xffff_ffff
}

fn put_u32(dst: &mut Vec<u8>, value: u32) {
    dst.extend_from_slice(&value.to_le_bytes());
}

fn read_u32(bytes: &[u8]) -> u32 {
    u32::from_le_bytes([bytes[0], bytes[1], bytes[2], bytes[3]])
```

listing 262: the castagnoli table as a const fn, the reflected update loop, and the little-endian helpers (source: rust/api/src/store/wal.rs, lines 40-77)

```rust
pub fn payload_of(rec: &Record) -> Vec<u8> {
    let mut payload = Vec::new();
    payload.push(WAL_VERSION);
    payload.push(rec.op.tag());
    put_u32(&mut payload, rec.key.len() as u32);
    payload.extend_from_slice(&rec.key);
    if rec.op == Op::Put {
        put_u32(&mut payload, rec.value.len() as u32);
        payload.extend_from_slice(&rec.value);
    }
    payload
}

pub fn encode_into(dst: &mut Vec<u8>, rec: &Record) {
    let payload = payload_of(rec);
    put_u32(dst, payload.len() as u32);
    put_u32(dst, crc32c(&payload));
    dst.extend_from_slice(&payload);
}
```

listing 263: the payload writer: version, op, and the op's fields, key then value for put, key alone for tombstone (source: rust/api/src/store/wal.rs, lines 80-98)

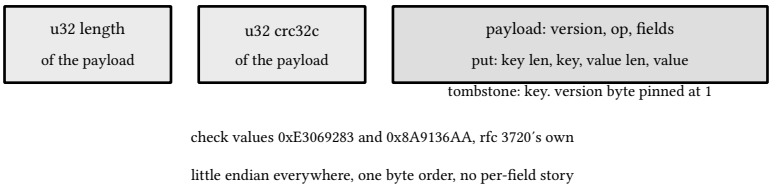


figure 39: one frame: the checksum covers exactly the payload, the payload carries its own version

The version byte is the codec's contract with the future: replay refuses a frame it cannot read rather than guessing, so a future format bumps the byte and old binaries fail closed. The two check values in the test are not derived from this implementation at all, they are the published crc32c constants, `"123456789"` to `0xE3069283` and thirty-two zero bytes to `0x8A9136AA`, the numbers rfc 3720 carries for exactly this purpose.

24.2. replay and the torn tail

Decoding walks frames until the bytes run out or a frame is torn: a remaining tail shorter than the header, a length that overruns, a checksum that lies, or a payload this build cannot parse, and every refusal is the same stop, the offset naming where:

```rust
fn decode_payload(payload: &[u8]) -> Option<Record> {
    if payload.len() < 2 || payload[0] != WAL_VERSION {
        return None;
    }
    let op = Op::of(payload[1])?;
    let mut rest = &payload[2..];
    if rest.len() < 4 {
        return None;
    }
    let key_len = read_u32(rest) as usize;
    rest = &rest[4..];
    if rest.len() < key_len {
        return None;
    }
    let key = rest[..key_len].to_vec();
    rest = &rest[key_len..];
    let value = if op == Op::Put {
        if rest.len() < 4 {
            return None;
        }
        let value_len = read_u32(rest) as usize;
        rest = &rest[4..];
        if rest.len() != value_len {
            return None;
        }
        rest.to_vec()
    } else if rest.is_empty() {
        Vec::new()
    } else {
        return None;
    };
    Some(Record { op, key, value })
}

pub fn decode(bytes: &[u8]) -> (Vec<Record>, usize) {
    let mut records = Vec::new();
    let mut off = 0usize;
    while off + HEADER_LEN ≤ bytes.len() {
        let len = read_u32(&bytes[off..off + 4]) as usize;
        let sum = read_u32(&bytes[off + 4..off + HEADER_LEN]);
        if len < 2 || off + HEADER_LEN + len > bytes.len() {
            return (records, off);
        }
        let payload = &bytes[off + HEADER_LEN..off + HEADER_LEN + len];
        if crc32c(payload) != sum {
            return (records, off);
        }
        let Some(rec) = decode_payload(payload) else {
            return (records, off);
        };
        records.push(rec);
        off += HEADER_LEN + len;
    }
    (records, off)
```

listing 264: the payload checks and the walk: every refusal stops the decode, the records before it stand (source: rust/api/src/store/wal.rs, lines 100-153)

```
impl Engine {
    pub fn open(path: &Path) -> std::io::Result<Engine> {
        if let Some(parent) = path.parent() {
            fs::create_dir_all(parent)?;
        }
        let bytes = match fs::read(path) {
            Ok(bytes) => bytes,
            Err(err) if err.kind() == std::io::ErrorKind::NotFound => Vec::new(),
            Err(err) => return Err(err),
        };
        let (records, offset) = wal::decode(&bytes);
        if offset < bytes.len() {
            let file = fs::OpenOptions::new().write(true).open(path)?;
            file.set_len(offset as u64)?;
        }
        let mut data = BTreeMap::new();
        for rec in records {
            let key = String::from_utf8_lossy(&rec.key).into_owned();
            match rec.op {
                Op::Put => {
                    data.insert(key, rec.value);
                }
                Op::Tombstone => {
                    data.remove(&key);
                }
            }
        }
        let appender = wal::Appender::open(path)?;
        Ok(Engine {
            inner: Mutex::new(Inner {
                data,
                wal: appender,
```

listing 265: open: parent directories, decode, and the torn tail truncated away before the appender ever positions after it (source: rust/api/src/store/mod.rs, lines 29-60)

The truncation is the chapter's own answer to a failure both siblings had to fence separately: an append that lands after a torn tail is unreachable forever, because replay stops at the tear and never reaches the frames behind it. Java poisoned the engine after a failed append, go relied on the writer never tearing, and here the open sequence simply buries the artifact, `set_len` to the last good offset, so the appender's position after recovery always points at readable bytes. The test writes the tear, opens, appends past it, reopens, and finds both the survivor and the new record.

The exhaustive test is java's geometry, and geometry is not timing: a four-frame log of seventy-five bytes, three puts and a tombstone, reopened at every one of its seventy-six byte lengths, from the empty file to the whole one. At the five frame boundaries the recovered state is compared against a hand-computed expectation written out in the test, so the oracle is not the implementation agreeing with itself, and between the boundaries the reopened state must equal the state at the last boundary, which is frame atomicity, the whole point of the checksum:

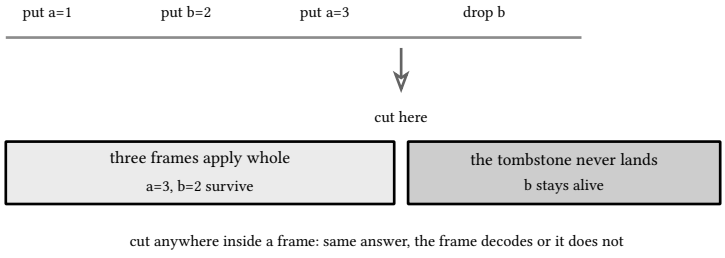


figure 40: a log recovers to the last whole frame, whatever the cut

24.3. append then mutate, one writer

One mutex is the engine's whole concurrency story, the same contract sqlite's wal mode gives at harder cost. A write appends the frame, and only then mutates the map, so the log is always at least as current as memory: a crash between the two recovers to the committed state, and the inverse order would leave memory holding a state the log cannot rebuild, the exact failure recovery exists to prevent:

```
            }),
        })
    }

    pub fn put(&self, key: &str, value: &[u8]) -> std::io::Result<()> {
        let mut inner = self.inner.lock().unwrap();
        let rec = Record {
            op: Op::Put,
            key: key.as_bytes().to_vec(),
            value: value.to_vec(),
        };
        inner.wal.append(&[rec])?;
        inner.data.insert(key.to_string(), value.to_vec());
        Ok(())
    }

    pub fn delete(&self, key: &str) -> std::io::Result<bool> {
        let mut inner = self.inner.lock().unwrap();
        if !inner.data.contains_key(key) {
            return Ok(false);
        }
        let rec = Record {
            op: Op::Tombstone,
            key: key.as_bytes().to_vec(),
            value: Vec::new(),
        };
        inner.wal.append(&[rec])?;
        inner.data.remove(key);
```

listing 266: append then mutate, under one lock, and delete only writes a tombstone for a key that stands (source: rust/api/src/store/mod.rs, lines 62-89)

One design decision deletes a whole subsystem the siblings carried: go and java wrapped multi-record groups in begin and commit markers so a group applied atomically, because their adapters wrote several keys per user. Here one record is one frame, the user with all its indexes rides a single json value under a single key, so the frame is the atomicity unit and the marker pair would be machinery without a customer. The cost is stated too: a whole record rewrites on every change, fine at book scale, and the compaction section is what keeps the rewrites from growing without bound.

The write reaches the operating system but is not forced, the line sqlite's synchronous NORMAL draws: durable across application crashes, relaxed about power loss, with `sync` as the operator's stronger call, one method on the engine nobody in this chapter's wiring calls automatically.

24.4. compaction, rewrite and atomic swap

The log only grows, and compaction is the answer. The compacted form is the same format, the live records as fresh put frames in key order, written to a tmp file, forced, closed, and moved over the live log atomically, `std::fs::rename`, which on windows rides `MoveFileEx` with replace-existing and on posix is rename(2), both atomic for this purpose. Only then does the appender reopen on the new file:

```rust
use std::fs;
use std::io::Write;

use super::Inner;
use super::Op;
use super::Record;
use super::encode_into;

pub(super) fn run(inner: &mut Inner) -> std::io::Result<()> {
    let tmp = inner.path.with_extension("tmp");
    let mut buf = Vec::new();
    for (key, value) in &inner.data {
        let rec = Record {
            op: Op::Put,
            key: key.as_bytes().to_vec(),
            value: value.clone(),
        };
        encode_into(&mut buf, &rec);
    }
    let mut file = fs::File::create(&tmp)?;
    file.write_all(&buf)?;
    file.sync_all()?;
    drop(file);
    inner.wal.close()?;
    fs::rename(&tmp, &inner.path)?;
    inner.wal = super::wal::Appender::open(&inner.path)?;
    Ok(())
}
```

listing 267: the whole compaction: live puts to tmp, force, rename over the log, reopen (source: rust/api/src/store/compact.rs, lines 1-28)

Every interruption of that order recovers by replay, because the file on disk is always a complete wal: a torn tmp never gets renamed, the next open ignores it and the next compaction truncates it, and a rename that landed simply leaves the compacted log for the next open. Go needed a snapshot file plus a rotation dance because its snapshot also carried a migration registry, a second channel this engine does not have, the wal is the only format and one swap suffices. Measured on this machine, dated 2026-10-13, by the test's own sequence: one surviving key beside sixty put and sixty delete frames grows the log to 3170 bytes, and compaction leaves 25 bytes holding the survivor, with the post-compaction put replaying over the swapped file on reopen.

24.5. the user store over the engine

The adapter implements chapter 21's port, `create`, `get`, `list`, and the key scheme is the documentation the keys enforce. The record lives under `u:rec:`, its key the creation millisecond sign-flipped into fixed-width hex beside the id, exactly the pair the pagination cursor carries, so storage order and keyset order are one notion of position and the b-tree hands the pages over without a sort:

```
fn order_key(created_at_millis: i64, id: &str) -> String {
    format!(
        "{USER_PREFIX}{:016x}:{id}",
        (created_at_millis as u64) ^ (1u64 << 63)
    )
}

fn cursor_bound(after: &CursorKey) -> String {
    format!(
        "{USER_PREFIX}{:016x}:{}\x00",
        (after.t as u64) ^ (1u64 << 63),
        after.i
    )
}
```

listing 268: the order key: sign-flipped millis keep ascending time ascending in hex, the id tiebreak beside it, the cursor bound one byte past (source: rust/api/src/store/users.rs, lines 27-40)

```
impl UserStore for UserEngine {
    fn create(&self, user: &mut User) -> Result<(), StoreError> {
        let mut indexes = self.indexes.lock().unwrap();
        if indexes.by_email.contains_key(&user.email) {
            return Err(StoreError::Conflict);
        }
        if user.roles.is_empty() {
            if indexes.by_id.is_empty() {
                user.roles = vec![ROLE_ADMIN.to_string(), ROLE_USER.to_string()];
            } else {
                user.roles = vec![ROLE_USER.to_string()];
            }
        }
        user.version = 1;
        let key = order_key(user.created_at_millis, &user.id);
        let value = record_json(user);
        self.engine
            .put(&key, &value)
            .expect("wal append is the one failure a frozen port cannot say");
        indexes.by_id.insert(user.id.clone(), key.clone());
        indexes.by_email.insert(user.email.clone(), key);
        Ok(())
    }

    fn get(&self, id: &str) -> Result<User, StoreError> {
        let key = {
            let indexes = self.indexes.lock().unwrap();
            indexes.by_id.get(id).cloned().ok_or(StoreError::NotFound)?
        };
        self.engine
            .get(&key)
            .as_deref()
            .and_then(record_of)
            .ok_or(StoreError::NotFound)
    }

    fn list(&self, after: CursorKey, limit: usize) -> Vec<User> {
        let from = if after.t == 0 && after.i.is_empty() {
            USER_PREFIX.to_string()
        } else {
            cursor_bound(&after)
        };
        self.engine
            .scan(&from, USER_END)
            .iter()
            .filter_map(|(_, value)| record_of(value))
            .take(limit)
            .collect()
    }
}
```

listing 269: the port: email claim and bootstrap rules under one lock, engine first then index, and list as a range strictly after the cursor (source: rust/api/src/store/users.rs, lines 110-159)

The stored form is not the wire form, and the split is worth naming. The record serializes all seven components including the password hash, because the store must verify logins after a restart, while the wire writer chapter 21 pinned lists five and never the hash: the hash is stored, never served, and the restart test asserts the stronger form, the same record yields the same etag forever, because the writer is deterministic and the bytes did not move

across a close and reopen. The by-id and by-email indexes are derived state, rebuilt by the replay scan on open and maintained inside create under the same lock that checked the claim, engine written first, index published second, so a reader either misses both or sees both. The bootstrap rule and the conflict semantics are the mem store's, error for error, and the keyset test drives both implementations over the same three records, ties included, and asserts the same page.

One expectation the port cannot say: an io failure inside create answers through the panic lane, recover's 500, because chapter 21 froze two outcomes and this chapter does not touch a landed module. The expect is stated in the code's own message rather than hidden in a `unwrap`.

24.6. the wiring and the reboot proof

`main` swaps the mem store for the engine-backed one at `RUSTAPI_DATA/users.wal`, three lines and an error lane that exits like the bind lane does, and nothing else in the program changes: the roles table adopts from the recovered records, the handlers cannot tell. Sessions stay in memory this chapter, stated as the staging it is: the `SessionStore` port froze in chapter 22 for exactly this moment, and a later wave implements it over the engine without a handler learning of the change:

```
    let wal_path = std::path::Path::new(&cfg.data).join("users.wal");
    let user_engine = match rustapi::store::UserEngine::open(&wal_path) {
        Ok(engine) => engine,
        Err(err) => {
            eprintln!("rustapi: open store {}: {err}", wal_path.display());
            std::process::exit(1);
        }
    };
    let engine = user_engine.engine();
    let users: Arc<dyn user::UserStore> = Arc::new(user_engine);
    let roles = Arc::new(authz::MemRoles::new());
    roles.adopt(&*users);
```

listing 270: the seam switch: the engine store at the configured path, the grants table adopted from the recovered records (source: rust/api/src/main.rs, lines 59-70)

The end-to-end test is the chapter's proof, two boots over one directory through the full guarded stack: register ada, log in, shut down, open a fresh engine and a fresh stack over the same wal, and the second login verifies against the persisted hash, the fetched record answers the first boot's etag byte for byte, and the list holds one item. A pre-restart token dies with the memory session family, which is the staging above, not a loss:

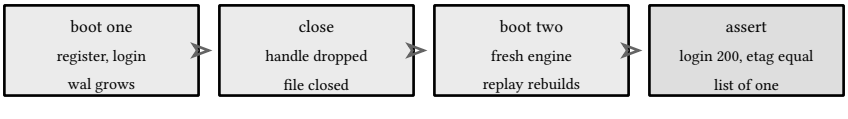


figure 41: the reboot proof: two processes, one directory, everything but the sessions comes back

On the pinned 1.99.0 build, dated 2026-10-13, the store test file carries 10 green tests: the two published crc32c check values, the frame round trip with its six refusals from the lying checksum to the future version under an honest one, the seventy-six-length torn-tail walk with its hand-computed boundaries, the burial that keeps later appends reachable, the restart durability with the stable etag and the surviving email claim, the compaction swap with its measured 3170 to 25 bytes and the post-compaction replay, the adapter's keyset parity against the mem store over the same ties, two hundred concurrent puts through eight threads all accounted after a reopen, the reboot over the real guarded stack, and the parent directory created on open. The whole engine is four files, 493 lines, small enough to be read whole, which is the point of owning it, and the crate's whole is 102 tests this wave, the 66 landed ones green unchanged, which is the point of the seam.

sources: RFC 3720 section B.4 for the crc32c check values `0xE3069283` over `"123456789"` and `0x8A9136AA` over thirty-two zero bytes, at rfc-editor.org, accessed 2026-10-13. The castagnoli reflected polynomial `0x82F63B78` verified against the same document. `std::fs::rename`'s windows behavior read from the offline 1.99.0 html share under `~/.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html`, accessed 2026-10-13. The wal framing, the torn-tail recovery argument, the append-then-mutate order, the synchronous NORMAL analogy, and the compaction trade from the sibling store chapters, go book 4 chapter 23 and java book 4 chapter 24 of this corpus, including go's begin and commit markers and java's poisoned engine, the two mechanisms this lane's one-frame-per-record design does not need. The byte measurements, 3170 to 25 and the 75-byte log walked at 76 lengths, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` and a probe binary on the same machine, dated 2026-10-13. All other measurements, 10 tests and the crate's 102 this wave, produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned build, dated 2026-10-13.

25. concurrency control

The pool of chapter 19 runs up to 32 workers over one shared handler and the engine of chapter 24 serializes every write behind one mutex, and still nothing stops two requests from racing on the same resource: two readers fetch the same version, both write back, and one overwrite silently wins, the oldest bug in any concurrent system. This chapter owns the service's answer in `rustapi::conc`, four mechanisms over one new resource family: strong etags computed from the landed sha256 primitive, `If-None-Match` answered with a 304 and `If-Match` with a 412, idempotency keys that replay a stored response instead of double-creating, and a version-gated compare-and-swap over the engine whose commit refuses on version drift. The go book walked the same territory over `golang.org/x/sync`, book 4, ch 24 (go 1.27, concurrency control), the java book over its virtual-thread executor, book 5, ch 25 (java 27, concurrency control), and this lane owns the same contracts with the std mutex, the std condvar, and nothing else.

25.1. the note, and the tag it carries

The user family is read-only at the wire, chapter 21 froze its port at create, get, and list, so the update story needs a resource this chapter owns outright. A note is the smallest thing that versions: an id, a body of at most 512 characters, and a version that starts at 1 and moves only through the swap. The wire writer is deterministic, the record under `n:rec:` in the engine is the same json, and the etag is the strong sha256 of those bytes through the primitive chapter 22 already rode for hmac, promoted here to the one public constructor:

```
pub fn strong_etag(body: &[u8]) -> String {
    format!("\"{}\"", hex(&sha256(body)))
}

pub fn if_match_allows(header: &str, etag: &str) -> bool {
    header.split(',').any(|candidate| {
        let token = candidate.trim();
        if token == "*" {
            return true;
        }
        if token.starts_with("W/") {
            return false;
        }
        token == etag
    })
}

pub fn if_none_match_hits(header: &str, etag: &str) -> bool {
    header.split(',').any(|candidate| {
        let token = candidate.trim();
        if token == "*" {
            return true;
        }
        let bare = token.strip_prefix("W/").unwrap_or(token);
        bare == etag
    })
}
```

listing 271: strong_etag over the landed primitive, and the two rfc comparison functions, strong for If-Match, weak for If-None-Match (source: rust/api/src/conc.rs, lines 25-51)

The two comparators are the whole of rfc 9110's section 13.1 in code. `If-Match` runs the strong comparison, the rfc's MUST: a star is true when any current representation exists, a list is true when the etag appears in it, and a `W/` form never matches, however close its opaque bytes. `If-None-Match` runs the weak comparison, the same walk

with the W/ prefix stripped before the compare, which is the one word of difference between them and the reason a weak tag can serve a 304 but never guard a write. The test pins both ladders against hand-built headers, star, list, weak, and stale, and the digest test asserts the two published values, e3b0c442... over the empty input and ba7816bf... over abc, re-derived from python 3 hashlib on this machine, dated 2026-10-14, the same independent oracle chapter 21 used for the primitive itself.

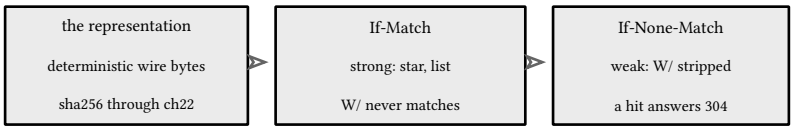


figure 42: one hash, two comparison rules

25.2. the conditional read

The read is one engine get, one etag computation, and one comparison against the request's If-None-Match. A hit answers 304 with the etag stamped and an empty body, the kernel's own bodyless law from chapter 19 refusing to write a content-length onto the status, and a miss or a stale header runs the full 200:

```
fn get(req: &mut Request, notes: &Arc<Notes>) -> Result<Response, ApiError> {
    let id = req.param("id").unwrap_or_default();
    let Some(note) = notes.read(id) else {
        return Err(ApiError::fail(Code::NotFound, "note not found"));
    };
    let etag = note.etag();
    if let Some(header) = req.header("if-none-match")
        && if_none_match_hits(header, &etag)
    {
        return Ok(Response::empty(304).with_header("etag", &etag));
    }
    Ok(Response::json(200, &note_json(&note)).with_header("etag", &etag))
}
```

listing 272: the conditional read: one read, one hash, one weak comparison, 304 bodyless on the hit (source: rust/api/src/conc.rs, lines 400-412)

The etag is computed, never stored, which is the property the cache chapter inherits next: the same record produces the same bytes and the same digest forever, so nothing can drift between a representation and its validator. Chapter 21's user route answers the same 304 with the same construction, and the note route exists because a family with a writer is needed before the write side of the contract can be pinned. The test walks the exact lane, the weak lane, the stale lane, and the unknown id, and the exact lane asserts the 304 carries the etag header and zero body bytes.

25.3. the guarded write

The write order is the contract: existence first, so an unknown id answers 404 before any precondition work, then the header requirement, then the strong comparison, and only then the decode. The missing If-Match answers 412 rather than the 428 rfc 6585 suggests for repeat-demanding clients, the same choice both sibling lanes made, because a 412 is the status the precondition rules already speak and the stricter code would police client behavior rather than resource state:

```
fn put(req: &mut Request, notes: &Arc<Notes>) -> Result<Response, ApiError> {
    let id = req.param("id").unwrap_or_default().to_string();
    let Some(current) = notes.read(&id) else {
        return Err(ApiError::fail(Code::NotFound, "note not found"));
    };
    let etag = current.etag();
    let Some(if_match) = req.header("if-match") else {
        return Err(ApiError::fail(
            Code::PreconditionFailed,
            "if-match header required",
        ));
    };
    if !if_match_allows(if_match, &etag) {
        return Err(ApiError::fail(Code::PreconditionFailed, "etag mismatch"));
    }
    let body = bind_json(req, &["body"])?;
    let text = member(&body, "body")?;
    let details = validate_note(&text);
    if !details.is_empty() {
        return Err(ApiError::validation(details));
    }
    let note = notes
        .commit(&id, current.version, &text)
        .map_err(|err| match err {
            CasError::NotFound => ApiError::fail(Code::NotFound, "note not found"),
            CasError::VersionDrift => ApiError::fail(
                Code::PreconditionFailed,
                "version drifted under the write, reread and retry",
            ),
        })?;
    Ok(Response::json(200, &note_json(&note)).with_header("etag", &note.etag()))
}
```

listing 273: existence, the required header, the strong compare, only then the decode, and the swap at the bottom (source: rust/api/src/conc.rs, lines 414-445)

Three refusals live in that order and each costs less than the work it skips. The missing header refuses before the body is parsed. The stale etag refuses after one engine read and before the decode. And the version drift at the bottom is the one refusal no ordering of checks can eliminate, which is the next section's whole subject. The rfc's evaluation order in section 13.2.2 is honored exactly: `If-Match` is tested before the request method runs, false answers 412, and the star's truth is evaluated at the read, so a writer landing between the read and the swap still meets the swap's own refusal.

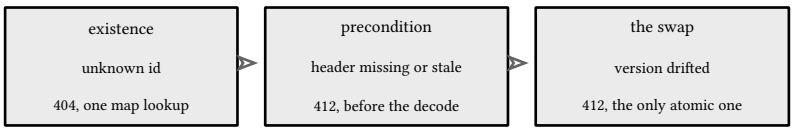


figure 43: three refusals, each cheaper than the work it skips

25.4. compare-and-swap over the engine

The engine's `put` is atomic on its own, one frame under one lock, but read-then-put is two lock acquisitions with a window between them, and the lost update lives exactly in that window. The fix is one gate, a mutex in `Notes` held across the read, the version check, and the write, so the compare and the swap become one critical section. The caller's half is optimistic: read the version, prepare the mutation, commit against the version read, and refuse on drift:

```
    pub fn commit(&self, id: &str, expect_version: i64, body: &str) -> Result<Note, CasError> {
        let _gate = self.gate.lock().unwrap();
        let current = self.read(id).ok_or(CasError::NotFound)?;
        if current.version != expect_version {
            return Err(CasError::VersionDrift);
        }
        let next = Note {
            id: id.to_string(),
            body: body.to_string(),
            version: expect_version + 1,
        };
        self.deps
            .engine
            .put(&note_key(id), &next.wire())
            .expect("wal append is the one failure a frozen port cannot say");
        Ok(next)
    }
```

listing 274: the swap under one gate: read the version, refuse the drift, write version plus one, the wal append before the map mutate is the engine's own order (source: rust/api/src/conc.rs, lines 318-334)

The proof is the duel. Two writers carrying the same valid etag are released together by one barrier through the real pool, and every run answers exactly one 200 and one 412 with version 2 standing after, five consecutive runs on the pinned build, dated 2026-10-14. Which 412 the loser collects depends on where it was when the winner landed: the precondition's `etag mismatch` if its read happened after, the swap's `version drifted under the write, reread and retry` if it passed the check before, and the test accepts both because both are honest refusals, the precondition is a gate and never a reservation. The engine under the gate is the chapter 24 one untouched, and the restart test reopens the wal to find version 2 survived with its drift check intact, a stale expect still refused after the process died between them.

25.5. idempotency keys, the stored response window

A retried post is the client's most ordinary race: the timeout that hides a 201, the retry that creates twice. The keyed layer answers it for `POST /api/notes`. A request carrying `Idempotency-Key` has its exact body bytes hashed, the row looked up under the composite of route, caller identity, and key, and a stored match replays the snapshot byte for byte, status, headers, and body. A different body under the same key is refused with the 422 both siblings chose, and the window is 24 hours, expired lazily by the read that notices:

```
    pub fn lookup(
        &self,
        key: &str,
        route: &str,
        identity: &str,
        request_hash: &[u8; 32],
    ) -> Lookup {
        let mut rows = self.rows.lock().unwrap();
        let composite = composite_key(key, route, identity);
        let Some(row) = rows.get(&composite) else {
            return Lookup::Miss;
        };
        if (self.now_millis)() ≥ row.expires_at {
            rows.remove(&composite);
            return Lookup::Miss;
        }
        if &row.request_hash ≠ request_hash {
            return Lookup::Mismatch;
        }
        Lookup::Replay(row.snap.clone())
    }

    pub fn save(
        &self,
        key: &str,
        route: &str,
        identity: &str,
        request_hash: &[u8; 32],
        snap: Snapshot,
    ) {
        let mut rows = self.rows.lock().unwrap();
        rows.insert(
            composite_key(key, route, identity),
            KeyRow {
                request_hash: *request_hash,
                snap,
                expires_at: (self.now_millis)() + IDEMPOTENCY_WINDOW_MILLIS,
            },
        );
    }
```

listing 275: lookup and save: the composite key, the lazy expiry that deletes on the noticing read, the hash carried beside the snapshot (source: rust/api/src/conc.rs, lines 116-155)

The flight settles who runs the handler under a key. The first caller inserts the composite into the flight map and owns a `Releaser` whose drop removes the entry and wakes the condvar, and a second caller under the same key while the first runs parks on the condvar instead of racing the handler into its own conflict:

```
struct Releaser {
    store: Arc<KeyStore>,
    composite: String,
}

impl Drop for Releaser {
    fn drop(&mut self) {
        self.store.flights.lock().unwrap().remove(&self.composite);
        self.store.wake.notify_all();
    }
}

fn composite_key(key: &str, route: &str, identity: &str) -> String {
    format!("{route}\u{0}{identity}\u{0}{key}")
}
```

listing 276: the releaser: drop removes the flight and wakes every waiter (source: rust/api/src/conc.rs, lines 90-104)

```
    fn claim(self: &Arc<Self>, composite: &str) -> Option<Releaser> {
        let mut flights = self.flights.lock().unwrap();
        if flights.contains_key(composite) {
            while flights.contains_key(composite) {
                flights = self.wake.wait(flights).unwrap();
            }
            return None;
        }
        flights.insert(composite.to_string(), ());
        Some(Releaser {
            store: Arc::clone(self),
            composite: composite.to_string(),
        })
    }
```

listing 277: claim: present means wait until absent, absent means insert and run (source: rust/api/src/conc.rs, lines 157-170)

The loop is the settle semantics the go lane taught: a waiter that woke re-reads the table before it computes, so the stored response the first caller saved is found and replayed, and only a waiter whose row is still absent, because the first run failed and stored nothing, runs the handler itself. Failures are never stored, a status of 400 or more passes through unstored so a retry after a validation error deserves a fresh run, and the test drives an invalid body under a key and then the fixed body under the same key to a fresh 201:

```
fn run_keyed(
    req: &mut Request,
    next: &Handler,
    store: &Arc<KeyStore>,
    governed: &Arc<Vec<String>>,
) -> Result<Response, ApiError> {
    let Some(key) = req.header(HEADER_IDEMPOTENCY).filter(|k| !k.is_empty()) else {
        return next(req);
    };
    let key = key.to_string();
    if !governed
        .iter()
        .any(|pattern| authz::match_pattern(pattern, &req.method, &req.path).is_some())
    {
        return next(req);
    }
    let route = format!("{} {}", req.method, req.path);
    let identity = identity_of(req);
    let request_hash = sha256(&req.body);
    loop {
        match store.lookup(&key, &route, &identity, &request_hash) {
            Lookup::Replay(snap) => return Ok(response_of(&snap)),
            Lookup::Mismatch => {
                return Err(ApiError::validation(vec![
                    "idempotency key already used with a different body".to_string(),
                ]));
            }
            Lookup::Miss => {}
        }
        let Some(_release) = store.claim(&composite_key(&key, &route, &identity)) else {
            continue;
        };
        let resp = next(req)?;
        if resp.status < 400 {
            store.save(&key, &route, &identity, &request_hash, Snapshot::of(&resp));
        }
        return Ok(resp);
    }
}
```

listing 278: run_keyed: the loop, replay or refuse or claim, the store before the release (source: rust/api/src/conc.rs, lines 188-226)

The concurrent test is counted, not timed: 8 posts under one key through the real pool all answer 200 with the identical body {"run":1}, and the handler's counter reads exactly 1, because whoever arrives first claims the flight and every later arrival either waits and replays or finds the stored row directly. The replay test adds the composition fact: the keyed layer sits inside the request-id layer, so the snapshot carries the handler's headers and never the request id, and one byte-equal replay shows two different `x-request-id` values, two real requests answered by one execution.

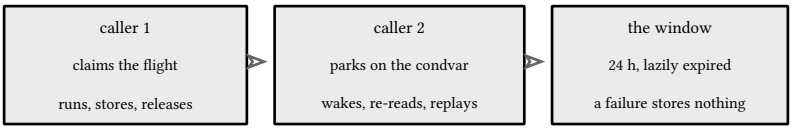


figure 44: the flight: the first runs, the later arrivals wait, then re-read

25.6. the count

The module is 445 lines and its test file 534, and the 14 tests land green five consecutive runs under the full gate, fmt, clippy at `-D warnings`, and the locked offline build, dated 2026-10-14: the two published digests, the two comparison ladders, the store's replay, mismatch, and exact window boundary, the swap's read, commit, drift, and reopen, the post and get etag contract, the conditional lanes, every `If-Match` lane including the star and the carried list, the duel, the byte-equal replay over two request ids, the conflicting body, the unstored failure, the expired window, and the counted single execution under 8 concurrent posts. The crate holds 116 tests this wave, the 102 landed ones green unchanged, which is the point of adding a family instead of touching a landed one.

sources: RFC 9110 sections 13.1.1 and 13.1.2 at rfc-editor.org, the strong comparison MUST, the two star rules, the list membership rule, and the weak comparison for `If-None-Match`, plus section 13.2.2′s evaluation order and section 15.5.13′s 412 definition, accessed 2026-10-14. RFC 6585 section 3 for status 428, the alternative this lane declined with both siblings, accessed 2026-10-14. The two sha256 digests re-derived from python 3 `hashlib` on this machine as the independent oracle, dated 2026-10-14, the same method chapter 21 used. The 24 hour window, the 422 refusal, the flight settle semantics, and the stored-snapshot replay follow the go book's concurrency chapter, book 4 chapter 24 of this corpus, and the strong-comparison edge lanes follow the java book's chapter 25, read for the parallel contracts. The duel and the counted single execution produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build, five consecutive green runs, dated 2026-10-14.

26. caching

A cache is a promise with two prices. The promise is freshness inside a stated window, the first price is memory, which must be bounded, and the second price is the stampede, the crowd of concurrent misses that all decide to fill the same key and hammer the handler they were supposed to protect. This chapter builds both answers in `rustapi::cache`: a bounded table of expiring entries over one mutex, and a single flight so one fill runs per key no matter how many callers arrive together. The snapshot type is chapter 25's, the same status, headers, and body triple the idempotency store replays, and that reuse is the point: one stored-response shape serves both chapters. The go book built its layer over `container/list` and `golang.org/x/sync`, book 4, ch 25 (go 1.27, caching), the java book over one `ConcurrentHashMap` with a deadline-rank eviction, book 5, ch 26 (java 27, caching), and this lane takes java's rank with go's wire posture, the std collections, and nothing else.

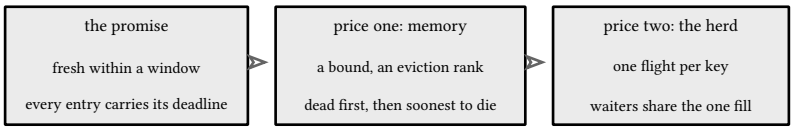


figure 45: the two prices and their two answers

26.1. ttl from first principles, lazy on the read

An entry carries its own deadline, and the read is its own eviction decision: a deadline that has passed means the entry is gone, removed by the very read that noticed it, and no thread ever wakes on a timer to do the same work. The clock is injected, a millis counter the whole plane shares, so every boundary test is arithmetic:

```
    pub fn get(&self, key: &str, now_millis: i64) -> Option<Snapshot> {
        let mut items = self.inner.lock().unwrap();
        match items.get(key) {
            Some(entry) if now_millis < entry.expires_at => Some(entry.snap.clone()),
            Some(_) => {
                items.remove(key);
                None
            }
            None => None,
        }
    }
```

listing 279: the read: live values only, the noticing read reaps, the boundary at-or-after means gone (source: rust/api/src/cache.rs, lines 42-52)

The boundary is exact and the test pins it: an entry with a 100 ms ttl answers at 99 and is dead at 100, `now < expires_at` with no tolerance and no rounding, and after the noticing read the table holds one fewer entry, the corpse never answers twice. A ttl of zero or less is refused by `set` before anything is written, an entry born dead is a defect, not a policy, and the constructor refuses a bound of zero the same way, a table that holds nothing cannot serve.

26.2. the bound and the eviction rank

Memory is capped at `max_entries`, and a new key arriving at a full table makes room in a fixed order. First the dead: the same retain the sweep runs reclaims every expired entry for free, because a corpse is space nobody is owed. Then, when everything held is live, the entry dying soonest goes, the one whose remaining life is shortest, so the table sheds what it was about to lose anyway:

```
    pub fn set(
        &self,
        key: &str,
        snap: Snapshot,
        now_millis: i64,
        ttl_millis: i64,
    ) -> Result<(), &'static str> {
        if ttl_millis ≤ 0 {
            return Err("an entry born dead is a defect, not a policy");
        }
        let mut items = self.inner.lock().unwrap();
        if let Some(entry) = items.get_mut(key) {
            entry.snap = snap;
            entry.expires_at = now_millis + ttl_millis;
            return Ok(());
        }
        items.retain(|_, entry| now_millis < entry.expires_at);
        if items.len() ≥ self.max_entries
            && let Some(victim) = items
                .iter()
                .min_by_key(|(_, entry)| entry.expires_at)
                .map(|(k, _)| k.clone())
        {
            items.remove(&victim);
        }
        items.insert(
            key.to_string(),
            Entry {
                snap,
                expires_at: now_millis + ttl_millis,
            },
        );
        Ok(())
    }
```

listing 280: the write: replace refreshes in place, the dead are swept before any live one, then the soonest deadline is the victim (source: rust/api/src/cache.rs, lines 54-87)

That rank is java's, not go's, and the trade deserves its sentence. An lru list tracks access order with a queue every read must move, and its payback needs a workload that revisits old keys, while the deadline the entry already carries is a free eviction rank whose behavior deterministic tests pin exactly. The edge table is the contract: a full table of three with deadlines 100, 50, and 200 loses the 50 on the fourth insert, a dead entry is swept in preference to any live one, and replacing an existing key refreshes its value and its deadline without evicting a neighbor, because a replace is not an admission. The eviction scan is linear in the table and runs only when a new key arrives at a full table of live entries, the same cost the java chapter owned at book scale, with the shard and the deadline heap named there as the escape hatches here.

26.3. the eager sweep

Lazy expiry keeps correctness, every read reaps what it notices, and correctness is not memory discipline: an unread hot key squatting on its bytes forever is invisible to the read path. The eager half is one function the wiring can drive on a cadence, the same retain under the same lock, answering how many it took:

```
    pub fn sweep(&self, now_millis: i64) -> usize {
        let mut items = self.inner.lock().unwrap();
        let before = items.len();
        items.retain(|_, entry| now_millis < entry.expires_at);
        before - items.len()
    }

    fn delete_matching(&self, drop_key: impl Fn(&str) -> bool) -> usize {
        let mut items = self.inner.lock().unwrap();
        let before = items.len();
        items.retain(|key, _| !drop_key(key));
        before - items.len()
    }
```

listing 281: sweep and delete_matching: the same retain twice, one for the janitor, one for invalidation (source: rust/api/src/cache.rs, lines 89-101)

Go ran its sweep on a ticker inside the cache itself and exercised the janitor's whole life in a synctest bubble, and java declined the timer entirely. The rust lane splits the difference the wiring decides: the sweep is a plain method, the main of chapter 27 spawns one thread that sleeps and sweeps, and no test ever needs the thread because the method is the contract and the clock is injected. The sweep test reaps one of three unread entries at a chosen tick and answers zero the second time, nothing left to reap.

26.4. single flight, one fill per key

The stampede, precisely: a popular key expires, 8 requests miss in the same instant, and each one innocently walks the handler because each innocently observed a miss. The fix is one flight per key. The first caller registers a slot in the flight map and becomes the filler, every caller that arrives while the fill runs parks on the slot's condvar, and when the fill lands the entry is stored and every parked caller wakes into the stored value:

```
struct FlightSlot {
    done: Mutex<bool>,
    wake: Condvar,
    parked: AtomicUsize,
}
```

listing 282: the slot: a done flag, a condvar, and a parked count the tests read (source: rust/api/src/cache.rs, lines 112-116)

```
    pub fn get_or_fill(
        &self,
        key: &str,
        fill: impl FnOnce() -> Result<Snapshot, ApiError>,
    ) -> Result<Snapshot, ApiError> {
        if let Some(snap) = self.table.get(key, (self.now_millis)()) {
            return Ok(snap);
        }
        loop {
            let (slot, first) = self.register(key);
            if !first {
                {
                    let mut done = slot.done.lock().unwrap();
                    slot.parked.fetch_add(1, Ordering::SeqCst);
                    while !*done {
                        done = slot.wake.wait(done).unwrap();
                    }
                    slot.parked.fetch_sub(1, Ordering::SeqCst);
                }
                if let Some(snap) = self.table.get(key, (self.now_millis)()) {
                    return Ok(snap);
                }
                continue;
            }
            let outcome = fill();
            if let Ok(snap) = &outcome {
                let _ = self
                    .table
                    .set(key, snap.clone(), (self.now_millis)(), self.ttl_millis);
            }
            *slot.done.lock().unwrap() = true;
            slot.wake.notify_all();
            self.retire(key, &slot);
            return outcome;
        }
    }
```

listing 283: get_or_fill: hit first, register under the lock, fill outside every lock, store before done, the waiter's recheck (source: rust/api/src/cache.rs, lines 144-179)

Two rules carry the weight. The fill runs outside every lock, a fill that held the flights mutex would serialize every unrelated key behind one slow handler, the same reason java kept its loader out of `computeIfAbsent`'s bin lock, and rust's one-liner for that lesson is that the closure runs between `register` and the store with nothing held. And the store happens before `done` is set, which is the whole correctness argument for the waiter's recheck: a parked caller wakes only after the flag flipped, the flag flipped only after the store, so the table read the waiter performs next must find the entry, unless the fill failed and stored nothing, which is the next section. The `retire` removes the key only when the map still holds this slot, `Arc::ptr_eq`, so a fresh flight a late arrival may have started is never clobbered, and the next wave after a lull starts a new flight, which is what keeps the plane a guard and not a second cache:

```
    fn register(&self, key: &str) -> (Arc<FlightSlot>, bool) {
        let mut flights = self.flights.lock().unwrap();
        if let Some(existing) = flights.get(key) {
            (Arc::clone(existing), false)
        } else {
            let slot = Arc::new(FlightSlot {
                done: Mutex::new(false),
                wake: Condvar::new(),
                parked: AtomicUsize::new(0),
            });
            flights.insert(key.to_string(), Arc::clone(&slot));
            (slot, true)
        }
    }

    fn retire(&self, key: &str, slot: &Arc<FlightSlot>) {
        let mut flights = self.flights.lock().unwrap();
        if flights
            .get(key)
            .is_some_and(|current| Arc::ptr_eq(current, slot))
        {
            flights.remove(key);
        }
    }

    pub fn waiting_on(&self, key: &str) -> usize {
        self.flights
            .lock()
            .unwrap()
            .get(key)
            .map(|slot| slot.parked.load(Ordering::SeqCst))
            .unwrap_or(0)
    }
```

listing 284: register and retire: present means join, absent means start, retirement checks the pointer, waiting_on reads the parked count (source: rust/api/src/cache.rs, lines 181-213)

The collapse test is counted, not timed. The fill blocks on a channel it controls, 8 callers arrive, and the test watches `waiting_on` until exactly 7 are parked, program state, not a stopwatch. The release lands the fill, all 8 callers answer the identical stored body, the fill counter reads exactly 1, and the table holds exactly 1 entry.

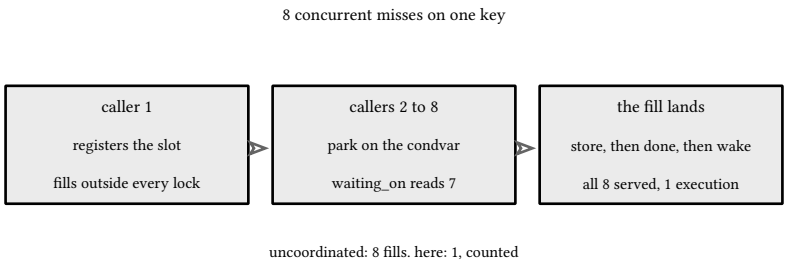


figure 46: the herd collapsed: one fill, seven parked, everyone served from the store

26.5. the recheck window and the failed fill

A failed fill is never stored. The waiter wakes into a table that still misses, loops, and registers again, so the herd that waited on a failure re-runs the fill serially, at most one at a time, never concurrently, the same settle semantics the go lane chose and the java lane declined in favor of sharing the error. Sharing the failure saves the recomputes and teaches every rider the origin's error, recomputing keeps every rider's answer its own, and the rust lane

follows go because the idempotency flight of chapter 25 already made the same call one module over. The test drives the failure lane exactly: a loader that fails once answers the error, the table holds zero entries, the next call refills to a success, and the call after that answers from the store with the loader never reached, 2 attempts in total across the three calls.

26.6. the layer at the wire

The plane becomes a layer over the router for the governed patterns, the same `match_pattern` the authz guard reads, and the lane order is the whole policy. A governed GET without a conditional header hits the table first, a hit answers the snapshot with `x-cache: hit`, a miss fills under the flight and answers `x-cache: miss`, both stamped with `cache-control: private, max-age=0, must-revalidate` and `vary: authorization`, because the key folds the actor in beside the path and the vary header is the wire telling the truth about the key:

```
fn key_of(req: &Request) -> String {
    let actor = authz::actor_of(req)
        .map(|actor| actor.user_id)
        .unwrap_or_else(|| "anonymous".to_string());
    format!("{}\u{0}{actor}", req.path)
}
```

listing 285: the key: path and actor, one nul between, identities never share an entry (source: rust/api/src/cache.rs, lines 243-248)

```
fn serve(
    req: &mut Request,
    next: &Handler,
    plane: &Arc<Plane>,
    governed: &Arc<Vec<String>>,
) -> Result<Response, ApiError> {
    let cacheable = req.method == "GET"
        && governed
            .iter()
            .any(|pattern| authz::match_pattern(pattern, &req.method, &req.path).is_some());
    if cacheable {
        if req.header("if-none-match").is_some() {
            return with_control(next(req), REVALIDATE);
        }
        let key = key_of(req);
        if let Some(snap) = plane.hit(&key) {
            return Ok(snapshot_response(&snap, REVALIDATE, true, "hit"));
        }
        let filled = plane.get_or_fill(&key, || next(req).map(|resp| Snapshot::of(&resp)));
        return with_control(
            filled.map(|snap| snapshot_response(&snap, REVALIDATE, true, "miss")),
            REVALIDATE,
        );
    }
    if !matches!(req.method.as_str(), "GET" | "HEAD") {
        let outcome = next(req);
        if let Ok(resp) = &outcome
            && resp.status < 400
        {
            plane.invalidate(&req.path);
        }
        return with_control(outcome, NO_STORE);
    }
    with_control(next(req), NO_STORE)
}
```

listing 286: the serve: hit, miss, the conditional bypass, writes invalidate on success, everything else no-store (source: rust/api/src/cache.rs, lines 291-325)

A conditional GET carries `if-none-match` and skips the snapshot entirely, the handler answers its own 304 with a bodyless response and no `x-cache` at all. The go layer served its cached 200 to a conditional request, the entry short-circuited the handler before its own revalidation could run, and this lane declines that: the 304 saves the body, the cache saves the rebuild, and letting the conditional request through composes them without a cached 200 ever answering a client that asked a question. The composition is safe because chapter 25 computes the etag, a stored representation and a fresh one carry the same digest, so nothing the layer does can make a 304 lie.

Writes do better than waiting out the ttl. A non-GET, non-HEAD response under 400 invalidates the path it touched, the written path itself, its parent collection, and any child under it, the delete that is idempotent against the update that can leave a stale window mid-flight. Everything else, ungoverned GETs included, answers `private, no-store`, a private api caches nothing unless a route says otherwise:

```
    pub fn invalidate(&self, path: &str) -> usize {
        let parent = match path.rfind('/') {
            Some(at) if at > 0 => path[..at].to_string(),
            _ => "/".to_string(),
        };
        let prefix = format!("{path}/");
        self.table
            .delete_matching(|key| match key.split_once('\u{0}') {
                Some((key_path, _)) => {
                    key_path == path || key_path == parent || key_path.starts_with(&prefix)
                }
                None => false,
            })
    }
```

listing 287: invalidate: the path, the parent, and the children, dropped by one retain (source: rust/api/src/cache.rs, lines 215-228)

The wire test drives the whole lane over a real socket with a counter in the handler: miss, hit off one execution, the conditional 304 bypassing with the counter moving, the hit still standing after the bypass because the bypass never touched the table, the PUT answering no-store and emptying the table to zero, and the next GET refilling. The invalidation test holds four keys and drops exactly the three the geometry names, the entry, its child, and the parent collection, with the unrelated key untouched.

26.7. the count

The module is 325 lines and its test file 404, and the 12 tests land green three consecutive runs under the full gate, fmt, clippy at `-D warnings`, the locked offline build, dated 2026-10-14: the exact tick, the refused zero ttl and the refused zero bound, the eager sweep count, the deadline rank, the dead-before-live room, the replace without eviction, the counted 8-to-1 collapse with 7 observed parked, the failed fill refilled, the invalidation geometry, the wire lane, the no-store default, and the error arms carrying the route policy. The crate holds 128 tests this wave, the 116 landed ones green unchanged.

sources: RFC 9111, HTTP Caching, section 3 on storing responses and section 5.2 on the no-store and private directives at rfc-editor.org, accessed 2026-10-14, and RFC 9110 section 12.5.5 for the Vary field the governed responses carry, same access date. The lazy-reap read, the deadline-rank eviction, the single-flight ordering argument, and the failed-fill policy follow the java book's cache chapter, book 4 chapter 26 of this corpus, and the wire posture, the hit and miss marks, the vary-keyed layer, and delete-after-write invalidation follow the go book's cache chapter, book 4 chapter 25, read for the parallel contracts. The 8-to-1 collapse, the parked count of 7, and the sweep counts produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build, three consecutive green runs, dated 2026-10-14.

27. rate limiting

Concurrency control answered how much at once and the cache answered how long a representation may stand, and neither answers the client that asks too fast forever. Rate limiting is the fairness layer: a budget per caller, spent one token per request, refilled as time passes, and a refused request answered with the one status code built for it, 429, through the envelope the kernel already owns. This chapter builds `rustapi::limit`, a token bucket per key behind one middleware layer, the state swept out after an idle window so the limiter's memory is bounded the moment it is born. The go book built its limiter on `golang.org/x/time/rate` and then owned the same header questions, book 4, ch 26 (go 1.27, rate limiting), the java book built the bucket on nanos and doubles, book 5, ch 27 (java 27, rate limiting), and this lane builds it on integer microtokens, where one period elapsed is exactly one token by construction and no float ever rounds.

27.1. why limits exist

Three reasons, and they rate differently. Fairness first: one client's unbounded appetite is every other client's latency, a shared server that does not budget its callers is arbitraged by whoever asks fastest. Cost second: every request spends real compute and real downstream work, and a scraper or a runaway retry loop can spend a day of budget in an hour. Abuse third, the deliberate flood, where the limit is a defense and the attacker's goal is exactly to be the loudest caller. One mechanism answers all three, which is why it lives as one layer at the stack's edge rather than as per-handler counters, and why its refusals must be cheap: a limiter that spends as much refusing as the requests cost asking protects nothing. The budget is client-visible, not internal policy. The 429 carries the shape of the budget in its headers and `Retry-After` carries the wait, and a well-behaved client backs off and spaces its work, the cheapest capacity upgrade a service ever ships.

27.2. the token bucket, all integer

One bucket per key. It holds at most `burst` tokens, starts full, every request spends one, and one whole token accrues per `refill_period_millis` of elapsed time, capped at the burst. No timer drips anything: refill is computed lazily at the moment a take arrives, from the time elapsed since the last conversion, so a key nobody asks about costs one map entry and zero wakeups. The level lives in microtokens, `1_000_000` per whole token, and the refill converts only whole periods, leaving the sub-period remainder banked in the clock rather than the level:

```
    fn refill_to(&mut self, now_millis: i64) {
        let now_us = now_millis * 1000;
        if now_us ≤ self.last_us {
            return;
        }
        if self.level_micro ≥ self.cap_micro {
            self.last_us = now_us;
            return;
        }
        let whole = (now_us - self.last_us) / self.period_us;
        if whole > 0 {
            self.last_us += whole * self.period_us;
            self.level_micro = (self.level_micro + (whole as u64).saturating_mul(MICROS_PER_TOKEN))
                .min(self.cap_micro);
        }
    }
```

listing 288: refill: whole periods convert to exactly one token each, the remainder stays in last_us, a full bucket snaps and wastes its idle time (source: rust/api/src/limit.rs, lines 50-65)

Two rules deserve their sentence each. The remainder-in-clock rule: `last_us` advances only by the whole periods converted, so four half-period advances accrue to exactly one token at the fourth, the banking java needed epsilon tolerances for in doubles falls out of the integers here, and the boundary test walks the waits `2, 1, 1` across the halves and lands the token on the fourth tick exactly. The cap rule: a full bucket snaps `last_us` forward and discards the elapsed time, because tokens that could have accrued while the bucket stood full are gone, and the test pins the consequence, after a 2.5-period idle on a 2-burst the next token arrives one full period after the read, not 0.5 periods after it, the exact spot an implementation that banks through the cap would over-admit.

```
    pub fn allow(&mut self, now_millis: i64) -> Decision {
        self.refill_to(now_millis);
        let allowed = self.level_micro ≥ MICROS_PER_TOKEN;
        if allowed {
            self.level_micro -= MICROS_PER_TOKEN;
        }
        let remaining = self.level_micro / MICROS_PER_TOKEN;
        let wait_us = ((self.last_us + self.period_us) - now_millis * 1000).max(0);
        let wait_ms = wait_us / 1000;
        Decision {
            allowed,
            limit: self.cap_micro / MICROS_PER_TOKEN,
            remaining,
            retry_after_secs: if allowed {
                0
            } else {
                (wait_ms as u64).div_ceil(1000)
            },
            reset_at_millis: now_millis + wait_ms,
        }
    }
```

listing 289: allow: spend within the level, the denial names the next whole token's arrival, the deficit rounded up to whole seconds (source: rust/api/src/limit.rs, lines 72-92)

The denial's arithmetic is one line: the wait is the distance from now to `last_us + period`, the instant the next whole token arrives, and `Retry-After` carries it rounded up to whole seconds, the delay-seconds form rfc 9110 defines as a non-negative decimal integer. The boundary table is the whole contract, measured under a frozen clock:

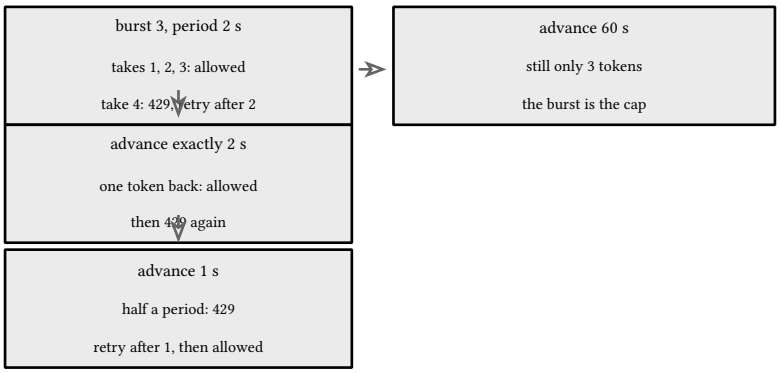


figure 47: the bucket's answers, every boundary exact under a frozen clock

At the burst, below it, far past it, exactly one period later, half a period later, and one millisecond before the boundary, where the refusal names one second because 1 ms of deficit rounds up and the next tick lands it. The race test closes the section: 10 threads against a burst of 5 at one frozen instant, and exactly 5 are admitted no matter the interleaving, the mutex making the spend monotone while the frozen clock makes refill a no-op, arithmetic where a wall-clock test would have proven nothing on a loaded machine.

27.3. the 429 contract through the ch19 writer

Rfc 6585 defines the status, the user "has sent too many requests in a given amount of time", says a response MAY carry `Retry-After`, and says a 429 MUST NOT be stored by a cache, which chapter 26's table satisfies by construction, a 429 is an answer about the request, never a stored value. The layer writes the envelope itself through the kernel's one failure writer, `ApiError` with the rate-limited code, so the 429 rides the same body shape as every other failure and carries the request id like them all:

```
pub fn layer(limiter: Arc<Limiter>, keyer: Keyer) -> Layer {
    let message = limiter.rule.message;
    Arc::new(move |next: Handler| {
        let limiter = Arc::clone(&limiter);
        let keyer = Arc::clone(&keyer);
        Arc::new(move |req: &mut Request| {
            let decision = limiter.allow(&keyer(req));
            if !decision.allowed {
                return Err(ApiError::fail(Code::RateLimited, message)
                    .with_header(HEADER_LIMIT, &decision.limit.to_string())
                    .with_header(HEADER_REMAINING, &decision.remaining.to_string())
                    .with_header(HEADER_RESET, &decision.reset_at_millis.to_string())
                    .with_header(HEADER_RETRY_AFTER, &decision.retry_after_secs.to_string()));
            }
            match next(req) {
                Ok(mut resp) => {
                    resp.headers
                        .push((HEADER_LIMIT.to_string(), decision.limit.to_string()));
                    resp.headers
                        .push((HEADER_REMAINING.to_string(), decision.remaining.to_string()));
                    resp.headers.push((
                        HEADER_RESET.to_string(),
                        decision.reset_at_millis.to_string(),
                    ));
                    Ok(resp)
                }
                Err(mut err) => {
                    err = err
                        .with_header(HEADER_LIMIT, &decision.limit.to_string())
                        .with_header(HEADER_REMAINING, &decision.remaining.to_string())
                        .with_header(HEADER_RESET, &decision.reset_at_millis.to_string());
                    Err(err)
                }
            }
        }) as Handler
    })
}
```

listing 290: the layer: denial answers the envelope with all four headers, every allowed or failed response carries the three budget headers (source: rust/api/src/limit.rs, lines 187-223)

The three budget headers ride every limited response, allowed, denied, or failed deeper in the stack, and `retry-after` rides only the denial, exactly the asymmetry the rfc's MAY permits and the tests pin: `x-ratelimit-limit`, `x-ratelimit-remaining`, `x-ratelimit-reset` beside a 200, and the same three plus `retry-after: 60` beside the 429 whose body is the envelope, code `rate_limited`, the message, and a parseable request id. The refused request costs the handler nothing, asserted with a counter that reads exactly the admitted count after the flood.

27.4. what to key on, and the address seam

The keyer is an injected function, the same construction every layer in this service takes, and the choice is policy. Keying on the resolved actor gives per-account budgets that follow the token, keying on the client address gives one budget per machine that an authenticated abuser cannot dodge by dropping their token, and routes can be folded into the key to budget an expensive family apart from a cheap one. Both siblings keyed on the remote address, and here the honest fact is a seam: chapter 19's `Request` carries no peer address, the wave's rule is wiring-only at the kernel, so the shipped keyer budgets per actor with one shared pool for the anonymous, and the address lane is the one-line `httpx` change a later wave owes, stated here rather than papered over:

```
pub fn actor_or_shared() -> Keyer {
    Arc::new(|req| {
        authz::actor_of(req)
            .map(|actor| actor.user_id)
            .unwrap_or_else(|| "shared".to_string())
    })
}
```

listing 291: the shipped keyer: the actor when authn has run, one shared pool before it (source: rust/api/src/limit.rs, lines 179-185)

The tests key on a header for the same reason production keys on an identity, one server serving many independent budgets, and the isolation test is the contract: address A exhausted to the last token leaves address B's bucket untouched, two seats held, the denial of one never spends the other.

27.5. bounded per-key state

Every key that asks once gets a bucket, so the limiter is a memory leak unless something reaps. Each seat carries its last-use millis, and the sweep that retires idle keys runs amortized inside the allow path, one retain when the window has elapsed since the last sweep, the same trick the go lane used to delete the janitor goroutine, with `held` exposed for the tests and for operators probing memory:

```
    pub fn allow(&self, key: &str) -> Decision {
        let now = (self.now_millis)();
        let mut inner = self.inner.lock().unwrap();
        if now - inner.last_sweep ≥ self.idle_window_millis {
            sweep_locked(&mut inner, now, self.idle_window_millis);
        }
        let rule = &self.rule;
        let seat = inner
            .seats
            .entry(key.to_string())
            .or_insert_with(|| BucketSeat {
                bucket: Bucket::new(rule.burst, rule.refill_period_millis, now),
                last_millis: now,
            });
        seat.last_millis = now;
        seat.bucket.allow(now)
    }
```

listing 292: allow: the amortized sweep, the seat created on first use, the touch, then the take (source: rust/api/src/limit.rs, lines 139-155)

```
fn sweep_locked(seats: &mut Seats, now: i64, idle_window_millis: i64) -> usize {
    let before = seats.seats.len();
    seats
        .seats
        .retain(|_, seat| now - seat.last_millis < idle_window_millis);
    seats.last_sweep = now;
    before - seats.seats.len()
}
```

listing 293: the sweep: one retain, idle keys out, the sweep clock moved (source: rust/api/src/limit.rs, lines 168-175)

The constructor refuses an idle window shorter than one refill period, because a window that tight would reap buckets between their own refills, an assert the test drives with the `should_panic` lane. The resurrection edge is owned rather than hidden: a key evicted while quiet comes back to a fresh full burst, an over-admission bounded by one burst per resurrection, and the test walks it exactly, one address swept out mid-window, the holder count at 1, and the returning address served from a full bucket.

27.6. the wiring and the stack order

The stack order is contract. The rate decision runs first, a map lookup and an integer take, so a flooding caller collects 429s without ever being granted a cache fill, then the cache, then the idempotency flight, all inside the access log so every refusal is a logged line a client can be pointed at, the chapter 20 order rule applied to three more layers:

```
    let mut layers = middleware::stack(middleware::stderr_sink());
    layers.insert(0, observing);
    layers.push(limiting);
    layers.push(caching);
    layers.push(idem);
    let stack = middleware::chain(layers);
    let guard = authz::guard(policy_rows);
```

listing 294: the wiring: rate outside cache outside idempotency, all inside the log layer, the guard inside the whole stack (source: rust/api/src/main.rs, lines 169-175)

The cache's janitor rides here too, one named thread sleeping 10 seconds between sweeps of the 512-entry, 30-second plane, and the limiter runs burst 30 with one token per second and the default 10-minute idle window. The whole lane was driven through the real binary, dated 2026-10-14: a fresh boot serves `POST /api/notes` with the etag and the budget headers, `x-ratelimit-remaining` walking down from 28 across the run, the second GET answers `x-cache: hit`, a conditional GET answers the bodyless 304, a star `If-Match` PUT lands version 2, and the same idempotency key replayed returns the identical `location` in 29 microseconds without the handler running.

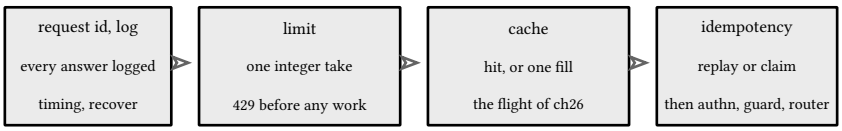


figure 48: the request path: the cheap integer refusal sits outside every expensive lane

27.7. the count

The module is 223 lines and its test file 265, and the 8 tests land green three consecutive runs under the full gate, fmt, clippy at `-D warnings`, the locked offline build, dated 2026-10-14: the boundary table with its six cells and the millisecond-short cell, the partial-advance banking at waits 2, 1, 1, the cap that wastes its idle time, the isolation of two addresses' budgets, the refused constructor lane, the idle sweep with the resurrection, the frozen race admitting exactly 5 of 10, and the wire test's envelope, headers, untouched handler, and second address. The

crate holds 136 tests this wave, the 128 landed ones green unchanged, and the whole gate, 85 sample files at 902 checks, the vectors hash, and the zero-dep manifest, green end to end.

sources: RFC 6585 section 4 at rfc-editor.org, the 429 definition quoted, the MAY on `Retry-After`, and the MUST NOT on storing 429 responses, accessed 2026-10-14. RFC 9110 section 10.2.3 at rfc-editor.org, “The Retry-After field value can be either an HTTP-date or a number of seconds to delay after receiving the response” and the delay-seconds grammar as a non-negative decimal integer, accessed 2026-10-14. The lazy refill, the burst ceiling, the amortized idle sweep, and the header set follow the go book’s rate limiting chapter, book 4 chapter 26 of this corpus, and the integer-exact accrual and the deficit rounding follow the java book’s chapter 27, read for the parallel contracts, with java’s floating-point tolerance discussion consulted for the divergence this lane’s microtokens avoid. The boundary table, the frozen race, and the binary run produced by `cargo test --locked` under `CARGO_NET_OFFLINE=true` and the built `rustapi.exe` on the pinned 1.99.0 build, three consecutive green runs, dated 2026-10-14.

28. observability

The access log answers who did what, one line per exchange with its request id, and nothing answers how much and how slow. This chapter builds `rustapi::obs` on nothing outside the crate: the 2 series a service actually reads, one counter row per route and status and one latency histogram per route, a bounded ring of request events, the percentile arithmetic the next chapter's load reports quote, and a second listener on `RUSTAPI_DEBUG_ADDR` that serves the prometheus text exposition beside the api port. The go book built the same layer with client floats and a hand-rolled traceparent, book 4, ch 27 (go 1.27, observability), the java book over `LongAdder` and `BigDecimal`, book 5, ch 28 (java 27, observability), and this lane builds it over whole microseconds in `u64`, the same integer discipline the token bucket chapter kept for its refill, where no float ever decides which bucket a latency lands in and no float ever rounds a percentile's rank.

28.1. the two series

A counter only ever goes up and a histogram only ever answers which bucket. `rustapi_requests_total` is one row per route and status code and `rustapi_request_duration_seconds` is one per route with the sum and count a scraper needs beside the buckets, both held in one registry under one mutex:

```
    pub fn count_request(&self, route: &str, code: u16) {
        let mut inner = self.inner.lock().unwrap();
        let hits = inner
            .counts
            .entry((route.to_string(), code.to_string()))
            .or_insert(0);
        *hits += 1;
    }

    pub fn observe(&self, route: &str, micros: u64) {
        let mut inner = self.inner.lock().unwrap();
        let hist = inner.hists.entry(route.to_string()).or_default();
        for (i, bound) in DURATION_BOUNDS_MICROS.iter().enumerate() {
            if micros ≤ *bound {
                hist.cum[i] += 1;
            }
        }
        hist.cum[DURATION_BOUNDS_MICROS.len()] += 1;
        hist.sum_micros += micros;
        hist.n += 1;
    }
```

listing 295: one observation: the counter row, the route's histogram, and the bucket walk over integer micros (source: rust/api/src/obs.rs, lines 106-126)

Three decisions in that walk earn their sentences. The buckets are cumulative, an observation fills every bound at or above it, which is the prometheus shape and what makes the counts monotone in the bound. The bounds live and compare in whole microseconds, so the 2.5 millisecond bound is the integer 2500 and a 2500 microsecond observation fills it exactly, the boundary java needed a comparison rule to state and go needed a float to almost get right. And the per-route cells are a fixed array inside the map entry, created with the entry itself, because a lazily created cell is a check-then-act race under a route's first concurrent observations, the hole java's adversarial round measured at 7 of 8 first observations counted. Here one mutex owns the whole registry, so the eager array is enough, and the contention test pins it: 10 threads, 8000 observations, the count and the `+Inf` cell both exactly 8000.

The ladder itself is the same policy statement both siblings made, tight at 1, 2.5, and 5 milliseconds where service latencies live, geometric through the seconds:

```
pub const DURATION_BOUNDS_MICROS: [u64; 11] = [
    1_000, 2_500, 5_000, 10_000, 25_000, 50_000, 100_000, 250_000, 500_000, 1_000_000, 2_500_000,
];
```

listing 296: 11 finite bounds in micros, plus Inf implicit as the last cell (source: rust/api/src/obs.rs, lines 30-32)

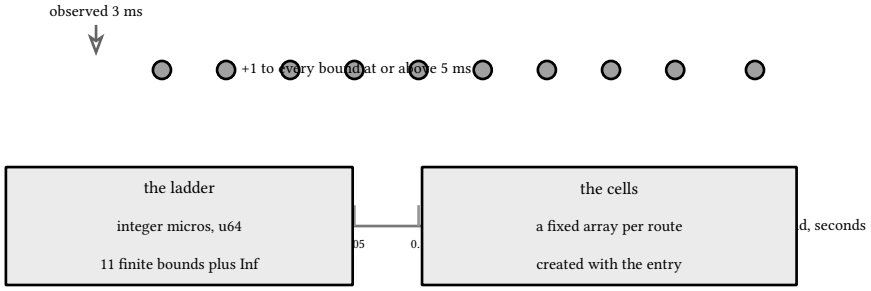


figure 49: cumulative buckets: one observation fills every bound above it

28.2. the text exposition

The registry renders the prometheus text format directly, `#` comment lines carrying the types, series sorted by label tuple, bucket lines ascending with `+Inf` last, then the sum and the count. The sort is not a sort: the registry holds `BTreeMap` keyed on the label tuple, so the iteration order is the sorted order, the ordering go and java paid for at render time is paid here once at insert. Both families elide entirely when empty, and zero buckets elide their lines, the one divergence from the client libraries, which print every finite bucket at zero as well:

```
    pub fn exposition(&self) -> String {
        let inner = self.inner.lock().unwrap();
        let mut out = String::new();
        if !inner.counts.is_empty() {
            out.push_str(&format!("# HELP {REQUESTS_NAME} {REQUESTS_HELP}\n"));
            out.push_str(&format!("# TYPE {REQUESTS_NAME} counter\n"));
            for ((route, code), hits) in &inner.counts {
                out.push_str(&format!(
                    "{REQUESTS_NAME}{{route=\"{route}\",code=\"{code}\"}} {hits}\n"
                ));
            }
        }
        if !inner.hists.is_empty() {
```

listing 297: the counter family: BTreeMap order is the label tuple order, the family absent when nothing was counted (source: rust/api/src/obs.rs, lines 136-148)

```
            out.push_str(&format!("# TYPE {DURATION_NAME} histogram\n"));
            let inf = DURATION_BOUNDS_MICROS.len();
            for (route, hist) in &inner.hists {
                for (i, bound) in DURATION_BOUNDS_MICROS.iter().enumerate() {
                    if hist.cum[i] > 0 {
                        out.push_str(&format!(
                            "{DURATION_NAME}_bucket{{route=\"{route}\",le=\"{}\"}} {}\n",
                            format_secs(*bound),
                            hist.cum[i]
                        ));
                    }
                }
                out.push_str(&format!(
                    "{DURATION_NAME}_bucket{{route=\"{route}\",le=\"+Inf\"}} {}\n",
                    hist.cum[inf]
                ));
                out.push_str(&format!(
                    "{DURATION_NAME}_sum{{route=\"{route}\"}} {}\n",
                    format_secs(hist.sum_micros)
                ));
                out.push_str(&format!(
                    "{DURATION_NAME}_count{{route=\"{route}\"}} {}\n",
                    hist.n
                ));
            }
        }
        out
```

listing 298: the histogram family: zero cells elided, +Inf always present, the sum and count after the buckets (source: rust/api/src/obs.rs, lines 150-176)

The seconds render without a float anywhere. The sum is microseconds in a `u64`, the renderer divides once for the whole part, formats the remainder as 6 digits, and pops trailing zeros:

```
pub fn format_secs(micros: u64) -> String {
    let secs = micros / MICROS_PER_SECOND;
    let frac = micros % MICROS_PER_SECOND;
    if frac == 0 {
        return secs.to_string();
    }
    let mut digits = format!("{frac:06}");
    while digits.ends_with('0') {
        digits.pop();
    }
    format!("{secs}.{digits}")
}
```

listing 299: the shortest round trip from u64 micros: 903400 prints 0.9034, 2500000 prints 2.5, 1000000 prints 1 (source: rust/api/src/obs.rs, lines 63-74)

Go reached the same bytes through `strconv.FormatFloat` and java through `BigDecimal`, both correct, and this lane reaches them through integer division, which cannot be anything else. The frozen vector test drives 3 observations of 3 ms, 0.4 ms, and 900 ms through a bare registry and pins the exact 16 lines, the 2 type comments and 14 series lines, including the `0.001` bucket the 0.4 ms observation fills and the `0.9034` sum. The elision has its own test: a registry whose smallest observation is 3 ms renders no `0.001` or `0.0025` line at all, and a scraper reading `+Inf` and `_count` still gets the totals.

28.3. the ops port

Where the exposition serves from is a placement decision the guard forces. The metrics route is unauthenticated by convention, a scraper carries no bearer token, and chapter 23's guard denies by default, every registered route needs a policy row. So the exposition belongs on a listener the guard never wraps. The kernel's `serve` takes one router, and the answer is a second `serve`, one route, on `RUSTAPI_DEBUG_ADDR`, whose default is `127.0.0.1:19590` beside the api's 19580:

```
pub fn serve_metrics(
    listener: TcpListener,
    metrics: Arc<Metrics>,
    version: &str,
) -> std::io::Result<ServerHandle> {
    let mut router = Router::new();
    let scraped = Arc::clone(&metrics);
    router.route(
        "GET",
        METRICS_PATH,
        Arc::new(move |_| {
            Ok(Response {
                status: 200,
                headers: vec![("content-type".to_string(), EXPOSITION_TYPE.to_string())],
                body: scraped.exposition().into_bytes(),
            })
        }),
    );
    serve(
        listener,
        router.handler(),
        Arc::new(Lifecycle::new()),
        ServerConfig::new(version),
    )
}
```

listing 300: the ops listener: the kernel's own serve, one route, the exposition's content type (source: rust/api/src/obs.rs, lines 287-311)

The second listener reuses the chapter 19 server wholesale, worker pool, keep-alive, envelope and all, so the exactness questions answer themselves the router's way: `/metrics/anything` matches no route and answers the 404 envelope, a `POST /metrics` answers 405 with `allow: GET`, and the one route that exists serves the format's own `text/plain; version=0.0.4; charset=utf-8` content type, which scrapers read before the body. The test scrapes a real port and pins all three answers.

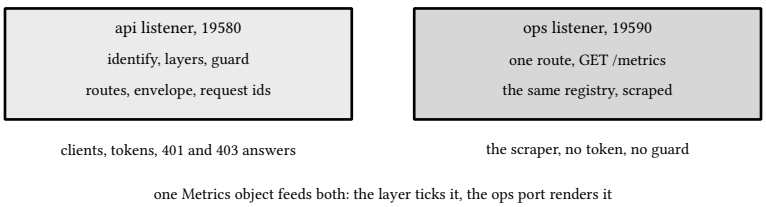


figure 50: two listeners, one process: the api port behind the guard, the ops port beside it

28.4. the layer at the edge

The layer seats first in the stack vector, which the chain applies outermost, so its numbers cover every layer below it and its counter ticks at the status that really went out. The route label comes from the app's own pattern table, the same `match_pattern` call the guard makes, the method prefix stripped so the label reads `/healthz` and not `GET /healthz`:

```
pub fn route_label(patterns: &[String], method: &str, path: &str) -> String {
    for pattern in patterns {
        if let Some(m) = authz::match_pattern(pattern, method, path)
            && let Some((_, rest)) = m.pattern.split_once(' ')
        {
            return rest.to_string();
        }
    }
    UNMATCHED.to_string()
}
```

listing 301: the label: the guard's own matcher, method stripped, one honest unmatched bucket (source: rust/api/src/obs.rs, lines 244-253)

```
pub fn layer(
    metrics: Arc<Metrics>,
    ring: Arc<Ring>,
    patterns: Vec<String>,
    now_micros: Arc<dyn Fn() -> i64 + Send + Sync>,
) -> Layer {
    Arc::new(move |next: Handler| {
        let metrics = Arc::clone(&metrics);
        let ring = Arc::clone(&ring);
        let patterns = Arc::new(patterns.clone());
        let now_micros = Arc::clone(&now_micros);
        Arc::new(move |req: &mut Request| {
            let start = now_micros();
            let label = route_label(&patterns, &req.method, &req.path);
            let outcome = next(req);
            let code = match &outcome {
                Ok(resp) => resp.status,
                Err(err) => err.status,
            };
            let micros = (now_micros() - start).max(0) as u64;
            metrics.count_request(&label, code);
            metrics.observe(&label, micros);
            ring.push(RequestEvent {
                route: label,
                code,
                micros,
            });
            outcome
        }) as Handler
    })
}
```

listing 302: the layer: label, outcome, status read off either arm of the Result, duration from the injected micros clock (source: rust/api/src/obs.rs, lines 255-285)

Reading the status off the `Result` is the quiet gift of this kernel's shape: chapter 19's connection loop owns error rendering, so a layer sees either a response or the `ApiError` that will become one, and both arms carry the status. A request the limiter refused ticks a 429 row, a path nobody routed ticks `unmatched` with a 404, and a known path under the wrong method answers 405 and lands under `unmatched` beside the 404s, because the label matcher honors the method exactly as the guard does, and the code dimension separates them. The honest reading of that row is requests the router did not serve. The clock is the same injected seam every layer takes, here in micros, and the duration clamps at zero so a clock that steps backwards between the two reads records 0 rather than a `u64` underflow panic.

28.5. the ring and the exact tail

Metrics answer aggregates, and the aggregate says the tail moved without saying which request moved it. Go answered with pprof and a hand-rolled traceparent, java with one custom JFR event the runtime records, and the rust runtime ships neither. The lane's answer is a bounded ring of the last 1024 request events, each one route, code, and micros, readable at any moment without a recording running and forgetting by overwriting its own oldest slot:

```
    pub fn push(&self, event: RequestEvent) {
        let mut state = self.state.lock().unwrap();
        let at = state.next;
        state.slots[at] = Some(event);
        state.next = (at + 1) % self.cap;
    }

    pub fn len(&self) -> usize {
        let state = self.state.lock().unwrap();
        state.slots.iter().filter(|slot| slot.is_some()).count()
    }

    pub fn is_empty(&self) -> bool {
        self.len() == 0
    }

    pub fn events(&self) -> Vec<RequestEvent> {
        let state = self.state.lock().unwrap();
        let mut out = Vec::with_capacity(self.cap);
        for i in 0..self.cap {
            let at = (state.next + i) % self.cap;
            if let Some(event) = &state.slots[at] {
                out.push(event.clone());
            }
        }
        out
    }

    pub fn quantile(&self, permille: u64) -> Option<u64> {
        let mut latencies: Vec<u64> = self.events().iter().map(|e| e.micros).collect();
        latencies.sort_unstable();
        nearest_rank(&latencies, permille)
    }
```

listing 303: the ring: overwrite the oldest, read the window in age order, quantile by sorting a copy (source: rust/api/src/obs.rs, lines 209-241)

The ring is the exact half of the chapter's percentile story, the latency window a human asks about at three in the morning, and its cost is stated plainly: 1024 events of 3 fields, one mutex, no background thread, no wakeup. The quantile over it sorts a copy and takes the nearest rank:

```rust
pub fn nearest_rank(sorted: &[u64], permille: u64) -> Option<u64> {
    if sorted.is_empty() {
        return None;
    }
    let n = sorted.len() as u64;
    let rank = permille.saturating_mul(n).div_ceil(1000).clamp(1, n) as usize;
    Some(sorted[rank - 1])
}
```

listing 304: nearest rank: the rank is ceil of permille times n over 1000, in u64 division, clamped at both ends (source: rust/api/src/obs.rs, lines 34-41)

The percentile argument is a permille integer, 500, 990, 999, and the rank is one integer ceiling. This is the boundary both siblings learned the hard way: java's adversarial round measured a double product sitting an ulp above an integral rank and ceiling one too far at 2 vectors, forcing exact decimal arithmetic, and go's round caught a harness accepting a NaN rate. The rust answer is the type. A `u64` permille cannot be NaN, the product cannot round, `div_ceil` is exact, and the boundary test pins the contrasts: p50 of 1, 1, 2, 3, 5, 8 milliseconds is 2, the third smallest, p99 of the same 6 is 8, the largest, because a percentile past the data is a bound, and 99 samples at 0.9 ms plus one at 1.1 gives an exact p99 of 0.9.

28.6. p99 from buckets

The scrape cannot answer nearest rank. A histogram only says which bound holds the rank, and the honest answer is the bound itself, never a value invented inside it:

```rust
pub fn bound_for_rank(cum: &[u64], bounds: &[u64], permille: u64) -> Option<u64> {
    if bounds.is_empty() || cum.len() != bounds.len() + 1 {
        return None;
    }
    let total = *cum.last()?;
    if total == 0 {
        return None;
    }
    let rank = permille
        .saturating_mul(total)
        .div_ceil(1000)
        .clamp(1, total);
    for (i, bound) in bounds.iter().enumerate() {
        if cum[i] ≥ rank {
            return Some(*bound);
        }
    }
    None
}
```

listing 305: the bucket form: the smallest bound holding the rank, None when only +Inf sees it (source: rust/api/src/obs.rs, lines 43-61)

The `None` case is the honesty lever. Past the last finite bound only `+Inf` sees the samples and infinity is not a latency, so the function answers nothing rather than inventing the top bound's value, and the caller states it. The 99-at-0.9-plus-one-at-1.1 vector pins both halves: the exact p99 is 900 micros, the bucket p99 is the 1000 micros bound, and the bound form can never undercut the exact one, the property the load chapter's report asserts between its 2 rows.

28.7. one request fully observed

The wiring is 5 lines in the program's table. The policy rows are built once and feed two consumers, the guard and the layer's label table, the registry and ring are built beside them, the ops server starts serving before the api does, and the layer is inserted at the stack's position zero:

```
        .collect();
    let observing = obs::layer(
        Arc::clone(&metrics),
        Arc::clone(&ring),
        route_patterns,
        Arc::new(wall_clock_micros),
    );
    let ops_listener = match TcpListener::bind(&cfg.debug_addr) {
        Ok(listener) => listener,
        Err(err) => {
            eprintln!("rustapi: bind {}: {err}", cfg.debug_addr);
            std::process::exit(1);
        }
    };
    let ops = match obs::serve_metrics(ops_listener, Arc::clone(&metrics), &version) {
        Ok(server) => server,
```

listing 306: the wiring: pattern rows feed the guard and the label table, the ops listener binds beside the api's (source: rust/api/src/main.rs, lines 148-163)

```
        identify(stack(guard(router.handler()))),
        lifecycle,
        ServerConfig::new(&version),
    ) {
        Ok(server) => server,
        Err(err) => {
```

listing 307: the layer at position zero, outermost in the stack, covering every layer below it (source: rust/api/src/main.rs, lines 178-183)

One request through the whole surface, read back: the access log carries its line, the counter row for its route and code ticks, its latency fills the histogram's tail cells, its event sits in the ring, and a scrape of 19590 renders all of it. The budget capture of chapter 29 shows the same readback against the real binary, 200 fires offered plus the readiness probe as the 201st, the exposition counting 30 twos hundreds and 171 fours, one registry observed from both ports. The module is 311 lines and its test file 315, and the 11 tests land green three consecutive runs under the full gate, fmt, clippy at `-D warnings`, the locked offline build, dated 2026-10-14: the sorted counter exposition, the frozen 16-line histogram vector, the elided zero buckets, the shortest round trip at 5 vectors, the nearest rank and bound tables with their clamps, the ring's overwrite and quantile, the 10-thread contention, the label table, the layer over a real listener with exact 100 micros durations under a tick clock, and the ops port's three answers. The crate count after this wave is chapter 31′s inventory, and the 135 landed tests stayed green unchanged through it.

The lane's boundary is stated as plainly: distributed tracing is not built here. The go chapter owns the W3C traceparent parse and the span tree, and a rust copy would be the same arithmetic wearing a new name, while this service still has one hop and no tree to print.

sources: the prometheus exposition format documentation, the content type line, the histogram bucket semantics and the optionality of HELP lines, at prometheus.io/docs/instrumenting/exposition_formats, accessed 2026-10-14. The ladder, the cumulative cells, the elision divergence, and the unmatched label follow the go book's observability chapter, book 4 chapter 27 of this corpus, and the integer bounds, the elided family comment, and the ulp-high rank lesson follow the java book's chapter 28, read for the parallel contracts. The div_ceil rank, the permille type, and the micros ladder verified live 2026-10-14 by the rustapi obs tests, 11 green three consecutive runs under `cargo test --locked` with `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build, and the crate at 157 tests the same three runs.

29. load and p99

A service whose p99 is a guess is a service shipped on faith. This chapter builds the tool that replaces the guess, `rustapi::load`, and points it at the real stack: the client on plain `std::net`, the closed loop and the open loop over 3 injected seams, the report whose every column renders from integers, and the dated captures this machine produced, ladder and budget, committed beside the code. The go book built the same harness around an `http.Client` seam with one goroutine per arrival, book 4, ch 28 (go 1.27, load and p99), the java book around one virtual thread per worker, book 5, ch 29 (java 27, load and p99), and this lane builds it around one OS thread per fire and a zero-dependency client, because the crate answers every want the same way: nothing outside the standard library.

29.1. the client, plain std::net

The whole client is one function. Resolve the authority, connect with a timeout, write one `GET` with `connection: close`, read to the end, parse the status out of the first line:

```
pub fn http_get(base_url: &str, path: &str) -> Outcome {
    let authority = base_url.trim_start_matches("http://").trim_end_matches('/');
    let candidates: Vec<std::net::SocketAddr> = authority
        .to_socket_addrs()
        .map(|addrs| addrs.collect())
        .unwrap_or_default();
    let mut stream = None;
    for addr in candidates {
        if let Ok(connected) = TcpStream::connect_timeout(&addr, CONNECT_TIMEOUT) {
            stream = Some(connected);
            break;
        }
    }
    let Some(mut stream) = stream else {
        return Outcome::failed();
    };
    let _ = stream.set_read_timeout(Some(IO_TIMEOUT));
    let _ = stream.set_write_timeout(Some(IO_TIMEOUT));
    let head = format!("GET {path} HTTP/1.1\r\nhost: {authority}\r\nconnection: close\r\n\r\n");
    if stream.write_all(head.as_bytes()).is_err() {
        return Outcome::failed();
    }
    let mut raw = Vec::new();
    if stream.read_to_end(&mut raw).is_err() && raw.is_empty() {
        return Outcome::failed();
    }
    let text = String::from_utf8_lossy(&raw);
    match text.split_whitespace().nth(1).and_then(|s| s.parse().ok()) {
        Some(status) => Outcome::at(status),
        None => Outcome::failed(),
    }
}
```

listing 308: one fire: resolve every address, connect with a timeout, one request, read to the close, every failure an Outcome never a panic (source: rust/api/src/load.rs, lines 67-98)

Three decisions earn their sentences. The connection asks to close, rfc 9112 section 9.6, so the server's answer ends at the connection close and `read_to_end` terminates without parsing lengths, the cheapest correct body read a zero-dep client can make. Every failure is a returned `Outcome::failed()`, never a panic and never a skipped request, because a load tool that dies on the first refused connection loses exactly the data the cliff produces. And the

resolver loop tries every address the name resolves to, a lesson the compose healthcheck taught the hard way: inside the container `localhost` resolves v6 first and the listener holds v4, and a client that dials only the first answer reported a healthy service as dead. And the trade is stated rather than hidden: one fresh connection per fire, no pooling, so every latency in the report pays the connect and the throughput ceiling measures the connect-plus-serve path, not the served path alone. The go and java clients pooled their connections and their ladders measured higher, and the two sets of numbers are not comparable to each other, a fact the capture header states by naming the client.

29.2. the two loops

The closed loop holds concurrency constant: `workers` threads, each firing one request at a time and starting the next only after the previous response lands, so offered load never exceeds the worker count no matter how slow the server gets. The open loop holds arrival rate constant: arrival `i` is scheduled at `i` periods past the start and fires on its own thread whether or not anything has come back, which is the loop that measures the overload cliff, because offered load stays fixed even as latency explodes:

```
    pub fn closed(
        &self,
        workers: usize,
        per_worker: u64,
        warmup_n: u64,
        stop: Arc<AtomicBool>,
    ) -> Run {
        let sink = Arc::new(Mutex::new(Run::default()));
        warmup(self, &sink, warmup_n);
        let start = (self.now_micros)();
        let mut handles = Vec::new();
        for _ in 0..workers {
            let fire = Arc::clone(&self.fire);
            let now_micros = Arc::clone(&self.now_micros);
            let sink = Arc::clone(&sink);
            let stop = Arc::clone(&stop);
            handles.push(std::thread::spawn(move || {
                for _ in 0..per_worker {
                    if stop.load(Ordering::SeqCst) {
                        break;
                    }
                    let begin = now_micros();
                    let outcome = fire();
                    let micros = (now_micros() - begin).max(0) as u64;
                    record(&sink, micros, &outcome);
                }
            }));
        }
        for handle in handles {
            let _ = handle.join();
        }
        let mut run = sink.lock().unwrap();
        run.elapsed_micros = (self.now_micros)() - start;
        run.stopped = stop.load(Ordering::SeqCst);
        run.clone()
    }
```

listing 309: the closed loop: one thread per worker, joined, warmup first and unrecorded (source: rust/api/src/load.rs, lines 140-175)

```
    pub fn open(&self, rate_per_sec: u64, total: u64, warmup_n: u64, stop: Arc<AtomicBool>) -> Run {
        assert!(rate_per_sec > 0, "an open loop divides by its rate");
        let sink = Arc::new(Mutex::new(Run::default()));
        warmup(self, &sink, warmup_n);
        let start = (self.now_micros)();
        let mut handles = Vec::new();
        for i in 0..total {
            let deadline = schedule_micros(i, rate_per_sec);
            let fire = Arc::clone(&self.fire);
            let now_micros = Arc::clone(&self.now_micros);
            let sleep_micros = Arc::clone(&self.sleep_micros);
            let sink = Arc::clone(&sink);
            let stop = Arc::clone(&stop);
            handles.push(std::thread::spawn(move || {
                let wait = deadline as i64 - ((now_micros)() - start);
                if wait > 0 {
                    (sleep_micros)(wait);
                }
                if stop.load(Ordering::SeqCst) {
                    return;
                }
                let begin = (now_micros)();
                let outcome = fire();
                let micros = (now_micros() - begin).max(0) as u64;
                record(&sink, micros, &outcome);
            }));
        }
        for handle in handles {
            let _ = handle.join();
        }
        let mut run = sink.lock().unwrap();
        run.elapsed_micros = (self.now_micros)() - start;
        run.stopped = stop.load(Ordering::SeqCst);
        run.clone()
    }
```

listing 310: the open loop: one thread per arrival, the deadline from absolute i, late tolerance (source: rust/api/src/load.rs, lines 177-211)

Little's law, in flight equals arrival rate times service time, predicts what the open loop shows: at fixed arrival rate, rising latency turns directly into a rising in-flight count. The late tolerance is one line, the wait computed as deadline minus now, so an arrival whose deadline has passed fires immediately and never sleeps backwards, and the test pins it with a clock that steps 10 seconds per read: both arrivals of a 10-per-second schedule fire with zero sleeps. A thread per arrival costs a real stack, some hundred kilobytes against a goroutine's few, and the tool's totals stay in the thousands, stated as the lane's own cost of the std-only ruling.

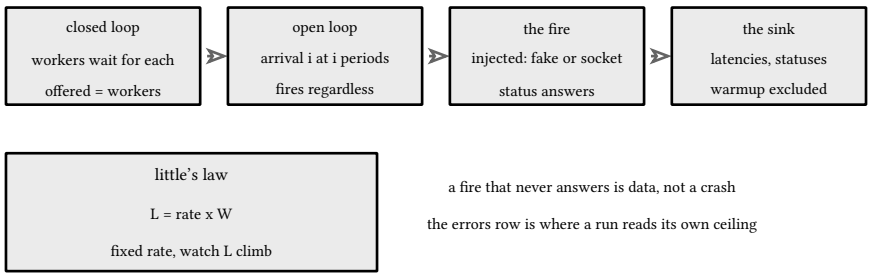


figure 51: workers, pacer, server, sink: one harness, two loops

29.3. the schedule is pure integer math

Arrival `i` of `rate` per second fires at `i` seconds over `rate`, computed in `u128` and returned as micros:

```
pub fn schedule_micros(i: u64, rate_per_sec: u64) -> u64 {
    ((i as u128 * MICROS_PER_SECOND as u128) / rate_per_sec as u128) as u64
}
```

listing 311: the deadline from absolute i: truncation error under one micro per arrival, never accumulated (source: rust/api/src/load.rs, lines 56-58)

Two properties fall out of the integers. The deadline is computed from the absolute arrival index, not by accumulating periods, so a rate like 7 that divides no power of ten truncates each deadline by less than one microsecond, `schedule_micros(1, 7)` is 142857, and the error never compounds across arrivals. And the rate is a `u64`, so the NaN rate go's adversarial round caught its harness accepting cannot exist here, answered by the type rather than by a validation branch, while the division by zero the type cannot keep out is refused by the assert, `rate_per_sec > 0`, in the lane's `should_panic`-testable style. The schedule table is pinned as pure math, six cells including the non-dividing rate.

29.4. warmup and the stop signal

Warmup fires first, unpaced and unrecorded, so the first measurement does not pay for cold caches and the kernel's fresh connection setup, and a warmup that cannot reach the server is counted in `warmup_errors` rather than failing the run, because the run that follows still measures what it measures:

```
fn warmup(harness: &Harness, sink: &Arc<Mutex<Run>>, warmup: u64) {
    for _ in 0..warmup {
        let outcome = (harness.fire)();
        if !outcome.ok {
            sink.lock().unwrap().warmup_errors += 1;
        }
    }
}
```

listing 312: warmup: fired, never recorded, failures counted apart (source: rust/api/src/load.rs, lines 112-119)

The stop signal is the ctrl-c equivalent, an `Arc<AtomicBool>` every loop consults before offering the next fire and never inside one. An in-flight fire always completes, a worker blocks on its own fire by construction, so setting the flag is a graceful drain: offering stops, the pending land, the run reports `stopped`. The tests pin both ends deterministically, a fire closure that sets the flag on its third call leaves exactly 3 recorded fires out of 10 offered, and a pre-set flag fires nothing at all, 0 fires, 0 latencies, `stopped` true.

29.5. the report, every number an integer

One run folds into one report, and the report renders without a float anywhere. The rate is computed in milli-requests per second from a `u128` product, the percentiles are chapter 28's nearest rank over a sorted copy, and the cells print by integer division with 3 fraction digits:

```rust
pub fn summarize(name: &str, run: &Run) -> Report {
    let mut report = Report {
        n: run.n,
        errors: run.errors,
        ..zero_report(name, "exact")
    };
    if run.elapsed_micros > 0 {
        report.rps_milli = (run.n as u128 * 1_000_000_000 / run.elapsed_micros as u128) as u64;
    }
    let mut sorted = run.latencies.clone();
    sorted.sort_unstable();
    if sorted.is_empty() {
        return report;
    }
    let total: u64 = sorted.iter().sum();
    report.mean_micros = total / sorted.len() as u64;
    report.p50_micros = obs::nearest_rank(&sorted, P50).unwrap_or(0);
    report.p90_micros = obs::nearest_rank(&sorted, P90).unwrap_or(0);
    report.p99_micros = obs::nearest_rank(&sorted, P99).unwrap_or(0);
    report.max_micros = *sorted.last().unwrap_or(&0);
    report
}
```

listing 313: summarize: the exact form, percentiles by permille, the rate in milli from u128 (source: rust/api/src/load.rs, lines 242-263)

```rust
pub fn summarize_buckets(name: &str, cum: &[u64], sum_micros: u64) -> Report {
    let top = *obs::DURATION_BOUNDS_MICROS.last().unwrap_or(&0);
    let mut report = zero_report(name, "buckets");
    let Some(total) = cum.last().copied() else {
        return report;
    };
    if total == 0 {
        return report;
    }
    report.n = total;
    report.mean_micros = sum_micros / total;
    report.p50_micros = obs::bound_for_rank(cum, &obs::DURATION_BOUNDS_MICROS, P50).unwrap_or(top);
    report.p90_micros = obs::bound_for_rank(cum, &obs::DURATION_BOUNDS_MICROS, P90).unwrap_or(top);
    report.p99_micros = obs::bound_for_rank(cum, &obs::DURATION_BOUNDS_MICROS, P99).unwrap_or(top);
    report.max_micros = obs::bound_for_rank(cum, &obs::DURATION_BOUNDS_MICROS, 1000).unwrap_or(top);
    report
}
```

listing 314: the buckets form: count and sum exact, percentiles as bounds, src names the approximation (source: rust/api/src/load.rs, lines 265-281)

```rust
fn ms_cell(micros: u64) -> String {
    format!("{}.{:03}", micros / 1000, micros % 1000)
}

fn rps_cell(rps_milli: u64) -> String {
    format!("{}.{:03}", rps_milli / 1000, rps_milli % 1000)
}
```

listing 315: the cells: every column rendered by integer division, no float formatting in the table (source: rust/api/src/load.rs, lines 283-289)

The golden test pins the bytes from a fixed vector, 6 latencies of 1, 1, 2, 3, 5, and 8 milliseconds over a 60 millisecond window, the same window the go lane's golden pinned at 100.0 rps where the java lane's 8 millisecond window read 750: here it renders 100.000 rps, p50 of 2.000, p90, p99, and max all clamped to 8.000 because a percentile past the data is a bound, and the bucket twin of the same observations reads p50 2.500 and p99 10.000, the bounds the ranks land in, never anything inside them. The test asserts the ordering property between the 2 rows, the bucket p99 can never undercut the exact one, which is the whole honesty of the src column in one line of code.

29.6. the ladder, measured

The capture is dated, machine-named, and committed at `books/rust/api/captures/2026-10-14-load.txt`. The stack is the spine's own, kernel routes, the notes resource, the wal engine, the limiter raised to burst 100000 through the same rule constructor the wiring uses, its default untouched, because the ladder's subject is the stack, not the limiter:

```
client: one fresh connection per fire, no pooling, connection close per request
name                  n  errors      rps    p50_ms    p90_ms    p99_ms    max_ms  src
closed-1            500       0  398.643     2.525     2.640     2.824     3.467  exact
closed-8           1600       0 3140.198     2.540     2.755     3.076     3.724  exact
closed-32          3200       0 9939.400     3.158     3.612     4.025     4.463  exact
open-5000          5000       0 4993.333     1.898     2.851     3.262     3.772  exact
```

listing 316: the ladder: 3 closed loops and one open loop, this machine, 2026-10-14 (source: rust/api/captures/2026-10-14-load.txt, lines 9-14)

Reading it: 1 worker answers 398.643 rps at a p50 of 2.525 ms, and the connect dominates, the client's stated trade, not a server number. 8 workers reach 3140, 32 reach 9939, and the growth from 8 to 32 is where the pool earns its keep. The open loop at 5000 offered per second achieves 4993.333 with all 5000 arrivals served and 0 errors, 99.87 percent of schedule, the unremarkable number that is the point: below capacity the open loop confirms the closed loop's story, and only above capacity do the 2 designs differ. The tail tells it from the other end, p50 moves 2.525 to 3.158 ms across the ladder while p99 moves 2.824 to 4.025 and max 3.467 to 4.463, the queue grows and the percentile ladder is where it shows.

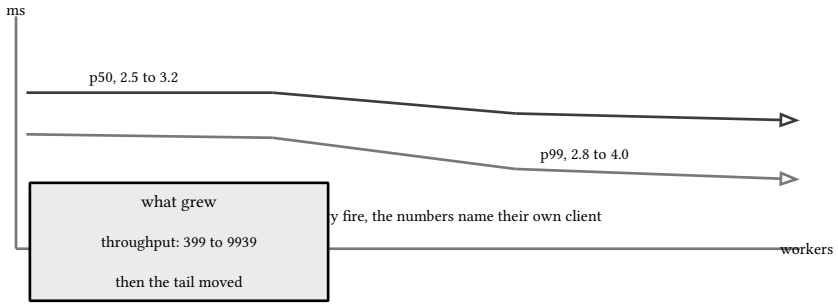


figure 52: the ladder's p50 and p99 against offered load, the tail is the story

29.7. the budget, measured on the shipped binary

The ladder raised the budget through the rule's constructor, and the wiring's default is unchanged from chapter 27, burst 30 with one token per second in the shared anonymous pool. Against the shipped binary itself, the one `main.rs` builds, booted on ephemeral ports with its wal in a scratch directory, the same closed loop meets the layer chapter 27 built and the layer chapter 28 seats outside it:

```
the shipped binary, wired defaults, burst 30 one token per second, shared anonymous pool
name                  n  errors      rps    p50_ms    p90_ms    p99_ms    max_ms  src
budget-200          200       0 3089.852     2.517     2.799     3.925     4.238  exact
status 200: 29
status 429: 171
```

listing 317: 8 workers offer 200, the budget answers 29, one probe burns the thirtieth (source: rust/api/captures/2026-10-14-load.txt, lines 16-20)

29 of 200 measured fires answered 200 and 171 answered 429, and the exposition scraped off the binary's own ops port counts exactly 30 and 171, 201 requests in all, because the harness's readiness probe was itself a `GET /healthz` the registry counted, one registry observing every request including the tool's own. The pooled p99 of 3.925 ms covers both statuses together, the limiter's design claim measured: refusing is cheap, the flood costs the handler nothing, and the budget is spent exactly where the token bucket said it would be. The `_count` of 201 and the `_sum` of 0.014603 are in the committed capture, and the scrape's content type is the one chapter 28 pinned.

NOTE: the knob, not a bypass

The ladder boot raises the budget through the same rule constructor the wiring calls, burst 100000 against the default 30, and nothing about the deployed service changes. The capture header names the stack and the client, so neither number can be mistaken for the default's behavior or a pooled client's throughput.

29.8. what the run leaves behind

Two artifacts outlive the session, and they are the point of the chapter. The capture, dated and committed, so the next run on the next machine has a baseline to diff against and no one re-derives today's numbers from memory. And the tool itself, 10 tests green three consecutive runs, which is what makes next quarter's number comparable to this one: a load report nobody can rebuild byte for byte is a slide, not a measurement. The suite drives the loops under tick clocks and fake sleepers, counts deterministic, zero wall-clock assertions, and one test walks the real client against a real in-process listener to prove the socket path, the only place the wall clock appears is where it is the subject.

sources: rfc 9112 section 9.3, persistence, and section 9.6, tear-down and the close option, at rfc-editor.org, accessed 2026-10-14, and rfc 9110 section 7.6.1 for the Connection field itself, same date. Little's law as stated across the corpus, book 4, ch 11 (go 1.27, profiling: pprof, the tracer, benchstat, p99) owns the profiling-side treatment, and the loop pair, the warmup discipline, and the report columns follow the go book's load chapter and the java book's chapter 29, read for the parallel contracts, with the integer schedule and the u64 rate being this lane's own divergence. The ladder and budget captures are books/rust/api/captures/2026-10-14-load.txt, produced by `cargo test --locked --test load -- --ignored --nocapture` under `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build, dated 2026-10-14, and the load tests ran green 10 of 10 three consecutive runs the same day.

30. ship it

The spine ends as a directory of tests unless it also ships. This chapter turns the crate into an artifact and a lane: `deploy.rs`, the ship facts held as data so tests can hold the files to them, the multi-stage image with a digest-pinned builder and a non-root runtime, the compose file that brings it up healthy on host ports 19580 and 19590, the `--health` flag the healthcheck drives, and the make gate that replays the wire contract over that stack in the next chapter. Everything leans on what the spine already built: the health route chapter 19 owns, the ops listener chapter 28 binds, the load client chapter 29 probes with. The go book shipped the same service shape and carried the platform envelopes, helm, terraform, ci, as taught copies, book 4, ch 29 (go 1.27, ship it), and the java book derived a jlink runtime from the pinned jdk, book 5, ch 30 (java 27, ship it): a rust service ships one statically honest binary and no runtime to derive, which makes this the narrowest ship chapter of the three.

30.1. the ship as data

Digests, dates, uid, ports, and build flags live once, in a module, not in a comment a file can outlive:

```
pub const BUILDER_IMAGE: &str =
    "rust:1.99@sha256:81015672d6939b0265f09adf96d5de125482bb1330106076956b0518e74edc90";
pub const BUILDER_RESOLVED: &str = "2026-10-14";
pub const RUNTIME_IMAGE: &str =
    "debian:trixie-slim@sha256:a29215f6a35e51e22adffa17f89e9d2ef06214e64a2bad10d765c46aea49f11f";
pub const RUNTIME_RESOLVED: &str = "2026-10-14";
pub const RUN_UID: u32 = 65532;
pub const RUN_GID: u32 = 65532;
pub const BIN_PATH: &str = "/rustapi";
pub const DATA_DIR: &str = "/data";
pub const HEALTH_FLAG: &str = "--health";
pub const BUILD_FLAGS: &str = "--locked --release --bin rustapi";
```

listing 318: the ship: both image refs at their manifest digests with resolved dates, the uid, the paths, the build flags (source: rust/api/src/deploy.rs, lines 1-12)

```
pub fn ship() -> Ship {
    Ship {
        builder: BUILDER_IMAGE,
        builder_resolved: BUILDER_RESOLVED,
        runtime: RUNTIME_IMAGE,
        runtime_resolved: RUNTIME_RESOLVED,
        uid: RUN_UID,
        gid: RUN_GID,
        binary: BIN_PATH,
        data_dir: DATA_DIR,
        build_flags: BUILD_FLAGS,
    }
}
```

listing 319: the whole story as one value, the single place a chapter or a test reads it from (source: rust/api/src/deploy.rs, lines 27-39)

The builder is `rust:1.99` resolved 2026-10-14 at manifest digest `sha256:81015672...edc90`, and the pin is stronger than it looks: the manifest list fixes the exact builder image whose `rustc` reports 1.99.0 (`b940084d7 2026-09-28`), the same build the lane's every gate asserts, so the image compiles with the bits the tests ran on or not at all. The runtime is `debian:trixie-slim`, resolved the same day at `sha256:a29215f6...f11f`, and trixie is not a preference: `rust:1.99` was measured to ride debian 13 trixie, `/etc/os-release` read inside the pulled image on 2026-10-14, so the binary links against trixie's glibc and a bookworm-slim runtime, one debian older, would be smaller and unable to load it. A pin is only a pin if it is checked:

```
pub fn digest_pinned(reference: &str) -> bool {
    let Some((name, digest)) = reference.split_once('@') else {
        return false;
    };
    let Some(hex) = digest.strip_prefix("sha256:") else {
        return false;
    };
    !name.is_empty()
        && hex.len() == 64
        && hex
            .bytes()
            .all(|b| b.is_ascii_hexdigit() && !b.is_ascii_uppercase())
}
```

listing 320: a reference earns its pin: name, sha256, 64 lowercase hex, five negative vectors in the test (source: rust/api/src/deploy.rs, lines 41-53)

The deploy tests hold the tree to these values: the `Dockerfile`'s two `FROM` lines parse, through `from_ref`, to exactly the two constants, the compose file maps the lane's ports and probes with the flag, and the `.dockerignore` keeps the build cache out of the context, so the story cannot drift from the ship.

30.2. the image, two stages

```
FROM rust:1.99@sha256:81015672d6939b0265f09adf96d5de125482bb1330106076956b0518e74edc90 AS build
WORKDIR /src
COPY Cargo.toml Cargo.lock ./
COPY src ./src
ENV CARGO_NET_OFFLINE=true
RUN cargo build --locked --release --bin rustapi \
    && mkdir /data \
    && chown 65532:65532 /data

FROM debian:trixie-slim@sha256:a29215f6a35e51e22adffa17f89e9d2ef06214e64a2bad10d765c46aea49f11f
COPY --from=build --chown=65532:65532 /src/target/release/rustapi /rustapi
COPY --from=build --chown=65532:65532 /data /data
USER 65532:65532
EXPOSE 19580 19590
ENTRYPOINT ["/rustapi"]
```

listing 321: the whole image: pinned builder, offline locked release build, trixie runtime, non-root, both ports (source: rust/api/Dockerfile, lines 1-15)

Four decisions in the file earn their sentences. The build is offline and locked, `CARGO_NET_OFFLINE=true` and `--locked`, the zero-dep law carried into the image: the lock holds exactly one package, nothing vendors, nothing resolves from the network, and a `--locked` build that drifts from the lock fails loudly instead of quietly updating it. The runtime copies one file, the release binary, measured 2026-10-15 at a 1.22 MB layer on this machine, against a 119 MB image whose remainder is the trixie base, so the shipped thing is one executable and a libc. The user is numeric, 65532, the same convention both siblings set, so the platform and the image agree on who runs the process without leaning on the base's user table. And `/data` is created and chowned in the builder then copied with its owner, because a named volume inherits the image content's ownership on first mount and a root-owned directory would deny the container user its wal store.

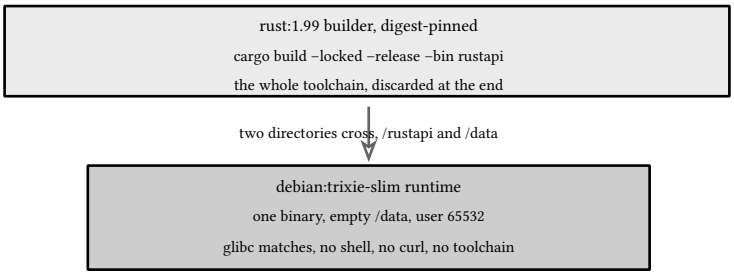


figure 53: two stages, two directories cross: the builder dies with the build, trixie runs them

A `.dockerignore` trims the context to what the image reads, and the rust tree needs it more than most: `target/` can hold gigabytes of build cache the builder would upload only to ignore, and the tests, the contract vectors, and the captures ride none of it into the image.

30.3. the binary as its own doctor

The image carries no shell, no curl, no wget, and installing a diagnostic would grow the runtime for a question the binary can ask itself. The `--health` flag answers it before any wiring runs, probing the configured url's healthz with chapter 29's client and exiting 0 on a 200, 1 on anything else:

```
    if std::env::args().any(|arg| arg == "--health") {
        let probe = load::http_get(&cfg.url, "/healthz");
        std::process::exit(if probe.ok && probe.status == 200 {
            0
        } else {
            1
        });
    }
```

listing 322: the self-check: one client call before any bind, the exit code the healthcheck reads (source: rust/api/src/main.rs, lines 41-48)

The probe earned its resolver lesson in this lane's own compose wait: the first stack came up serving and stayed unhealthy, because inside the container `localhost` resolves v6 first, the listener holds v4, and a client dialing only the first answer pronounced a healthy service dead. The fix landed in the client, chapter 29's try-every-address loop, not in the compose file, and the same probe then went healthy against the same image.

30.4. compose, one command

The compose file is one service and its volume, and the contract it enforces is the return value of a single command. `docker compose up -d --build --wait` builds the image, creates the container, starts it, and blocks until the healthcheck passes, so a green up means the image built offline, the wal engine opened its volume, and both listeners answer:

```yaml
services:
  api:
    build: .
    environment:
      RUSTAPI_ADDR: 0.0.0.0:19580
      RUSTAPI_DEBUG_ADDR: 0.0.0.0:19590
      RUSTAPI_URL: http://localhost:19580
      RUSTAPI_METRICS_URL: http://localhost:19590/metrics
      RUSTAPI_SECRET: dev-only-not-a-secret
      RUSTAPI_DATA: /data
    ports:
      - "19580:19580"
      - "19590:19590"
    volumes:
      - data:/data
    healthcheck:
      test: ["CMD", "/rustapi", "--health"]
      <<: *hc

volumes:
  data:
```

listing 323: the whole system: the env the wiring reads by name, both ports published, the volume, the probe (source: rust/api/compose.yml, lines 9-29)

The environment block carries exactly what construction demands: both listeners bound wide for the container, both urls pointed at localhost for the probe, the secret present so authn signs with a known key, and the data directory on the named volume. `down -v` removes the volume in the same breath, so every lane run starts from store zero, the wal engine applying its log from nothing. Measured 2026-10-14, this machine: the stack went healthy on the wait, the api answered `{"status":"ok","version":"0.1.0"}` on 19580, and the first scrape of 19590 counted 6 requests on `/healthz`, every one of them the probe's own fires, one registry counting its own doctor.

30.5. the docker lane as a gate

The lane is one make target, committed before this wave's suite landed, and its shape is the infrastructure book's two-gate rule, book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems) owns the compose depth and book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1) the pid 1 and signal story. The plain crate lane never needs a daemon, and the docker lane never runs silently, a missing daemon is a loud red with the instruction to start it, never a skip:

```
verify-rustapi-docker:
  @docker info >/dev/null 2>&1 || { \
    echo "docker daemon not reachable." >&2; \
    echo "start Docker Desktop (or the engine), then rerun: make verify-rustapi-docker" >&2; \
    exit 1; \
  }
  @cd books/rust/api && docker compose down -v >/dev/null 2>&1 || true && \
    trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    PATH="$$HOME/.rustup/toolchains/stable-x86_64-pc-windows-msvc/bin:$$PATH" \
    CARGO_NET_OFFLINE=true \
    RUSTAPI_URL=http://127.0.0.1:19580 \
    RUSTAPI_METRICS_URL=http://127.0.0.1:19590 \
    cargo test --locked --test docker_it -- --ignored; rc=$$?; exit $$rc
```

```
fragment, illustrative, not executed
```

The suite it runs lives in `tests/docker_it.rs`, every test carrying `#[ignore = "docker lane only: make verify-rustapi-docker"]`, so a plain `cargo test` skips the file and the target runs it with `--ignored` against the live stack, then the trap tears the stack down with volumes on any exit. Chapter 31 walks the tests themselves and states the two-lane contract in full.

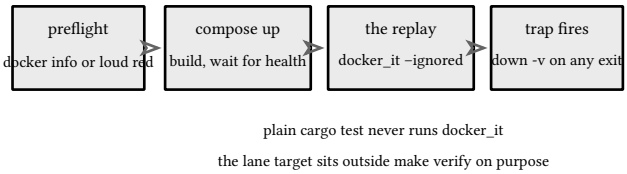


figure 54: the gate: preflight, compose, replay, teardown, no skips

30.6. the shipped checklist

The ship lane ends where it started, with the property each spine chapter contributed, every row a behavior the lane or the module gate observes:

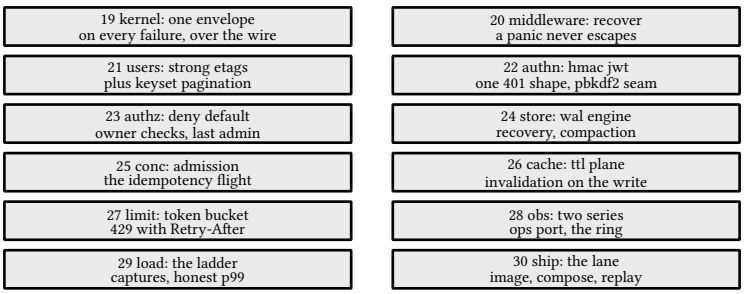


figure 55: the readiness table: twelve chapters, twelve properties, all proven

The chapter also declares what it does not teach again. Docker's storage model, the container runtime, and networking are the infrastructure book's chapters, and the cloud side stays with the go book's ship chapter, whose helm chart and terraform slice are the same service's platform envelopes taught line by line. What this chapter owns is the band between a green crate and a running service: 59 lines of ship data, a 15-line image, a 29-line compose file, and an 8-line probe.

sources: docker docs on multi-stage builds and `docker compose up --wait` semantics, at docs.docker.com, accessed 2026-10-14. The manifest digests resolved from the docker hub registry with `docker buildx imagetools inspect` on 2026-10-14 and recorded in `deploy.rs` with their dates, the builder `rust:1.99` at `sha256:81015672d6939b0265f09adf96d5de125482bb1330106076956b0518e74edc90` and the runtime `debian:trixie-slim` at `sha256:a29215f6a35e51e22adffa17f89e9d2ef06214e64a2bad10d765c46aea49f11f`. The builder's base measured by reading `/etc/os-release` inside the pulled image, debian gnu/linux 13 trixie, and its `rustc --version` matching 1.99.0 (`b940084d7 2026-09-28`), both 2026-10-14. The image and binary sizes from `docker images` and `docker history` the same day, re-measured 2026-10-15 through the lane with the numbers unchanged, 1.22 MB layer against a 119 MB image. Verified live 2026-10-14: `docker compose up -d --build --wait` healthy on this machine, both ports scraped from the host, the healthcheck's own probes counted on the ops port, and `docker compose down -v` clean. The make target quoted from the Makefile as committed at 01cca1f8.

31. the test suite

The service spine ends with the service tested, and this chapter turns the suite around and examines it as its own artifact. Every chapter from 19 on applied one technique to its own component, frozen clocks, injected seams, barrier duels, the ignored lane, and what is left is the discipline the whole suite runs on: the inventory matched to the tree, the one lane gate and its legs, the two-lane contract this wave added, the seams that keep time and chance deterministic, and the adversarial history each wave recorded, including this wave's own. The go book closed its spine over six fuzz targets and a strict recording double, book 4, ch 30 (go 1.27, the test suite), the java book over a vendored junit console, `xref-to` ("java", "suite"), and the rust lane's honest difference is the smallest vendor count of the three: `cargo test` is the runner, the toolchain brings it, and the crate contributes no dependency at all.

31.1. the inventory

The counts are taken from the tree, not from memory, and they are this commit's counts. The crate holds 162 tests in 13 integration files, and 5 more live behind the docker gate:

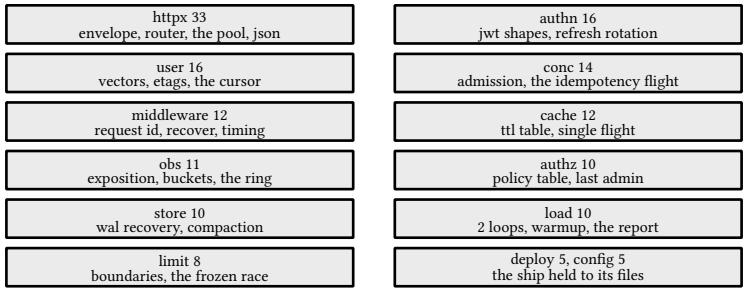


figure 56: the inventory by file, each row the tests that pin it

Beside the crate suite stands the samples contract, the manual arc's own gate: 85 sample files printing 902 dated checks across chapters 1 through 18, run by the same lane script as the multi-language suites, 3 of them present and green this wave. The two contracts answer different questions: a sample teaches a language fact and proves it on stdout, an integration test pins a service behavior, and neither substitutes for the other.

31.2. the one lane gate

Everything runs through `tools/verify-rust.ps1`, and its legs are the whole anatomy. The toolchain leg resolves the rustup-managed stable msvc directory and asserts the exact `rustc 1.99.0 (b940084d7` prefix before anything compiles, so no machine default toolchain can quietly satisfy a gate. The samples leg walks every chapter directory and fails on any file that prints no check or exits non-zero. The crate legs run fmt, clippy at `--all-targets -D warnings`, and the tests, all under `CARGO_NET_OFFLINE=true` and `--locked`, then the contract vectors hash against the go oracle file by file, and finally the zero-dep leg reads the manifest for any dependency section and the lock for exactly one package. The docker lane is the one leg that lives outside the gate, `make verify-rustapi-docker`, its absent daemon a loud red, never a skip:

```
cargo clippy --all-targets --locked -- -D warnings
cargo test --locked
# then: the vectors hash against books/go/api/contract/testdata
# then: Cargo.lock holds exactly one package
```

fragment, illustrative, not executed

The clippy invocation is the one that keeps the ignored file honest: `--all-targets` compiles `docker_it.rs` like any other target, so the docker-gated tests carry the same lint law as everything else, ignored at run time, never exempt at compile time.

31.3. the two-lane contract

The docker-gated suite is one file, `tests/docker_it.rs`, and every test in it carries the same attribute:

```
#[test]
#[ignore = "docker lane only: make verify-rustapi-docker"]
```

listing 324: the ignored-by-default law: the string names the one command that runs the test (source: rust/api/tests/docker_it.rs, lines 75-76)

A plain `cargo test` compiles the file, lints it, and runs none of it, measured: `0 passed; 5 ignored`. The lane target runs the same file with `--ignored` against the compose stack the target itself brings up, `RUSTAPI_URL` and `RUSTAPI_METRICS_URL` pointing at the published ports, and the trap tears the stack down with volumes on any exit. The five tests replay the contract over the real image: health and routing with the 405 and its `allow`, register and login with the one 401 shape that must not enumerate accounts, the notes resource with its idempotent create, the etag duel over one `If-Match` resolving to exactly one 200 and one 412, the exposition scraped off the ops port with its content type and its one exact path, and the actor budget drained to an honest 429 with `retry-after` and both budget headers, spent under a registered actor's own bucket so the drain can never starve the anonymous pool the other four tests share.

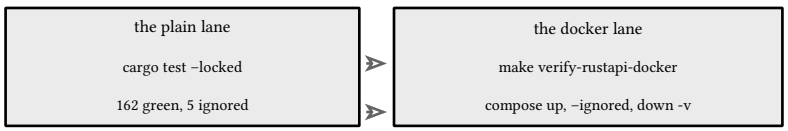


figure 57: two lanes, one file: plain cargo never runs it, the lane target runs nothing else

31.4. the deterministic seams

Every boundary the suite proves is injected, never waited for. The clocks freeze or tick on arithmetic: frozen millis drive the token bucket's boundary table, a mutex-held micros counter advances 100 per read so the observability layer measures exactly 100 micros per request, and the load loops run whole schedules under fake sleepers that record their waits into a vector. The ids are injected functions, so the vector tests pin byte-identical cursor and etag responses. The hashers are injected, so pbkdf2′s cost never runs inside a table test. The fire seam is a function, so the open loop's schedule is stepped, not slept. The wall clock appears in exactly one test, the client against a real listener, and that test asserts counts and statuses only, never a duration, because a latency asserted against a wall clock proves nothing on a loaded machine.

The one sleep in the tree sits behind `--ignored`: the capture lane's readiness poll waits up to 10 seconds for the real binary's first healthz, an availability wait with a deadline, and it is the scenario itself, a measurement tool booting a measurement subject, not a boundary proof.

31.5. the adversarial history

Each wave ran its own attackers before landing, and the record is the suite's scar tissue. The earlier waves' catches are their chapters' own: the frozen race admitting exactly the burst, the single flight measured at 6390 double fills before the fix in the java lane's telling, the wal truncate-at-stop law, the substring pins surviving clang drift. This wave's development half caught its own bugs as they surfaced, and the list is short because the tests came with the code: the nearest-rank rank computed with both an add-999 ceiling and a `div_ceil`, a double ceiling that landed one rank too far and failed the boundary table before any run went green; the golden table's column

literals, wrong by one space until the first render; the ring's push borrowing its own cursor while mutating it; the elapsed window miscounted by one clock read. The expensive one arrived from outside the suite: the compose healthcheck pronounced a healthy service dead, because inside the container `localhost` resolves v6 first and the load client dialed only the first address, a defect no in-process test could see because the test loopback resolves one family. The fix landed in the client's resolver loop, chapter 29 states it, and the lane that caught it is the same lane that now proves it green on every run.

NOTE: what the rounds are for

The rounds are recorded because an unfixed finding that survives into the book is a lie with a publication date, and a fixed one that names its test is a suite that grew. The healthcheck catch is the wave's own proof of the two-lane contract: the plain lane was green while the ship was broken, and only the docker lane could say so.

31.6. what the suite does not hold

The inventory's gaps are stated as plainly as its contents. No property or fuzz targets ride this lane, the go spine's 6 fuzz engines have no siblings here or in java, so the arithmetic surfaces the lane does hold, the json codec, the bucket refill, the rank, the ring's overwrite, are pinned by boundary vectors and adversarial rounds rather than by a mutation engine. No coverage percentage is printed anywhere in the book, the figure moves with every edit and a pinned number would be a lie with a date on it. The capture lane is one test behind `--ignored` and its outputs live in committed, dated files rather than in assertions, because a measurement's artifact is a document, not a gate. And the capstone crate's own suite is chapter 32′s subject, walked there beside the code it pins.

31.7. what the suite proves

The closer maps each layer to the fact it establishes. The httpx tests prove the envelope and the router's exactness, the user tests the vectors and the strong etags, the authn tests the one 401 shape and the refresh rotation, the authz tests the deny default and the last admin, the store tests the wal's recovery and compaction, the conc tests the admission cap and the idempotent replay, the cache tests the ttl plane and the single flight, the limit tests the honest 429, the obs tests the exposition and the exact rank, the load tests the harness itself, the deploy tests hold the image and the compose file to the ship's own constants, and the docker lane proves all of it again over a real image on real ports with a real clock. That surface is contract, not preference, and the spine's last word is the same as its first: a green lane means the behaviors hold, and every number in this chapter was counted from the tree on 2026-10-14.

sources: the lane anatomy read from tools/verify-rust.ps1, tools/run-rust-samples.ps1, and the Makefile's verify-rustapi-docker target as they stand at this commit, the target quoted in chapter 30 from its 01cca1f8 commit. The inventory counted from the tree with `cargo test --locked` per target under `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build, 2026-10-14: 162 plain-lane tests green, 5 docker-gated green under `make verify-rustapi-docker` with the compose teardown observed firing, and the samples leg reporting 85 files at 902 checks with all 3 multi-language suites present. The two-lane contract verified live the same day: a plain `cargo test --locked --test docker_it` run printing `0 passed; 5 ignored`, then the lane target running the same 5 green against the stack it built.

32. capstone: an async runtime from scratch

The service part of this book runs on threads: chapter 19′s kernel is a bounded pool with one thread per connection, and it holds that shape through chapter 31 because std gives no other road. This chapter builds the other road. The crate `runcap` under `books/rust/capstone` is an async runtime with zero dependencies: a work-stealing-free executor over one task queue, a hashed timing wheel on its own thread, a fixed `spawn_blocking` pool bridged over channels, a poll-loop reactor for nonblocking tcp, and a fan-out server composed from all four, 1765 lines of source across 7 files and 36 tests, every one green under `cargo test --locked` with `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build, dated 2026-10-13. The suite is 783 lines of tests in five files plus a 66-line shared line-client harness, and the crate carries no `unsafe` and no dependency, one package in `Cargo.lock`. Chapter 14 built the parts by hand in miniature, poll and waker and `block_on` as a loop. Everything here is those parts grown to load-bearing size, and nothing else.

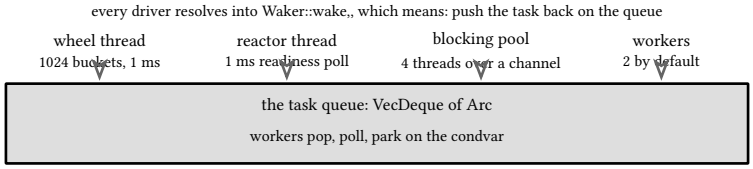


figure 58: the runtime in one picture: four driver threads around one task queue

32.1. the smallest executor

A task is a boxed future plus a pointer home. The `Wake` trait, stable since 1.51, turns an `Arc<Task>` into a `Waker` whose wake is one operation, push the task on the executor queue, and that single equivalence is the whole scheduler:

```
struct Task {
    fut: Mutex<Option<Pin<Box<dyn Future<Output = ()> + Send>>>>,
    exec: Arc<ExecutorShared>,
}

impl Wake for Task {
    fn wake(self: Arc<Self>) {
        let exec = self.exec.clone();
        exec.push(self);
    }
}

impl Task {
    fn run(self: &Arc<Self>) {
        let waker = Waker::from(self.clone());
        let mut cx = Context::from_waker(&waker);
        let mut slot = lock(&self.fut);
        let Some(fut) = slot.as_mut() else {
            return;
        };
        let outcome = panic::catch_unwind(AssertUnwindSafe(|| fut.as_mut().poll(&mut cx)));
        match outcome {
            Ok(Poll::Ready(())) | Err(_) => {
                *slot = None;
            }
            Ok(Poll::Pending) => {}
        }
    }
}
```

listing 325: the task: wake is a requeue, run is one poll under catch_unwind (source: rust/capstone/src/runtime.rs, lines 73-101)

Three disciplines live in those 29 lines. The future sits behind a `Mutex<Option<...>>` so a task parked in the queue and a task being polled are the same object, and `None` marks completion so a stale wake of a finished task is a no-op pop. The poll runs under `catch_unwind`, so a panicking task dies alone, its slot cleared, its siblings untouched, the worker keeps looping. And the waker is built from the task's own `Arc`, so cloning it into a reactor slot is one refcount, the property that makes wakers cheap enough to store everywhere.

`block_on` is chapter 14's spin loop with the spin replaced by a condvar park, the same `Arc` state and flag protocol the timer bridge used there, and workers drain the queue with the same condvar discipline the chapter 19 pool used for connections:

```
    pub fn block_on<F: Future>(&self, fut: F) -> F::Output {
        let mut fut = std::pin::pin!(fut);
        let park = Arc::new(ParkWaker {
            flag: Mutex::new(false),
            cv: Condvar::new(),
        });
        let waker = Waker::from(park.clone());
        let mut cx = Context::from_waker(&waker);
        loop {
            *lock(&park.flag) = false;
            match fut.as_mut().poll(&mut cx) {
                Poll::Ready(v) => return v,
                Poll::Pending => {
                    let mut f = lock(&park.flag);
                    while !*f {
                        f = park.cv.wait(f).unwrap_or_else(PoisonError::into_inner);
                    }
                }
            }
        }
    }
```

listing 326: block_on: clear flag, poll, park until the waker fires, repeat (source: rust/capstone/src/runtime.rs, lines 374-394)

`spawn` wraps the user future in a `JoinWrap` that forwards the output into a shared slot and wakes whoever awaits the handle, and the handle is a future itself, so `.await` on a `JoinHandle` parks the awaiting task until the child finishes. Cancellation is the same drop as chapter 14 taught, now with a witness: if the task dies before completing, the wrapper's `Drop` marks the state done with no result, and the awaiting side panics with a message instead of hanging:

```rust
struct JoinWrap<F: Future> {
    fut: Pin<Box<F>>,
    state: Arc<JoinState<F::Output>>,
}

impl<F: Future> Future for JoinWrap<F> {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        match this.fut.as_mut().poll(cx) {
            Poll::Ready(v) => {
                this.state.finish(Some(v));
                Poll::Ready(())
            }
            Poll::Pending => Poll::Pending,
        }
    }
}

impl<F: Future> Drop for JoinWrap<F> {
    fn drop(&mut self) {
        let waker = {
            let mut g = lock(&self.state.inner);
            if g.done {
                None
            } else {
                g.done = true;
                g.waker.take()
            }
        };
        if let Some(w) = waker {
            w.wake();
        }
    }
}
```

listing 327: the join bridge: output forwarded on ready, cancellation surfaced on drop (source: rust/capstone/src/runtime.rs, lines 136-171)

Measured on the pinned build, dated 2026-10-13: the 3-step self-waking future from chapter 14's executor, regrown on `runcap`, completes in exactly 4 polls, 3 pending that each self-wake plus 1 ready. Two tasks over `Runtime::new(2)` meet a `Barrier(2)` and report exactly 2 distinct thread ids. A task that panics isolates cleanly: its siblings sum to 16, its own handle panics on await, and the runtime still serves new spawns afterwards. A handle awaited after its runtime dropped panics with the cancel message rather than parking forever, which is the post-shutdown contract tokio also chose: fail loud, never hang.

32.2. timers, the wheel, and select ordering

Chapter 14's timer was one thread per sleep. The capstone wheel serves every timer from one thread: entries hash into 1024 buckets by deadline millisecond, the ticker advances a tick cursor and fires the bucket whose millisecond has arrived, and the thread sleeps on a condvar until exactly the earliest pending deadline, so an empty wheel costs nothing and a far deadline parks rather than spins:

```
    fn insert(&mut self, deadline: Instant, waker: Waker) -> (u64, u128) {
        let id = self.next_id;
        self.next_id += 1;
        let ms = self.deadline_ms(deadline);
        self.buckets[(ms & MASK) as usize].push_back(Entry {
            id,
            ms,
            waker: Some(waker),
        });
        (id, ms)
    }
    fn bucket_of(&mut self, ms: u128) -> &mut VecDeque<Entry> {
        &mut self.buckets[(ms & MASK) as usize]
    }
    fn remove(&mut self, id: u64, ms: u128) {
        self.bucket_of(ms).retain(|e| e.id ≠ id);
    }
    fn refresh_waker(&mut self, id: u64, ms: u128, waker: Waker) -> bool {
        let bucket = self.bucket_of(ms);
        match bucket.iter_mut().find(|e| e.id == id) {
            Some(entry) => {
                entry.waker = Some(waker);
                true
            }
            None => false,
        }
    }
    fn advance(&mut self, now: Instant, wakes: &mut Vec<Waker>) {
        let now_ms = now.saturating_duration_since(self.base).as_millis();
        if now_ms ≤ self.last_ms {
            return;
        }
        for tick in self.last_ms + 1..=now_ms {
            let bucket = self.bucket_of(tick);
            if bucket.is_empty() {
                continue;
            }
            bucket.retain(|e| {
                if e.ms ≤ tick {
                    if let Some(w) = &e.waker {
                        wakes.push(w.clone());
                    }
                    false
                } else {
                    true
                }
            });
        }
        self.last_ms = now_ms;
    }
    fn earliest_ms(&self) -> Option<u128> {
        self.buckets.iter().flatten().map(|e| e.ms).min()
    }
```

listing 328: the wheel: hash by deadline ms, advance fires due buckets, earliest drives the sleep (source: rust/capstone/src/timer.rs, lines 52-104)

`Sleep::poll` is the chapter 14 double-check pattern under one lock: due means ready, otherwise the entry is inserted or its waker refreshed, and a missing entry means the ticker already fired it, so a wakeup is never lost while a value sits queued. Dropping the future removes the entry, and cancellation of a timer is again just a drop:

```
impl Future for Sleep {
    type Output = Instant;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Instant> {
        let this = self.get_mut();
        if Instant::now() ≥ this.deadline {
            return Poll::Ready(this.deadline);
        }
        let mut g = lock(&this.shared.state);
        if g.shutdown {
            panic!("runcap: timer driver shut down before the deadline");
        }
        match this.slot {
            None ⇒ {
                this.slot = Some(g.insert(this.deadline, cx.waker().clone()));
            }
            Some((id, ms)) ⇒ {
                if !g.refresh_waker(id, ms, cx.waker().clone()) {
                    return Poll::Ready(this.deadline);
                }
            }
        }
        drop(g);
        this.shared.cv.notify_one();
        Poll::Pending
    }
}

impl Drop for Sleep {
    fn drop(&mut self) {
        if let Some((id, ms)) = self.slot.take() {
            lock(&self.shared.state).remove(id, ms);
        }
    }
}
```

listing 329: sleep: due check first, insert or refresh under the wheel lock, drop removes (source: rust/capstone/src/timer.rs, lines 212-246)

Measured 2026-10-13, debug build on windows: a 2 ms sleep delivered in 9.5406 ms and a 2 ms entry with a 10 s entry parked delivered in 13.3590 ms, the honest lateness of a 1 ms wheel under a 15 ms os timer granularity, and the floor is the only asserted property, delivery never precedes the deadline. Twenty-six sleeps spanning 1250 ms, more than one full 1024 ms revolution, fire in exact deadline order with zero entries left. Five entries at one deadline all fire. A sleep dropped after one manual poll leaves `pending() == 0`.

Ordering against a second future is the biased race, chapter 14's `Select2` at two arms, now the runtime's own combinator. It polls the left arm first, always, which makes ties reproducible and gives shutdown its shape: a reader racing `read` against `stop` still drains bytes that arrived before the signal, because readiness on the left wins the tie:

```rust
impl<A: Future + Unpin, B: Future + Unpin> Future for Race<A, B> {
    type Output = RaceOut<A::Output, B::Output>;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
        let this = self.get_mut();
        if let Poll::Ready(v) = Pin::new(&mut this.a).poll(cx) {
            return Poll::Ready(RaceOut::Left(v));
        }
        if let Poll::Ready(v) = Pin::new(&mut this.b).poll(cx) {
            return Poll::Ready(RaceOut::Right(v));
        }
        Poll::Pending
    }
}
```

listing 330: the biased race: left arm polled first, first ready wins, the loser is dropped (source: rust/capstone/src/runtime.rs, lines 210-223)

32.3. the blocking pool and the io bridge

Std file and socket calls block, a worker must never block, so the runtime owns a fixed pool of 4 threads fed by one `mpsc` channel, the bridge chapter 14 built for one timer generalized to any closure. The worker loop is the whole pool: try the channel, park on the condvar when empty, drain what is queued before exiting at shutdown, so a dropping pool finishes its accepted work:

```rust
fn worker_loop(shared: Arc<PoolShared>) {
    loop {
        let job = {
            let mut g = lock(&shared.state);
            loop {
                let Some(rx) = g.rx.as_mut() else {
                    break None;
                };
                match rx.try_recv() {
                    Ok(job) => break Some(job),
                    Err(TryRecvError::Empty) => {
                        if g.shutdown {
                            break None;
                        }
                        g = shared.cv.wait(g).unwrap_or_else(PoisonError::into_inner);
                    }
                    Err(TryRecvError::Disconnected) => break None,
                }
            }
        };
        match job {
            Some(job) => job(),
            None => break,
        }
    }
}
```

listing 331: the pool worker: try_recv under the lock, condvar park, drain before exit (source: rust/capstone/src/blocking.rs, lines 78-103)

`spawn_blocking` returns a future over the same slot-and-waker seam the join bridge uses, with one addition: a panicking job stores its payload and the awaiting side `resume_unwinds` it, so panics cross the thread boundary as

panics. Measured 2026-10-13: three jobs on a 3-worker pool meet a `Barrier(3)` and report exactly 3 distinct thread ids, the fixed pool proven by construction, five jobs on a 1-worker pool run in fifo order 0 through 4, and a write-then-read of a temp file rides the pool end to end. Connects take this road too, because std exposes no nonblocking connect: `AsyncTcpStream::connect` parks the connect on the pool, then registers the arrived socket with the reactor. The honest statement is the chapter 19 one restated: what std cannot do asynchronously, the runtime does on a thread you are not waiting on.

32.4. the reactor and the concurrent server

Std has no readiness API on windows, no epoll, no kqueue, so the reactor is a level-triggered poll loop: every 1 ms tick it accepts on registered listeners, peeks registered streams whose readers parked, and drains outbound queues with nonblocking writes. Readiness is `peek`, one syscall, `WouldBlock` means not yet, anything else wakes the parked reader:

```rust
fn reactor_loop(shared: Arc<ReactorShared>) {
    let mut g = lock(&shared.inner);
    loop {
        if g.shutdown {
            let listeners = std::mem::take(&mut g.listeners);
            let streams = std::mem::take(&mut g.streams);
            drop(g);
            for (_, entry) in listeners {
                wake_entry_wakers(Some(entry), None);
            }
            for (_, entry) in streams {
                wake_entry_wakers(None, Some(entry));
            }
            break;
        }
        let mut wakes = Vec::new();
        for entry in g.listeners.values_mut() {
            if drain_listener(entry)
                && let Some(w) = entry.waker.take()
            {
                wakes.push(w);
            }
        }
        for entry in g.streams.values_mut() {
            if entry.read_waker.is_some()
                && probe_read(entry)
                && let Some(w) = entry.read_waker.take()
            {
                wakes.push(w);
            }
            if !entry.outbound.is_empty() {
                pump_write(entry, &mut wakes);
            }
        }
        drop(g);
        for w in wakes {
            w.wake();
        }
        g = lock(&shared.inner);
        if g.shutdown {
            continue;
        }
        g = shared
            .cv
            .wait_timeout(g, TICK)
            .unwrap_or_else(PoisonError::into_inner)
            .0;
    }
}
```

listing 332: the reactor tick: accept, probe with peek, pump writes, wake outside the lock (source: rust/capstone/src/net.rs, lines 126-174)

Two laws in that loop cost real debugging, both dated 2026-10-13. Map values are never dropped under the reactor lock: an entry's stored waker can hold the last reference to a task whose future owns a socket handle, and that handle's `Drop` re-enters the reactor lock, a self-deadlock that hung the first server test until the maps were cleared by taking them out and dropping after the unlock. And every removal wakes its parked futures, so a read parked on a closed entry observes `NotConnected` instead of sleeping forever.

Reads and writes are futures over the registry, not over sockets. A `Read` parks by storing the task waker in the entry, a `WriteAll` commits its bytes to the connection's own outbound queue and completes when the reactor has drained it, which is exactly the per-connection outbound queue the fan-out server needs, broadcasts enqueue without awaiting and shutdown flushes what is queued:

```
pub struct Read<'a> {
    reactor: &'a ReactorHandle,
    id: u64,
    buf: &'a mut [u8],
}

impl Future for Read<'_> {
    type Output = io::Result<usize>;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
        let this = self.get_mut();
        let mut g = lock(&this.reactor.shared.inner);
        let Some(entry) = g.streams.get_mut(&this.id) else {
            return Poll::Ready(Err(not_connected()));
        };
        if let Some(e) = &entry.err {
            return Poll::Ready(Err(clone_err(e)));
        }
        match entry.stream.read(this.buf) {
            Ok(n) => Poll::Ready(Ok(n)),
            Err(ref e) if e.kind() == ErrorKind::WouldBlock => {
                entry.read_waker = Some(cx.waker().clone());
                Poll::Pending
            }
            Err(e) => Poll::Ready(Err(e)),
        }
    }
}

#[derive(Clone)]
pub struct ConnWriter {
    reactor: ReactorHandle,
    id: u64,
}

impl ConnWriter {
```

listing 333: read: one nonblocking syscall under the lock, park by storing the waker (source: rust/capstone/src/net.rs, lines 425-460)

The write side is the same seam turned around: a `ConnWriter` is a cheap clone of the reactor handle plus the connection id, so the server can hold one per connection without moving the socket, and its whole surface is five methods over the registry. `enqueue` appends bytes to the entry's queue and pokes the reactor, `write_all` is a future that finishes when the reactor has drained what it enqueued, and `close` unregisters the entry and wakes whatever was parked on it, the same wake-after-unlock law the loop obeys:

```
    pub fn enqueue(&self, data: &[u8]) {
        let mut g = lock(&self.reactor.shared.inner);
        if let Some(entry) = g.streams.get_mut(&self.id) {
            entry.outbound.extend(data);
        }
        drop(g);
        self.reactor.shared.cv.notify_one();
    }

    pub fn queued_bytes(&self) -> usize {
        let g = lock(&self.reactor.shared.inner);
        g.streams.get(&self.id).map_or(0, |e| e.outbound.len())
    }

    pub fn write_all<'a>(&'a self, data: &'a [u8]) -> WriteAll<'a> {
        WriteAll {
            writer: self,
            data: Some(data),
        }
    }

    pub fn shutdown_write(&self) {
        let g = lock(&self.reactor.shared.inner);
        if let Some(entry) = g.streams.get(&self.id) {
            let _ = entry.stream.shutdown(Shutdown::Write);
        }
    }

    pub fn close(&self) {
        let removed = {
            let mut g = lock(&self.reactor.shared.inner);
            g.streams.remove(&self.id)
        };
        wake_entry_wakers(None, removed);
        self.reactor.shared.cv.notify_one();
    }
```

listing 334: the writer handle: enqueue and poke, await the drain, close wakes the parked (source: rust/capstone/src/net.rs, lines 469-504)

The server composes every module. Each connection runs a reader task that races `read` against the stop signal, splits complete lines, and broadcasts each line to every registered connection. The socket itself lives in the connection map, not in the task, so a stopped reader leaves its outbound queue alive for the drain:

```
    handle.spawn(async move {
        let mut acc = Vec::new();
        let mut buf = [0u8; 4096];
        let mut eof_or_err = false;
        loop {
            let quit = StopFuture {
                signal: stop.clone(),
            };
            match Race::new(reader.read(&mut buf), quit).await {
                RaceOut::Left(Ok(0)) | RaceOut::Left(Err(_)) => {
                    eof_or_err = true;
                    break;
                }
                RaceOut::Right(()) => break,
                RaceOut::Left(Ok(n)) => {
                    stats.bytes_in.fetch_add(n as u64, Ordering::Relaxed);
                    acc.extend_from_slice(&buf[..n]);
                    while let Some(p) = acc.iter().position(|&b| b == b'\n') {
                        let line: Vec<u8> = acc.drain(..=p).collect();
                        stats.messages.fetch_add(1, Ordering::Relaxed);
                        broadcast(&conns, &stats, &line);
                    }
                }
            }
        }
        if eof_or_err {
            let removed = lock(&conns).remove(&id);
            drop(removed);
        }
        done.store(true, Ordering::Release);
    });
```

listing 335: the reader task: biased race against stop, lines broadcast to the map (source: rust/capstone/src/server.rs, lines 144-174)

Shutdown is the contract the api lane established in chapter 19, restated for futures: stop intake first, then wait until every reader has finished and every queue has drained, bounded by a grace deadline, then close. The stop signal itself is a waker list, not a slot, one entry per parked task with `will_wake` dedupe, because the first version held a single `Option<Waker>` and only the last reader to park ever woke, measured as one reader of three breaking while the drain waited on the other two:

```
    pub async fn shutdown(self, handle: &Handle, grace: Duration) -> ShutdownReport {
        self.stop.set();
        let live: Vec<(ConnWriter, Arc<AtomicBool>)> = lock(&self.conns)
            .values()
            .map(|rec| (rec.stream.writer(), rec.done.clone()))
            .collect();
        let deadline = Instant::now() + grace;
        let drained;
        loop {
            let accept_done = self.accept_done.load(Ordering::Acquire);
            let readers_done = live.iter().all(|(_, d)| d.load(Ordering::Acquire));
            let queues_empty = live.iter().all(|(w, _)| w.queued_bytes() == 0);
            if (accept_done && readers_done && queues_empty) || Instant::now() ≥ deadline {
                drained = accept_done && readers_done && queues_empty;
                break;
            }
            handle.sleep(TICK).await;
        }
        for (writer, _) in &live {
            writer.close();
        }
        let snap = self.stats.snapshot();
        ShutdownReport {
            connections: snap.connections,
            messages: snap.messages,
            bytes_in: snap.bytes_in,
            bytes_out: snap.bytes_out,
            drained,
        }
    }
```

listing 336: graceful shutdown: stop, drain readers and queues under the grace, then close (source: rust/capstone/src/server.rs, lines 252-281)

The demo binary drives the whole stack over its own loopback: 4 client tasks on the runtime connect, publish 3 lines each, and read all 12 broadcasts, then the server drains. Measured 2026-10-14, 10 runs of 10 green at 0.020 to 0.033 s each: 4 connections, 12 messages, 192 bytes in, 768 bytes out, drained true, 48 lines delivered. The first version of that demo hung one run in two, and the fault was not in the runtime but in the protocol around it, which is the last lesson the capstone teaches. The barrier that fixed it is four lines of the demo itself:

```
        let join_deadline = Instant::now() + Duration::from_secs(5);
        while server.stats().connections < CLIENTS {
            assert!(
                Instant::now() < join_deadline,
                "demo: not all clients accepted in time"
            );
            handle.sleep(Duration::from_millis(1)).await;
        }
        all_joined.store(true, Ordering::Release);
```

listing 337: the join barrier: no client writes until the server has accepted all four (source: rust/capstone/src/main.rs, lines 45-53)

PITFALL: broadcast reaches accepted connections only, measured

The demo's first version hung at 10 of 12 lines, and the byte ledger showed why: two broadcast waves fired in the window after the fourth client's connect returned and before the accept task registered that connection in the map. The client could write the instant its connect completed, the server could only broadcast to

> connections it had accepted, and the lines in between reached the other three and not the fourth. The fix is the join barrier every real chat protocol carries: the demo waits for `stats().connections == 4` before any client writes, exactly the `wait_for_connections` helper the test suite already used. A tcp accept is not a join, it is a race you must barrier yourself.

32.5. what tokio adds

Tokio is this chapter's runtime scaled, and the corpus rule applies: the library is explained by the std primitives it composes. Chapter 14 built the primitives sidebar, this section only maps.

```
let rt = tokio::runtime::Runtime::new().unwrap();
rt.block_on(async {
    let listener = tokio::net::TcpListener::bind(addr).await?;
    loop {
        let (sock, _) = listener.accept().await?;
        tokio::spawn(handle_conn(sock));
    }
});
```

fragment, illustrative, not executed

The scheduler is the task queue plus workers, widened with stealing: tokio's runtime docs, read 2026-10-14, state it executes futures on a thread pool using a work-stealing strategy, that stealing moves half of the tasks in one local queue to another, and that a worker thread starts per cpu core by default. `runcap` has one global queue and no stealing, which is the simpler correct shape and the reason its `Runtime::new(2)` is a parameter instead of a cpu count.

The timer is the wheel with levels. Tokio's time driver source, read 2026-10-14, describes six wheels of 64 slots each, level 0 slots of one millisecond, each level covering the whole wheel below it, entries cascading down as time advances, one millisecond resolution with sub-millisecond durations rounded up, the design of Varghese and Lauck's hashed and hierarchical timing wheels. `runcap` is one level of 1024 slots at the same 1 ms resolution, which fires correctly at any distance, measured past one full revolution, and simply scans more buckets when deadlines spread.

The reactor is the poll loop replaced by the os: tokio's net driver subscribes sockets to epoll on linux, kqueue on macos, iocp on windows, and wakes exactly the task whose readiness arrived. `runcap` polls every tick because std exposes none of those, trading 1 ms of latency and a quiet tick of cpu for zero unsafe and zero ffi, the same trade chapter 19′s accept loop made at 2 ms.

`spawn_blocking` is the pool, `JoinHandle` is the join bridge with `abort` added on top of the same drop, `select!` is the biased race with a fairness twist chapter 14 verified: random branch order by default, `biased;` polls top to bottom, the exact order `Race` bakes in. Graceful shutdown in tokio compositions, task cancellation plus a drain, is the server's stop signal plus queue drain, and the demo's join barrier is the property no runtime gives you for free: admission and notification must be ordered by the protocol, not by the scheduler.

32.6. what the capstone proves

Chapter 1 pinned the toolchain that builds all of this and asserts its own version. Chapters 4 through 7 gave the value model the executor stands on: `Poll` is an enum, `Option` in a `Mutex` is the task slot, the `Arc` clones that make wakers cheap are ownership and moves doing scheduler work. Chapter 8′s traits are the contract, `Future` and `Wake` both, one vtable each. Chapter 10′s closures are the pool's jobs. Chapter 11′s error model is why `WouldBlock` is a kind, not a failure, the single decision that makes readiness polling composable. Chapter 12′s `Pin` is why `pin!` and

`Box::pin` appear exactly three times in the crate and nowhere else. Chapter 13′s mutexes, condvars, atomics, and barriers are every synchronization primitive the runtime uses, and no other kind appears.

Chapter 14 is the direct ancestor: poll by hand, the waker, block_on the loop, join and select, the timer bridge. The capstone is that chapter with the miniatures promoted to modules, and its tests are the chapter's observations promoted to assertions, the 4-poll countdown and the biased tie among them. Chapter 18′s discipline governs the suite: fixed thread counts, joins before asserts, no timing assertions beyond the never-early floor, and 36 tests that attack the runtime from outside through its real interfaces.

The service part contributed its laws. Chapter 19′s kernel taught the socket front door, nonblocking mode set on everything registered, eof as a quiet close, and the windows inheritance trap the reactor's register path still guards against. Its bounded pool and 50 ms worker waits are the condvar discipline the executor and the blocking pool both reuse, and its drain, flip readiness, refuse work, wait for busy within a grace, is the shutdown contract the fan-out server restates over queues. Chapters 20 through 31 built the service above a thread pool and never needed async to be correct, which is the capstone's closing claim: async rust is not a better way to serve, it is a cheaper way to park, one heap enum instead of a megabyte of stack per waiting connection, and now the whole mechanism is code you have read at 1.99.0, line by line, with the threads showing.

sources: the rust standard library surface read from the offline 1.99.0 html share under /.rustup/toolchains/stable-x86_64-pc-windows-msvc/share/doc/rust/html, accessed 2026-10-13 and 2026-10-14: std::task for the Wake trait and Waker (doc.rust-lang.org/stable/std/task/trait.Wake.html), std::sync for Condvar including wait_timeout and for the mpsc module (doc.rust-lang.org/stable/std/sync/index.html), std::thread (doc.rust-lang.org/stable/std/thread/index.html), and std::net for TcpStream including peek, set_nonblocking, and shutdown, and TcpListener accept (doc.rust-lang.org/stable/std/net/struct.TcpStream.html). The tokio mapping verified live 2026-10-14: the runtime module page for the thread pool, the half-queue steal, and the per-core worker default (docs.rs/tokio/latest/tokio/runtime/index.html), the select macro page for random-first order and biased top to bottom (docs.rs/tokio/latest/tokio/macro.select.html), and the time driver source for the six-wheel, 64-slot, 1 ms leveled wheel after Varghese and Lauck's hashed and hierarchical timing wheels (github.com/tokio-rs/tokio/blob/master/tokio/src/runtime/time/mod.rs). All measurements produced by `cargo test --locked` and `cargo run --locked --bin runcap-demo` under `CARGO_NET_OFFLINE=true` on the pinned rustc 1.99.0 (b940084d7), dated 2026-10-13 and 2026-10-14: 36 tests, 8 runtime, 8 timer, 7 blocking, 7 net, 6 server, the 4-poll countdown, 2 ms sleep at 9.5406 ms and 13.3590 ms with a far entry parked, 26 sleeps ordered across one 1024 ms revolution, the 3-thread barrier pool, the 1 MiB enqueue at 1048576 bytes queued, the demo's 10 of 10 runs at 0.020 to 0.033 s with 12 messages, 192 bytes in, 768 bytes out, drained true, and the three debugging findings, the drop-cascade deadlock, the single-waker stop slot, and the 10-of-12 acceptance gap, each observed live in the test and demo logs before its fix.

33. appendices: coverage and sources

The coverage matrices are the book's audit trail. Every strict, reserved, and weak token enumerated from the rust reference keyword chapter, every stdlib component family the book touches mapped to the chapter that teaches it, and the official sources pinned with their access dates. The build enforces honesty from the other side: every chapter listing reads a real file the lanes compile and test, 162 api-lane tests plus 5 behind the docker gate and 36 in the capstone, 85 sample files at 902 checks, zero skipped anywhere, the inventory chapter 31 recorded on 2026-10-14, every count from `cargo test --locked` under `CARGO_NET_OFFLINE=true` on the pinned 1.99.0 build.

33.1. keyword coverage

Chapter 3 owns the census pointer and this table is its accounting: 39 strict keywords, 14 reserved for future use with `gen` joining through the 2024 edition, and the 5 weak words that parse only in specific positions. All 39 strict words map to a teaching site, and the borderline ones map to their one honest home: `if` and `else` ride chapter 7′s pattern forms, guards and `if let` and let-else, plain `while` is the condvar predicate loop of chapter 13, the only place the book states its discipline, `loop` is chapter 14′s `block_on` driver, `break` is chapter 10′s iterator measurement, and `continue` appears in exactly one teaching position, chapter 7′s let-else divergence rule beside `return`, `break`, and panic. Of the reserved set only `gen` and `try` map, both to chapter 2, the 2024 reservation and the word still waiting since 2018. The other 12 reserved words and the weak `safe`, whose `unsafe(...)` mirror chapter 2 teaches while the `safe(...)` spelling goes unmentioned, stay unmapped, 13 rows in all, named nowhere outside this matrix.

word	kind	since	taught in
_	strict		ch 7
as	strict		ch 4
async	strict	1.39	ch 14
await	strict	1.39	ch 14
break	strict		ch 10
const	strict		ch 4
continue	strict		ch 7
crate	strict		ch 16
dyn	strict	1.27	ch 8
else	strict		ch 7
enum	strict		ch 7
extern	strict		ch 17
false	strict		ch 4
fn	strict		ch 5
for	strict		ch 10
if	strict		ch 7
impl	strict		ch 7
in	strict		ch 10
let	strict		ch 4
loop	strict		ch 14
match	strict		ch 7
mod	strict		ch 16
move	strict		ch 5
mut	strict		ch 6
pub	strict		ch 16
ref	strict		ch 7

word	kind	since	taught in
return	strict		ch 11
self	strict		ch 7
Self	strict		ch 8
static	strict		ch 4
struct	strict		ch 7
super	strict		ch 16
trait	strict		ch 8
true	strict		ch 4
type	strict		ch 4
unsafe	strict		ch 17
use	strict		ch 16
where	strict		ch 9
while	strict		ch 13
abstract	reserved		unmapped
become	reserved		unmapped
box	reserved		unmapped
do	reserved		unmapped
final	reserved		unmapped
gen	reserved	1.85, 2024 ed.	ch 2
macro	reserved		unmapped
override	reserved		unmapped
priv	reserved		unmapped
try	reserved	2018 ed.	ch 2
typeof	reserved		unmapped
unsized	reserved		unmapped
virtual	reserved		unmapped
yield	reserved		unmapped
union	weak		ch 17
macro_rules	weak		ch 9
raw	weak	1.30	ch 3
safe	weak		unmapped
'static	weak		ch 6

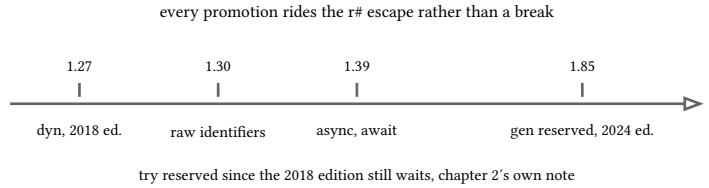


figure 59: the edition promotions, words that arrived after 1.0 and the escape that softened them

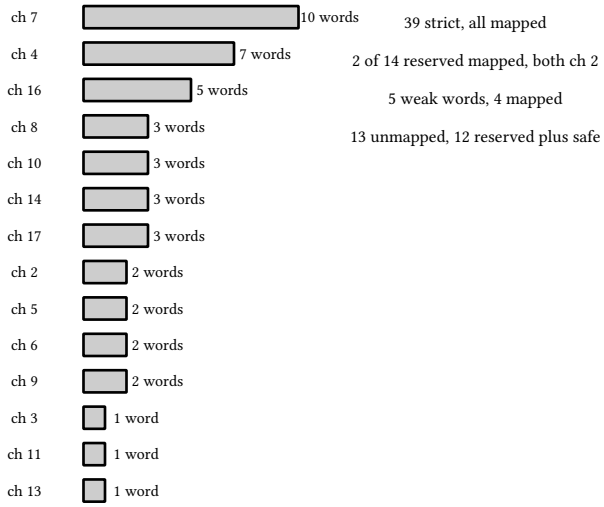


figure 60: the census by teaching chapter, counted from the table above

33.2. stdlib coverage

Component families against chapters, with the honest status column. The std pages were read from the offline 1.99.0 rust-docs share, the same oracle every chapter read, and each row below was grepped against its chapter on 2026-10-14: taught with checks means the chapter's own samples or its lane's tests exercise the family, stated means the chapter names it without a demonstration.

component family	chapter	status
collections, Vec VecDeque HashMap BTreeMap	ch 15	taught with checks, hashbrown swap stated at 1.36
str and String, the utf-8 boundary	ch 15	taught with checks, the kernel's parser rides it
io, Read Write BufReader read_exact copy	ch 15	taught with checks, in memory and deterministic
path, Path PathBuf components	ch 15	taught with checks, windows component walk measured
time, Duration Instant SystemTime	ch 15	taught with checks, ns per iteration measured
net, TcpListener TcpStream	ch 15	taught with checks, the kernel and the capstone reactor reuse it
env, args vars set_var	ch 15	taught with checks, set_var unsafe in edition 2024
process, Command ExitCode Termination	ch 15	taught with checks, self spawn exit codes measured
thread and join, spawn Builder scope	ch 13	taught with checks, scoped threads 1.63
sync, Mutex RwLock Condvar mpsc	ch 13	taught with checks, the capstone executor rides it
atomics, the orderings	ch 13	taught with checks
cell and refcell	ch 12	taught with checks
pin	ch 12	taught in one section, chapter 14 and the capstone ride it
future and task, Poll Wake	ch 14	taught with checks, xref to the capstone
box rc arc, Weak and cycle breaking	ch 12	taught with checks
alloc	ch 12	stated, the allocator api named as an escape hatch, not demonstrated

component family	chapter	status
fs, File OpenOptions read create_dir_all rename	ch 24	taught with checks, the wal and compaction lane
ffi, CStr c_char extern C	ch 17	taught with checks, strlen and qsort on msvc

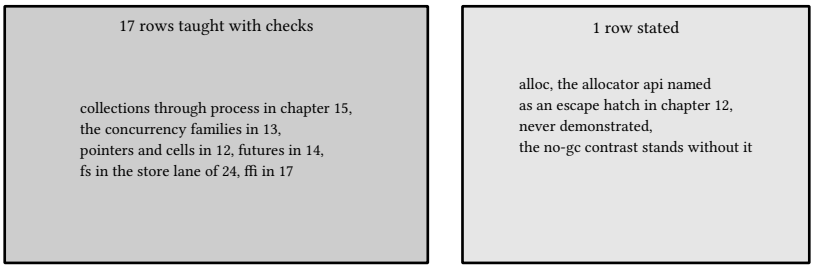


figure 61: stdlib coverage, 17 families taught with checks against the one honest stated row

33.3. pinned sources

Every chapter ends with its own sources line naming what it consulted and how it verified live. The book level pins:

topic	url	accessed
the rust book, the rust programming language, 2nd edition	https://doc.rust-lang.org/stable/book/	2026-10-13
the rust reference	https://doc.rust-lang.org/stable/reference/	2026-10-13
the rust reference, keywords	https://doc.rust-lang.org/stable/reference/keywords.html	2026-10-13
the rust reference, tokens and literal forms	https://doc.rust-lang.org/stable/reference/tokens.html	2026-10-13
the rust reference, identifiers and raw identifiers	https://doc.rust-lang.org/stable/reference/identifiers.html	2026-10-13
the rust reference, comments	https://doc.rust-lang.org/stable/reference/comments.html	2026-10-13
the rust reference, char, a unicode scalar value	https://doc.rust-lang.org/stable/reference/types/char.html	2026-10-13
the rust reference, numeric types and their ranges	https://doc.rust-lang.org/stable/reference/types/numeric.html	2026-10-13
the rust reference, operators, casts, and overflow	https://doc.rust-lang.org/stable/reference/expressions/operator-expr.html	2026-10-13
the rust reference, type coercions	https://doc.rust-lang.org/stable/reference/type-coercions.html	2026-10-13
the rust reference, constant items	https://doc.rust-lang.org/stable/reference/items/constant-items.html	2026-10-13
the rust reference, static items and static mut	https://doc.rust-lang.org/stable/reference/items/static-items.html	2026-10-13
the rust reference, destructors and drop order	https://doc.rust-lang.org/stable/reference/destructors.html	2026-10-13
the rust reference, special types and traits, the Copy boundary	https://doc.rust-lang.org/stable/reference/special-types-and-traits.html	2026-10-13
the rust reference, lifetime elision	https://doc.rust-lang.org/stable/reference/lifetime-elision.html	2026-10-13

topic	url	accessed
the rust reference, subtyping and variance	https://doc.rust-lang.org/stable/reference/subtyping.html	2026-10-13
the rust reference, patterns	https://doc.rust-lang.org/stable/reference/patterns.html	2026-10-13
the rust reference, statements and let chains	https://doc.rust-lang.org/stable/reference/statements.html	2026-10-13
the rust reference, implementations and the orphan rule	https://doc.rust-lang.org/stable/reference/items/implementations.html	2026-10-13
the rust reference, generics	https://doc.rust-lang.org/stable/reference/items/generics.html	2026-10-13
the rust reference, associated items	https://doc.rust-lang.org/stable/reference/items/associated-items.html	2026-10-13
the rust reference, trait objects	https://doc.rust-lang.org/stable/reference/types/trait-object.html	2026-10-13
the rust reference, closure types	https://doc.rust-lang.org/stable/reference/types/closure.html	2026-10-13
the rust reference, type layout, repr C against repr Rust	https://doc.rust-lang.org/stable/reference/type-layout.html	2026-10-13
the rust reference, macros by example	https://doc.rust-lang.org/stable/reference/macros-by-example.html	2026-10-13
the rust reference, modules and visibility	https://doc.rust-lang.org/stable/reference/items/modules.html	2026-10-13
the rust reference, unsafe behavior	https://doc.rust-lang.org/stable/reference/unsafe-behavior.html	2026-10-13
the rust reference, testing attributes	https://doc.rust-lang.org/stable/reference/attributes/testing.html	2026-10-13
rust by example	https://doc.rust-lang.org/stable/rust-by-example/	2026-10-13
the rustonomicon, unsafe rust	https://doc.rust-lang.org/stable/nomicon/	2026-10-13
the edition guide, 2015 through 2024	https://doc.rust-lang.org/stable/edition-guide/	2026-10-13
edition 2018, module system path changes	https://doc.rust-lang.org/stable/edition-guide/rust-2018/path-changes.html	2026-10-13
edition 2018, new keywords, dyn and reserved	https://doc.rust-lang.org/stable/edition-guide/rust-2018/new-keywords.html	2026-10-13
edition 2021, disjoint capture in closures	https://doc.rust-lang.org/stable/edition-guide/rust-2021/disjoint-capture-in-closures.html	2026-10-13
edition 2021, IntoIterator for arrays	https://doc.rust-lang.org/stable/edition-guide/rust-2021/IntoIterator-for-arrays.html	2026-10-13
edition 2024, gen keyword	https://doc.rust-lang.org/stable/edition-guide/rust-2024/gen-keyword.html	2026-10-13
edition 2024, rpit lifetime capture	https://doc.rust-lang.org/stable/edition-guide/rust-2024/rpit-lifetime-capture.html	2026-10-13
edition 2024, unsafe attributes	https://doc.rust-lang.org/stable/edition-guide/rust-2024/unsafe-attributes.html	2026-10-13
edition 2024, static mut references	https://doc.rust-lang.org/stable/edition-guide/rust-2024/static-mut-references.html	2026-10-13
edition 2024, match ergonomics	https://doc.rust-lang.org/stable/edition-guide/rust-2024/match-ergonomics.html	2026-10-13

topic	url	accessed
edition 2024, newly unsafe functions	https://doc.rust-lang.org/stable/edition-guide/rust-2024/newly-unsafe-functions.html	2026-10-13
the rust book, ch04, what is ownership	https://doc.rust-lang.org/stable/book/ch04-01-what-is-ownership.html	2026-10-13
the rust book, ch04, references and borrowing	https://doc.rust-lang.org/stable/book/ch04-02-references-and-borrowing.html	2026-10-13
the rust book, ch05, structs	https://doc.rust-lang.org/stable/book/ch05-00-structs.html	2026-10-13
the rust book, ch06, enums and match	https://doc.rust-lang.org/stable/book/ch06-00-enums.html	2026-10-13
the rust book, ch07, packages, crates, and modules	https://doc.rust-lang.org/stable/book/ch07-00-managing-growing-projects-with-packages-crates-and-modules.html	2026-10-13
the rust book, ch09, error handling	https://doc.rust-lang.org/stable/book/ch09-00-error-handling.html	2026-10-13
the rust book, ch10, generics, traits, and lifetimes	https://doc.rust-lang.org/stable/book/ch10-00-generics.html	2026-10-13
the rust book, ch11, testing	https://doc.rust-lang.org/stable/book/ch11-00-testing.html	2026-10-13
the rust book, ch13, closures and iterators	https://doc.rust-lang.org/stable/book/ch13-00-functional-features.html	2026-10-13
the rust book, ch15, smart pointers	https://doc.rust-lang.org/stable/book/ch15-00-smart-pointers.html	2026-10-13
the rust book, ch16, concurrency	https://doc.rust-lang.org/stable/book/ch16-00-concurrency.html	2026-10-13
the rust book, ch17, async await	https://doc.rust-lang.org/stable/book/ch17-00-async-await.html	2026-10-13
the rust book, ch19, unsafe rust	https://doc.rust-lang.org/stable/book/ch19-01-unsafe-rust.html	2026-10-13
the rust book, appendix c, derivable traits	https://doc.rust-lang.org/stable/book/appendix-03-derivable-traits.html	2026-10-13
rustc error codes, the borrow checker vocabulary, E0382 E0502 E0499 E0597 E0106 E0204 E0184	https://doc.rust-lang.org/stable/error_codes/E0382.html	2026-10-13
the rustc book, lint listing	https://doc.rust-lang.org/stable/rustc/lints/listing/index.html	2026-10-13
the rustc book, platform support, windows tiers	https://doc.rust-lang.org/stable/rustc/platform-support.html	2026-10-13
the cargo book, reference, profiles	https://doc.rust-lang.org/stable/cargo/reference/profiles.html	2026-10-13
the cargo book	https://doc.rust-lang.org/stable/cargo/	2026-10-13
the rustc book, editions and flags	https://doc.rust-lang.org/stable/rustc/	2026-10-13
std, the standard library api, 1.99.0	https://doc.rust-lang.org/stable/std/	2026-10-13
clippy, the lint collection	https://doc.rust-lang.org/stable/clippy/	2026-10-13
rust release notes, 1.0 through 1.99	https://github.com/rust-lang/rust/blob/master/RELEASES.md	2026-10-13

topic	url	accessed
tokio, the select macro and the biased option, quoted for the x-libs sidebar	https://docs.rs/tokio/latest/tokio/macro.select.html	2026-10-13
tokio, the runtime, work stealing and the bundled drivers, quoted for the x-libs sidebar	https://docs.rs/tokio/latest/tokio/runtime/index.html	2026-10-13
RFC 9112, http/1.1, message syntax, chunked transfer and trailers	https://www.rfc-editor.org/rfc/rfc9112	2026-10-13
RFC 9110, http semantics, if-match and etags	https://www.rfc-editor.org/rfc/rfc9110	2026-10-13
RFC 8259, the json grammar	https://www.rfc-editor.org/rfc/rfc8259	2026-10-13
RFC 8018, pbkdf2	https://www.rfc-editor.org/rfc/rfc8018	2026-10-13
RFC 2104, hmac	https://www.rfc-editor.org/rfc/rfc2104	2026-10-13
RFC 4231, hmac test vectors	https://www.rfc-editor.org/rfc/rfc4231	2026-10-13
FIPS 180-4, the secure hash standard, sha-256	https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf	2026-10-13
RFC 7519, json web token	https://www.rfc-editor.org/rfc/rfc7519	2026-10-13
RFC 3720, crc32c and its check values	https://www.rfc-editor.org/rfc/rfc3720	2026-10-13
OWASP, idor prevention cheat sheet	https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html	2026-10-13
OWASP, access control cheat sheet	https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html	2026-10-13
std, task, the wake trait, the capstone executor's waker unit	https://doc.rust-lang.org/stable/std/task/trait.Wake.html	2026-10-14
std, sync primitives, condvar and mutex in the executor and timer	https://doc.rust-lang.org/stable/std/sync/index.html	2026-10-14
std, threads, scoped driver threads in the capstone runtime	https://doc.rust-lang.org/stable/std/thread/index.html	2026-10-14
std, net, tcpstream nonblocking reads in the reactor	https://doc.rust-lang.org/stable/std/net/struct.TcpStream.html	2026-10-14
tokio, the runtime time wheel, the varghese and lauck attribution	https://github.com/tokio-rs/tokio/blob/master/tokio/src/runtime/time/mod.rs	2026-10-14
RFC 6585, additional http status codes, 429 and retry-after	https://www.rfc-editor.org/rfc/rfc6585	2026-10-14
RFC 9111, http caching, no-store and private	https://www.rfc-editor.org/rfc/rfc9111	2026-10-14
prometheus, the exposition format, content type, buckets, optional help	https://prometheus.io/docs/instrumenting/exposition_formats	2026-10-14
docker docs, multi-stage builds	https://docs.docker.com/build/building/multi-stage/	2026-10-14
docker docs, compose up and the --wait flag	https://docs.docker.com/reference/cli/docker/compose/up/	2026-10-14

NOTE: the offline oracle

The pin under every row is the 1.99.0 rust-docs component under `~/.rustup/toolchains/stable-x86_64-pc-windows-msvc`, its `share/doc/rust/html` tree reached through `rustup doc --book`, the same pages the api and capstone lanes compile against offline. Each url row was verified against that offline copy the same day it was accessed, so a moved upstream page is a recorded delta here, never a silent rewrite of a chapter.

NOTE: the pinned build

Every lane in this book compiles and runs on rustc 1.99.0 (b940084d7 2026-09-28), resolved through the rustup lane of chapter 1 and asserted by `tools/verify-rust.ps1` before anything runs, because the machine's own default toolchain can drift a release from the book. The receipts are `cargo test --locked` under `CARGO_NET_OFFLINE=true`, the same invocation every chapter quotes. Under the corpus version currency law the pin rides the newest installable ga build.

NOTE: the std-only stance

Std is the only dependency the book allows itself, chapter 15′s standing rule: the api lane's manifest declares no dependencies at all and the capstone crate is one package in its `Cargo.lock`, with no `unsafe` anywhere in its 1765 lines. The one external surface is deployment, and chapter 30 pins it by digest, the builder `rust:1.99` and the runtime `debian:trixie-slim`, both recorded in `deploy.rs` with the dates they were resolved.

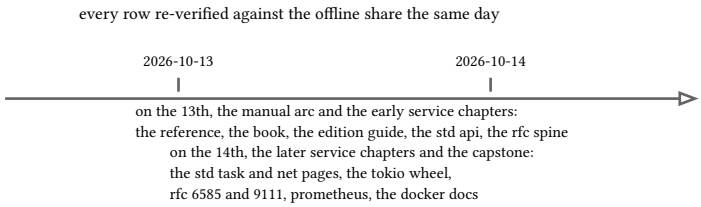


figure 62: the access window, when the pins were read

sources: the three coverage files this chapter renders, `books/rust/coverage/keywords.typ`, `stdlib.typ`, and `sources.typ`, enumerated 2026-10-13 and filled 2026-10-14 from the chapters' own recorded sources paragraphs, the taught-in map grepped from the 32 written chapters for each word's teaching site, and the sources table checked row by row against every chapter 19 through 32 trailing paragraph, the prometheus and docker rows the only additions the sweep found. The test counts quoted in the opening paragraph are chapter 31′s and 32′s own recorded runs of 2026-10-14 and 2026-10-15: 162 plain-lane tests plus 5 behind the docker gate, 36 capstone tests, 85 sample files at 902 checks with all 3 multi-language suites present.